PHYSICAL AI, TOGETHER

Online supplement

Implementation records, simulator configurations, detailed diagnostics and provenance accompanying the book. The address of the companion repository is `https://github.com/Mehdi-Heydari-Shahna/Physical-AI-Together`.

Mehdi Heydari Shahna

Companion to the book · version of 31 October 2026

Contents

S0 About the companion materials **1**

S3 The smallest complete example: records and details **2**
S3.1 Parameters of the example 2
S3.2 How the numbers are computed 2

S5 How a control researcher contributes: records and details **4**
S5.1 The numbers of the certificate 4
S5.2 The two scripts and how they check the runs 5
S5.3 Numerical details of the MPC package 5
S5.4 The constants of the sampled tracking law 5

S6 How a learning researcher contributes: records and details **8**
S6.1 Proposed and executed references in seven periods 8
S6.2 Reproducing the numbers of Chapter 6 9
S6.3 The published algorithms: implementations, environment and protocol 9
S6.4 The published SAC in detail 12
S6.5 The published TD-MPC2 in detail 18
S6.6 The published diffusion policy in detail 21
S6.7 What the reward pays for, and a second protocol 25
S6.8 Variants of the off-policy learner 26

S7 Predicting capability and demand: records and details **28**
S7.1 Reproducing the numbers of Chapter 7 28
S7.2 The record of a demand 29
S7.3 Errors injected into the demand forecast 29
S7.4 Errors injected into the capability forecast 29
S7.5 The calibrated set in worlds unlike the calibration worlds . . . 29
S7.6 What the closed loop selects, world by world 30

S8 The ALLOVI governor: records and details **33**
S8.1 The decision trace 33
S8.2 What the trace shows over a run 34

S8.3 The alarm runs in detail 34

S10 An architecture researchers can extend: records and details 36
S10.1 The data of one period 36
S10.2 The calls of one period, and what the core checks 37
S10.3 Registration and the build of a configuration 39
S10.4 How the check suite reaches its verdicts 40
S10.5 The template of a capability predictor 41
S10.6 The cart's robot model 41
S10.7 The numerics of a simulator adapter 42
S10.8 The counts of RoboCompiler's export 44
S10.9 The interfaces 45
S10.10 Compatibility checks and verification 51
S10.11 An optional model provider 56

S11 MuJoCo and Isaac Sim workflows: records and details 61
S11.1 The records of the two experiments 61
S11.2 Releases and installation 61
S11.3 The cart as MuJoCo draws its model 64
S11.4 The diagnostics in Isaac Sim 64
S11.5 The comparison in Isaac Sim at three time steps 66
S11.6 Traces 66
S11.7 The learning tool, a separate dependency 74
S11.8 Sliding in MuJoCo 75
S11.9 The cart in MuJoCo 75
S11.10 The cart in Isaac Sim 83
S11.11 The same declared experiment in two simulators 92

S12 Newton, Genesis and numerical credibility: records and details 97
S12.1 The per-period comparison 97
S12.2 Releases and the support matrix 97
S12.3 Newton's changes, imposed on MuJoCo itself 99
S12.4 The renderer 99
S12.5 The checks of the numerical model 100
S12.6 Single precision and the time step 101
S12.7 Failures and their reproduction 102
S12.8 The record, the report and one reproduced experiment 105
S12.9 Engine, solver and renderer 106
S12.10 Numerical validation and failures in the record 111

S13 Wheeled robots and the Husky: records and details 118
S13.1 The task and the asset in detail 118
S13.2 The contacts in detail 119
S13.3 The record of the Husky in MuJoCo 122
S13.4 The Husky in Isaac Sim 125
S13.5 The governed Husky in detail 132
S13.6 The record of the governed Husky 133
S13.7 The substitutions in detail 135
S13.8 The record of the substitutions 138
S13.9 Asset conversion details 139
S13.10 The contact-loss calculation 140
S13.11 Detailed capability and substitution tables 141

S14 Arms and heavy-duty mobile manipulators: records and details 146
S14.1 The UR5e in its sources 146
S14.2 The UR5e's stops at its labels 147
S14.3 The records of the two machines 149
S14.4 Further checks of the mobile manipulator's rigid model 149
S14.5 The lift and the turns 151
S14.6 The record of the labels, reserves and demands 152
S14.7 The governed runs: limits, details and provenance 152
S14.8 The record of the governed runs 155
S14.9 Bounds for the two reserve norms and the reserve proof 155

S15 A governed quadruped: records and details 158
S15.1 The Go2 as published 158
S15.2 The package's numbers 158
S15.3 Touchdowns on two contact models 158
S15.4 Provenance 159
S15.5 The record 159
S15.6 The Apollo without the beam's reference following the pelvis . 161
S15.7 An ungovened humanoid illustration 162
S15.8 Detailed claims ledger 163

S16 Comparisons that identify the contribution: records and details 166
S16.1 The record of the anticipation benchmark 166
S16.2 The runs of the benchmark and their timing 167
S16.3 When ALLOVI began the operation on time 167
S16.4 Paired statistics 167
S16.5 The differences under equal information 170
S16.6 The failed runs 171

S16.7 The interaction in the substitution grid 172

S17 How readers add their own research: records and details 173
S17.1 The helpers that the two contributions reuse 173
S17.2 Documented evaluation tasks 174
S17.3 The diagnoses of F4 and E4 174
S17.4 The worlds that are not valid 175
S17.5 The reproductions made by the author 176
S17.6 Detailed contribution records 177
S17.7 Procedural record: Contribution paths 177
S17.8 Procedural record: A contribution checklist 183
S17.9 Procedural record: Reproductions, negative results and releases 184

S19 Rendering and media provenance 188

SA Safety standards and regulation: records and details 189
SA.1 The status of the standards and acts cited 189
SA.2 Routes to conformity under the Machinery Regulation 193
SA.3 Per-robot source map 193
SA.4 Public-source notes on the robot families 194

References 201

CHAPTER S0

About the companion materials

This companion holds detailed run records, provenance, release-specific software instructions, diagnostic tables and implementation examples for *Physical AI, Together*. It accompanies the book's code in its repository: `https://github.com/Mehdi-Heydari-Shahna/Physical-AI-Together`. The instructor's manual, a separate companion with exercise solutions, is distributed to instructors separately.

Chapters follow the book's chapter numbers: Chapter S13 extends Chapter 13. Source labels are shared, so that references resolve when the three documents are built together.

Version pins identify the environments of the reported runs, not the current releases of the simulators. Some figure panels are static images taken from earlier renderings of the reported runs; Chapter S19 states their provenance. The executable release record is Section 17.3.4. The book states the mathematical assumptions and what its evidence establishes; this companion supplies the detail for inspection and reproduction.

CHAPTER S3

The smallest complete example: records and details

This chapter collects the parameters of the example of Chapter 3 in one table, and states which of its quantities the script of the chapter computes exactly and which in floating point.

S3.1 Parameters of the example

Table S3.1 collects the parameters that Sections 3.1.1 to 3.1.3 define one by one.

S3.2 How the numbers are computed

The script `code/ch03/cart_example.py` computes every number of Chapter 3, every planned and certified quantity in exact rational arithmetic. Only a

Table S3.1. Parameters of the smallest complete example.

Quantity	Value
Period; length of a run	$T = 0.1\,\mathrm{s}$; $22\,\mathrm{s}$
State; command	(p, v); acceleration u with $\lvert u\rvert \le 2\,\mathrm{m/s^2}$, and operation mode ℓ, both held for one period
Operation mode	$\ell = 0$, tool folded, or $\ell = 0.8\,\mathrm{m}$, tool unfolded
Declared constraint	$p + \ell \le 25\,\mathrm{m}$
Braking available (true world)	$2\,\mathrm{m/s^2}$ for $p < 20\,\mathrm{m}$; $0.5\,\mathrm{m/s^2}$ for $p \ge 20\,\mathrm{m}$; it limits every acceleration towards the start
Global capability	braking of at least $0.5\,\mathrm{m/s^2}$ everywhere
Reserve floor; levels	$0.5\,\mathrm{m}$; multiples of $1\,\mathrm{cm}$ from $0.5\,\mathrm{m}$
Ambitions offered	1, 0.75, 0.5, 0.25 and 0
Forecast horizon	400 periods, longer than a run
Task	reach the work zone, $p \ge 24\,\mathrm{m}$, and hold there
Operation	announced at $14\,\mathrm{s}$, onset at $20\,\mathrm{s}$; requires a reserve of $1.3\,\mathrm{m}$; mode $\ell = 0.8\,\mathrm{m}$ from $20.1\,\mathrm{s}$
Initial state and plan	at rest at $p = 0$; the rest segment, in mode $\ell = 0$
Measurement	exact (error bound zero)

few quantities are computed in floating point: the instants and speeds at which the cart crosses a position, the proposals executed without the governor, motion under an invalid label, and the checks of physical recovery, including the point of no return of Section 3.3. Run with the option `--check`, the script confirms every simulation result quoted in Chapter 3; the remaining numbers follow from these by the arithmetic shown there. The runs of Figures 3.2 to 3.5 are drawn directly from its output files.

The traces of the valid and the invalid run of Section 3.3, in the files `trace-forecasting.csv` and `trace-persisting.csv`, list for every period the executed command, its operation mode and its label. In the second, the row of period 49 shows the command -2 with the label 2 and the alarm.

CHAPTER S5

How a control researcher contributes: records and details

This chapter collects the numbers of the certificate of Section 5.1 in one table, describes how the two scripts of Chapter 5 integrate and check the runs, and gives the numerical details of the MPC package of Section 5.3.3.

S5.1 The numbers of the certificate

Table S5.1 collects the numbers of the certificate of the tracking segments, which Sections 5.1.3 to 5.1.5 derive.

Table S5.1. The certificate of the tracking segments of the cart, with the numbers that the handover and the rest segment use (Sections 5.1.3 to 5.1.5). Bounds marked with an asterisk are rounded up from the exact values, which `cart_feedback.py` checks in rational arithmetic. Data: `code/ch05/results/summary.txt`, section [certificate].

Quantity	Value
Disturbance bound, in every world	$\bar{w} = 0.05\,\mathrm{m/s^2}$
Feedback gains; closed-loop eigenvalues	$k_\mathrm{p} = 8/\mathrm{s^2}$, $k_\mathrm{v} = 4/\mathrm{s}$; $-2 \pm 2\mathrm{i}$ per second
Lyapunov function; decrease rate	$V = \sqrt{e^\top P e}$ with $P = \left(\begin{smallmatrix} 6.8 & 2 \\ 2 & 1 \end{smallmatrix}\right)$; $\gamma = 1.61/\mathrm{s}$
Disturbance gain; tube level	$\delta = 0.05\,\mathrm{m/s^2}$; $\varphi = \delta/\gamma = 5/161 \approx 0.0311\,\mathrm{m/s}$
Level on which the bounds hold	$\bar{\varphi} = \frac{1}{32}\,\mathrm{m/s}$
Error and input bounds for $V \le \bar{\varphi}$	$0.019\,\mathrm{m}^*$; $0.049\,\mathrm{m/s}^*$; $0.125\,\mathrm{m/s^2}$
Reference acceleration and braking	at most $1.875\,\mathrm{m/s^2}$; braking at most $1.875\,\mathrm{m/s^2}$ under label 2 and $0.375\,\mathrm{m/s^2}$ under label 0.5
Reference speed of a tracking segment	at least $0.049\,\mathrm{m/s}$ at both ends of the period
Handover speed to the rest segment (Section 5.1.6)	$0.05\,\mathrm{m/s}$
Least net braking of the rest segment under $\bar{c}$	$0.5 - 0.05 = 0.45\,\mathrm{m/s^2}$

S5.2 The two scripts and how they check the runs

Every simulation number of Chapter 5 comes from one of two scripts in the folder `code/ch05`; only the results of Chapter 3 that Table 5.1 repeats come from that chapter's script. `cart_feedback.py` serves Sections 5.1 and 5.2, and `cart_methods.py` serves Section 5.3. The first implements the control method of Sections 5.1 and 5.2 together with the governor, the predictors and the proposal method of Chapter 3. The second imports it and replaces only the control method and, for one package, the prediction from which the initial set is chosen. Both check every certificate in exact rational arithmetic and integrate the realized motion in floating point. Run with the option `--check`, each confirms every number of the chapter that it computes. Every figure of the chapter that shows data is drawn directly from the output files of these scripts and, for the comparison with Chapter 3, of its script.

In the governed runs of Section 5.2.5, the realized motion is integrated in floating point, with 200 steps per period. The script `cart_feedback.py` checks every executed segment against its funnel at every step, with a tolerance of 10^{-9} for rounding. The tolerance matters only where a motion runs along the edge of its funnel, as the launch does under a constant disturbance. The six further runs of Section 5.2.5 hold the disturbance at $+0.05$ or -0.05, or choose it at every instant to make V grow as fast as possible, each with the fixed and with the forecasting predictor; `cart_methods.py` runs each of its four packages under the same eight conditions.

S5.3 Numerical details of the MPC package

The MPC package of Section 5.3.3 computes the recursion of Equation (5.17) with the rational lower bound $38378/46875 \approx 0.81873067$ of ϑ and rounds the scales up. Its prediction of the motion that the executed segment produces from the measured state uses a rational approximation of the transition matrix $e^{A_K T}$, whose error is at most 2.1×10^{-23} in the maximum row-sum norm, and it rounds the predicted tracking error to a grid of 10^{-9}; both errors are added to the tube.

S5.4 The constants of the sampled tracking law

The paragraph 'The cart, sampled' after Lemma 5.2 uses the contraction factor λ, the increment δ_h and the settling level $\delta_h/(1-\lambda)$ of the cart's tracking law held over intervals of h seconds. For each interval, the listing also checks that the level stays below $\bar{\varphi}$ throughout an interval that begins at the highest level the sampled funnels reach, $\max\{5/161, \delta_h/(1-\lambda)\}$: it takes the largest level at

201 evenly spaced instants of the interval, and bounds how far the level can rise between two of them by a bound on its rate of change times half their spacing. Last, it gives the level after the samples of one period from $\varphi_0 = 5/161$, which decides whether consecutive segments chain at that level. The computation is in floating point; it is a check of a design, not part of the exact test of the control package, and not a certificate that the shipped code issues.

Listing S5.1. The constants of the sampled tracking law.

```
import numpy as np
from scipy.linalg import sqrtm
from scipy.integrate import quad
P = np.array([[6.8, 2.0], [2.0, 1.0]])    # the matrix of V
K = np.array([[8.0, 4.0]])                # the gains k_p and k_v
wbar, T = 0.05, 0.1                       # disturbance bound, period
phibar, phi0 = 1 / 32, 5 / 161            # the levels of the funnel
S = np.real(sqrtm(P)); Si = np.linalg.inv(S)
def M(t):        # the error map over an input held for t seconds
    return np.array([[1, t], [0, 1]]) - np.array([[t * t / 2], [t]])
    @ K
def lam(t):
    return np.linalg.norm(S @ M(t) @ Si, 2)
def dlt(t):
    return wbar * quad(lambda s: np.linalg.norm(S @ [t - s, 1.0]), 0,
     t)[0]
for ms in (0.25, 0.5, 1, 2, 5, 10, 20):
    h = ms / 1000; N = round(T / h); l, d = lam(h), dlt(h)
    top = max(phi0, d / (1 - l))          # the highest level reached
    grid = np.linspace(0, h, 201)
    worst = max(lam(t) * top + dlt(t) for t in grid)
    dM = max(np.linalg.norm(S @ (np.array([[0, 1], [0, 0]])
             - np.array([[t], [1]]) @ K) @ Si, 2) for t in grid)
    L = top * dM + wbar * (np.linalg.norm(S[:, 1]) + h * np.linalg.
    norm(S[:, 0]))
    phiN = l ** N * phi0 + d * (1 - l ** N) / (1 - l)
    print(f"{ms:5} {l:.6f} {d:.4g} {d / (1 - l):.5f}"
          f" {worst:.6f} {L * h / 400:.1e} {phiN:.6f}")
```

It prints, for h in milliseconds: λ, δ_h, the settling level, the largest level on the grid, the most that the level can exceed it between instants of the grid, and the level after one period from 5/161, all in metres per second:

```
0.25 0.999590 1.25e-05 0.03048 0.031056 9.4e-08 0.030968
0.5 0.999180 2.501e-05 0.03049 0.031056 1.9e-07 0.030971
1 0.998360 5.005e-05 0.03052 0.031056 3.7e-07 0.030975
2 0.996724 0.0001002 0.03059 0.031056 7.5e-07 0.030985
5 0.991837 0.0002513 0.03078 0.031056 1.9e-06 0.031014
10 0.983766 0.000505 0.03111 0.031108 3.8e-06 0.031064
20 0.967914 0.00102 0.03179 0.031795 7.9e-06 0.031167
```

With $\bar{\varphi} = 1/32 = 0.03125$, the largest level plus its margin stays below $\bar{\varphi}$ up to 10 ms, and not at 20 ms; the level after one period stays at most $5/161 = 0.031056$ up to 5 ms, and exceeds it at 10 ms.

CHAPTER S6

How a learning researcher contributes: records and details

This chapter holds the records and details that Chapter 6 refers to: the record of seven periods of Section 6.1.3, how the script of that chapter computes and draws its random numbers, the implementations, environment, checks and protocol of the published algorithms of Section 6.2.4, the results of the three published methods in detail, with SAC's exploratory runs at the stand-in's discount, an exploratory check of what the reward of SAC and TD-MPC2 pays for, with the second protocol that it led to, and the variants of the off-policy learner of Sections 6.3.3 and 6.3.4.

S6.1 Proposed and executed references in seven periods

Table S6.1 lists seven periods of the runs that Figure 6.1 plots, around the instant at which the governor begins to change the rule's proposals.

Up to the period that starts at 3.3 s, the rule's request is executed in full: as a launch at $2\,\mathrm{m/s^2}$ in the first period, and limited to $1.875\,\mathrm{m/s^2}$ after it. For the period that starts at 3.4 s, the governor commits the blend of ambition $1/4$.

Table S6.1. Proposed and executed reference accelerations, in metres per second squared, in seven periods of the world of Chapter 5, for the rule and the off-policy learner of the first seed. Each row concerns the period that starts at the instant given; the proposal for it was formed one period earlier. Data: `code/ch06/results/summary.txt`, sections [trace rule] and [trace off-policy learner].

	Rule			Off-policy learner		
Period starts	q_k	α_k	a^{ex}_{k+1}	q_k	α_k	a^{ex}_{k+1}
3.1 s	2	1	1.875	1.875	1	1.875
3.2 s	2	1	1.875	1.875	1	1.875
3.3 s	2	1	1.875	1.5	1	1.5
3.4 s	2	1/4	−0.9375	−1.875	1	−1.875
3.5 s	2	0	−1.875	−1.875	1	−1.875
3.6 s	2	0	−1.875	−1.875	1	−1.875
3.7 s	2	0	−1.875	−1.875	1	−1.875

From 3.5 s on, it commits the blend of ambition 0, except for the period that starts at 6.5 s, again 1/4. The cart brakes at 1.875 m/s^2 on firm ground and later at 0.375 m/s^2 on the ramp. The off-policy learner proposes 1.5 m/s^2 for the period that starts at 3.3 s and, from 3.4 s on, braking in all but three periods.

S6.2 Reproducing the numbers of Chapter 6

The script `code/ch06/cart_learning.py` implements the proposal methods and the protocol of Section 6.2.2, and reproduces every number of Chapter 6; with the option `--check`, it also checks every number quoted there, and the option `--quick` runs a smoke test with small budgets, whose numbers differ (Exercise 6.4). The governor and the test compute exactly. Elsewhere, the script uses only floating-point additions, subtractions, multiplications, divisions, square roots and rounding, in a fixed order, and computes its means exactly. Two exceptions call the platform's mathematical library, whose last digits may differ between platforms: an exponential in the funnel check of the contraction-metric package, and the sine disturbance of the world of Chapter 5. The script draws its random numbers only through the method `random()` of generators seeded with integers. Python promises only that `random()` keeps producing the same sequence when the compatible seeder is given the same seed (Python Software Foundation, 2026); the script relies on integer seeding being that seeder. Its results were identical, bit for bit, in runs with CPython 3.8, 3.13 and 3.14 on one x86-64 Linux machine; other processors, operating systems and versions have not been tested.

S6.3 The published algorithms: implementations, environment and protocol

This section describes the experiment of Section 6.2.4, which trains the published implementations of the three families on the cart through the adapters of Section 6.2.1 and under the protocol of Section 6.2.2. The package `code/ch06/published` holds it, with its protocol, `code/ch06/published/PROTOCOL.md`, written before any run of the three training scripts. No externally timestamped registration is cited, so claims about its chronology are provenance statements of the author, not independently verified preregistration. All three methods are reported (Table S6.2); their results are in Sections S6.4 to S6.6.

The implementations. The harness imports each implementation unchanged, at a pinned version, with the settings stated below. SAC is that of Stable-Baselines3 (Raffin et al., 2021; Raffin and Stable-Baselines3 Contributors, 2026), with the

Table S6.2. The published learning methods under the protocol: adapter, budget, and their status.

Method	Proposal adapter	Budget	Status
SAC	One reference acceleration, blended with fallback	40 and 1000 episodes	Run, three seeds: reported (Table 6.4)
TD-MPC2	First planned action only; one reference, blended with fallback	40 and 1000 episodes	Run, three seeds: reported (Table 6.4)
Diffusion policy	Chunks of eight references; prefixes	Expert demonstrations and one aggregation round	Run, three seeds: reported (Table 6.4)

library's default hyperparameters. TD-MPC2 is the official implementation (Hansen, 2026), with its default configuration and its single-task model of five million parameters. The diffusion policy is the authors' implementation for low-dimensional observations (Chi and Diffusion Policy Contributors, 2024), with the settings of its configuration for that case except the horizons, which are the chunk imitator's: $T_o = 1$, $T_p = 8$ and $T_a = 4$. The protocol lists every other setting that the experiment fixes, with its reason. For TD-MPC2, these are the settings for tasks that end, which its code provides, because the task ends at arrival. SAC is trained by the library's own loop. For TD-MPC2 and the diffusion policy, the scripts are described as following the training loops of the official code step by step, except that they stop at the protocol's budget and do not evaluate during training.

The environment. The three methods meet the governed loop through an environment with the interface of Gymnasium (Towers et al., 2024), `code/ch06/published/cart_env.py`. An episode is one run of the protocol in one world. The world is drawn from its seed as before, and the tube package, the forecasting predictor, the governor, the reserve floor, the task and the period are those of `code/ch06/cart_learning.py`, imported unchanged. The action of a step is the proposal q_k, and the governor answers it before the step returns. The observation is that of Table 6.1, extended by v_{env} and by whether the cart is on the low-braking region. SAC and TD-MPC2 propose one reference acceleration within the declared limit of $1.875\,m/s^2$ and receive the reward of the off-policy learner. TD-MPC2's configuration plans over three steps and exposes only the first action of its plan, as in receding-horizon execution. It therefore proposes one reference through the blending adapter of SAC, rather than the sequence-prefix adapter of the teaching planner, and its interventions compare with those of SAC and the off-policy learner, not with the planner's.

The diffusion policy proposes chunks of eight references through the prefixes, served with the timing of the chunk imitator. Like the imitator, it learns from the expert planner's chunks, with one round of dataset aggregation.

What has been checked. The environment reproduces the protocol of Chapter 6. The script `code/ch06/published/test_cart_env.py` trains and evaluates the proposal methods of the nine rows of Table 6.3 through the environment, as `code/ch06/cart_learning.py` does. Its 440 training episodes and 340 evaluation runs equal, value for value, the rows of the chapter's results files, so every cell of Table 6.3 is reproduced. The training episodes include the 50 that the two rows with delays repeat. Three paths that only the published methods use leave these runs unchanged. The first ends the learner's episode at arrival and lets the environment finish the run, as the libraries train. The second gives the proposals as single-precision numbers, as the libraries give them. The third serves chunks with the code that serves the diffusion policy's. In 150 episodes with random proposals, the governor committed a plan in every period, no alarm was raised, no executed segment left its funnel, and the cart never passed the reserve floor. A second script, `code/ch06/published/test_harness.py`, runs the three training scripts with stand-ins for the libraries, SAC's and TD-MPC2's also with the reward of the second protocol (Section S6.7); it checks the scripts' own steps, not the libraries. The first script also checks, in nine test runs of the rule, the braking curve and random proposals, each in three worlds, that the two rewards give the same run for the same proposals, and that every period's reward is what its formula gives.

Budgets, evaluation and decisions. The protocol fixes the following budgets. SAC and TD-MPC2 are trained for 40 episodes, the stand-ins' budget in the stand-ins' worlds, and for 1000 episodes, 25 passes over the same worlds. An episode has at most 200 periods before arrival, so the larger budget is at most 2×10^5 environment steps, and about 10^5 once the cart arrives in time. The policy of the smaller budget is a snapshot of the same run. The diffusion policy learns, as the chunk imitator does, from five episodes of the expert and five of aggregation in the same worlds, with the training schedule of its configuration. Each method is trained with seeds 1, 2 and 3, which select the training worlds as for the stand-ins. Every frozen policy runs in the same 20 evaluation worlds, with the metrics of Table 6.3. The protocol records one prediction and eight comparisons. The prediction is that of Proposition 6.1(b): no alarm, no step outside a funnel, and rest before the reserve floor, in every training episode and every evaluation run of every method. Each comparison sets a published method against its stand-in, world by world over the 20 evaluation worlds, by the sign test with Holm's correction over the eight (Holm, 1979). SAC is compared with the off-policy learner in arrival and intervention, at both budgets; TD-MPC2

with the planner with a learned model in arrival only; and the diffusion policy with the chunk imitator in arrival and intervention. The protocol also states what counts as doing better or worse, and that every outcome will be reported, including no difference and a difference in favour of a stand-in.

How the report decides. The rules are applied by the script `code/ch06/published/report.py`, which writes the file `code/ch06/published/results/summary.txt`. Its first version took Holm's correction over the primary tests that had been run, which agrees with the protocol only once all eight are in. Its second version, written after the SAC runs and listed under the protocol's deviations with its reason, takes the correction over the registered eight. While some of them have not been run, each adjusted p-value lies between two bounds: Holm over the eight with every missing p-value set to 1, and Holm over the tests run alone. A difference is stated only when the first bound is below 0.05, since it then holds whatever the missing tests give; 'no difference shown' only when even the second bound is not below 0.05; and otherwise the decision is pending. The second version also withholds a method until the folders of all three seeds hold their files, because the protocol's unit is the mean over the three trained policies in each world, and reports what the seeds in so far show as interim, apart from the results. With all eight tests in, the two versions decide alike.

S6.4 The published SAC in detail

Table S6.3 gives SAC's runs seed by seed, Table S6.4 its registered comparisons, Figure S6.1 the course of its training, and Table S6.5 the record of its runs; Table S6.6, at the end of the section, its exploratory runs with the stand-in's discount.

The four runs that do not arrive are those of seed 2's policy of budget L in the worlds 1006, 1009, 1015 and 1019. In training, 39 of the first 40 episodes of each seed reached the work zone within 20 s, and 787, 813 and 858 of the 1000. The three seeds took 168 174, 160 953 and 161 257 environment steps. Seed 1's policy of budget L proposed, in every period before arrival of its 20 runs, a reference that passed unchanged.

SAC with the stand-in's discount. The check of Section 6.2.4 that trains the stand-in with SAC's discount has a converse, chosen at the same time, after SAC's registered results: SAC itself trained with the stand-in's discount of 0.5, every other setting unchanged (`train_sac.py --gamma 0.5`, run on 30 October 2026 on the laptop's processor; its results apart from the registered ones, in `code/ch06/published/results/sac-gamma0.5`). Table S6.6 gives its runs and its comparisons, all exploratory and uncorrected. In training, 117 of

Table S6.3. SAC seed by seed: the evaluation runs in the 20 evaluation worlds with the columns of Table 6.3 other than the largest position, the share of the periods before arrival in which the proposal passed unchanged, and the measured inference time per proposal, median and largest. Arrival in seconds, over the runs that arrived. Data: the files `evaluation.csv` and `record.json` of `code/ch06/published/results/sac`, `seed1` to `seed3`.

Seed	Budget	Runs arrived	Mean arrival	Earliest, latest	Mean ι_k	Passed unchanged	Inference (ms)
1	M	20 of 20	10.10	9.7, 10.4	0.499	49.1 %	0.49, 2.81
1	L	20 of 20	15.57	13.0, 18.7	0.000	100.0 %	0.44, 2.70
2	M	20 of 20	10.07	9.7, 10.4	0.509	48.0 %	0.91, 2.76
2	L	16 of 20	18.93	16.5, 20.0	0.007	99.3 %	0.86, 1.93
3	M	20 of 20	10.17	9.8, 10.5	0.480	51.0 %	0.55, 1.75
3	L	20 of 20	16.07	14.9, 17.9	0.005	99.5 %	0.41, 1.38

the first 120 episodes and 2997 of the 3000 reached the work zone within 20 s, in 95 838, 96 332 and 96 428 environment steps; in its 3000 training episodes and 120 evaluation runs, no alarm was raised, no step left its funnel and the cart never passed the reserve floor. Its proposals took a median of 0.35 to 0.72 ms to compute, and at most 7.32 ms.

Table S6.4. The registered comparisons of SAC: the primary tests P1 to P4, against the off-policy learner, and the secondary tests that involve SAC alone. The unit is the evaluation world; in each, the mean over the three trained policies. Each test split the 20 worlds 20 to 0, for a two-sided sign-test p of $2^{-19} \approx 1.9 \times 10^{-6}$; Holm's correction over the eight primary tests raises it to 1.5×10^{-5} for P1 to P4, and the secondary tests are uncorrected. A mean over the worlds is undefined where a run in some world did not arrive. P5 and P6, of TD-MPC2, are in Table S6.8, and P7 and P8, of the diffusion policy, in Table S6.11. Data: `code/ch06/published/results/summary.txt`, groups [primary] and [secondary].

Test	SAC against	Quantity	Means over the worlds	Decision
P1	off-policy learner, budget M	arrival	10.110 s against 9.648 s	later
P2	off-policy learner, budget M	intervention	0.496 against 0.041	higher
P3	off-policy learner, budget L	arrival	undefined against 9.648 s; every run arrived in 16 worlds	later
P4	off-policy learner, budget L	intervention	0.004 against 0.041	lower
S1–S3	rule, braking curve, expert planner; budget M	arrival	10.110 s against 9.545, 9.555 and 9.660 s	later
S4, S5	rule, braking curve; budget M	intervention	0.496 against 0.656 and 0.246	lower; higher
S6–S8	rule, braking curve, expert planner; budget L	arrival	undefined against 9.545, 9.555 and 9.660 s	later
S9, S10	rule, braking curve; budget L	intervention	0.004 against 0.656 and 0.246	lower
S31	its own budget M; budget L	arrival	undefined against 10.110 s	later
S32	its own budget M; budget L	intervention	0.004 against 0.496	lower

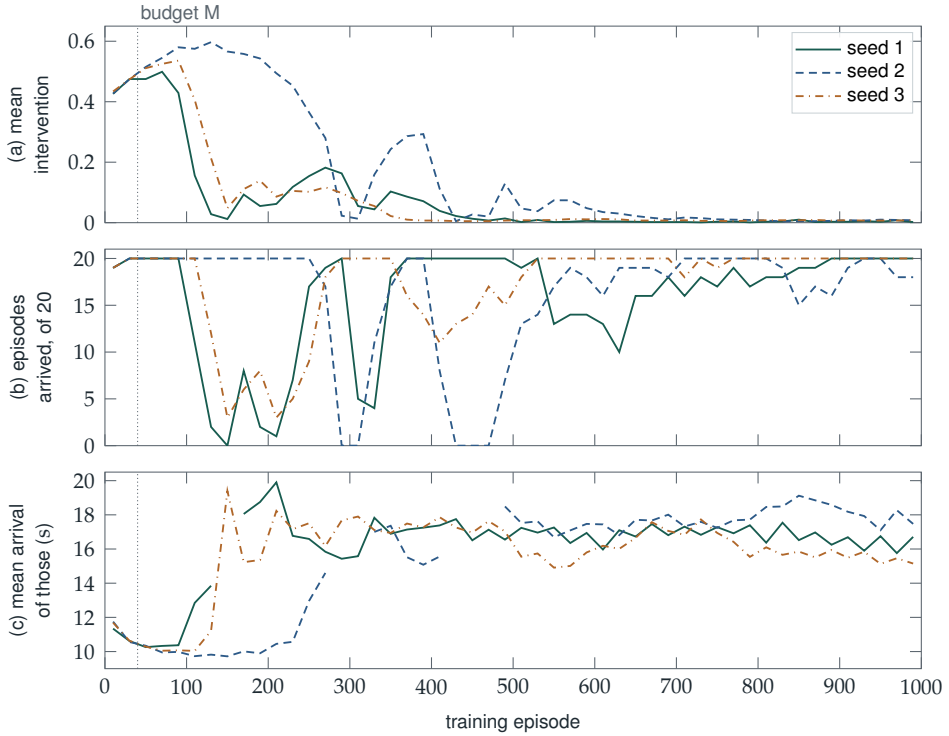


Figure S6.1. SIMULATION The training of the published SAC in its three seeds, in blocks of 20 training episodes: (a) the mean intervention, (b) how many of the 20 episodes reached the work zone within 20 s, and (c) the mean arrival time of those that did; no point is drawn where none did. The dotted line marks the snapshot of budget M. In every seed the policy first proposes more than the governor commits, and arrives at about 10 to 11 s; over the next few hundred episodes, with relapses, it changes to proposals that the governor commits unchanged, and arrives at about 15 to 19 s. While it changes, many episodes do not arrive in time. The policy of budget M is the snapshot after 40 episodes, that of budget L after 1000. No episode raised an alarm. Data: `code/ch06/published/results/sac`, `seed1` to `seed3`, `training.csv`.

Table S6.5. The record of the runs of SAC, after the files `record.json` of `code/ch06/published/results/sac`, which `train_sac.py` writes, and the group [record] of `code/ch06/published/results/summary.txt`.

Item	Record
Implementation	SAC of Stable-Baselines3 2.9.2, policy `MlpPolicy`, with the library's defaults as it holds them: learning rate 3×10^{-4}, replay buffer of 10^6 transitions, 100 random steps before learning, batches of 256, $\tau = 0.005$, discount 0.99, one gradient step per environment step, entropy coefficient tuned automatically towards a target entropy of -1, two hidden layers of 256 units; only the seed and the device set
Software	CPython 3.11.18 (conda-forge), torch 2.8.0 for the CPU, one thread; gymnasium 1.3.0, NumPy 2.2.6, cloudpickle 3.1.2; the environment `code/ch06/published/cart_env.py`, with the package, the predictor and the governor of `code/ch06/cart_learning.py` unchanged
Hardware	a Windows laptop (build 10.0.26100), x86-64 Intel processor with 20 logical processors; its NVIDIA RTX 4060 Laptop GPU was not used
Seeds and worlds	seeds 1, 2 and 3; seed s trains in the worlds $100s + i$ mod 40 of its episodes i; evaluation in the worlds 1000 to 1019
Budgets	M, the snapshot after 40 training episodes; L, after 1000
Wall-clock time	training 4491, 14 666 and 5486 s; evaluation 38 to 75 s for the 20 runs of a policy; the records written on 27 October 2026
Inference	per proposal, median 0.41 to 0.91 ms, largest 2.81 ms (Table S6.3); not charged against the deadline
Report	`report.py`, version 2 (29 October 2026); version 1 gives the same decisions
Guarantee status	H1 supported: in the 3120 runs, no alarm, no step outside a funnel, every run that arrived at rest, none beyond the reserve floor

Table S6.6. SIMULATION SAC with the stand-in's discount of 0.5, every other setting as in Table S6.5: the evaluation runs of the three trained policies in the 20 evaluation worlds, with the columns of Table 6.3 other than the largest position, which is at most 24.477 m, the share of the periods before arrival in which the proposal passed unchanged, and its comparisons world by world, by the sign test, with the off-policy learner at the same discount and with the baselines. Exploratory, chosen after SAC's registered results, and uncorrected; the registered decisions stand. Data: `code/ch06/published/results/sac-gamma0.5`, `seed1` to `seed3`, and `code/ch06/results/discount-check.txt`.

Item	Budget M	Budget L
Runs arrived	60 of 60	60 of 60
Mean arrival; earliest, latest	9.84 s; 9.3, 10.3	9.52 s; 9.0, 10.0
Mean ι_k; passed unchanged	0.598; 39.0 %	0.478; 51.2 %
Against the off-policy learner (discount 0.5)	later in 20 worlds; intervention higher in 20	earlier in 20 worlds; intervention higher in 20
Against the rule	later in 20 worlds; intervention lower in 20	earlier in 7, later in 1, the same in 12, $p = 0.070$: no difference shown; intervention lower in 20
Against the braking curve	later in 20 worlds; intervention higher in 20	earlier in 8, later in 3, the same in 9, $p = 0.23$: no difference shown; intervention higher in 20
Against the expert planner	later in 20 worlds	earlier in 20 worlds

S6.5 The published TD-MPC2 in detail

Table S6.7 gives TD-MPC2's runs seed by seed, Table S6.8 its registered comparisons, Figure S6.2 the course of its training, and Table S6.9 the record of its runs.

The three seeds' policies differ more than SAC's did. At budget M, seed 1's never moved the cart, seed 2's moved it at most 2.18 m and seed 3's as far as 7.58 m; at budget L, seed 2's never moved it, seed 1's took it to just short of the work zone and seed 3's to about 7 m. The three runs that arrived, seed 1's in world 1013 and seed 3's in worlds 1007 and 1013, came to rest at 24.469, 24.473 and 24.472 m. In training, 19, 16 and 21 of the first 40 episodes of the three seeds reached the work zone within 20 s, all after the five episodes of random proposals, and 180, 279 and 258 of the 960 that followed. The three seeds took 184 021, 175 539 and 180 652 environment steps, and as many updates less one.

Table S6.7. TD-MPC2 seed by seed: the evaluation runs in the 20 evaluation worlds; the runs in which the cart never moved by more than 1 cm; the range of the largest positions of the runs that moved but did not arrive, in metres; the mean intervention; the share of the periods before arrival in which the proposal passed unchanged; and the measured inference time per proposal, median and largest. Data: the files `evaluation.csv` and `record.json` of `code/ch06/published/results/tdmpc2`, `seed1` to `seed3`.

Seed	Budget	Runs arrived, at (s)	Did not move	Others reached (m)	Mean ι_k	Passed unchanged	Inference (ms)
1	M	0 of 20	20	—	0.000	100.0 %	86.7, 361.9
1	L	1 of 20, 19.8	0	20.86 to 23.80	0.132	85.1 %	70.7, 245.1
2	M	0 of 20	9	0.01 to 2.18	0.011	89.9 %	69.7, 201.4
2	L	0 of 20	20	—	0.000	100.0 %	69.7, 148.5
3	M	0 of 20	16	6.15 to 7.58	0.003	98.7 %	49.2, 143.3
3	L	2 of 20, 11.6 and 11.7	0	6.11 to 7.38	0.023	98.5 %	49.0, 131.0

Table S6.8. The registered comparisons of TD-MPC2: the primary tests P5 and P6, against the planner with a learned model, and the secondary tests that involve TD-MPC2. The unit is the evaluation world; in each, the mean over the three trained policies. TD-MPC2's mean arrival is undefined in every world at both budgets, since in every world at least one of its policies did not arrive. Unless stated, a test split the 20 worlds 20 to 0, for a two-sided sign-test p of $2^{-19} \approx 1.9 \times 10^{-6}$; Holm's correction over the eight primary tests raises it to 1.5×10^{-5} for P5 and P6, and the secondary tests are uncorrected. Data: `code/ch06/published/results/summary.txt`, groups [primary] and [secondary].

Test	TD-MPC2 against	Quantity	Means over the worlds	Decision
P5	planner with a learned model, budget M	arrival	undefined against 9.717 s	later
P6	planner with a learned model, budget L	arrival	undefined against 9.717 s	later
S11–S13	rule, braking curve, expert planner; budget M	arrival	undefined against 9.545, 9.555 and 9.660 s	later
S14, S15	rule, braking curve; budget M	intervention	0.004 against 0.656 and 0.246	lower
S16–S18	rule, braking curve, expert planner; budget L	arrival	undefined against 9.545, 9.555 and 9.660 s	later
S19, S20	rule, braking curve; budget L	intervention	0.052 against 0.656 and 0.246	lower
S25	off-policy learner; budget M	intervention	0.004 against 0.041	lower
S26, S27	SAC, budget M	arrival; intervention	undefined against 10.110 s; 0.004 against 0.496	later; lower
S28	off-policy learner; budget L	intervention	0.052 against 0.041; lower in 8 worlds, higher in 12, $p = 0.503$	no difference shown
S29, S30	SAC, budget L	arrival; intervention	both undefined, later in 16 worlds and the same in 4, $p = 3.05 \times 10^{-5}$; 0.052 against 0.004, higher in 19 worlds of 20, $p = 4.01 \times 10^{-5}$	later; higher
S33, S34	its own budget M; budget L	arrival; intervention	both undefined, the same in all 20 worlds; 0.052 against 0.004, higher in 17 worlds of 20, $p = 0.00258$	no difference shown; higher

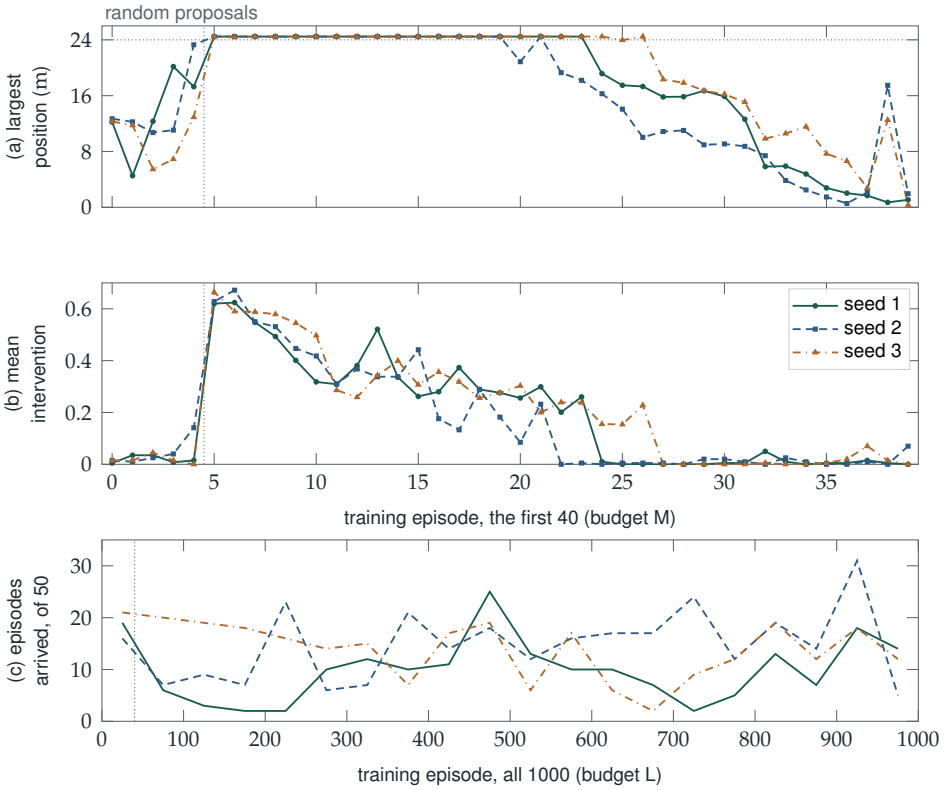


Figure S6.2. SIMULATION The training of the published TD-MPC2 in its three seeds: (a) the largest position of each of the first 40 training episodes, (b) their mean interventions, and (c) how many episodes of each block of 50 reached the work zone within 20 s. The first five episodes are TD-MPC2's random proposals. In every seed, the first policies after them bring the cart into the work zone, at 24 m, as the rule does, with a mean intervention near 0.6; over the next episodes they arrive later and the governor intervenes less, until the cart no longer reaches the work zone before the snapshot of budget M and, by episode 39, hardly moves. Over the 1000 episodes, in which the planner adds random noise to its actions, between 2 and 31 episodes of each block of 50 arrive. The policy of budget M is the snapshot after 40 episodes, episodes 0 to 39, marked by the dotted line in (c); that of budget L, after 1000. No episode raised an alarm. Data: `code/ch06/published/results/tdmpc2`, `seed1` to `seed3`, `training.csv`.

Table S6.9. The record of the runs of TD-MPC2, after the files `record.json` of `code/ch06/published/results/tdmpc2`, which `train_tdmpc2.py` writes, and the group [record] of `code/ch06/published/results/summary.txt`.

Item	Record
Implementation	the official TD-MPC2 at commit `332f19e4`, imported unchanged, with its configuration's defaults and the settings of the protocol: the single-task model of size 5 (5 461 601 parameters), tasks that end, $\rho = 0.7$ and a largest discount of 0.99; the discount 0.975, which its rule derives from the 200 periods of an episode (the largest discount does not bind); planning over three steps with 6 iterations of 512 samples, 64 elites and 24 trajectories of its policy prior; batches of 256, learning rate 3×10^{-4}; a buffer of 200 000 transitions, the smaller of its configured 10^6 and the steps that the configuration declares; random actions up to and including step 1000, the 1001st step, at which 1000 updates are made, then one update per step; `torch.compile` off
Software	CPython 3.11.18 (conda-forge), torch 2.7.1 with CUDA 12.8, tensordict 0.8.3, torchrl 0.8.1, hydra-core 1.3.2, omegaconf 2.3.0, NumPy 1.24.4, gymnasium 0.29.1; the environment `code/ch06/published/cart_env.py`, with the package, the predictor and the governor of `code/ch06/cart_learning.py` unchanged
Hardware	the Windows laptop of SAC's runs; the agent ran on its NVIDIA GeForce RTX 4060 Laptop GPU (8188 MiB)
Seeds and worlds	seeds 1, 2 and 3; seed s trains in the worlds $100s + i$ mod 40 of its episodes i; evaluation in the worlds 1000 to 1019, torch seeded with the world's seed before each run
Budgets	M, the agent saved after 40 training episodes; L, after 1000
Wall-clock time	training 47 756, 27 030 and 25 479 s, about 13, 7.5 and 7 hours; evaluation 222 to 401 s for the 20 runs of a policy; the records written on 28 and 29 October 2026
Inference	per proposal, median 49 to 87 ms, largest 362 ms (Table S6.7); not charged against the deadline
Report	`report.py`, version 2, as for SAC
Guarantee status	H1 supported: in the 3120 runs, no alarm, no step outside a funnel, every run that arrived at rest, none beyond the reserve floor

S6.6 The published diffusion policy in detail

Table S6.10 gives the diffusion policy's runs seed by seed, Table S6.11 its registered comparisons, and Table S6.12 the record of its runs.

The demonstrations of each seed are the five episodes of the expert planner that the chunk imitator learned from, 470, 483 and 475 records, in which the governor committed less than the full chunk in 342 of the 1428 periods and none of it in 219. In the five aggregation episodes of each seed, the policy trained on

Table S6.10. The diffusion policy seed by seed: the evaluation runs in the 20 evaluation worlds with the columns of Table 6.3 other than the largest position, which is at most 24.474 m; the share of the periods before arrival in which the governor committed the full chunk; the aggregation episodes, in which the policy trained on the demonstrations acted, that reached the work zone; and the measured inference time per chunk, median and largest. Arrival in seconds. Data: the files `evaluation.csv`, `training.csv` and `record.json` of `code/ch06/published/results/diffusion`, `seed1` to `seed3`.

Seed	Runs arrived	Mean arrival	Earliest, latest	Mean ι_k	Committed in full	Aggregation episodes arrived	Inference (s)
1	20 of 20	9.75	9.2, 10.2	0.285	64.8 %	3 of 5	1.42, 4.15
2	20 of 20	9.79	9.3, 10.3	0.392	53.7 %	2 of 5	1.46, 1.70
3	20 of 20	9.74	9.3, 10.2	0.378	55.2 %	5 of 5	1.46, 3.88

them acted and the expert labelled the states it visited, 690, 803 and 505 more records. Five of those fifteen episodes did not reach the work zone within 20 s: in seed 1, in the worlds 155 and 158, the furthest the cart went was 23.97 and 23.55 m, and in seed 2, in the worlds 256 to 258, 23.96, 23.96 and 23.49 m. Each training made 5000 passes over its data in batches of 256: 10 000 gradient steps on the demonstrations and 25 000, 30 000 and 20 000 on all the records, with a last mean loss of 0.0008 to 0.0061. The final policies, trained on all the records, arrived in every evaluation run; the governor committed the full chunk in 57.9 % of their periods before arrival.

Table S6.11. The registered comparisons of the diffusion policy: the primary tests P7 and P8, against the chunk imitator, and the secondary tests that involve it. The unit is the evaluation world; in each, the mean over the three trained policies. With all eight primary tests in, Holm's correction gives P7 0.804 and P8 1.5×10^{-5}; the secondary tests are uncorrected. Interventions compare only with methods that propose sequences. Data: `code/ch06/published/results/summary.txt`, groups [primary] and [secondary].

Test	Diffusion policy against	Quantity	Means over the worlds; worlds	Decision
P7	chunk imitator	arrival	9.758 s against 9.750 s; earlier in 7, later in 9, the same in 4; $p = 0.804$	no difference shown
P8	chunk imitator	intervention	0.352 against 0.139; higher in 20; $p = 2^{-19}$	higher
S21	rule	arrival	9.758 s against 9.545 s; later in 20; $p = 2^{-19}$	later
S22	braking curve	arrival	against 9.555 s; later in 19, the same in 1; $p = 3.81 \times 10^{-6}$	later
S23	expert planner	arrival	against 9.660 s; later in 16, earlier in 2, the same in 2; $p = 0.00131$	later
S24	expert planner	intervention	0.352 against 0.203; higher in 20; $p = 2^{-19}$	higher

Table S6.12. The record of the runs of the diffusion policy, after the files `record.json` of `code/ch06/published/results/diffusion`, which `train_diffusion.py` writes, and the group [record] of `code/ch06/published/results/summary.txt`.

Item	Record
Implementation	the authors' Diffusion Policy at commit `5ba07ac6`, imported unchanged: the low-dimensional U-Net policy, with channels 256, 512 and 1024, kernel 5 and FiLM scaling, 65 337 089 parameters; 100 diffusion steps with the squared-cosine schedule and noise prediction, and 100 denoising iterations per chunk; the observation as global condition; an exponential moving average with power 0.75; AdamW with learning rate 1×10^{-4} and the cosine schedule with 500 warm-up steps; batches of 256, 5000 epochs; $T_o = 1$, $T_p = 8$, $T_a = 4$
Software	CPython 3.11.18 (conda-forge), torch 2.7.1 with CUDA 12.8, diffusers 0.36.0, einops 0.8.1, zarr 2.18.7, NumPy 1.26.4, huggingface-hub 1.2.1, gymnasium 1.3.0; the environment `code/ch06/published/cart_env.py`
Hardware	the laptop of the other runs; the policy ran on its NVIDIA GeForce RTX 4060 Laptop GPU
Seeds and worlds	seeds 1, 2 and 3; seed s learns from the worlds $100s + 50$ to $100s + 54$ (demonstrations) and $100s + 55$ to $100s + 59$ (aggregation); evaluation in the worlds 1000 to 1019, torch seeded with the world's seed before each run
Wall-clock time	training 7832, 9672 and 7259 s, about 2.2, 2.7 and 2.0 hours, of which 1828, 2273 and 2263 s for the demonstrations and the first training; evaluation 1861 to 2208 s for the 20 runs of a policy; the records written on 29 and 30 October 2026
Inference	per chunk, median 1.42 to 1.46 s, largest 4.15 s, against the period of 0.1 s; not charged against the deadline
Report	`report.py`, version 2, as for SAC and TD-MPC2
Guarantee status	H1 supported: in the 30 training episodes and 60 evaluation runs, no alarm, no step outside a funnel, every run that arrived at rest, none beyond the reserve floor

S6.7 What the reward pays for, and a second protocol

The reward that the protocol gives SAC and TD-MPC2, that of the off-policy learner, is in every period before arrival the executed reference acceleration of the next period divided by $1.875\,\mathrm{m/s^2}$, minus 0.5 times the intervention, and 0 after arrival. A check written after TD-MPC2's results, and listed among the protocol's deviations as exploratory, `code/ch06/published/reward_check.py`, runs four reference policies through the environment in the 20 evaluation worlds and adds up the rewards of each run (Table S6.13): the rule, the braking curve, the off-policy learner of the three seeds, and a policy that proposes no acceleration and so keeps the cart at rest. Its runs of the first three are those of Table 6.3, run for run: their arrivals and mean interventions are those of the chapter's results file. It also adds up, for each run, a second reward, $-1 - 0.5\,\iota_k$ in every period before arrival and 0 after, which charges for time instead of paying for acceleration.

Without a discount, the registered reward pays little for arriving. Its first term adds up, over a run, to the change of the reference speed divided by $0.1875\,\mathrm{m/s}$, except that the reward counts the head of a rest segment as no acceleration: the last braking of each standstill, from at most $0.05\,\mathrm{m/s}$, the speed at which the rest segment takes over, is not charged. A run that arrives without stopping on the way has a reference speed of at most $0.612\,\mathrm{m/s}$ from 24 m on,

Table S6.13. SIMULATION What two rewards pay four reference policies in the 20 evaluation worlds (the stand-in is the off-policy learner, its three seeds pooled): the runs that arrive; the mean over the runs of S, the sum of a run's interventions before arrival; and the mean return of a run from its first period, without a discount and with the discounts of SAC, 0.99, and of TD-MPC2 under the protocol, 0.975. The registered reward is the protocol's; the time reward, $-1 - 0.5\,\iota_k$ before arrival, is that of the second protocol. A run that does not arrive is truncated at 20 s; under the time reward, its discounted return includes the $-\gamma^{200}/(1-\gamma)$ that a learner which continues past the truncation assigns to staying where it is. Over the runs that arrive, none of which stops on the way, the registered reward's first term ranges from 2.30 to 2.95, below the bound of 3.27 for such runs. Exploratory: not a result of the protocol. Data: `code/ch06/published/results/reward-check.txt`, written by `code/ch06/published/reward_check.py`.

			Registered reward			Time reward		
Policy	Arrived	S	None	0.99	0.975	None	0.99	0.975
Rule	20 of 20	62.61	−28.49	−6.86	6.82	−126.76	−78.49	−43.39
Braking curve	20 of 20	23.49	−9.07	2.34	9.86	−107.29	−69.26	−40.33
Stand-in	60 of 60	3.92	0.74	8.76	13.22	−98.44	−63.19	−37.02
At rest	0 of 20	0.00	0.00	0.00	0.00	−200.00	−100.00	−40.00

the largest from which the ramp's declared braking of $0.375\,\mathrm{m/s^2}$ still stops the cart before the reserve floor, so it collects at most 3.27 from that term; each standstill on the way can add up to 0.27. The reference policies' runs, which do not stop, collect from 2.30 to 2.95 from it, and a run at rest nothing. In all, the stand-in's runs collect 0.74 on average, the braking curve's -9.07 and the rule's -28.49. The published methods' rewards are not recorded, and their files do not say whether a run stopped on the way, so the check gives no bound for their runs, except that TD-MPC2's 65 runs in which the cart never moved collect exactly 0. A discount separates arriving from standing still through the timing of the accelerations, early ones counting more than the late braking. With TD-MPC2's discount, every reference policy that arrives collects more than a run at rest, so a policy that keeps the cart at rest, as TD-MPC2's did in 45 of the 60 runs of budget M, is not what its own discounted objective favours; the check says nothing about the policies that moved but did not arrive. With SAC's discount, the rule collects less than a run at rest. Under the time reward, the three policies that arrive are ordered alike at every discount, the stand-in first and the rule last; but with TD-MPC2's discount of 0.975 the rule and the braking curve collect less than a run at rest, which pays -1 per period for ever, while with 0.99 every policy that arrives collects more.

These readings led to a second protocol, `code/ch06/published/PROTOCOL-R2.md`, written after the results of SAC and TD-MPC2 and before any run of its own. It trains SAC and TD-MPC2 again with the time reward and gives TD-MPC2 SAC's discount of 0.99, through the official configuration; everything else, from the seeds and budgets to the evaluation, which uses no reward, is that of the first protocol. Its four primary comparisons set each method and budget against the same method and budget under the first protocol, in arrival, with Holm's correction over the four; for TD-MPC2 they compare the reward and the discount together. The scripts `train_sac.py` and `train_tdmpc2.py` run it with `--reward time`, and `report_r2.py` writes its results apart, in `results/summary-r2.txt`. Its runs have not been made; the results of the first protocol stand as reported, whatever the second gives.

S6.8 Variants of the off-policy learner

Table S6.14 lists the off-policy learner of Section 6.2.2 and the three variants of Sections 6.3.3 and 6.3.4: one records the executed references instead of its proposals, one ignores the label summary, and one computes its features at the state predicted for t_{k+1}. Figure S6.3 shows the mean intervention of every training episode of the learner and of the first two variants.

Table S6.14. The off-policy learner and three variants, each trained three times and evaluated in the 20 evaluation worlds with the tube package: arrival in seconds, mean and range; mean intervention; share of the periods before arrival in which the proposal passes unchanged; mean intervention of the first and the last ten training episodes; share of the training periods in aliased pairs (Section 6.3.4). Data: `code/ch06/results/summary.txt`.

Variant	Mean arrival	Earliest, latest	Mean ι_k	Passed unchanged, %	Training ι_k, first, last	Aliased periods, %
As in Section 6.2.2	9.65	9.1, 10.2	0.041	94.8	0.352, 0.127	22.3
Executed references recorded	9.54	9.0, 10.0	0.651	33.8	0.603, 0.599	61.1
Label summary ignored	9.69	9.1, 10.3	0.355	63.7	0.456, 0.375	92.9
State at t_{k+1} predicted	9.62	9.2, 10.1	0.033	95.9	0.357, 0.153	24.2

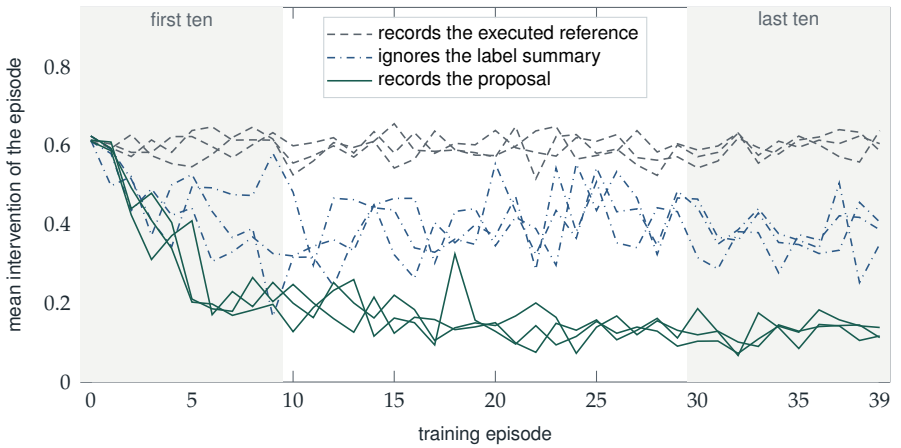


Figure S6.3. SIMULATION Mean intervention of every training episode of the off-policy learner of Section 6.2.2, which records its proposals (green), and of two variants of Table S6.14: one that records the executed references instead (grey, dashed) and one that ignores the label summary (blue, dash-dotted). Each line is one of the three trained policies over its 40 training episodes, each episode in a different world of the family. When the learner records its proposals, its interventions fall within the first ten episodes and then stay low; when it records the executed references, they stay near 0.6. The grey bands mark the first and the last ten episodes, whose means Table S6.14 lists: 0.352 and 0.127, 0.603 and 0.599, 0.456 and 0.375. No episode raised an alarm. Data: `code/ch06/results/training.csv`.

CHAPTER S7

Predicting capability and demand: records and details

This chapter holds the details that Chapter 7 refers to: how its script computes and checks its numbers, the record that turns the cart's operation into a demand (Section 7.2.1), the runs with an error injected into the demand forecast or into the capability forecast (Section 7.2.3), each world of the closed-loop runs of Section 7.3.3, and the calibrated set in worlds unlike the calibration worlds (Section 7.3.4).

S7.1 Reproducing the numbers of Chapter 7

Every number of Chapter 7 comes from the script `cart_forecasts.py` in the folder `code/ch07`. It imports the cart, its control method, the governor, the proposal method and the operation from the script of Chapter 3, unchanged, and also checks that, with the world and the predictors of Chapter 3, it reproduces every row of the runs of Chapter 3. The command `python3 cart_forecasts.py --check` runs every experiment of the chapter: the family of worlds of Table 7.3, the injected errors of Tables S7.2 and S7.3 and the calibration experiments of Section 7.3. It writes the numbers quoted in the text to `results/summary.txt`, with one file of comma-separated values per experiment beside it, and confirms each quoted number. The options `--bias`, `--onset-error`, `--level` and `--lead` inject errors of one's own choice into the world of Chapter 3, print the outcome and write no file (Exercises 7.1 and 7.4).

Planned and certified quantities, the realized motion under valid labels, and the calibrations and probabilities of Section 7.3 are computed exactly, in rational arithmetic. The confidence bounds are found by bisection in integer arithmetic and are never below the exact value. Motion under an invalid label and after an alarm, and one bound in Section 7.1.5, are computed in floating point. Random numbers come only from the method `random()` of generators seeded with integers, as in Chapter 6 (Python Software Foundation, 2026). The results were identical, bit for bit, in runs with CPython 3.8, 3.13 and 3.14 on one x86-64 Linux machine; other processors, operating systems and versions have not been tested.

Table S7.1. The record that turns the operation of the cart into a demand, and the check that the onset admits the operation.

Field	Value
Operation	Work with the tool unfolded, $\ell = 0.8\,\mathrm{m}$
Reserve consumed	Clearance in $p + \ell \leq 25\,\mathrm{m}$, in metres: 0.8 m
Level	Floor 0.5 m plus consumption: 1.3 m
Onset	Period 200: the plan committed then unfolds the tool, from 20.1 s
Announcement	Period 140, 14.0 s; lead time 60 periods
Source	Task plan
Check of Assumption 4.13	Plan in force at the onset: 1.3 m, cart at rest at 23.7 m; first plan with the tool unfolded: 0.5 m

S7.2 The record of a demand

Table S7.1 records the three steps of Section 7.2.1 for the operation of the cart, with the check that the onset admits the operation.

S7.3 Errors injected into the demand forecast

Table S7.2 lists the 14 runs of Section 7.2.3 with an error in the requirement, one wrong parameter each. The operation begins at its onset exactly when the lead time is at least 2.1 s, the level at least 1.3 m and the predicted onset not later than the real one. In all 14 runs, no alarm is raised, no motion crosses the constraint, and every committed plan is certified at the floor or above.

S7.4 Errors injected into the capability forecast

Table S7.3 lists the eleven runs of Section 7.2.3 in which the survey predictor, used without its offset, reads a survey biased by b. Every negative bias leaves every label valid; every positive bias makes some labels invalid, and one of them, executed and used, raises the alarm.

S7.5 The calibrated set in worlds unlike the calibration worlds

Table S7.4 lists the runs of Section 7.3.4: the calibrated set, $\hat{q} = 0.40\,\mathrm{m}$, in worlds as calibrated, after a shift to half unreliable surveys and in stale worlds, in which the ramp starts 0.5 m earlier than when the surveys were made, and the worst-case bound in the stale worlds. Its held-out miscoverages are those that Figure 7.2 plots.

Table S7.2. Errors injected into the requirement, one at a time, in the world and with the operation of Chapter 3; the capability predictor is the map. Back-away from: when the cart starts to reverse. Position at onset: where the front is at the real onset, period 200. At the work zone: the time that the cart spends at 24 m or beyond between 14.0 s, when the task plan announces the operation, and the real onset, whatever the lead time of the requirement.

Requirement	Operation at the onset	Back-away from, s	Position at onset, m	At the work zone, s
As stated	Begins	18.0	23.700	5.5
Lead time 3.0 s	Begins	18.0	23.700	5.5
Lead time 2.1 s	Begins	18.0	23.700	5.5
Lead time 2.0 s	Overload	18.1	23.710	5.6
Lead time 1.0 s	Overload	19.1	24.298	6.0
Lead time 0 s	Overload	—	24.500	6.0
Level 1.2 m	Overload	18.1	23.795	5.6
Level 1.25 m	Overload	18.2	23.740	5.7
Level 1.4 m	Begins	17.9	23.598	5.4
Level 1.6 m	Begins	17.7	23.378	5.2
Onset 1.0 s early	Begins	17.0	23.700	4.5
Onset 0.5 s early	Begins	17.5	23.700	5.0
Onset 0.1 s late	Overload	18.1	23.710	5.6
Onset 0.5 s late	Overload	18.5	23.938	6.0

S7.6 What the closed loop selects, world by world

Figure S7.1 plots, for each of the 50 worlds of Table 7.5, the score of the world and the share of invalid labels among the labels attached to candidate pairs in its run, with the two proposal methods of Section 7.3.3.

Table S7.3. Errors injected into the survey of the ramp start, used without offset, in the world and with the operation of Chapter 3; a negative bias places the ramp start too early. Invalid labels: among those attached to candidate pairs. Arrival: when the cart first reaches the work zone. The governed run ends at the alarm. Restart certified, stop and, after an alarm, violation are calculated from the measured state at the alarm, not simulated: the stop is where maximal braking, $0.5\,\mathrm{m/s^2}$ on the ramp, brings the cart to rest, and a restart can be certified once the cart is at rest exactly when the stop is at or before 24.5 m.

Survey bias b	Invalid labels	Arrival, s	Alarm	Restart certified	Stop, m	Violation
−1 m	0	8.8	None	—	—	No
−0.5 m	0	8.7	None	—	—	No
−0.2 m	0	8.6	None	—	—	No
0 m	0	8.6	None	—	—	No
0.05 m	43	—	5.6 s	Yes	24.50	No
0.1 m	77	—	5.6 s	Yes	24.36	No
0.2 m	127	—	5.6 s	Yes	24.48	No
0.3 m	167	—	5.6 s	Yes	24.48	No
0.35 m	237	—	5.6 s	No	24.99	No
0.5 m	335	—	5.5 s	No	25.59	Yes
1 m	568	—	5.4 s	No	27.12	Yes

Table S7.4. The calibrated set, $\hat{q} = 0.40$ m, in worlds as calibrated and, with the worst-case bound, in worlds unlike them. Shift: half of the surveys are unreliable. Stale: the ramp starts 0.5 m earlier than when the surveys were made. Columns as in Table 7.4; restarts: alarms after which the cart came to rest at or before the floor, from where a restart can be certified. Restarts and violations are calculated from the state at the alarm, as in Section 7.2.3.

Rule, worlds	Held-out miscoverage	Runs miscovered	Alarms, restarts	Violations	Runs arrived
Calibrated, as calibrated	3.4 % ($\leq$ 4.2 %)	0	0, 0	0	50
Calibrated, shift	8.3 % ($\leq$ 9.4 %)	1	1, 1	0	49
Calibrated, stale	70.4 % ($\leq$ 72.1 %)	34	28, 10	0	22
Worst case, stale	26.4 % ($\leq$ 28.1 %)	18	13, 10	0	37

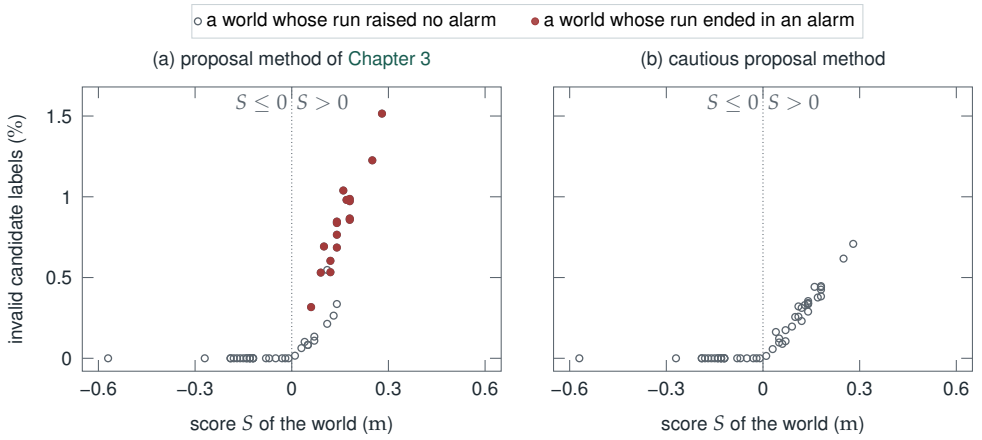


Figure S7.1. SIMULATION What the closed loop selects: the point estimate of Table 7.4 in its 50 worlds, without the operation, with two proposal methods. Each mark is one world: its score S, the distance by which the survey places the ramp start too far ahead, and the share of invalid labels among all the labels attached to candidate pairs in its run. Every world with $S > 0$ is miscovered and has invalid candidate labels, a small share of them in each; no world with $S \leq 0$ has any. Whether an alarm follows depends on the proposal method: (a) 17 of the 28 miscovered worlds end in an alarm with the method of Chapter 3, which brakes hardest just before $\hat{s}$; (b) none does with the cautious method, whose commands never use the braking that an invalid label overstates. Data: `code/ch07/results/loop.csv`.

CHAPTER S8

The ALLOVI governor: records and details

This chapter holds what Chapter 8 moved out of its text: the fields of the decision trace and the record of its runs, the counts of the forecasting run, and the detail of the alarm runs.

S8.1 The decision trace

The script `cart_governor.py` in `code/ch08` (Section 8.1.2) records, for every candidate of every period:

- its index i, 0 for the fallback, its ambition $\alpha^{(i)}$, its head with the label of the head, its number of pairs and its point of rest;
- its level $\hat{\varepsilon}_i(0)$ and its onset level $\hat{\varepsilon}_i(n_{k+1})$, each with its reason: the first pair that the test does not certify and the condition that it fails, or the pair or point of rest with the least clearance, which may lie below the floor;
- whether it is admitted, its two keys and its outcome: committed; admitted but with a lower first key; with the same first key but a lower ambition; tied on both keys and passed over, the fallback for a fresh candidate or a fresh candidate for a smaller index; or not admitted, because a pair is not certified, because its clearance falls below the floor or, in the budget runs of Section 8.3.3, because it was not evaluated within the budget.

For every period, the trace also records the state, the initial set, the proposal, the requirement, the committed candidate and the reason for the choice; in the runs of Section 8.3, it records as well the mode of the governor, the number of fresh candidates evaluated and, after an alarm, the search for a restart and its reason.

Checks. The script imports the cart, its generator, its test, its capability predictors and its proposal method from the script of Chapter 3, unchanged. With the configuration of that chapter, it reproduces every row of its runs with the fixed, reactive and forecasting capability predictors, selects in every period the candidate that the selection function of that chapter selects, and checks in every period that evaluating the fallback would give its shifted profile. The survey predictor of Section 7.2.3 and the rest feedback are restated in the script, which

checks the alarms and the stops against the script of Chapter 7. Every number of Chapter 8 comes from the script, which is run in one of two ways:

- `python3 cart_governor.py --check` runs every experiment of the chapter, writes the traces and the numbers quoted in the text to `results`, and confirms each quoted number;
- `python3 cart_governor.py --period 179` prints the trace of one period of the forecasting run.

Precision and reproducibility. Every planned and certified quantity is computed in exact rational arithmetic, as in Chapter 3. In the runs of Sections 8.1 and 8.2 and in the budget runs of Section 8.3.3, every label is valid, so the realized motion is exact as well. In the alarm runs of Section 8.3.4, the motion of the period of the alarm is computed in floating point and the point of rest exactly. The results were identical in runs with CPython 3.8, 3.13 and 3.14 on one x86-64 Linux machine.

S8.2 What the trace shows over a run

Table S8.1 counts the outcomes of all candidates and the reasons for all choices in the 220 periods of the forecasting run. In 160 periods no demand was pending and the most ambitious admitted candidate was committed. In the 60 periods from the announcement to the onset, the first key narrowed the choice: the most ambitious candidate holding the required reserve from the onset on was committed in 44 periods, the fallback in 14, and a tie on both keys went to a fresh candidate in 2. In each of these 60 periods, the committed plan held 1.3 m from the onset on. Of the 1540 candidates, 260 were not admitted. In 212 of them the clearance fell below the floor, as for the blends of period 32 in Table 3.2; in the other 48 a head braked harder than its label allowed, all in periods 86 to 97, while the cart braked on the ramp.

S8.3 The alarm runs in detail

Section 7.2.3 ended the two runs of Table 8.4 at the alarm and calculated the stop; Section 8.3.4 simulates the response, and the script checks that the alarms and the stops are the same. With the survey biased by 0.2 m, the cart passes into the work zone at 8.4 s under the rest feedback, without a guarantee.

The script runs both surveys a second time with a rest feedback that brakes at $0.5\,\mathrm{m/s^2}$, which every world consistent with $\bar{c}$ realizes, instead of maximally; its files in `code/ch08/results` end in `-global`. With the bias of 0.2 m, the motion is practically the same, because the ramp brakes at that rate, but the

Table S8.1. The outcomes of the 1540 candidates, fallbacks included, and the reasons for the 220 choices of the forecasting run, as the trace records them.

Outcome of a candidate	Candidates
Committed	220
Admitted; same first key, lower ambition	835
Admitted; lower first key	99
Admitted; tied on both keys, the fallback passed over for a fresh candidate	64
Admitted; tied on both keys, passed over for a smaller index	62
Not admitted; clearance below the floor	212
Not admitted; a pair not certified: acceleration towards the start beyond its label	48

Reason for the choice	Periods
No demand pending: the most ambitious admitted candidate	160
Demand: the most ambitious candidate with the highest first key	44
Demand: tied on both keys, broken for a fresh candidate	2
Fallback: the only candidate with the highest first key	14

restart comes one period after the alarm: a restart candidate that brakes at that rate is certified at 0.51 m and committed at 5.7 s, so the guarantees resume 4.2 s earlier. Its plan brakes in whole periods and stops the cart within the last one at a lower rate, so the cart comes to rest 0.47 mm farther, at 24.480 47 m; this is why its level is 0.51 m rather than 0.52 m. With the bias of 0.35 m, no restart candidate is admitted with this rest feedback either.

CHAPTER S10

An architecture researchers can extend: records and details

This chapter holds the reference material of Chapter 10 for a reader who implements a module against the code of `code/ch10`: the data of one period; the calls of one period, with their defaults, and what the core checks in an output; registration and the configuration of one run; how the check suite reaches its verdicts; the template of a capability predictor; the cart's robot model; the numerics of a simulator adapter; and the counts of RoboCompiler's export.

S10.1 The data of one period

Four records cross the interfaces in every period (Listing S10.1). The core forms the observation o_k at t_k: the period, its time $t_k = kT$, the measured state, the level of the plan in force, which is $-\infty$ after an alarm, the ambition committed at t_k, even if an alarm is raised then, as in Algorithm 8.1, and whether the rest feedback acts after an alarm. The proposal method and both predictors receive it. The demand predictor returns a requirement (Definition 4.12), a level and an onset. The core passes the generator a step: the period, the proposal or `None`, the labelling map λ_k as a function of a period and a segment, the requirement, what the task asks the generator to begin, and the measured state; the plan in force and the initial set are passed beside it. The generator returns candidates, each a plan with its ambition, listed in the order of evaluation.

The profile that the test returns holds the largest levels $\hat{\varepsilon}(0), \ldots, \hat{\varepsilon}(N)$ at which the plan is certified from each pair on, constant after the last, with $-\infty$ where nothing is certified (Definition 4.9). The vector reserves of Section 4.2.7 would need a level space that admits component by component and whose key is ψ of the componentwise minimum, a separable test, and one norm for each family of constraints; the state declaration provides one norm only, and the core marks a plan that is not certified with a scalar $-\infty$. The decisions are the core's own records: the committed plan with its profile and ambition, the reason for the choice, and, for every candidate, its profile, its keys and its outcome.

Listing S10.1. The data of one period (`allovi/contract.py`, without docstrings and comments).

```
@dataclass(frozen=True)
class Observation:
    k: int
    t: Any
    measured: Any
    level: Any
    ambition: Any
    alarm: bool

@dataclass(frozen=True)
class Requirement:
    level: Any
    onset: int = 0

@dataclass(frozen=True)
class Step:
    k: int
    proposal: Any
    labelling: Callable[[int, Any], Any]
    requirement: Requirement
    operation: Any
    measured: Any

@dataclass(frozen=True)
class Candidate:
    plan: Any
    ambition: Any
```

S10.2 The calls of one period, and what the core checks

Table S10.1 lists the calls of one period of the loop (Section S10.9.3) in order, with what replaces an output that is missing or unusable in operation (Section S10.9.4).

Before it acts on an output, the core checks it against the contract: candidates must form a list, each with a plan that can be shifted and an ambition in $[0, 1]$; a profile must have levels of the level space or $-\infty$ as entries and must not decrease; a requirement must have a level of the space and an onset that is a whole number of at least 0; a labelling map must be callable; a distance must be a number. Outputs that it only passes on, such as initial sets, feedback laws and states, it does not inspect.

Every output that has a default in the contract (Section 4.3.5) can be replaced by it (Table S10.1). A generator that fails offers no fresh candidate, so the fallback

Table S10.1. The calls of one period in the loop of the core, in order, with what replaces an output that is missing or unusable, or whose call fails, in operation (`strict=False`). With strict checking, every failure stops the run.

Call	Returns	If missing or failing, in operation
In every period		
`task.begin_period`(k, level in force)	—	no default
`proposal.propose`(o_k)	proposal q_k	recovery candidates only
`demand.requirement`(o_k)	requirement	$(\varepsilon_{\min}, 0)$: no demand
`task.operation`(k)	what to begin	no default
In normal operation		
`capability.labels`(o_k)	λ_k	$\bar{c}$; always $\bar{c}$ after a restart
`control.initial_set`(P_k)	I_{k+1}	no default
`control.generate`(step, P_k, I_{k+1})	candidates	none: the fallback alone
`control.profile`($P^{(i)}$, I_{k+1}) for $i = 1, 2, \ldots$ within the budget	profiles	not certified
fallback, admission, selection (the core)	P_{k+1}	—
`world.step`(x, `control.law`(P_k))	$x(t_{k+1})$	no default
`robot.measure`($x(t_{k+1})$)	$\tilde{x}(t_{k+1})$	no default
`control.distance`($\tilde{x}(t_{k+1})$, I_{k+1}) $> \varpi$	alarm	no default
After an alarm, until a restart		
`control.reach`($\tilde{x}(t_k)$, ϖ)	I_{k+1}, or none with a reason	none: no restart in the period
`control.restart`(step, I_{k+1})	restart candidates	none: no restart in the period
`control.profile`($P^{(i)}$, I_{k+1}) for $i = 1, 2, \ldots$ within the budget	profiles	not certified
`world.step`(x, `control.rest_law`($\tilde{x}(t_k)$)), `robot.measure`	$x(t_{k+1})$, $\tilde{x}(t_{k+1})$	no default
In every period		
`capability.observe`(k, $\tilde{x}(t_k)$, $\tilde{x}(t_{k+1})$)	—	the event is recorded

is committed. A generator that raises on a proposal that it cannot use has the effect of one that fails. A test that fails on a candidate leaves it not certified, and without restart candidates, no restart is possible in that period. In operation, each event is recorded with the period and what went wrong.

After an alarm, the control package may answer that it cannot represent the set that its rest feedback can reach, with no set and a reason, as the cart's package does while the cart moves under maximal braking. That is a legitimate answer, not a failure: it is recorded with its reason, and no restart is attempted in that period. A call of `reach` that raises or returns something else is a failure; in operation, the core replaces it by the same answer and records the event.

Listing S10.2. The configuration of one run of the cart (`cart/__init__.py`, without its docstring).

```
def configuration(name, capability="cart.forecasting",
                  capability_args=None, rest="maximal", budget=None,
                  world="cart.track", **operation):
    task = modules.Operation(**operation)
    return allovi.build(name, T, {
        "control": ("cart.constant-command", {"rest": rest}),
        "proposal": ("cart.naive",),
        "capability": (capability, capability_args or {}),
        "demand": ("cart.task-plan", {"task": task}),
        "task": task,
        "robot": ("cart.robot",),
        "world": (world,),
        "evaluator": ("cart.metrics", {"task": task}),
    }, budget=budget)
```

Either way, the rest feedback continues and nothing is claimed until a restart plan is committed.

Every other call has no default: the calls to the control package for the initial plan, the initial set, the law, the distance and the rest feedback, the call of a plan's `shift`, which gives the fallback, and the calls to the robot and simulator adapters and to the task. An exception there, or an output that the core cannot use, stops the run in both modes, with an error that names the period and the call.

S10.3 Registration and the build of a configuration

The decorator `register` adds a class to the default registry, which checks that

- the role is one of the eight;
- the name is a nonempty string, not yet used by another module of the role;
- the first number of the contract version is that of the core;
- every method of the role is present and accepts the number of positional arguments of the interface.

A registered module can then be named in a configuration without any change to the core. Listing S10.2 assembles the modules of one run of the cart. Each entry names a role and a registered module with its arguments, or gives a module already created, as for the task, which the demand predictor and the evaluator also use.

Building a configuration creates each module with its arguments and checks that the control package, the robot adapter and the simulator adapter are present, since without them nothing can be certified, executed or measured; the proposal method, the predictors, the task and the evaluator are optional, as Section 4.3.5 allows. It also checks that every module has the attributes of its role, that the level space and the capability space of the control package have their attributes and methods, and that the declarations agree (Section S10.9.2).

A few further methods are optional and serve the records only: the control package can explain each level and describe its plans, sets and proposals, and the robot adapter its states; they add columns to the traces and never influence a decision. Five more, also optional, serve only the checks (Section S10.4).

S10.4 How the check suite reaches its verdicts

The module `checks.py` of the core runs the configuration once with strict checking and diagnostics (Figure S10.1). Wrappers around the capability predictor and the simulator adapter record what those modules receive and return, and change nothing; to check labels and inputs, they also call labelling maps and laws once more, which assumes that these calls have no side effects. The suite then runs the configuration a second time and compares every record, and it asks the oracle about the first run.

If the first run stops because a module fails or a call without a default raises, the check of the outputs fails, naming the period and the call, and the fallback check, the second run and the falsification checks are not applicable; a second run or an oracle that raises fails its checks. The falsification checks need three optional methods: `head(plan)` of the control package, the pair that a plan executes in its first period, and `admits(label, segment, k)` and `clearance(state)` of the simulator adapter. The cart's track implements the last two from the true braking of its ground and the position of the wall.

To check labels, the suite records every label that the generator asks for. Just before the capability predictor learns the measurements of the period, the suite asks the labelling map of the period again for each of those labels, to see whether the answer has changed, and for the label of each recorded segment in the first period beyond the declared horizon. The second request is the one that the predictor of Example 9.19 fails: it keeps an informative label beyond its horizon. Whether a label is at least as informative as the global capability is asked of the capability space, through its optional method `informative`, which the cart's has; a module may also supply checks of its own through a method `checks()`. These five methods are the only ones that the suite needs beyond the interfaces of Section 10.2.

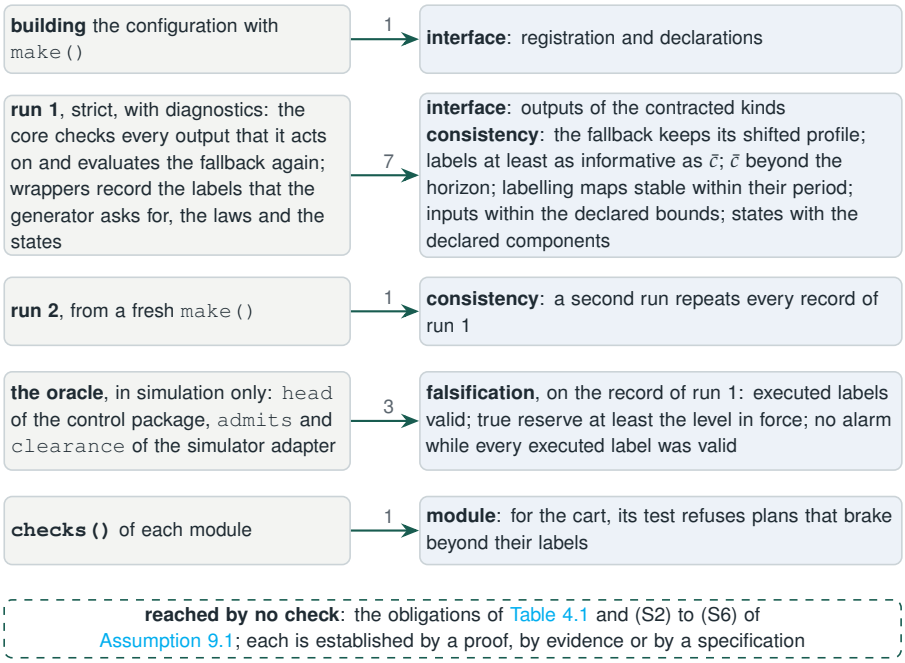


Figure S10.1. ILLUSTRATION How the check suite of `code/ch10/allovi/checks.py` reaches its verdicts: where each check takes its evidence from (left), and the checks, with their kinds (right); the numbers count the checks that the suite runs on the cart, 13 in all. Each check reports passed, failed or not applicable, with the number of cases that it examined. If run 1 stops, the check of the outputs fails, and the fallback check, the second run and the falsification checks are not applicable. Without the three optional methods of the oracle, the falsification checks are not applicable; they never run on the machine. No check reaches the obligations of the contract (Table S10.2).

S10.5 The template of a capability predictor

Listing S10.3 is the template of one role, as `code/ch10/templates` holds it for each of the eight (Section S10.10.4). The docstring of its file, left out here, states the obligations of the role, which no check discharges, and the checks that the suite runs on it.

S10.6 The cart's robot model

Listing S10.4 is the record through which the cart's model crosses the boundary of Section S10.11.1. Its state and actuation are the declarations of Listing 10.1.

Listing S10.3. The template of a capability predictor, without the docstring of its file (`templates/capability_predictor.py`).

```
class MyCapabilityPredictor:
    ROLE, CONTRACT = "capability", "1.0"
    NAME = "template.capability-predictor"
    # the NAME of the capability space of the test
    SPACE = "cart.braking"
    # periods; beyond them, every label is the global capability
    horizon = 0

    def labels(self, observation):
        """The labelling map of period observation.k: a function
        labelling(j, segment) that returns the label of the segment
        for period j, used only during period k; or None if no map
        is ready."""
        raise NotImplementedError

    def observe(self, k, before, after):
        """The measured states at the start and at the end of period
        k, reported after the decision of the period."""
        raise NotImplementedError
```

S10.7 The numerics of a simulator adapter

The thirteen runs of Section 10.1.3 repeat the records of Chapter 8 to the last digit, which needs more than a correct core. In the period of each alarm, the head of the plan brakes harder than the ground allows, and its motion is computed in floating point, as in Chapter 3; only the four alarm runs contain such a period, one each. The simulator adapter as first written kept the state at the end of that period as computed, with the rounding error of its position. The cart then came to rest beyond the exact point of rest: by 1.7×10^{-15} m with the survey biased by 0.2 m, and by 1.8×10^{-15} m with the survey biased by 0.35 m. An error of this size changes a level by one step whenever it moves a clearance across a level of the grid. In the run with the survey biased by 0.2 m and maximal braking, the restart candidate that holds the cart at rest, evaluated in period 98, was certified at 0.51 m instead of 0.52 m, because its clearance fell just below a level of the grid. In both runs with the survey biased by 0.35 m, a restart candidate of period 140 had the onset level 1.38 m instead of 1.39 m. Four of the eight files of the alarm runs differed, with the core and every other module unchanged. In both versions, every committed plan is certified for the states that the adapter produces; the versions differ in those states.

Listing S10.4. The robot model of the cart, as supplied with the application (`cart/model.py`).

```
def _motion(x, u, h):
    """The nominal state after an interval h under inputs u held
    over it: a constant acceleration, and the commanded reach."""
    p, v, _ = x
    return (p + v * h + u[0] * h * h / 2, v + u[0] * h, u[1])


# The robot model, supplied with the application as in Chapter 3.
# The robot adapter, the simulator adapter and the control package
# declare its state and actuation.
MODEL = RobotModel(
    name="cart.chapter-3",
    provider="supplied with the application (Chapter 3)",
    state=STATE,
    actuation=ACTUATION,
    points=(Quantity("bumper", "m", "track"),
            Quantity("tool tip", "m", "track")),
    kinematics=lambda x: (x[0], x[0] + x[2]),
    dynamics=_motion,
    assumptions=(
        "every input is realized as commanded; where the ground "
        "brakes less than a command asks, a capability label says "
        "so (Section 4.2)",
        "the reach is set at the start of a period and held"),
    sources=SOURCES)
```

The adapter now keeps the velocity as computed and corrects the position by its rounding error, so that the point of rest that the state implies is exact; the script `cart_allovi.py` runs both versions. The guarantees do not need the correction, but the modular runs need it to repeat the levels that Chapter 8 records. The script of Chapter 8, which saw the whole loop at once, also kept the floating-point state, but whenever the rest feedback brought the cart to a stop it set the exact point of rest, computed from the start of the period, both in the motion and in the set that the rest feedback can reach. A control package sees only the measurement, so it computes that set exactly only if the state implies the exact point of rest. The two scripts therefore record the same numbers in every printed digit, but their states differ, from the alarm on, by at most 1.8×10^{-15} m. They differ at 42 to 165 of the instants of each alarm run, and until the end of the run in which a restart plan is committed while the cart still moves: the run with the survey biased by 0.2 m and braking at 0.5 m/s^2. For a control package that computes exactly, as that of the cart does, the numerics of

the simulator adapter are thus what can still change a recorded number; they belong to the record of an experiment.

S10.8 The counts of RoboCompiler's export

The export of RoboCompiler's compile step for its six-UPS Stewart platform (Section S10.11.3) was read at commit `a4e4437`. The input graph has 21 bodies and 25 joints (12 revolute, 6 prismatic, 6 spherical and one fixed) and declares 6 actuators, forces in newtons on the prismatic joints. A graph with more joints than a tree needs has cycles, $25 - 21 + 1 = 5$ of them here, and a compiler keeps a spanning tree of the joints and cuts one joint, a *chord*, in each cycle, whose closure it then imposes as a constraint. The compiled model closes the 5 physical cycles with 5 spherical chords on a tree of 21 coordinates, and its plan counts the chords in two representations. As a point closure on the tree coordinates, a chord adds 3 position rows, 15 in all, and the plan's dimensional mobility is 21 less 15, or 6. In the augmented coordinates on which RoboCompiler's assembly runs, a chord adds 3 chart coordinates, 36 coordinates in all, and a full rigid-body residual of 6 rows, 30 in all, the count that Listing 3 of the RoboCompiler manuscript reports (cited in Section S10.11.3); 36 less 30 is also 6. As independent coordinates the compiled model takes the 6 actuated leg lengths, which the input declares; their rank is checked when a configuration is assembled.

The adapter takes the independent coordinates and their rates as the state; another adapter could take the pose of the platform, which RoboCompiler's assembly computes from them. For the dynamics, RoboCompiler's closure maps and curvature term take the tree inertia and bias from a rigid-body evaluator: the compiler workflow's own NumPy evaluator, checked against Pinocchio when Pinocchio is installed, or, in the simulation studies, Pinocchio itself. For the checks of Section 10.4, the simulator of the platform could be the MuJoCo model that RoboCompiler's package generates from the graph of its simulation studies, with its state mapped to the leg lengths and a tolerance for its soft loop closures.

Implementation and diagnostic record

The following sections hold the release-specific implementation and diagnostic material of the book's runs. They are records of those runs, not installation advice for later releases.

S10.9 The interfaces

A module meets the core only at its interface: the methods of its role, the data that they exchange, and the declarations that a configuration checks. The interfaces are written in the file `contract.py` of the core as Python protocols (classes of `typing.Protocol`). A module implements one by having the methods and attributes listed there, with the meanings stated there; it need not inherit from anything.

S10.9.1 Data and their shapes

Four records cross the interfaces in every period: the observation o_k, which the core forms at t_k and passes to the proposal method and both predictors; the requirement that the demand predictor returns (Definition 4.12); the step that the core passes to the generator, with the proposal, the labelling map λ_k as a function of a period and a segment, the requirement, what the task asks the generator to begin and the measured state; and the candidates that the generator returns, each a plan with its ambition, in the order of evaluation. Listing S10.1 lists their fields. Section 4.1.5 lets an observation contain the labels in force as well; the reference implementation leaves them out, because labels are attached to plans, which are opaque to the core, and a labelling map is valid only during its period.

The test returns a profile (Definition 4.9), which the core checks, shifts, compares and records but never computes. It handles levels only through the level space of the test, an object with the floor and four operations: whether a value is a level, whether a level is admitted, the first key of Equation (4.2) as a number, and the order of levels, in which a profile must be nondecreasing. The core provides one level space, the scalar grid that the cart uses. It does not support the vector reserves of Section 4.2.7, which would need another level space, a separable test and one norm for each family of constraints.

Every other datum is opaque and has whatever form its modules give it, so that robots with different states and segments can share the core. A plan must provide one method, `shift`, which returns the plan without its head (Section 8.3.2). The form of a proposal, and of what the task asks the generator to begin, is agreed between the modules that exchange them, with the meaning that the application states (Definition 4.11). Two forms are part of the interface. A feedback law, from the control package to the simulator adapter, can be called with the time within the period and the state, and returns an input in the declared actuation (Definition 4.4); a simulator adapter may also use a structure of the laws that the package documents, as the cart's track uses their constant commands to compute the motion exactly. A state, from the simulator adapter

to the robot adapter, and its measurement, from the robot adapter to the other modules, have the components that the state declaration lists, in that order.

S10.9.2 Units, frames and actuator semantics

The declarations of the cart are given in Listing 10.1 of the book.

Units fix the scale of every number that crosses an interface; frames name the body and axis of each component, so that a package that assumes joint angles about one axis does not meet a simulator that reports them about another; and the norm is declared because the alarm check compares a distance in it with ϖ, and every level is a reserve in it.

When a configuration is built, the declarations must agree: the states of the robot adapter, of the simulator adapter and, if it declares one, of the control package must have the same components in the same order, with the same units, frames and norm; their actuations must have the same inputs, bounds, hold and meaning; and the capability space of the capability predictor's labels must be the one that the test accepts. The script refuses a simulator adapter that declares the commanded acceleration in millimetres per second squared with

```
refusal: actuation, world vs robot: u_a: unit 'mm/s^2' differs from
  'm/s^2'
```

where `refusal` is the name of the configuration, and it refuses a capability predictor that issues friction coefficients, since the test accepts labels of the space `cart.braking` only.

These checks compare declarations, not physics. A simulator that declares metres and reports millimetres passes them, and so does a package whose segments assume a hold that the actuator does not realize, if both declare the same hold. Units and meanings are compared as written, so that `m/s^2` and `m s^-2` would be refused as different, which errs on the side of refusal. Two checks of Section 10.3 compare what modules do with what they declare, the inputs of every law with the declared bounds and every state with the declared number of components, and neither can see a unit. A check of the motion against a robot model can (Section S10.11.2).

S10.9.3 The clock

The clock of the core is the period index k; the time of period k is $t_k = kT$, with the period T of the configuration, 0.1 s for the cart. One iteration of the loop is one period of Figure 4.1. Before the first period, the core measures the initial state and asks the control package for an initial plan certified on a set that contains every state within ϖ of the measurement, as (S4) requires; if its profile is unusable or its level does not reach the floor, the run does not start.

In each period, the core then informs the task of the level in force, forms the observation, and asks for the proposal, the requirement and what the task asks the generator to begin. In normal operation, it asks for the labelling map, the initial set, the candidates and, within the budget, their profiles, selects, has the simulator adapter execute the head of the plan in force, measures, and checks for an alarm; after an alarm, it asks instead for the set that the rest feedback can reach and for restart candidates, while the rest feedback acts. At the end of every period, it reports the measurements to the capability predictor. Table S10.1 lists these calls, with what replaces each output in operation.

The order matters in three places. *Causality*: everything that the decision of period k uses is known at t_k, and the capability predictor receives the measurements of period k only after the decision. *Simulated and real time*: the loop computes the decision of period k before it advances the world through period k, whereas a robot executes the head of P_k while the decision is computed; both orders give the same decisions and motion, because the decision uses only what is known at t_k, and the motion during period k depends only on P_k. *Commitment*: the plan selected during period k is in force from t_{k+1}, and nothing else changes the plan in force except an alarm, after which none is in force until a restart; the task learns its level at the start of period $k+1$, which decides whether an operation due then can begin (Assumption 4.13).

The reference implementation does not measure time. A deadline is represented by a budget, the number of fresh candidates that the test can evaluate before it, given for each period by a function of k in the configuration, as in Section 8.3.3. The budget applies after an alarm as well: restart candidates not evaluated count as not certified, as (S6) requires, and if none is admitted, the rest feedback continues; no run of Chapter 8 limits a period after an alarm. A run with a budget does not depend on the speed of the machine, and with deterministic modules, such as those of the cart, it repeats exactly. On a robot, a deployment must enforce the deadline itself, in seconds: stop the evaluation at t_{k+1}, count every evaluation not completed as not certified, and make sure that the common steps, and the calls without a default on which they depend, finish in time, as (S6) assumes. The reference implementation provides none of this.

S10.9.4 Errors

The core distinguishes three kinds of event: a module that fails (Figure S10.2), an alarm, and an implementation check that fails.

A module that fails. A module fails when its call raises an exception or returns an output that the core cannot use; Section S10.2 lists what the core checks in each output, and the calls that have no default. A proposal method or a predictor that returns `None` has not failed: its output is not ready, and the default replaces

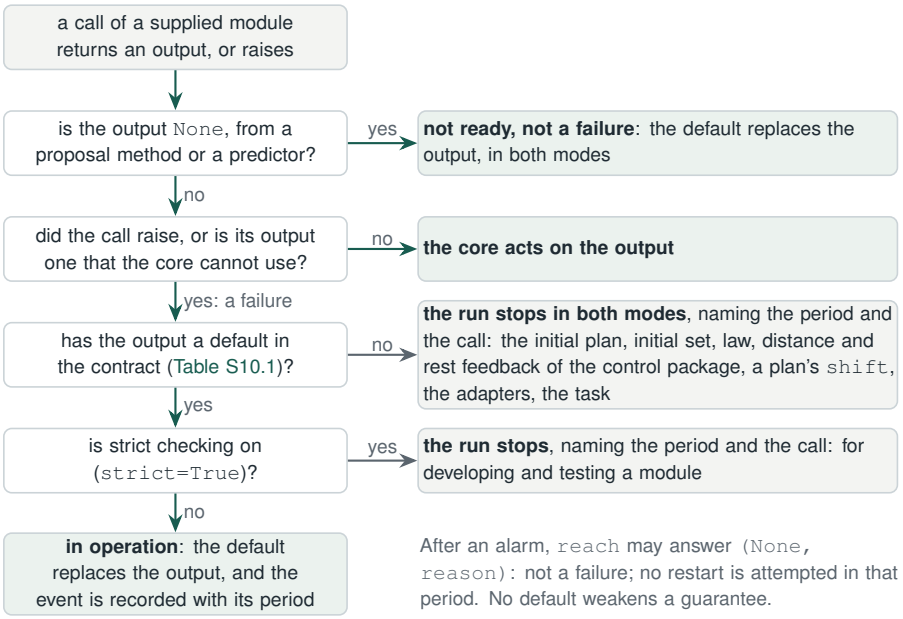


Figure S10.2. ILLUSTRATION What the core does with the output of a call, as the loop of `code/ch10/allovi` implements it (`governor.py` and `loop.py`). An output that is not ready, `None` from a proposal method or a predictor, is replaced by its default in both modes. A call that raises, or an output of the wrong kind, is a failure: if the output has a default in the contract, strict checking stops the run and operation replaces the output by the default and records the event; every other call stops the run in both modes.

it. The core cannot tell that an output is late, except for the evaluations of the test, which the budget cuts off, so a module that cannot answer in time must return `None`, and on a robot the deployment must stop waiting at the deadline. What a proposal that the generator cannot use does is decided by the generator, which alone reads proposals: the cart's generator reads every proposal other than 'accelerate' as 'hold'.

An output that has a default in the contract (Section 4.3.5) is treated in one of two modes. With strict checking, `strict=True`, for developing and testing a module, a failure stops the run with an error that names the call and the period. In operation, `strict=False`, the default replaces the output, and the event is recorded. A default does not weaken a guarantee, provided that the common steps still finish before the deadline and the previously certified recovery remains executable (Section 10.2): the results of Chapter 9 hold for the defaults without any assumption on the module that failed, so a failure can cost progress, as a short budget does, but not certification. A failure of a call without a default stops the run in both modes, because no output could replace

it without an argument that the core cannot make; what the robot does then is outside the framework, as after an alarm without a restart. After an alarm, by contrast, a control package may answer that it cannot represent the set that its rest feedback can reach, with a reason; that is a legitimate answer, and no restart is attempted in that period.

The script shows both modes with a capability predictor whose labels are never ready: every call raises an exception. With strict checking, the run stops in its first period with

```
period 0: labels raised TimeoutError('labels not ready before the
  deadline')
```

In operation, it continues and records 220 events, one in each period, and, as the default implies, its decisions are those of the run with the fixed predictor: the cart arrives in the work zone at 9.9 s instead of 8.6 s, and its least clearance is 0.505 m.

An alarm. An alarm is not a failure of the software but an event of the loop: the measurement at t_{k+1} lies farther from the initial set than $\mathcal{O}$, which, with a valid $\mathcal{O}$, a sound test and an admissible initial set, shows that a label was invalid (Section 4.3.2). The core responds as Algorithm 8.1 prescribes, and the records show the alarm and its outcome.

An implementation check that fails. With `diagnostics=True`, the core also has the test evaluate the fallback again in every period of normal operation, and checks that it gives the shifted profile (Lemma 4.10(c)). The check suits a test that decides the inclusions exactly, as the cart's does; a sound test that cannot decide (P1) on the new initial set may certify the fallback lower, and the check reports that too. The guarantees, which assume a sound test (S3), do not need it; it detects mistakes in the implementation of a test, and in the thirteen runs it held in every period.

S10.9.5 Registration

A module is a class with three class attributes, `ROLE`, `NAME` and `CONTRACT`, and the methods and attributes of its role (Table 10.1). Listing S10.5 is a complete module, the survey predictor of Section 7.2.3. The decorator `register` adds the class to the default registry, which checks the role, the name, the version of the contract, and every method of the role with its number of positional arguments (Section S10.3). A control package without the method `restart`, for example, is refused when it is registered, with

```
NoRestart: missing method restart()
```

Listing S10.5. A complete module: the survey predictor (`cart/modules.py`).

```
@register
class Survey:
    """The survey predictor of Section 7.2.3 without its offset:
    2 m/s^2 for a segment whose region lies before the surveyed start
    of the ramp, 0.5 m/s^2 for the others."""
    ROLE, NAME, CONTRACT = "capability", "cart.survey", "1.0"
    SPACE, horizon = "cart.braking", ce.HORIZON

    def __init__(self, ramp_start):
        self.ramp_start = ramp_start

    def labels(self, observation):
        k = observation.k

        def labelling(j, segment):
            if j > k + 1 + self.horizon:
                return ce.B_GLOBAL
            if segment.hi >= self.ramp_start:
                return ce.B_RAMP
            return ce.B_FIRM
        return labelling

    def observe(self, k, before, after):
        pass
```

where `NoRestart` is the name of its class. A registered module can then be named in a configuration without any change to the core. Building the configuration checks that the control package and the two adapters are present, that every module has the attributes of its role, and that the declarations agree (Section S10.9.2).

The interfaces are narrow: eight roles with nineteen methods in all, nine of them for the control package; six more methods for the objects that the control package supplies, its plans, its level space and its capability space; and four records, with the profiles that the test returns. Further methods are optional and serve the records or the checks only; they never influence a decision. The interfaces are kept stable by the version of the contract, which has two numbers. A change that every module must follow, such as a new required method or a changed meaning, increases the first, and the registry then refuses a module written for the old one; an addition that no module must follow, such as a new optional method, increases the second. The current version is 1.0.

S10.10 Compatibility checks and verification

Before a module meets the modules of others, its supplier must know what the software can check about it and what only the supplier can establish. This section separates the two, describes the check suite and what it finds in the cart, and follows a contribution from a template to a checked module.

S10.10.1 Specifications and checks

The obligations of Table 4.1, and assumptions (S2) to (S6) of Assumption 9.1, are statements about every world, every solution and every period. The guarantees of Chapter 9 rest on them and on nothing else, and they are established outside the software: by a proof, as for the test of Chapter 5; by evidence, as for the coverage of a predictor in Chapter 7; or by the specification of a sensor, as for ω. A program runs finitely many cases, so it cannot establish a statement about all of them.

What software can do is look for mistakes, with checks of four kinds, kept apart because they show different things (Table S10.2).

- *Interface checks* find a module that does not fit the core: registration, declarations, and every output that the core acts on, which must be of the contracted kind; a profile, for example, must not decrease (Definition 4.9).
- *Consistency checks* test outputs for further properties that the specification implies: a fallback that keeps its shifted profile when the test evaluates it again, for a test that decides the inclusions exactly (Section S10.9.4); labels at least as informative as the global capability and equal to it beyond the horizon (Assumption 4.3, Section 4.2.1); labelling maps that give the same label for the same arguments throughout their period; inputs of every law, at the start of its period, within the declared bounds; states with the declared number of components; and, for deterministic modules, runs that repeat exactly. A module can violate its specification without violating any of them.
- *Falsification checks* need a simulation, whose simulator adapter knows the world and can serve as an oracle: it tells whether an executed label admits the world of its period, and how far a state truly is from the constraints. While every executed label is valid, Theorem 9.5 predicts no alarm, by (b), and, as the proof of (b) shows, a true reserve at least the level of the plan in force throughout each period, which the suite examines at the start of every period. A run that contradicts either, with valid labels, is a counterexample to one of the assumptions (S3) to (S6) of the theorem, unless the checks themselves are wrong, through the oracle or the methods that they use. After a restart, the prediction holds again provided that

Table S10.2. What each kind of check tests, what its failure and its pass show, and what it needs; the obligations themselves are established outside the software.

Kind	Tests	A failure shows	A pass shows	Needs
Interface	registration, declarations, outputs that the core acts on	a module that does not fit	that the module fits, in the cases run	the configuration
Consistency	properties that the specification implies	a mistake in the module, where the property applies	no mistake of this kind in the cases run	runs of the configuration
Falsification	executed labels, and the predictions of Theorem 9.5 while they are valid, against an oracle	an invalid label; a false certificate, through an assumption of Theorem 9.5 that fails or, after a restart, an incomplete reach set; or a mistake in the oracle	no counterexample in the cases run	a simulator adapter that serves as oracle
Module	what its supplier aims at	a mistake there	no counterexample among the supplier's cases	`checks()` of the module
Obligations	the rows of Table 4.1 and (S2) to (S6) of Assumption 9.1; no			a proof, evidence or a specification

the restart plan is certified on a set that contains every state that the rest feedback can reach (Proposition 9.6), so a contradiction then can also show that the set misses one.

- *Module checks* are written by the supplier of a module, aimed where it could be wrong, and run by the suite with the others.

A failed check locates a mistake, or a claim that does not hold in the case run. A passed check bounds nothing beyond the cases that it ran, and no row of Table S10.2 turns a passed check into an obligation discharged.

S10.10.2 The check suite

The module `checks.py` of the core runs the checks on a configuration, given as a function that builds it, so that every run starts afresh. It runs the configuration once with strict checking and diagnostics, while wrappers record what the capability predictor and the simulator adapter receive and return; it runs it a second time and compares every record; and it asks the oracle about the first run. Every check reports 'passed', 'failed' or 'not applicable', with the number of cases that it examined and, on a failure, the first one. Of the three falsification checks, 'executed labels valid' examines every period of normal operation, and

'true reserve at least the level in force' and 'no alarm while every executed label was valid' only the periods in which every executed label has been valid since the start or the last restart. The cart's track serves as the oracle, from the true braking of its ground and the position of the wall; without an oracle, these checks are not applicable, and they never run on the machine. Section S10.4 shows how each check reaches its verdict.

The script `cart_checks.py` in the folder `code/ch10` runs the suite on every configuration of this section, each for the 220 periods of the forecasting run of Chapter 3, in one to two minutes; with `--check`, it also confirms each number quoted here. For the reference configuration, the forecasting run itself, the suite runs 13 checks, and all pass. It examines 124 276 labels, compares 1760 rows of records with those of a second run, finds every one of the 220 executed labels valid, and tries 16 plans in the package's own check.

S10.10.3 What the checks find

Table S10.3 gives the outcome of every check in six configurations, each differing from the reference in one or two modules: the capability predictor, the test of the control package, or the proposal method. The eager proposal method, one of the two examples of Section S10.10.4, asks to accelerate in every period. With the cart's own package, it passes all 13 checks, and the least clearance is 0.5 m: the governor alone keeps the cart within its constraint.

Invalid labels. The survey predictor of Section 7.2.3, biased by 0.2 m, and the contributed predictor that uses the same survey with a margin of only 0.1 m pass every interface and consistency check: their labels are at least as informative as the global capability, equal to it beyond the horizon and stable within their periods. Only the oracle finds that the label executed in period 55, 2 m/s^2, does not admit the world of that period, because its segment reaches the ramp; the alarm at 5.6 s follows (Section 8.3.4). The other falsification checks pass, as they should: they concern the periods in which every executed label was valid, and those after the restart. With a margin of 0.25 m, the contributed predictor passes all 13 checks. No check can tell, on the machine, which margin is enough: that is the evidence of Chapter 7.

A test that forgets the tool. This control package differs from the cart's in its test only, which computes every clearance as if the tool were folded. It fits the core and passes every consistency check, because the mistake is the same in every evaluation. With the naive proposal method, its decisions are those of the cart's package until the onset period, 200, in which the candidates first have the tool unfolded; from then on, the same plans are committed, with higher levels. Every plan in force from period 201 on has the level 1.3 m, while the true

Table S10.3. The outcome of every check of the suite in six configurations of the forecasting run of Chapter 3, and what the runs show. Clearances in metres.

Check	Reference	Survey, 0.2 m bias	Margin of 0.1 m	Forgets the tool	Forgets the tool, eager	Generous test, eager
Interface (2 checks) and consistency (7 checks)						
every check	passed	passed	passed	passed	passed	passed
Falsification						
executed labels valid	passed	failed in period 55	failed in period 55	passed	passed	passed
true reserve at least the level in force	passed	passed	passed	failed from period 202	failed from period 202	passed
no alarm while every executed label was valid	passed	passed	passed	passed	passed	passed
Module						
own test refuses plans that brake beyond their labels	passed	passed	passed	passed	passed	failed, 12 of 16
The run						
alarm	none	5.6 s	5.6 s	none	none	none
least true clearance	0.5	0.52	0.545	0.5	−0.29625	0.5

reserve is 0.5 m from t_{202} on, so the falsification check fails in each of the last 18 periods; nothing is violated, because the naive method holds the cart where it is. With the eager method, the governor accepts candidates that move the cart towards the wall with the tool unfolded. The check fails from period 202 on, where the level in force is 1.1 m and the true reserve 0.491 25 m; the tool reaches the wall 6.8 ms into period 210, and at the end of the run, still moving, it is 0.296 25 m beyond it. The check flags the false certificate eight periods before the first violation. The records locate the mistake: in period 202 of the run with the naive method, the committed candidate has the level 1.3 m but the reason 'clearance 0.5 m at the rest point 23.7 m', from the package's explanation, which still counts the tool. A mistake made in both would not show in the trace, but the falsification check would still find it.

A generous test. This package's test accepts braking up to 0.01 m/s^2 beyond the label. It is unsound: the label admits a world that brakes exactly as labelled, which does not realize such a command. Yet every generic check passes, with either proposal method, and the records with the naive method are those of the cart's package, row for row. The mistake is never exercised: with the naive method, 48 candidates brake beyond their labels, all by 0.031 25 m/s^2 or more,

so both tests refuse them, and with the eager method none does. Only the package's own check finds it. That check builds plans that brake at $0.5 + d$ m/s^2 under labels of 0.5 m/s^2, from four forward motions on the ramp and with four values of d, from 0.001 to 0.1 m/s^2, and requires that none be certified. The generous test certifies 12 of the 16, for example a plan from 20 m at 2 m/s that brakes at 0.501 m/s^2 and plans to rest at 23.9922 m: in a world that brakes at exactly 0.5 m/s^2, which the labels admit, the cart cannot stop before 24 m, outside the funnel of the plan. The cart's test certifies none of them.

The reach of the checks. Interface and consistency checks find modules that do not fit or contradict themselves; they found none of the three mistakes. Falsification in simulation found the invalid labels and the false certificates that the runs exercised. The mistake that the runs never exercised was found only by a check aimed at it, written by the supplier, and any number of further runs of these configurations would have missed it; the other 12 checks all passed on an unsound test, and only a proof that the test is sound excludes such a mistake. The checks verify declared properties of modules: that their outputs have the contracted kinds and shapes, that their units, frames and actuation agree as declared, and that they keep the timing contract as the simulation measures it, in periods and budgets of evaluations; in simulation, they also confront the runs with what the guarantees predict. They do not verify that a model is physically valid (Section S10.11.4), and they do not verify that a deadline is met in real time on a real computer (Section S10.9.3).

S10.10.4 A contribution, from template to report

The folder `code/ch10/templates` holds one template for each role of Table 10.1, and one for a robot model (Section 10.4), with a README that states the path below. The docstring of each template states the obligations of the role, which no check discharges, and the checks that the suite runs on it; its class lists every method of the role with what it receives and returns (Listing S10.3). A contribution takes four steps.

1. Copy the template of the role, rename the class, choose a `NAME` and fill in every method.
2. Add `@register`; registration checks the role, the name, the version of the contract and every method (Section S10.9.5).
3. Write down how each obligation of the role is established: by a proof, by evidence or by the specification of the hardware.
4. Run `python3 cart_checks.py --module FILE`, which places each module that the file registers in its role in the reference configuration and prints the report of the suite. A task, like a module of another robot, is

checked by calling the suite with a function that builds its configuration, because the cart's demand predictor and evaluator are built with the cart's task.

Both examples in `code/ch10/examples` took this path: the eager proposal method, a class with one method, which needs no evidence because a proposal method carries no obligation for safety; and the survey predictor with a margin, the predictor of Listing S10.5 with its threshold moved, written from the public interface and the documented fields of the cart's segments, whose bound on how often its labels are invalid needs the evidence of Chapter 7. With their defaults, a survey biased by 0.2 m and a margin of 0.25 m, both pass all 13 checks, with the reports of the corresponding configurations of this section. The script also confirms that each of the eight templates still has every method of its role, with the number of parameters of the interface, though not their docstrings.

> **A check can show that a module does not fit, that it contradicts itself or, in simulation, that it contradicts what the guarantees predict; it cannot show that a module keeps its promises, and a test that passes every generic check can still be unsound.**

S10.11 An optional model provider

Every module of the cart uses one description of the robot: the state (p, v, ℓ), the two inputs and what they mean, and the motion that a constant command produces. It came with the application, in the code of Chapter 3, short enough to read in full. A robot with closed linkages or coupled actuators needs more, and its controller, its predictors and its simulator must still describe the same robot. This section makes explicit the boundary at which a model enters the software, the robot-model adapter, and introduces RoboCompiler as one optional provider behind it; any other model that meets the same contract can take its place.

S10.11.1 The boundary

The module `model.py` of the core defines the record that crosses the boundary, `RobotModel` (Figure S10.3), with six items (Table 10.2). Four are what a mechanical model provides: the coordinates of the state, with their units and frames; the kinematics, the points that constraints and labels refer to, as functions of the state; the nominal dynamics, the state at the end of an interval in which the inputs are held; and the actuation, the inputs with their units, frames and bounds. Two are decisions of the application: the norm, which fixes the unit of

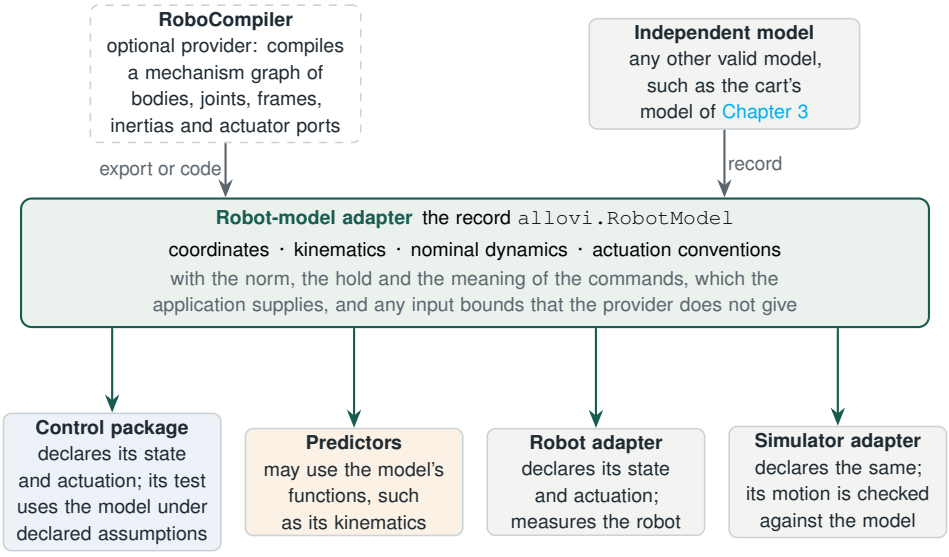


Figure S10.3. ILLUSTRATION The robot-model adapter. RoboCompiler or any independently supplied model reaches the modules through one record, which, once complete, carries the coordinates, the kinematics, the nominal dynamics and the actuation conventions of the robot; the application adds the norm, the hold and the meaning of the commands, and any input bounds that the provider does not give. The robot adapter, the simulator adapter and the control package declare the record's state and actuation, and the checks of Section 10.4 compare their declarations, and the motion that the simulator adapter realizes, with the record. Mechanical consistency establishes no obligation of Table 4.1 (Section S10.11.4).

the reserve (Section 4.1), and the hold and meaning of the commands, which depend on the robot's interface and on the drives behind it; when a provider gives no bounds, the application states them as well. The robot adapter, the simulator adapter and the control package declare the state and the actuation of the record, so the declarations of Section S10.9.2 have one source.

A model is nominal. What a world may do differently enters through the capability labels (Section 4.2), whose meaning the adapter does not change; the constraints of the task, the measurement-error bound and the test stay outside the record, with the modules that carry their obligations. For each item, the record either provides it, or names where the provider offers it outside the record, or leaves it missing; the checks below accept a model only when every item is provided.

The module offers three checks. `check_model` confirms that a complete model returns what it declares, as many points and state components as it

names, for sample states and inputs within the declared bounds. `agreement` compares the declarations of the robot adapter, the simulator adapter and the control package with those of the model. `motion_agreement` runs the simulator adapter for one period from sample states and compares its states with the nominal dynamics, within a tolerance that the caller sets. A command realized as the model assumes does not by itself give the nominal motion, since the same force moves different masses differently: the comparison needs a simulated world that matches the model's nominal world in everything that the motion depends on, or a tolerance that covers the variation among the worlds that the labels admit. Then a difference beyond the tolerance is a mistake of the simulator adapter or of the model, not a property of the world. The cart's commands are accelerations, which alone fix the motion wherever the ground realizes them, so its tolerance is zero.

S10.11.2 The cart's model through the adapter

The record of the cart, in the module `model.py` of its package, carries the declarations of Listing 10.1, the bumper and the tool tip as its points, and the motion under a constant acceleration as its dynamics (Listing S10.4). Its first assumption marks the boundary between model and labels: the model says that every command is realized, and where the ground brakes less, a label says so. The robot adapter, the simulator adapter and the control package declare its state and actuation, and the thirteen runs of Section 10.1.3 are unchanged, record for record.

The script `model_providers.py` in the folder `code/ch10` runs the checks. The cart's model provides all six items. For 5 sample states and 6 inputs, it returns what it declares; the declarations of the three modules agree with it; and in 26 cases the simulator adapter realizes its motion exactly. The cases accelerate the cart towards the wall, which the ground always realizes, keep its velocity, or brake it at the global capability, which the world of every period allows (S2), without reversing the motion within the period; every admitted world then moves the cart as the model does.

The last check is the one that the declarations cannot replace. A simulator adapter that declares metres and reports millimetres, defined in the script, passes the comparison of declarations, so the configuration is built, as Section S10.9.2 warned. Against the model, 21 of the 26 cases differ, 6 of them because the simulator adapter raises an error; the first differs already from rest:

```
from (0, 0, 0) under (2, 0): the model gives (0.01, 0.2, 0), the
  simulator adapter (10, 200, 0)
```

The five cases that agree command neither acceleration nor reach: the motion is then linear in the state, and a scale factor passes through it unseen, so the

samples must give every input a value other than zero. The check suite of Section 10.3 fails on this configuration too, but only through a consequence: an alarm is raised at 0.2 s, and the run stops in period 2, when the simulator adapter cannot realize the rest feedback; the report, 7 checks passed, 1 failed and 5 not applicable, does not name the unit.

S10.11.3 RoboCompiler as a provider

RoboCompiler compiles a canonical mechanism graph, made of bodies, joints, attachment frames, inertias and actuator ports, into one mechanical interface for assembly, motion, actuation and rigid-body dynamics, kinematic loops included (Shahna et al., 2026a); its repository (Shahna et al., 2026b) has examples for five robots, with workflows in MuJoCo, Pinocchio and Isaac Sim/PhysX. For a closed-chain robot, this is the consistency that the boundary asks for: a controller and a simulator derived from one graph share its bodies, joints, closures and actuator ports by construction.

What it provides at the boundary was confirmed on the repository at commit `a4e4437` (10 October 2026), with its six-UPS Stewart platform: a fixed base and a moving platform carrying a fixed payload, joined by six legs, each with a universal joint, an actuated prismatic joint and a spherical joint. The adapter `providers/robocompiler.py` reads, with the standard library, the three files that RoboCompiler's compile step writes, which `code/ch10/providers/robocompiler-stewart` holds unchanged with the command that wrote them; Section S10.8 gives their counts. The adapter takes as the state the 6 actuated leg lengths, the compiled model's independent coordinates, and their rates: 12 components, in metres and metres per second. Table 10.2 gives the result. Only the coordinates are provided. The kinematics and the dynamics are offered elsewhere, as RoboCompiler's code, which needs NumPy and SciPy. The actuation is offered elsewhere too: the export names each command, a force in newtons on its leg, and gives the strokes, 0.5 m to 0.91 m, but no force limit, which the mechanism graph of the simulation studies declares, 900 N, with a servo on the leg lengths (Shahna et al., 2026b). The norm and the hold are not given, and the meaning of the commands only in part: the export does not say what effort is realized. `check_model` refuses the record until the five items that it lacks are in it.

Table 10.2 of the book sets out the items.

An adapter that makes this model usable by an ALLOVI control package has five things to implement and check. It chooses the state and justifies a norm in which components of different units are commensurate. It wraps RoboCompiler's assembly and body poses as the kinematics, on the assembly branch that it selects, since the leg lengths fix the pose of the platform only

locally. It states the bounds, the hold and the meaning of the commands: forces at the ports, or length targets for a servo, whose drive dynamics RoboCompiler takes as inputs alongside the graph rather than deriving them (Shahna et al., 2026a). It turns the continuous reduced dynamics into the state after a period under the declared hold, with a bound on the error of the integration that the funnels of the control package must cover. And it runs the checks of this section, with a simulator for the platform (Section S10.8).

S10.11.4 What a model cannot establish

A consistent model removes a class of mistakes: modules that describe the robot differently. It establishes none of the obligations of Table 4.1. A test that uses the model must still be proved sound for every world that its labels admit, which includes showing that the model's assumptions hold in those worlds, and labels must still be backed by evidence on how often they are invalid. Nor does it make a simulator faithful: one that agrees with the model under the samples of `motion_agreement` may still differ from the robot, and from another simulator, in contact, friction, timing and numerics. The guarantees of Chapter 9 do not depend on who provides the model, and neither does the governor. RoboCompiler is one provider and an independently written model another; both are checked in the same way, and neither discharges an obligation of the contract.

> **A consistent model makes the modules describe the same robot; it does not make that description true, and it discharges no obligation of the contract, whoever provides it.**

CHAPTER S11

MuJoCo and Isaac Sim workflows: records and details

This chapter of the supplement holds what Chapter 11 moved out of its text: the records of its two experiments, the releases and installation of the simulators, the stills of the model, the table of the diagnostics in Isaac Sim, the comparison in Isaac Sim at three time steps, the step-by-step traces of Isaac Sim, the learning tool as a dependency of its own, and the sliding bodies of the diagnostics in MuJoCo. The releases named are those used for the runs.

S11.1 The records of the two experiments

Each experiment of Chapter 11 writes its record: what a simulated result depends on, from the model and its assets to the guarantee status of the run (Tables S11.1 and S11.2).

S11.2 Releases and installation

MuJoCo. The runs of Section 11.1 were made with MuJoCo 3.16.0, released on 14 October 2026 (Google DeepMind, 2026b, Changelog), with NumPy 2.5.3 under CPython 3.13.16 on one x86-64 Linux machine; the same results, digit for digit, came with CPython 3.11.15 and 3.14.6 (Table S11.1).

Isaac Sim. Release 6.2.0 of Isaac Sim, of October 2026, with which the runs of Section 11.2 were made, bundles Newton 1.6.1, and 'PhysX remains the default' (NVIDIA, 2026b, Release notes). Its MJCF importer, release 3.13.0, converts a text through the package `mujoco-usd-converter`, release 0.6.1, which Isaac Sim bundles (NVIDIA, 2026b, Release notes) and which its authors call an alpha product (Newton Developers, 2026a); the source of release 0.8.0 of the converter, read as well, also marks meshes given in a text as future work. The PhysX extension of Isaac Sim 6.2.0, `omni.physx` 110.4.4, does not state the version of the PhysX SDK that it contains; the documentation cited in Section 11.2 is that of version 5.6.1.

The script records, besides the versions, the default of every setting that the stage writes, read from fresh schemas, so that the settings that differ from the

Table S11.1. The record of the experiment of Section 11.1, after `results/record.txt`, which `mujoco_cart.py` writes; the size of the mesh is from `results/mujoco-summary.txt`, and the runs with other versions of Python are those of the folder's README.

Item	Record
Model and assets	MJCF text written by `cart_mujoco/scene.py` from its list of parameters and saved as `results/cart.xml`, with the ground as a convex mesh of 342 vertices; SHA-256 beginning `ed63fb3f13056da6`; no external assets
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3 and CPython 3.13.16; Newton solver, at most 100 iterations, tolerance 10^{-8}; elliptic friction cones; MuJoCo's defaults otherwise (impratio 1, no noslip iterations, contact parameters `solref` 0.02 1 and `solimp` 0.9 0.95 0.001 0.5 2)
Time steps	1 ms, integrator `implicitfast`; 100 steps, counted, per period of 0.1 s; the drive loop at every step; the comparison of Section S11.9.2 repeated at 0.5 ms and 0.25 ms
Actuators and sensors	per wheel, a torque motor limited to 40 N m; a drive loop with feedforward and speed feedback (gain 100/s), shared by load; tool servo 1000 N/m, 20 N s/m, 100 N; rest below 5 mm/s; 14 sensors
Frames and units	SI units; x towards the wall, z up; p the arc length of the curve at height r above the ground, to below the front axle, plus 0.1 m; v its rate, ℓ the tool's slide
Uncertainty bounds	labels of the forecasting predictor: 2 m/s^2 on regions that end before 20 m, 0.5 m/s^2 on the others; measurement-error bound 0
Constraints	$p + \ell \leq 25$ m; $\lvert u_a \rvert \leq 2$ m/s^2; $u_\ell \in \{0, 0.8\text{ m}\}$; in the simulator, contacts and the tool's stops
Terminal behaviour	rest feedback: a cart moving towards the wall braked at its command until at rest, then held; a cart at rest or moving back held at once; restart when certified
Training and evaluation split	none: no module of the configuration learns
Seeds	none: neither the configuration nor the simulator draws random numbers
Hardware	one x86-64 Linux machine; the same results, digit for digit, with CPython 3.11.15 and 3.14.6
Guarantee status	of the results of Chapter 9, only those that hold up to the first alarm, at 0.1 s: the funnels are single motions and the simulated cart leaves them; no restart plan, no restart; no violation; the task not completed

Table S11.2. The record of the experiment of Section 11.2, after the record and the summary that `isaac_cart.py` writes into `results/isaac`; the hash of the stage, the hardware beyond the record and the comparison of the two runs are from `isaac_check.py` and the console logs, kept with the results.

Item	Record
Model and assets	USD stage written by `cart_isaac/stage.py` from the parameters of Section 11.1, exported after the cart settled (`results/isaac/cart.usda`); the ground a surface mesh of the same profile, without approximation; SHA-256 without the renderer's settings beginning `b8df766ee0f7cf7b`; no external assets; the MJCF importer could not convert the text of Section 11.1
Simulator and solver	Isaac Sim 6.2.0, pip package `isaacsim` 6.2.0.0 (NVIDIA, 2026c), version file 6.2.0-rc.14 as in the release's source (NVIDIA, 2026d); Kit 110.4.0; `omni.physx` 110.4.4; CPython 3.12.15, NumPy 2.3.1; PhysX TGS, 32 position iterations and 1 velocity iteration, on the processor; patch friction; contact offset 0.02 m, rest offset 0
Settings that differ from the defaults (defaults in brackets)	1000 steps per second (60); processor and broad phase MBP (GPU, GPU); gravity 9.81 $\mathrm{m/s^2}$ (Earth's, in the stage's units); contact and rest offsets (set by PhysX); angular damping 0 (0.05); sleep and stabilization thresholds 0 (5×10^{-5}, 1×10^{-5}); no collisions between the cart's links (on)
Time steps	1 ms; 100 steps per period of 0.1 s, counted, one at a time through the simulation manager; the drive loop at every step; the comparison repeated at 0.5 ms and 0.25 ms
Actuators and sensors	per wheel, a joint effort limited to 40 N m, from the drive loop of Section 11.1.2; the tool's drive 1000 N/m, 20 N s/m, 100 N; link poses and velocities, joint positions and velocities, the wheels' contact forces, a ray cast as rangefinder
Frames and units	SI units, one metre and one kilogram per unit, z up; angles in degrees on the stage, in radians in PhysX; the state (p, v, ℓ) of Section 11.1.2
Uncertainty bounds, constraints, terminal behaviour	as in Table S11.1
Learning tool, training split, seeds	none used: no module learns, and nothing draws random numbers; Isaac Lab was not installed, and the components for reinforcement learning installed with the package, `isaacsim-rl` 6.2.0.0 with PyTorch 2.11.0, went unused
Hardware	a laptop: Windows 11 Enterprise 24H2, Intel Core i7-13700H, 32 GB; NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB, below the listed minimum of an RTX 4080 with 16 GB, and driver 596.08, where NVIDIA tested 595.97 (NVIDIA, 2026b, Requirements); the script uses no rendered output; two runs, the same in every value that both report, the renderer's settings aside; a follow-up with the same release and packages (Section S11.6)
Guarantee status	as in Table S11.1; the disturbance bound of Chapter 5 neither confirmed nor refuted, the reported speed not being the rate of the position (Section S11.6)

defaults can be named (Table S11.2). They are the time step, 1 ms where the default is 60 steps per second; the processor, where the default is the GPU, with its broad phase; the gravity; the contact and rest offsets, which PhysX otherwise sets itself, the contact offset from the size of each shape (NVIDIA, 2025b); the angular damping of the links, 0.05 by default; the thresholds below which PhysX lets an articulation sleep or stabilizes it; and collisions between the links of the cart, which the stage turns off. The script saves the text of the stage with the results. It exported the stage after the cart had settled at rest, so that the text holds the poses and velocities that PhysX wrote back, and Newton's schema, which Isaac Sim applied to the physics scene. The application also writes its renderer's settings into the stage, and they differed between the two runs: the summary and the record file of each run hash the whole text, and the hash in Table S11.2 is that of the stage without the renderer's settings, which was the same in both.

The results of Section 11.2 come from one laptop (Table S11.2), in two runs, the second with the comparisons at the smaller time steps that the first left out. Every value that both runs report is the same, except the hash of the whole text of the stage, and their period-by-period tables are identical, byte for byte. A follow-up on the same laptop, the script `isaac_trace.py`, traced step by step what the two runs could only bound (Section S11.6).

S11.3 The cart as MuJoCo draws its model

MuJoCo draws the bodies of Section 11.1.1 from the model itself: boxes for the chassis, the tool and the wall, spheres for the wheels, and the mesh of the ground (Figure S11.1).

S11.4 The diagnostics in Isaac Sim

Table S11.3 lists the diagnostics that Section S11.10.3 discusses, with their results and the mistakes that their variants reveal.

With speed servos, their damping written per degree, the encoders read 1.74, 18.6 and 6.74 mm/s above the cart's speed at 1, 0.5 and 0.25 ms. Halving the time step does not settle the encoders. At 1 ms, the traces of Section S11.6 show them following the motion of the rounded positions.

The clock of the simulation manager is the count of steps divided by the steps per second, computed afresh after every step (NVIDIA, 2026d), and 22,000 steps after the loop on the clock began, it had advanced by exactly 22.0 s. Yet that loop runs 101 steps in period 3 and 99 in period 4, and the same in every fifth period after: 88 of the 220. The loop measures the clock from its reading when

Table S11.3. The diagnostics of the simulator adapter `cart.isaac`, their results, and the mistakes that they reveal; MuJoCo's are in Table S11.6. The variants, written into the package, are used by nothing else. From `results/isaac/isaac-summary.txt` and `isaac-derived.txt`.

Diagnostic	Adapter	Variant with a mistake	Reveals
Wheel loads at rest	442.54 N and 443.96 N, together 886.504 N: 0.296 % above the weight of the 90.101 kg that the wheels carry, in one step; over 200 steps, the weight to within 0.0034 %	stage in centimetres: 88 390.3 in the stage's units of force, a hundred times the weight in newtons	mass, gravity, units
Collision surface at 14 positions	within 8.94×10^{-8} m of the profile; on the crest, within 1.31 µm	—	the profile behind the coordinate
Rangefinder on the slope, tool folded and unfolded	the clearance $25\,\mathrm{m} - (p + \ell)$, to within 1.17 µm and 2.78 µm	wall rotated by 0.1658, the slope's angle in radians, read in degrees: 56.9 mm more	a unit of angles
One period at 1 m/s from 5 m, no command	0.707 mm beyond 5.1 m	—	a reported speed that is not the rate of the position (text)
Steps in each of 220 periods	100, counted	loop on the simulation clock: 101 or 99 in 88 periods	rounding in a difference of the clock
Speed over the crest, braking at $0.5\,\mathrm{m/s^2}$ from 2.1 m/s	within 2.83 mm/s of the reference	arc length of the ground as the coordinate: the cart up to 22 mm/s too fast	a coordinate whose rate jumps
Coasting 1 s from 1 m/s, no torque	0.0985 mm/s lost	PhysX's defaults: 2.28 mm/s lost	a default that the model does not declare
Contacts over the crest, braking at the label from 2.12 m/s	a wheel without load in 7 steps; the total load within 86.8 % of the rigid cart's	—	the contact forces of single steps
Sled from 4 m/s, at 1, 0.5 and 0.25 ms	front wheel loaded at every step; mean deceleration $2.45\,\mathrm{m/s^2}$ at each time step	—	friction while sliding
Encoders, accelerating at $2\,\mathrm{m/s^2}$, at 1, 0.5 and 0.25 ms	2.57, 4.42 and 17.2 mm/s above the cart's speed	speed servos, their damping written per radian: PhysX reports 2606.96 for 45.5	a unit of gains; a numerical artefact

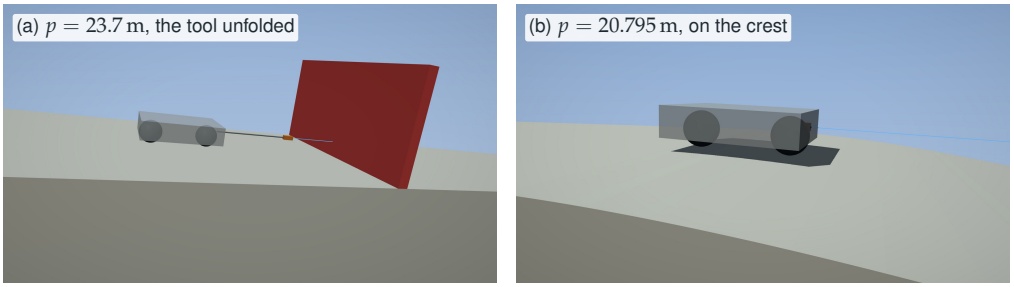


Figure S11.1. SIMULATION The cart of Figure 11.1 as MuJoCo draws its model, the text of `results/cart.xml`: boxes for the chassis (grey, drawn translucent to show the wheels), the tool (amber) and the wall (red), spheres for the wheels, and the convex mesh of the ground. (a) The state of periods 202 to 219 of the run of Chapter 3, at rest on the slope with the tool unfolded, its tip at 24.5 m; the rangefinder's ray (blue) crosses the clearance of 0.5 m to the wall, which the diagnostic of Table S11.6 reads in this state. (b) The state at the start of period 60, on the crest at 1.9 m/s, braking, with the tool folded; the ray reaches the wall beyond the frame. The adapter places the cart at each state with both wheels on the ground, sunk by the penetration at rest; the drawing simulates nothing. MuJoCo's renderer, through the book's `code/render`, adds a sky, two lights and colours; the tool's slide, for which the model has no geometry, is drawn as a thin grey line.

it began, about 35 s, after the diagnostics that ran before it, and the difference of two such readings is rounded. With the count of steps at the start of the loop that the diagnostics before it imply, 35 342, the clock's definition gives exactly these 88 periods. That is a consistency check, not a proof: it assumes that PhysX's own count of steps, from which the clock is computed, equals the manager's, and about one start count in five near 35 000 gives the same pattern.

S11.5 The comparison in Isaac Sim at three time steps

Table S11.4 gives the values of the comparison of Section S11.10.4 at the three time steps, which that section reads.

S11.6 Traces

The diagnostics and the comparison of Section 11.2 found three effects that their summaries could only bound: a reported speed that is not the rate of the position, loads at rest above the weight, and encoders whose reading changes with the step. This section follows them step by step; its last paragraph states what it established. A follow-up, the script `isaac_trace.py`, ran once on the same laptop, with the same release and the same packages, and none of its steps

Table S11.4. The cart in Isaac Sim against the cart's model, period by period, at three time steps of the simulator: the values of the comparison and what each difference is; MuJoCo's values are in Table S11.7. From `results/isaac/isaac-summary.txt`.

Difference	1 ms	0.5 ms	0.25 ms	What it is
End of a period, median (mm)	0.170	0.168	0.309	the folded tool's sag on the slope, then the reported speed
End of a period, 90th percentile (mm)	0.758	2.42	6.36	the reported speed, which departs further as the step shrinks, as does the bound of the rounding of the positions, by our reading the cause
End of a period, largest (mm); period	1.89; 49	3.81; 71	9.69; 87	the same
Position after accelerating from rest, period 1 (mm)	0.100	0.0498	0.0261	the integrator: $ahT/2$
Reach at the end of period 201, short of the stop (mm)	1.12	1.13	1.13	the servo's last approach, resolved at 1 ms: 1.13 and 1.14 mm at 0.125 and 0.0625 ms
Speed within a period, outside period 201 (mm/s)	3.64	5.25	≥ 10.1	the reported speed; doubling the drive loop's gain leaves the crest at 2.48 mm/s (2.56 at the gain of 100/s); at 0.25 ms, the largest lies in another period and is at least period 201's 10.1 mm/s, which the summary gives
Reported speed of the front axle less command, tool folded, over 1 ms (m/s^2); periods above the bound	3.02; 195	4.81; 160	9.77; 154	jumps of the reported speed, from one step to the next
The same over one step (m/s^2)	3.02	9.59	36.2	the same: jumps of about 3, 5 and 9 mm/s
Jump of the speed signal against the axle in one step (µm/s)	0.728	0.595	0.707	the coordinate, with the wheels on the surface

failed. It recorded the cart step by step in six traces: one period at 1 m/s from 5 m without a lead-in, as in the diagnostic, and the two periods after it; the cart at rest on the firm ground and on the slope, each after standing for a second; braking at 2 m/s^2 from 4 m/s; and periods 83 and 49 of the run of Chapter 3, with their lead-in. It also repeated the comparison of Section S11.10.4, with the recorded settings of Section 11.2 and with six variants (Table S11.5). With the recorded settings, it reproduced the comparison: all 220 periods, in the 19 columns that the two tables share, to the nine digits printed.

Positions in single precision. PhysX advances the positions in single precision at every substep. TGS divides the step of 1 ms into 32 substeps, one per position iteration, and in each, PhysX's source adds a prismatic joint's velocity times the substep to the joint's position, a sum of single-precision numbers (NVIDIA, 2026f). The joint that slides the cart along the track is horizontal, and on the firm ground its position is the track coordinate p itself (`cart_isaac/sim.py`). Each sum is rounded to the nearest number that single precision holds, and these lie 0.477 µm apart between 4 and 8 m from the origin, twice as far apart up to 16 m, and 1.91 µm apart beyond. At 1 m/s and 5 m, the velocity times a substep of 31.25 µs is 65.536 spacings, and the sum rounds to 66: each substep moves the cart 0.464 of a spacing further than its velocity does, as if it ran 7.08 mm/s faster, and a period of 3200 substeps takes it 0.708 mm further. This is the reckoning by which Section S12.10.2 predicts the ends of Newton's periods in single precision, applied to every substep instead of every step. The diagnostic of one period ended 0.707 mm beyond 5.1 m.

The trace repeats that diagnostic, and its first period ends 0.707 mm beyond 5.1 m again; rounding every substep's advance at the speed that PhysX reports after each step predicts 0.708 mm, two spacings more, within the micrometre that the check of arithmetic of Section S12.10.1 allows. Positions read in single precision differ by whole spacings, and the trace's do, to within 0.021 of a spacing at the nine digits printed. In 254 of its 300 steps, the cart advances by 2112 spacings, 32 times 66, and in all of them by 2110 to 2114; rounded once per step, the reported speed times the step would advance it by 2096 to 2098, by 2097 in 256 of the steps. On average it moves 7.01 mm/s faster than the reported speed, as the rounding of every substep predicts (Figure S11.2a). Step by step, the rate of the position lies 0.198 mm/s (RMS) from the rounding's prediction and 7.01 mm/s from the reported speed, and the rounding predicts the advance of 268 of the 300 steps to within one spacing and of all of them to within two. In period 49, braking at 2 m/s^2 from 3.4 m/s at 18.1 m, the spacing is 1.91 µm, and the advance per substep falls by a spacing, which is 61 mm/s of speed, every 30.5 ms. Each time, the rate of the position less the reported speed sweeps once across the range of half a spacing per substep, ±30.5 mm/s: from −29.9 to

29.8 mm/s over the trace and its lead-in (Figure S11.2b). It lies 17.1 mm/s from the reported speed and 5.09 mm/s from the rounding (RMS), which predicts 142 of the 150 steps to within a spacing; in the other eight, the advance per substep lies within 0.013 of half a spacing, where the substeps of a step can round either way. On the slope, the joint stays horizontal, and the cart, held to the slope, moves along it by the joint's advance divided by the cosine of the slope, whose angle is 0.1658 rad. Period 83, on the slope, ends furthest beyond the reported speed, by 1.28 mm. Over the period, the rate of the position lies 13.3 mm/s below the reported speed on average, and 13.8 mm/s from it (RMS); from the rounding of the joint's advance at each substep, it lies 0.000 73 mm/s on average and 1.13 mm/s (RMS).

Rounding alone thus moves the slider faster or slower than the velocity that PhysX adds, by up to half a spacing per substep: 7.63 mm/s between 4 and 8 m, 15.3 mm/s up to 16 m and 30.5 mm/s beyond. These bound the rate of the slider's horizontal coordinate, which on the firm ground is the track coordinate itself. Along the slope, the track coordinate advances by the slider's advance divided by the cosine of the slope, and the bound of its rate, with which the reported speed along the track is compared, is 1.4 % larger: 30.9 mm/s beyond 16 m. On the crest, where the ground turns from level to the slope, it lies between the two. Beyond 16 m lie the last four metres of the firm ground, the crest and the slope, and there begin 176 of the 220 periods, among them all 41 that end beyond the reported speed (Section S11.10.3). The traces cover the firm ground and the slope, where 35 of the 41 lie, but not the crest, where the other 6 lie.

The variants. The variants test this reading (Table S11.5), and Figure S11.3 shows the test period by period for the recorded settings and for the two variants that change the number of updates of the positions. Without a velocity iteration, or with four, 37 and 36 periods end beyond the reported speed, against 41, by up to 1.27 and 1.25 mm: the velocity iterations, whose velocities PhysX reports but does not integrate (Section S11.10.2), do not separate the two. A contact offset of 2 mm leaves every period as it was, and a drive loop that shares its force by loads filtered over 10 ms leaves 46 periods beyond. With 64 position iterations, and so 64 substeps, 70 periods end beyond the reported speed, and over ten steps the two differ by up to 58.2 mm/s, against 28.0 mm/s with 32. Each is close to its bound, half a spacing per substep beyond 16 m: 61.0 and 30.5 mm/s for the rate of the slider and, along the slope, 61.9 and 30.9 mm/s for the rate of the track coordinate, with which the reported speed is compared. And 64 position iterations at 1 ms, whose substeps are those of 32 at 0.5 ms, leave nine periods in ten within 2.42 mm of the model, as the step of 0.5 ms does (Table S11.4). PGS, which advances the positions once per step, leaves no period

Table S11.5. The comparison of Section S11.10.4, repeated by the follow-up with the recorded settings and with five variants: no velocity iteration or four, 64 position iterations, the PGS solver, and a drive loop that shares its force by the wheels' loads filtered over 10 ms; a sixth, a contact offset of 2 mm, gave every column of every period as with the recorded settings. Updates: the advances of the positions in a step, one per substep of TGS, one with PGS. Bounds: half a spacing per update beyond 16 m, for the rate of the horizontal slider, and along the slope, divided by cos 0.1658 rad, for that of the track coordinate, with which the reported speed is compared. Disturbance: the reported speed of the front axle read as a disturbance of the commanded acceleration over 1 ms, with the tool folded, against the bound of Chapter 5. From `results/isaac-derived.txt`, which `isaac_check.py` derives from `results/isaac-trace`.

Value	Recorded settings	No velocity iteration	Four velocity iterations	64 position iterations	PGS solver	Loads filtered
Updates of the positions per step	32	32	32	64	1	32
Periods beyond the reported speed by more than 0.01 mm, of 220	41	37	36	70	0	46
The largest excess (mm)	1.28	1.27	1.25	3.42	0.00227	1.32
Reported speed less rate of the position over ten steps, largest (mm/s)	28.0	28.3	28.2	58.2	0.844	27.8
Bound for the slider (mm/s)	30.5	30.5	30.5	61.0	0.954	30.5
Bound along the slope (mm/s)	30.9	30.9	30.9	61.9	0.967	30.9
Disturbance above the bound: periods of motion, of 119	119	119	119	119	72	119
Disturbance above the bound: periods of rest, of 82	76	82	64	72	0	76
Disturbance, median over the periods of motion (m/s^2)	1.63	1.95	2.02	2.05	0.0588	1.66
Disturbance, largest (m/s^2)	3.02	4.41	3.90	5.40	0.745	3.04
End of a period, median (mm)	0.170	0.174	0.166	0.232	0.164	0.194
End of a period, 90th percentile (mm)	0.758	0.958	0.833	2.42	0.276	0.758
End of a period, largest (mm)	1.89	2.53	2.72	3.78	1.53	1.62
The period of the largest	49	91	84	82	201	68
Start of a period, position and reach, largest (mm)	1.07	1.10	1.06	2.54	0.245	1.08
Start of a period, speed, largest (mm/s)	1.80	2.10	2.27	3.56	0.400	2.32
Speed within a period, outside period 201, largest (mm/s)	3.64	6.00	4.77	6.06	0.958	3.20

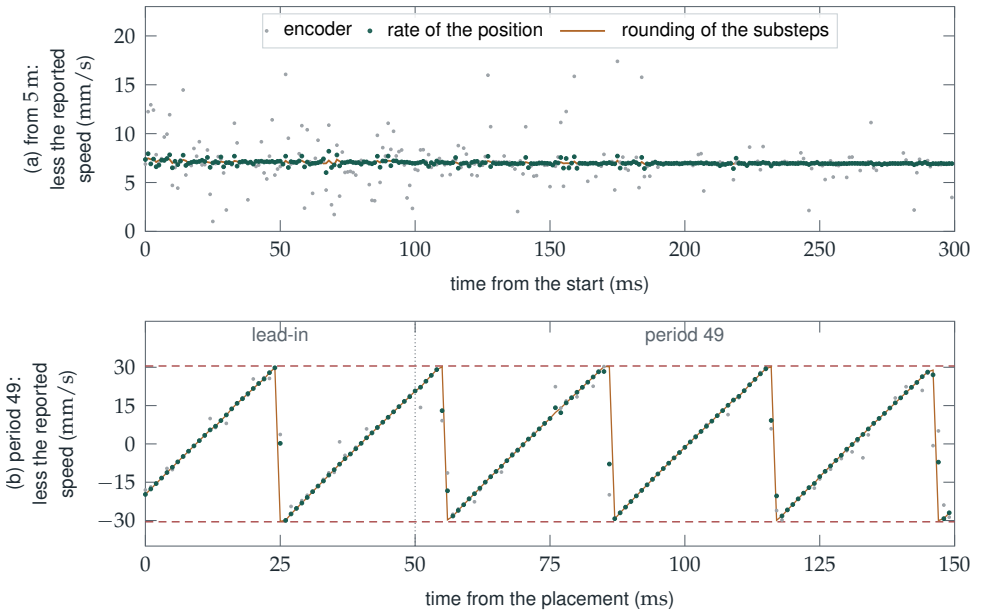


Figure S11.2. SIMULATION Positions advanced in single precision at every substep, from the traces of the follow-up. Per step of 1 ms: the rate of the cart's position over the step (green), the rate that rounding the advance of each of the 32 substeps to single precision predicts (amber), and the speed that the front wheel's encoder reads (grey), each less the speed that PhysX reports after the step. (a) One period at 1 m/s from 5 m with no command, and two more: in 254 of the 300 steps the cart advances 32 times 66 spacings of 0.477 µm, where the reported speed implies 65.5 per substep, and it moves 7.01 mm/s faster than reported; the encoder follows the motion. (b) Period 49 with its lead-in, braking at 2 m/s^2 near 18 m, where the spacing is 1.91 µm: each time the speed falls by one spacing per substep, 61 mm/s, every 30.5 ms, the difference sweeps once between the bounds of half a spacing per substep, ±30.5 mm/s (dashed), as the rounding predicts, except where the advance lies within 0.013 of half a spacing. Data: `code/ch11/results` (`isaac-rounding-5m.csv`, `isaac-rounding-period49.csv`).

beyond the reported speed by more than 2.27 µm, and 11 beyond it by more than a micrometre, and over ten steps the two differ by at most 0.844 mm/s, inside its bound of 0.954 mm/s for the slider and 0.967 mm/s along the slope. With PGS, the comparison comes close to MuJoCo's: nine periods in ten end within 0.276 mm of the model, against 0.758 mm with TGS and 0.26 mm in MuJoCo; the periods start within 0.245 mm and 0.4 mm/s of the exact state, against 0.248 and 0.294 mm/s in MuJoCo; and outside period 201 the speed stays within 0.958 mm/s of the nominal motion, against 0.347 mm/s.

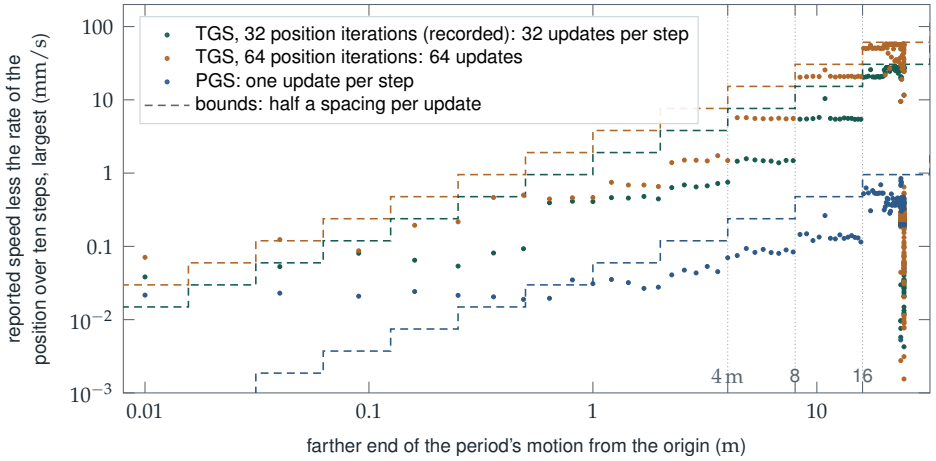


Figure S11.3. SIMULATION How far the speed that PhysX reports departs from the rate of the cart's position, in each period of the run of Chapter 3, under three settings of the solver: the largest difference over ten steps, against the farther end of the period's motion from the origin. Dashed, in the colour of each setting: half the spacing of single precision at that distance, per update of the positions, as a speed; the spacing doubles at 4, 8 and 16 m. The departure follows the number of updates per step and the spacing at the cart's position: beyond the first metre, every period lies under the bound of its setting, and with 32 or 64 updates per step the largest come within a tenth of it. Within the first metre, where the spacing is smallest, the differences do not fall below a few hundredths of a millimetre per second, which the rounding of the slider does not explain; they exceed the bound in 8 periods with PGS and in 1 or 2 with TGS. The periods of rest lie well below their bounds. Along the slope, the bound of the track coordinate is 1.4 % larger than the one drawn. Data: the comparison of Table S11.5, in `code/ch11/results/isaac-trace` (`periods-*.csv`).

The encoders. The encoders follow the motion that the rounding produces, not the reported speed. In the trace from 5 m, the front wheel's encoder reads 7.11 mm/s above the reported speed on average, and over ten steps it lies 0.662 mm/s (RMS) from the rate of the position; in the trace of period 49, over ten steps, it lies 0.901 mm/s from the rate of the position and 12 mm/s from the reported speed (RMS). The diagnostic of the encoders in Section S11.10.3 accelerates the cart from rest at 2 m and reads them at 2.25 m, where half a spacing per substep is 3.81, 7.63 and 15.3 mm/s at steps of 1, 0.5 and 0.25 ms. Its readings at the first two time steps, 2.57 and 4.42 mm/s, lie within these bounds; its reading at 0.25 ms, 17.2 mm/s, exceeds its bound by 1.9 mm/s. That is a single reading, and at 1 ms the encoder departs from the rate of the position over single steps by 1.97 mm/s (RMS); the traces, all at 1 ms, do not show the smaller steps.

The jumps of the reported speed. The jumps of the reported speed change with the updates of the positions. Read as a disturbance of the commanded acceleration, as in Section S11.10.4, they exceed the bound of Chapter 5 in every period of motion with the tool folded, with 32 position iterations as with 64. Their medians over these periods are 1.63 and 2.05 $\mathrm{m/s^2}$. With PGS, they exceed it in 72 of the 119, with a median of 0.0588 $\mathrm{m/s^2}$, and in none of the 82 periods of rest, where the recorded settings exceed it in 76. The velocity iterations change them too: without one, or with four, the median is 1.95 or 2.02 $\mathrm{m/s^2}$. At rest, the jumps seem to die out as the cart stands. In the comparison, which places the cart 0.05 s before each period, they reach 1.64 $\mathrm{m/s^2}$ in the periods of rest on the slope. After the cart has stood for a second at 24.5 m, where it rests in those periods, the reported speed jumps by at most 0.0342 mm/s in a step, and at 5 m on the firm ground by at most 0.0428 mm/s: read in the same way, 0.0342 and 0.0428 $\mathrm{m/s^2}$, below the bound. The traces show only these two moments after a placement. We read the jumps with TGS as an effect of the rounding: the wheels follow the rounded motion, as their encoders show, and the friction that holds them to it acts on the velocities that PhysX reports. The traces are consistent with this reading, but they do not establish it, and the velocity iterations, which leave the rounding as it is, change the jumps as well.

Loads. The loads that the contact views report scatter from step to step about those of a rigid cart. At rest on the firm ground, over 200 steps, the two wheels' loads add up to between 867.4 and 910.2 N, with a standard deviation of 3.13 N, and they tend to alternate from one step to the next: the totals of successive steps are correlated by -0.42. Their mean, 883.921 N, is the weight to within 0.0034 %, and their means over 10 and over 100 consecutive steps stay within 0.29 % and 0.029 % of the weight. The first of these steps comes after the same placement and the same second at rest as the diagnostic of the loads, and it reports the same loads, 442.54 and 443.96 N: the excess of 0.296 % was the reading of a single step. On the slope, the mean over 500 steps is the rigid cart's load to within 0.0002 %. Rolling at 1 m/s from 5 m, single steps range from 774 to 1016 N, about a mean of 883.89 N, the weight to within 0.0001 %; braking at 2 $\mathrm{m/s^2}$ from 4 m/s, from 742 to 1008 N, about 883.37 N, 0.059 % below the weight. A placement disturbs the loads briefly: in the trace of period 83, on the slope, they add up to between 596 and 1743 N in the first ten steps after the placement, and to between 833 and 903 N in the forty steps of the lead-in that follow. The traces did not cover the crest: the seven steps in which a wheel reported no load there remain unexplained.

What the record holds. The rounding belongs to the numerical model. The number of substeps, the distance from the origin and single precision together set how far the motion can depart from the velocities that PhysX adds, here up

to 30.5 mm/s for the slider beyond 16 m and 30.9 mm/s along the slope, and a record that names only the time step does not fix it. The record of Section 11.2 names the solver and its iterations, and the bound follows from them and from the track: half a spacing of single precision at the largest coordinate, per update of the positions, and, for the speed along the track, divided by the cosine of the steepest angle of the ground. The test of the speed signal catches the departure, as it did here; with PGS, no period ends beyond the reported speed by more than 0.01 mm, though 11 exceed the micrometre that the check of estimation of Section S12.10.4 allows. The configuration of Section 11.2 stays as recorded. The traces explain F11 of Section S12.7 on the firm ground and on the slope, and the excess load of F12; the readings of F13 at 1 and 0.5 ms lie within the bounds of the rounding. On the crest, which the traces did not cover, F11 rests on the bound and the unloaded steps of F12 remain unexplained, and F13 remains open at 0.25 ms.

S11.7 The learning tool, a separate dependency

Isaac Sim's documentation divides the work: 'Rig your robot and stage a scene in Isaac Sim so Isaac Lab can train policies on it' (NVIDIA, 2026b). Isaac Lab is released on its own: its version 3.0.0 appeared on 19 October 2026, six days after the pip package of Isaac Sim 6.2.0 (NVIDIA, 2026a,c). Isaac Sim 6.2.0 lists Isaac Lab 3.0.0 among its supported products, and warns that 'matching a release number alone does not establish compatibility for every physics backend, policy, or custom ROS interface'; for the policies of Isaac Lab, it refers to each policy's 'documented simulator and physics-backend requirements' (NVIDIA, 2026b, Cross-product compatibility).

For the record of an experiment, a learning tool is therefore a dependency of its own. A configuration whose module was trained with it records the release of the learning tool beside those of the simulator and the physics backend, the settings of the training, and the training and evaluation split. It also runs the diagnostics of Section 11.2 again in the configuration that trains, wherever that configuration departs from the one recorded here. The GPU pipeline, which Isaac Sim's PhysX schema enables by default and this experiment turned off, is one such departure. A module learns from the signals of the configuration that trains it, a reported speed among them. No module of the cart's configuration learns, and no learning tool was used: Isaac Lab was not installed, and the components for reinforcement learning installed with the package went unused (Table S11.2).

S11.8 Sliding in MuJoCo

Above what the ground allows, the wheels slide, and MuJoCo's contacts show another side of their nature (Section S11.9.1). In its convex model of contact, slip can begin only with some motion along the normal of the contact; the documentation calls such effects numerically small yet undesirable, and of little practical relevance, because the objection to them presumes hard contact (Google DeepMind, 2026b, Computation). A sled, the cart with its wheels welded to the chassis, sliding from 4 m/s on the firm ground hops: its front wheel touches the ground in 63 of 500 steps, with loads of up to 5.02×10^3 N, against 442.53 N at rest, and in 13 % of the steps at 0.5 ms and 0.25 ms as well; it decelerates at 2.46 m/s^2 on average. A box of the cart's mass sliding from 4 m/s on a plane, with the same options, touches it in 150 of 500 steps, with loads of up to 12.2 times its weight. The hops are consistent with that effect; we have not traced them to it.

Implementation and diagnostic record

The following sections hold the release-specific implementation and diagnostic material of the book's runs. They are records of those runs, not installation advice for later releases.

S11.9 The cart in MuJoCo

MuJoCo is a general-purpose physics engine for robotics and neighbouring fields, first described by Todorov et al. (2012). It represents the state of a system in joint coordinates, with the bodies organized in kinematic trees, and it computes contact forces by solving a convex optimization problem (Google DeepMind, 2026b, Overview). Its contacts are soft: a contact allows a small penetration, and it draws its acceleration towards a reference that follows a spring-damper law on the penetration and its rate (Google DeepMind, 2026b, Computation); a contact under load also slips gradually. The book builds the cart of Chapter 3 in MuJoCo and connects it to the reference implementation as a simulator adapter (Sections 11.1.1 and 11.1.2); this section records the diagnostics and the run that ask whether the simulated world realizes what the cart's model and labels assume. MuJoCo 3.16.0 is the release with which these runs were made; MuJoCo 3.17.0 followed on 22 October 2026 (Google DeepMind, 2026a), and the runs were not repeated with it. The code is in `code/ch11`: the package `cart_mujoco`, with the simulator adapter `cart.mujoco`, and the script `mujoco_cart.py`, which runs the diagnostics and comparisons of this

section and, with the option `--check`, confirms every number that the section quotes from its results. The record of the experiment is in Table S11.1.

S11.9.1 Diagnostics

An adapter can be wrong in ways that no declaration shows: a unit read differently by the simulator, a quantity read a step late, a clock that drifts, a coordinate that jumps, a numerical artefact that looks like physics. Each diagnostic of Table S11.6 checks one link between the model and the simulator, and the script runs most of them twice, on the adapter and on a variant with one such mistake, to show that the diagnostic reveals it. Others measure what the simulated contacts do where the model has nothing to say.

Units, timing and coordinates. The wheel loads at rest add up to the weight to the last of the six digits printed, and at 14 positions along the track the wheels overlap the ground by the penetration at rest, so the geometry that MuJoCo collides agrees with the profile behind the track coordinate. The rangefinder checks that coordinate, through MuJoCo's own ray cast, against the wall that the scene places; its variant writes the wall's rotation as an Euler angle of 0.1658, the slope's angle in radians, which the compiler reads in degrees, so that the wall stands almost upright. The variant that reads the state after `mj_step`, as if the derived quantities were current, reads the position at the start of the last step. A loop on the simulator's clock miscounts in four periods, because the clock adds 0.001 s at every step in binary floating point, which cannot represent 0.001 exactly: after 22,000 steps it reads 22.000 000 000 003 904 s. With the arc length of the ground as the coordinate, the speed that the drive loop follows drops by 20 mm/s at the first step on the crest, while the true speed is still on the reference; the loop restores the measured speed within 32 steps by speeding the cart up, and the true speed then stays more than 15 mm/s above the reference over the whole crest.

What the ground allows. The largest constant acceleration that the drive loop sustains, found by bisection to 0.005 m/s^2, is 2.45 m/s^2 when braking on the firm ground, against μg, and 0.796 m/s^2 on the slope, against 0.7997 m/s^2; an acceleration counts as sustained if, after 0.1 s, the error of the speed changes by no more than 1 mm/s in 0.4 s. At the labels themselves, the speed stays within 0.381 mm/s of the reference.

Contacts on the crest. Braking at the label from 2.12 m/s at 20 m, the highest speed there of a certified plan, the wheels keep their contacts over the crest, and their total load stays within 1.96 % of $M(g\cos\phi - v^2/(R+r))$, that of a rigid cart. Rolling freely from 3 m/s, however, the cart has a wheel out of contact in 39 steps, although the centripetal acceleration of an axle at that

Table S11.6. The diagnostics of the simulator adapter `cart.mujoco`, their results, and the mistakes that they reveal. Each variant carries one mistake; the variants are written into the package, where nothing else uses them. From `results/mujoco-summary.txt`.

Diagnostic	Adapter	Variant with a mistake	Reveals
Wheel loads at rest	442.53 N and 441.35 N, together 883.881 N: the weight of 90.1 kg	—	mass, gravity, units
Wheels on the ground, at 14 positions	each sunk by the penetration at rest, to within 5.34×10^{-8} m; on the crest, to within 1.29 µm	—	the profile behind the coordinate
Rangefinder on the slope, tool folded and unfolded	the clearance $25\,\text{m} - (p + \ell)$, to within 1.9×10^{-11} m	wall rotated by an Euler angle in radians, read in degrees: 56.9 mm more	a unit of angles
One period at 1 m/s from 5 m, no command	0.519 µm short of 5.1 m	state read after `mj_step`: 1.00 mm short, the distance of one step	quantities a step old
Steps in each of 220 periods	100, counted	loop on the simulator's clock: 101 in periods 10 and 98, 99 in periods 49 and 188	a clock in floating point
Speed over the crest, braking at $0.5\,\text{m/s}^2$ from 2.1 m/s	within 0.364 mm/s of the reference	arc length of the ground as the coordinate: the cart 20.1 mm/s too fast	a coordinate whose rate jumps
Accelerations that the drive loop sustains	braking $2.45\,\text{m/s}^2$ on firm ground and $0.796\,\text{m/s}^2$ on the slope; $0.796\,\text{m/s}^2$ up the slope from rest; $2.45\,\text{m/s}^2$ towards the wall	—	what the labels claim
Contacts over the crest	braking at the label from 2.12 m/s at 20 m: in contact throughout, loads within 1.96 % of the rigid cart's	—	soft contacts on a curve
Encoders, accelerating at $2\,\text{m/s}^2$, at time steps of 1, 0.5 and 0.25 ms	2.05 mm/s above the cart's speed at each time step	stiff speed servos: 40.9, 19.4 and 8.68 mm/s below	creep of the contacts; a numerical artefact

speed, $0.891\,\mathrm{m/s^2}$, is less than a tenth of gravity. The documentation explains a cause consistent with this: for contacts, MuJoCo omits the part of a constraint's acceleration that comes from the motion of the contact frame, an omission that 'introduces a velocity-dependent bias for any constraint whose Jacobian varies with configuration' (Google DeepMind, 2026b, Computation), and the frame of a wheel rolling over the crest turns with it. Our reading is that this bias, $v^2/(R+r)$, counts against the reference acceleration that the penetration supplies, $0.584\,\mathrm{m/s^2}$ for each wheel at rest: three quarters of it at 2.12 m/s, all of it at 2.43 m/s; the curvature also unloads the wheels, which reduces that reference further. At constant speeds over the crest, the contacts hold at 2.3 m/s, and a wheel is out of contact in 3 steps at 2.4 m/s and in 9 at 2.5 m/s. The certified plans reach the crest at no more than 2.12 m/s, braking, where the contacts hold; the bound of 3.46 m/s is not claimed.

Sliding. Above what the ground allows, the wheels slide, and a cart sliding on MuJoCo's soft contacts hops (Section S11.8). The bisection therefore measures what the drive loop sustains rather than how a sliding cart slows down.

Signals. While the cart accelerates at $2\,\mathrm{m/s^2}$, the encoders report a speed 2.05 mm/s above that of the cart at each of the three time steps: the wheels creep, as a soft contact must while it drives the cart, and the adapter's loop follows the cart's speed, which on a real robot would come from an estimator. The variant that replaces the loop's speed feedback with MuJoCo's velocity actuators, set as stiff speed servos of $45.5\,\mathrm{N\,m\,s/rad}$, has its encoders read *below* the cart's speed, by a slip that shrinks with the time step and has a sign that friction, which here drives the cart forward, cannot produce. The integrator `implicitfast` accounts for the derivatives of the actuators' forces but not for those of the constraint forces (Google DeepMind, 2026b, Computation); with a servo this stiff on a wheel this light, we read the slip as the mismatch between the two. With the Euler integrator, the variant's encoders read 2.06 mm/s above the cart's speed at 1 ms, as the drive loop's do. Halving the time step is the diagnostic that tells a numerical artefact from physics.

S11.9.2 The run of Chapter 3 in MuJoCo

Period by period. The first comparison runs each of the 220 periods of the run of Chapter 3, the configuration `chapter3` of Chapter 10 with the exact track, in MuJoCo. The script places the cart 0.05 s before the period, on the motion of the previous period, and drives it under the previous law until the period starts, so that its contacts carry the load of that law. It executes the period's law and compares the result, at the end of the period and within it, with the

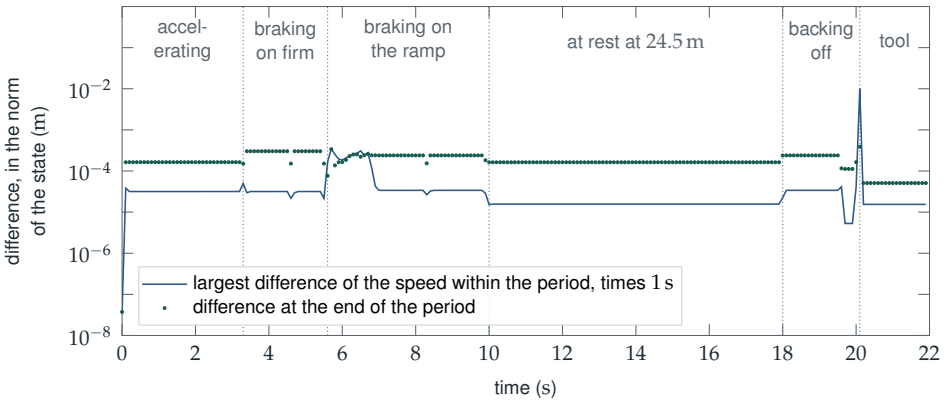


Figure S11.4. SIMULATION The simulated cart against the cart's model, period by period, in the run of Chapter 3. For each period, the cart starts near the state of the exact run, under the load of the previous period, and the period's law is executed in MuJoCo; the plot compares the result with the nominal motion of the model from the same start: the difference at the end of the period, in the norm of the state (green), and the largest difference of the speed within the period, times 1 s (blue). At the end of most periods, the folded tool and the integrator set the level: the tool's sag on the slope, 0.162 mm; while the cart brakes at $2\,\mathrm{m/s^2}$, the tool's yield, 0.2 mm, and the integrator's $ahT/2$; while it accelerates, the integrator's $ahT/2$ and the tool pressed into its soft stop. Over the crest, in periods 57 to 66, the difference at the end of a period is one of speed, which the drive loop leaves there. Within the periods, the speed differs most over the crest, where the drive loop's feedforward misses how the cart turns, and in period 201, in which the tool unfolds and its reaction pushes the cart back and forth. Data: `code/ch11/results/agreement-chapter3.csv`.

nominal motion of the cart's model from the state at which the period began (Figure S11.4), in the norm of the cart's state (Section 4.1.2),

$$\|(\Delta p, \Delta v, \Delta \ell)\| = \max\{|\Delta p| + |\Delta \ell|,\ |\Delta v| \cdot 1\,\mathrm{s}\}. \tag{S11.1}$$

The cart begins each period with its position and reach together at most 0.248 mm, and its speed at most 0.294 mm/s, from the exact state.

At the end of a period, the difference has a median of 0.163 mm, and nearly all of it is the folded tool: its servo, a spring of stiffness $k_p = 1000\,\mathrm{N/m}$, yields to the weight of the tool, of mass $m = 0.1\,\mathrm{kg}$, along the slope by $mg\sin\theta/k_p = 0.162\,\mathrm{mm}$. Nine periods in ten stay at or below 0.26 mm. Of the 22 above it, period 57 differs in speed on the crest, period 201 is that of the tool's unfolding, and twenty are periods of braking at $2\,\mathrm{m/s^2}$, at 0.302 to 0.303 mm. In them, the folded tool yields by $ma/k_p = 0.2\,\mathrm{mm}$, and the position differs from the nominal motion by 0.102 mm, almost all of it $ahT/2 = 0.1\,\mathrm{mm}$, where a is the acceleration, h the time step and T the period. This is the mark of an integrator

that advances the positions with the velocity at the end of each step, as the documentation states of MuJoCo's Euler integrator (Google DeepMind, 2026b, Computation); the measurements show the same of `implicitfast`. Under a constant acceleration a, that velocity exceeds the mean velocity over the step by $ah/2$, so each of the T/h steps adds $ah^2/2$ to the advance of the exact motion:

$$\frac{T}{h} \cdot \frac{ah^2}{2} = \frac{ahT}{2} = \frac{2\,\mathrm{m/s^2} \times 1\,\mathrm{ms} \times 0.1\,\mathrm{s}}{2} = 0.1\,\mathrm{mm}. \tag{S11.2}$$

The largest difference, 0.388 mm, is in period 201. The tool's servo, saturated at 100 N, drives the tool out and brakes it before the stop, and the reaction pushes the cart back, at up to 10 mm/s, and then forward, at up to 6.11 mm/s; the tool reaches 0.799 79 m at the end of the period, and the cart moves at 0.388 mm/s. Elsewhere, the speed differs from the nominal motion within a period by at most 0.347 mm/s, in period 57, just after the front wheel reaches the crest, where the drive loop's feedforward misses how the cart turns. The braking that each of the 220 labels of the exact run claims is available on the region of its segment, by the bound that the adapter computes at each wheel, $g(\mu \cos\alpha - \sin\alpha)$ at the angle α of the ground, less $\mu v^2/R$ on the crest, which the diagnostics support; the smallest margin, 0.232 m/s^2, is in period 65, at the end of the crest.

Three time steps. Halving the time step twice tells the kinds of difference apart (Table S11.7). What the simulator's physics produces does not change: the median stays at 0.163 mm, the sag of the simulated servo, and the encoders report the creep of the contacts as the same 2.05 mm/s at every step. What the integrator produces shrinks with the step: the differences in the periods of braking fall as $ahT/2$ halves. The adapter's own differences do not change with the step either, and other tests identify them: the speed differences at the crest halve when the speed gain of the drive loop is doubled, from 0.347 to 0.178 mm/s, because they are what the loop's feedback leaves of the error of its feedforward, which treats the cart as a point mass on the track; over the crest, where the chassis turns, the cart is not one. One value is not resolved at 1 ms: the tool's last approach to its stop in period 201 changes with the step, from 0.205 mm to 0.888 mm short of it, and smaller steps bracket it, 1.07 mm with `implicitfast` at 0.0625 ms and 1.25 mm with the Euler integrator at 0.125 ms. The value at the recorded step understates the shortfall.

> **Halving the time step tells a numerical artefact from what the simulation models: a value that stays as the step shrinks belongs to the simulated cart, its physics or its adapter, and a value that changes, whether it converges or grows, belongs to the computation.**

Table S11.7. The simulated cart against the cart's model, period by period, at three time steps of MuJoCo: the values of the comparison and what each difference is. The integrator's $ahT/2$ is that of Equation (S11.2). From `results/mujoco-summary.txt`.

Difference	1 ms	0.5 ms	0.25 ms	What it is
End of a period, median (mm)	0.163	0.163	0.163	the folded tool's sag on the slope
End of a period, 90th percentile (mm)	0.26	0.252	0.228	the periods over the crest and of braking; it does not grow
End of a period, braking at 2 m/s^2, period 40 (mm)	0.303	0.253	0.228	the tool's yield, 0.2 mm, and $ahT/2$
Position after accelerating from rest, period 1 (mm)	0.102	0.0521	0.0271	the integrator: $ahT/2$
Reach at the end of period 201, short of the stop (mm)	0.205	0.645	0.888	the servo's last approach, not resolved at 1 ms
Speed within a period, outside period 201 (mm/s)	0.347	0.345	0.347	the drive loop at the crest: what its feedback leaves of its feedforward's error, halved at twice the gain; period 57
Acceleration of the front axle less command, tool folded, over 1 ms (m/s^2)	0.0339	0.0315	0.0307	the drive loop where a period begins and its reference restarts from the measured speed: its feedforward's error, until its feedback takes it up again; period 65, near the end of the crest
The same over one step (m/s^2)	0.0339	0.0329	0.0328	the same
Jump of the speed signal where the front axle reaches the crest, read over one step (m/s^2)	0.0493	0.0987	0.197	the coordinate: a jump of 49.3 to 49.4 µm/s divided by the step

The cases of Section 10.4. With a tolerance of zero, all 26 cases of Section S10.11.2 differ from the model through the check `motion_agreement`; with 0.001, a millimetre and a millimetre per second, none does, the largest difference being 0.666 mm/s, in speed. A tolerance must cover the variation among the worlds that the labels admit (Section S10.11.1), which for these commands is none, so it cannot justify hiding differences that come from the simulated cart and the adapter.

The configuration of Chapter 3. The configuration of Chapter 3, with the exact track replaced by `cart.mujoco`, is built without complaint. In its run of 220 periods, the cart starts at rest and is to hold still in period 0, under the global capability of the rest pair of the initial plan, while the governor chooses the first plan that moves it. After that period, the measured state is $(-1.34 \times 10^{-9}\,\mathrm{m}, -1.09 \times 10^{-8}\,\mathrm{m/s}, 3.98 \times 10^{-10}\,\mathrm{m})$: the contacts have settled by about a nanometre. The funnel of the head is a single state, the measurement-error bound is zero, and the distance between them, $1.09 \times 10^{-8}\,\mathrm{m}$ in the norm of the state, raises an alarm at 0.1 s. The rest feedback then holds the cart. No restart is certified: with its maximal rest feedback, the control package represents the set that the rest feedback can reach only for a cart exactly at rest, and the measured speed is not zero. The cart stays within $4.59 \times 10^{-7}\,\mathrm{m}$ of the start for the whole run, and the tool operation is dropped after an overload at 20 s. The run therefore raises an alarm at 0.1 s, finds no certified restart and does not complete its task. (The evaluator `cart.metrics` reports that the tool unfolds at 0 s, because it counts any reach above zero; like the control package's set of the states that its rest feedback can reach, it was written for exact states.) The three outcomes are reported separately because they say different things. *Label validity*: formally, under the cart's model, no label is valid for the simulated world, as the next paragraph explains; the braking that the labels claim is available, for all 220 labels of the exact run and for the one label that this run executed, the global capability in period 0. *Alarm detection*: one alarm, at 0.1 s, raised correctly, because the measured state was not the predicted one. *Physical violations*: none; the least clearance is 25 m.

Guarantee status. No safety or recoverability result of Chapter 9 applies to this run, whose first alarm comes at its first measurement: the simulated cart is not a world of the cart's model, so Theorem 9.5(b)–(c) and Corollary 9.7 do not apply, and Proposition 9.6 needs a restart plan, which the run does not have. Up to the alarm, each plan that the governor committed was certified, by its test, under the model of Chapter 3: the property that Theorem 9.5(a) states, whose proof uses only (S3), (S4) and (S6). Parts (b) and (c), safety and recoverability, are claims about the world; they need the standing assumptions to hold of the simulated world, and they do not, for a reason that is not the braking. Two readings of the cart's model lead to the same place. The first keeps the model of Chapter 3, whose worlds differ in the braking that the ground allows and otherwise follow its motion rule exactly (Section 4.1.3). The simulated cart follows that rule only up to the differences measured above, so the simulated world is not a world of the model, and (S1) of Assumption 9.1 fails; formally, no label is then valid for it, not even the global capability, so (S2) fails as well. The second reading widens the model, as Chapter 5 did for the disturbed cart (Section 5.1.1), to worlds

whose motion departs from the rule by bounded amounts. The labels can then be valid, if the worlds are read along the motion that the cart executes, or if the operational design domain limits the speed on the crest, where the bound falls below $0.5\,\mathrm{m/s^2}$ above $3.46\,\mathrm{m/s}$ and the simulated contacts give way between 2.3 and $2.4\,\mathrm{m/s}$. But the test of Chapter 3, which certifies single motions, is unsound for such worlds, so (S3) fails, and the initial plan, whose rest segment holds a single state, is not certified, so (S4) fails too. Either way, the funnels do not contain the simulated motion. The alarm is the contract at work: at the first measurement, it found a motion that no certificate covered, and it turned to the rest feedback. Agreement does not change this: the agreement of the simulated cart with the model, like agreement between simulators (Section 11.3), is numerical agreement of declared quantities, not certified execution of a task.

A certificate in MuJoCo needs funnels that contain the simulated motion, such as the tubes of Chapter 5, and evidence that the simulated motion stays within them for every state and command that a certified plan can reach. What tests a tube is its hypothesis, a disturbance of the commanded acceleration bounded at every instant (Equation (5.1)), not the differences above, which with the tool folded stay far inside the tube's error bounds (Equations (5.9) and (5.10)) but start afresh in every period. Read as such a disturbance, the realized acceleration of the front axle stays within $0.0339\,\mathrm{m/s^2}$ of the command in all 201 periods with the tool folded, at each time step, below the bound of $0.05\,\mathrm{m/s^2}$. Read from the adapter's speed signal, however, the jump of the coordinate at the crest appears as $0.197\,\mathrm{m/s^2}$ over steps of 0.25 ms, and a bound at every instant cannot accommodate a jump: a tube for this adapter needs a coordinate without one. While the tool unfolds, which it never does in Chapter 5, the reaction reaches $1.11\,\mathrm{m/s^2}$, and the tube does not apply. Runs such as these can show that a bound fails; they cannot show that it holds.

S11.10 The cart in Isaac Sim

Isaac Sim, NVIDIA's simulator for robotics, calls itself 'USD-native': a scene is a stage of OpenUSD, 'one scene representation from asset import to deployment', which it simulates 'with PhysX or Newton' (NVIDIA, 2026b). This section runs the experiment of Section 11.1 a second time, in Isaac Sim with PhysX, through a second simulator adapter, `cart.isaac`, with the same contract, and asks what changes when a declared experiment moves to another simulator. Isaac Sim 6.2.0 is the release with which these runs were made. The script `isaac_cart.py` of `code/ch11` runs only inside Isaac Sim, on a computer with an NVIDIA RTX GPU. Its results come from two runs on one laptop, whose period-by-period tables are identical, byte for byte. The script `isaac_check.py` needs nothing

but Python: it derives the values that this section computes from those results and, with `--check`, confirms every number that the section quotes. The record, the releases, the laptop and the two runs are in Table S11.2 and Section S11.2.

S11.10.1 Asset import and the stage

Isaac Sim imports robots and scenes from URDF, MJCF and other formats (NVIDIA, 2026b). Its MJCF importer, which converts a text through the bundled package `mujoco-usd-converter`, did not convert the text of Section 11.1: that text defines the mesh of the ground by its vertices, and the converter stopped with 'Mesh ground has no file'. It converts meshes only from files, and its source marks meshes given in the text as future work, in release 0.6.1, which Isaac Sim bundles, as in release 0.8.0 (Newton Developers, 2026a). The script therefore writes the stage itself, with the schemas of USD Physics and of PhysX, from the list of parameters of Section 11.1 (`cart_isaac/stage.py`): the two simulators no longer read one text, but the same numbers, and what PhysX builds from them is part of its record. A text that one importer reads, another may reject or read differently; Section 12.1.2 returns to this.

USD carries units that MJCF leaves to its compiler. A stage states its metres and kilograms per unit. Its angles are in degrees, for the rotation of a prim as for the limits of a revolute joint, and so are the gains of a joint's drive: the damping of an angular drive is a torque per degree per second (Pixar Animation Studios, 2026). PhysX measures the same angles in radians (NVIDIA, 2025a, Articulations), and so does the interface through which the adapter reads and writes the cart. The stage states every unit that matters: metres and kilograms per unit; the gravity, $9.81\,\mathrm{m/s^2}$, which a stage that leaves it unset receives as Earth's gravity in its own units of length (Pixar Animation Studios, 2026); and the wall's rotation as a quaternion.

PhysX simulates the cart as an articulation, in joint coordinates as MuJoCo does, but with one link per joint: the chassis moves in its plane through two auxiliary links of 1 g each, which add 2 g to the mass that the drive loop moves. The contacts are rigid, as PhysX's are unless they are made compliant (NVIDIA, 2025a, Rigid Body Dynamics): collision detection generates contact points as soon as two shapes come closer than the sum of their contact offsets, 0.02 m here, and the solver brings them to rest at the sum of their rest offsets, zero here (NVIDIA, 2025a, Advanced Collision Detection), so at rest the wheels' axles stand at the wheels' radius, to within 8.94 nm. Friction is Coulomb friction with $\mu = 0.25$, static and dynamic alike, in PhysX's patch model. The solver is TGS, temporal Gauss–Seidel, with 32 position iterations and one velocity iteration in each step, and TGS divides each step into as many substeps as it runs position iterations (NVIDIA, 2025a, Rigid Body Dynamics). The tool's servo is a drive

of its slide, which PhysX solves implicitly (NVIDIA, 2025a, Articulations), and the stops of the slide are limits of the joint, which, unlike MuJoCo's, do not yield. PhysX runs on the processor, in single precision (NVIDIA, 2026f). The script records the default of every setting that the stage writes; one that the stage changes is the angular damping of the links, 0.05 by default, which the model does not declare.

S11.10.2 Sensing, actuation and timing

The simulator adapter `cart.isaac` declares the state and the actuation of the cart's robot model, as `cart.mujoco` does, and keeps the track coordinate, the drive loop, the rest feedback and the placement at a state of Section 11.1.2, line for line. What changes is how it reaches the simulator.

Sensing. The adapter reads the cart through PhysX's tensor interface, with NumPy on the processor: the poses and velocities of the links, the positions and velocities of the joints, and, from one contact view per wheel, the force of the ground on the wheel over the last step. The track coordinate comes from the position of the front axle and the speed from its velocity. Two properties of these readings matter below. They are in single precision, whose numbers lie 1.91 µm apart at 25 m. And the velocities that PhysX reports need not be those with which it moved the bodies: its position iterations correct positions and velocities, and their velocities advance the poses, while its velocity iterations correct only velocities, and theirs are the velocities that it carries to the next step and reports. In the words of its documentation, 'one cannot expect that the reported body velocity will match the position difference between two simulation steps' (NVIDIA, 2025a, Rigid Body Dynamics). For this cart, the traces (Section S11.10.4) find another cause, which the velocity iterations do not change: PhysX rounds the positions to single precision at every substep. Figure S11.5 sets one period and one step of each simulator side by side.

Actuation and timing. At every step, the drive loop writes the wheels' torques as efforts of their joints, each limited to 40 N m, and shares its force by the loads that the contact views reported in the previous step; the commanded reach is the target of the tool's drive. The adapter steps the physics one step at a time through Isaac Sim's simulation manager, a hundred steps per period, and counts them. It never steps by updating the application, which steps the physics by a clock of its own, 16 or 17 steps of 1 ms per update. Starting the simulation had stepped it already: when the adapter had taken 3000 steps, the simulation manager had counted 3018, two of them in the warm-up with which Isaac Sim starts the physics (NVIDIA, 2026d), and the rest, as we read it, in the application's update that followed. The adapter places the cart after these steps,

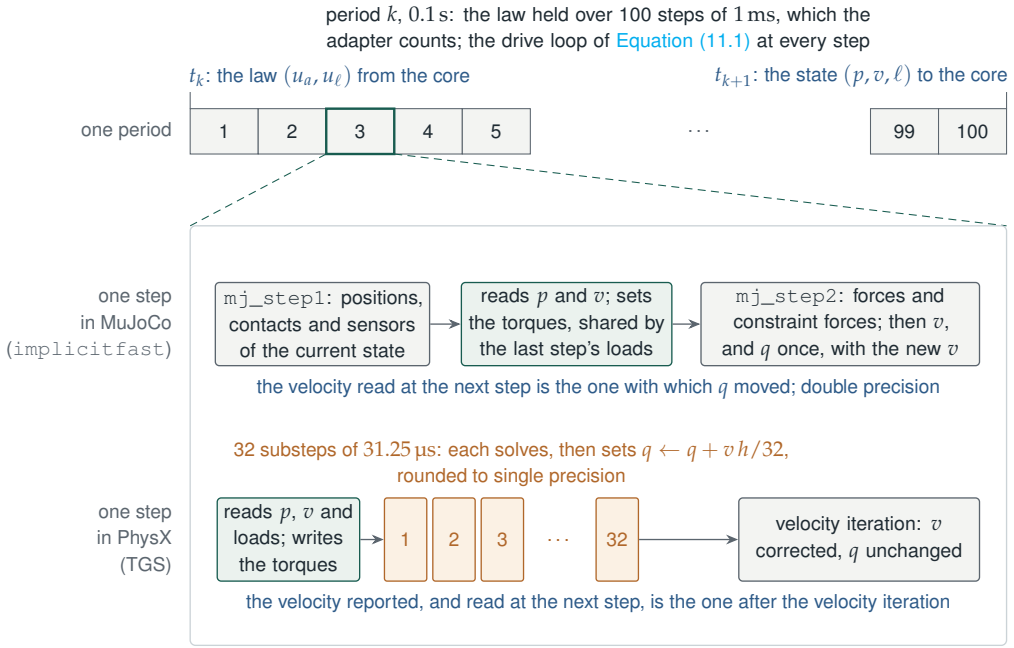


Figure S11.5. ILLUSTRATION One period of the governor and one step of each simulator, as the adapters of Sections 11.1.2 and S11.10.2 run them. The law of period k is held over 100 steps of 1 ms, which the adapters count; at every step, the drive loop reads the state and sets the wheels' torques (green). In MuJoCo (middle), `mj_step1` brings the derived quantities up to date before the adapter reads them, and `mj_step2` advances the velocity and then, once, the joint positions q with the new velocity, in double precision. In PhysX's TGS solver (bottom), the step of length h is divided into 32 substeps, one per position iteration; each advances the joint positions by their velocity times the substep and rounds the sum to single precision (amber). The velocity iteration that follows corrects the velocities without moving the positions, and its velocities are those that PhysX reports. After the documentation and source cited in Sections 11.1.2, S11.6 and S11.10.2.

so they change no result, but a record that counted the manager's steps would be 18 steps out.

S11.10.3 Diagnostics

The diagnostics of Section S11.9.1 ran in Isaac Sim, with variants that carry the mistakes that a USD stage invites; the online supplement lists them all (Table S11.3). Most check the same links between the model and the simulator as in MuJoCo, and reveal the same mistakes. Some show properties of PhysX that the cart's model does not have, and two, calibrated on MuJoCo's signals, do not transfer: the bisection of what the ground allows, and the loads over the crest.

Units. The wall rotated by the slope's angle in radians, where USD reads degrees, stands almost upright, and the rangefinder reads 56.9 mm more than the clearance, as in MuJoCo. Speed servos on the wheels whose damping is written per radian, where USD reads it per degree, get from PhysX a gain of 2606.96 N m s/rad, 57.3 times the 45.5 intended: reading a gain back from the simulator reveals such a mistake before any motion does. A stage in centimetres may keep the lengths of one in metres and leave the gravity unset, which it then receives in centimetres per second squared: the cart is a hundred times smaller, and its loads at rest add up to 88 390.3 in the stage's units of force, a hundred times the weight in newtons, which a check of the loads in SI units reveals. And a stage that leaves PhysX's defaults in place gives each link an angular damping of 0.05, which the model does not declare: coasting for 1 s from 1 m/s, the cart then loses 2.28 mm/s instead of 0.0985 mm/s.

Loads, geometry and timing. At rest on the firm ground, the wheels' loads add up to 0.296 % more than the weight, yet in the stage in centimetres they add up to the weight within 0.0014 %: the excess is no mistake of mass or gravity, but the reading of a single step (Section S11.10.4). Ray casts find the collision surface on the profile, and the rangefinder reads the clearance to within 2.78 µm, in single precision. The clock of the simulation manager does not drift: it is the count of steps divided by the steps per second (NVIDIA, 2026d). Yet a loop that steps while the clock, measured from the start of the loop, is below the end of the period runs 101 or 99 steps in 88 of the 220 periods. The difference of two readings of the clock is rounded: after the 400 steps of periods 0 to 3 it reads 0.399 999 999 999 998 6 s, and the loop takes a 101st step. That the rounding gives exactly these 88 periods is a consistency check, not a proof (Section S11.4). The adapter counts steps, as in MuJoCo, where the clock drifts instead.

The reported speed. Over one period at 1 m/s from 5 m with no command, the drive loop holds the reported speed at 1 m/s, and the bumper ends 0.707 mm beyond 5.1 m: as if the cart had moved, on average, 7 mm/s faster than the speed that the loop held. Suppose that the reported speed were the rate of the position, and that the simulator advanced the position by h times the speed at the end of each of the $N = T/h$ steps of a period. Let Δv_i be the difference of the reported speed after step i from the nominal speed. Then the end position would differ from the nominal motion by $\Delta p = h \sum_{i=1}^{N} \Delta v_i + ahT/2$, with the integrator's lead of Equation (S11.2), and it would lie in the band

$$T \min_i \Delta v_i + \frac{ahT}{2} \;\le\; \Delta p \;\le\; T \max_i \Delta v_i + \frac{ahT}{2}. \tag{S11.3}$$

In MuJoCo it does, in each of the 220 periods of the run of Section S11.9.2. In Isaac Sim it lies beyond that band by more than 0.01 mm in 41 periods, by more

than 0.1 mm in 31, and by 1.28 mm in period 83: 32 of the 41 are periods on the slope, 6 with a wheel on the crest and 3 on the firm ground, and all 41 begin beyond 16 m. The adapter's speed signal is the reported velocity, and the drive loop regulates that signal, not the rate of the cart's motion. With the arc length of the ground as the coordinate, the mistake of Section S11.9.1 shows as in MuJoCo; the jump of the track coordinate's own rate, which in MuJoCo comes from the penetration of the loaded wheels, all but disappears, at most 0.728 µm/s in one step.

Friction and contacts. The bisection of Section S11.9.1 finds 2.45 m/s^2 towards the wall from rest, as in MuJoCo, but 0.142 m/s^2 when braking on the firm ground and nothing when braking on the slope. These numbers do not measure the ground: at the labels themselves, the speed stays within 2.78 mm/s of the reference, but its error changes after the first 0.1 s by up to 4.03 mm/s, and the criterion, set for MuJoCo's speed, reads these jumps of the reported speed, not the friction. The sled measures the friction directly: sliding from 4 m/s on the firm ground, it keeps its front wheel loaded at every step, at each of the three time steps, and decelerates at 2.45 m/s^2, μg to three digits. PhysX realizes the friction of the labels while the cart slides, and the hops of MuJoCo's sled do not appear. Over the crest, braking at the label from 2.12 m/s, a wheel reports no load in 7 steps, and the total load departs from that of a rigid cart by up to 86.8 %, although a rigid cart would keep both wheels loaded over the crest up to about 9.9 m/s. The contact forces of single steps of a rigid contact are not the loads of a rolling cart, and in a step in which one wheel reports no load, the drive loop, which shares its force by them, asks the other for all of it. On the firm ground and on the slope, the traces show these forces scattering from step to step about the loads of a rigid cart; on the crest, which the traces did not cover, the unloaded steps remain unexplained.

Signals. Accelerating at 2 m/s^2, the encoders report a speed 2.57, 4.42 and 17.2 mm/s above the cart's at 1, 0.5 and 0.25 ms. A difference that grows as the step shrinks is, by the test of Section S11.9.1, a numerical artefact, not the creep of a soft contact.

S11.10.4 The run of Chapter 3 in Isaac Sim

Period by period. The comparison of Section S11.9.2 ran unchanged (Figure S11.6). The cart begins its periods with its position and reach together at most 1.07 mm, and its speed at most 1.8 mm/s, from the exact state. At the end of a period, the difference has a median of 0.170 mm. At rest, it is the folded tool's sag once more, and accelerating from rest the position leads the nominal motion by the integrator's $ahT/2$, while the folded tool stays at its

rigid stop. Nine periods in ten stay at or below 0.758 mm, and 10 exceed a millimetre. The largest difference, 1.89 mm, is one of speed, 1.89 mm/s at the end of period 49, braking on the firm ground, and the speed sets the norm in 75 of the 220 periods. Within the periods, outside period 201, the speed differs from the nominal motion by up to 3.64 mm/s, in period 186, and the position by up to 1.34 mm, in period 83, on the slope. In period 201 the tool unfolds, and it ends the period 1.12 mm short of its stop. The braking that each label of the exact run claims is available by the same bound as in MuJoCo, since both simulators build the same ground; what supports the labels here is the tracking at the labels and the friction of the sled.

The disturbance read from the reported speed. Read as a disturbance of the commanded acceleration, as Section S11.9.2 reads it, the reported speed of the front axle departs from the command by more than the bound of Chapter 5, 0.05 m/s^2, in 195 of the 201 periods with the tool folded, by up to 3.02 m/s^2, in period 67, as the cart leaves the crest. The departure tends to grow with the speed, and even at rest it reaches 1.64 m/s^2 on the slope. Yet the position and the speed at the end of every period of rest with the tool still stay within 0.036 mm and 0.304 mm/s of the nominal motion. These are jumps of the reported speed from one step to the next, of up to 3 mm/s, and a speed that is not the rate of the position does not show whether the cart's motion jumps with it.

Three time steps. Halving the time step twice (Table S11.4), the differences do not converge as the step shrinks; they grow. The 90th percentile grows from 0.758 to 2.42 and 6.36 mm, the largest difference from 1.89 to 3.81 and 9.69 mm, and the reported speed jumps by up to about 3, 5 and 9 mm/s in one step: by the test of Section S11.9.1, these differences are numerical. Two things converge: the integrator's $ahT/2$, which halves with the step, and the tool's last approach to its stop, 1.13 to 1.14 mm short at every step below 1 ms (Section S11.11.2). Doubling the drive loop's speed gain, which halved MuJoCo's speed differences over the crest, moves them here only from 2.56 to 2.48 mm/s: they are not what the loop's feedback leaves of its feedforward's error.

The traces. A follow-up on the same laptop, the script `isaac_trace.py`, traced the cart step by step and repeated the comparison with six variants of the solver and the drive loop (Section S11.6). PhysX advances the positions in single precision at each of the 32 substeps of a step, and each rounding moves the cart faster or slower than the velocity that PhysX adds, by up to half a spacing of single precision per substep. At 1 m/s from 5 m, the rounding moves the cart 7.08 mm/s faster, and over a period 0.708 mm further, where the diagnostic found 0.707 mm; beyond 16 m, its bound is 30.5 mm/s horizontally and 30.9 mm/s along the slope. The number of updates of the positions sets the

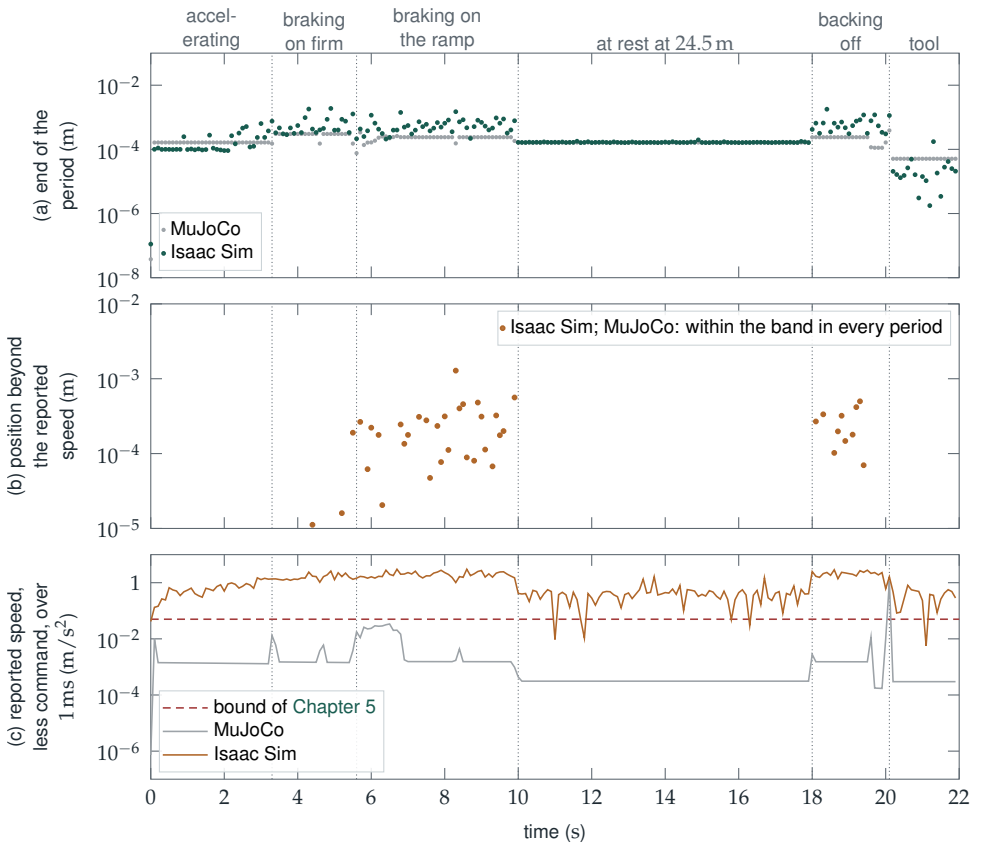


Figure S11.6. SIMULATION The run of Chapter 3 in Isaac Sim against the cart's model, period by period, with MuJoCo's run of Section S11.9.2 for comparison; every period starts afresh, 0.05 s early on the motion of the previous period. (a) The difference at the end of the period from the nominal motion from its start, in the norm of the state. At rest, the folded tool's sag on the slope sets it in both simulators; elsewhere, Isaac Sim's differences are larger, the largest of them in speed. (b) How far Isaac Sim's position at the end of the period lies beyond the band that its reported speed allows over the period, in the 41 of the 220 periods in which it lies beyond by more than 0.01 mm; in MuJoCo, it lies within the band in every period. (c) The reported speed of the front axle, read as a disturbance of the commanded acceleration: its largest departure from the command over a window of 1 ms, against the bound of Chapter 5. In Isaac Sim these are jumps of a reported speed that is not the rate of the position, far smaller with PGS (Section S11.6). Data: `code/ch11/results/isaac-periods.csv`.

effect: with 64 position iterations, 70 periods end beyond the reported speed by more than 0.01 mm, and with the PGS solver, which advances the positions once per step, none. The rounding explains the excess of the reported speed on the firm ground and on the slope, where 35 of the 41 periods lie; the encoders

follow the rounded motion; and the loads scatter from step to step about those of a rigid cart, their mean over 200 steps at rest being the weight to within 0.0034 %. The traces were all at 1 ms and did not cover the crest, so the growth with the step is attributed to the rounding, whose bound doubles each time the step halves, by our reading only.

The cases and the configuration of Chapter 3. With a tolerance of a millimetre and a millimetre per second, 15 of the 26 cases of Section S10.11.2 differ from the model, by up to 2.28 mm, in position; like the diagnostic of one period, they start from placed states without a lead-in, and since the traces did not follow them, their cause is our reading. The configuration of Chapter 3 with `cart.isaac` is built without complaint, and its run follows that of Section S11.9.2. After period 0, the measured state is $(5.96 \times 10^{-9}\,\mathrm{m}, 1.72 \times 10^{-9}\,\mathrm{m/s}, 1.87 \times 10^{-10}\,\mathrm{m})$: the position is that of the start, the base of the articulation standing at $-0.35\,\mathrm{m}$, which single precision stores as $-0.349\,999\,994\,039\,535\,5\,\mathrm{m}$. The speed, against the single state of the funnel of the head, raises an alarm at 0.1 s, and the rest feedback holds the cart. No restart is certified, because the measured speed is not zero; the cart stays within $5.42 \times 10^{-7}\,\mathrm{m}$ of zero, and the tool operation is dropped after an overload at 20 s. Reported separately: *label validity*, formally none, while the braking that the labels claim is available by the bound, with the friction that the sled measures; *alarm detection*, one alarm, raised correctly; *physical violations*, none, with a least clearance of 25 m.

Guarantee status. As in MuJoCo, the run raises an alarm at 0.1 s, finds no certified restart and does not complete its task, and no safety or recoverability result of Chapter 9 applies to it, for the reasons of Section S11.9.2. What a certificate would need is harder to test here, because the hypothesis of the tube, a bound on the disturbance at every instant, cannot be read from the adapter's signal, which is not the rate of the position; and in single precision the positions near the wall are 1.91 µm apart, while a disturbance of $0.05\,\mathrm{m/s^2}$ moves the cart by 0.025 µm in a step. A test of the tube in this simulator needs the disturbance of the motion itself, from positions over longer windows or from velocities that are their rate. With PGS, whose reported speed is the rate of the position to within the rounding over ten steps, the bound read from it still fails in 72 of the 119 periods of motion with the tool folded, by up to $0.745\,\mathrm{m/s^2}$, which suggests that the motion itself exceeds the bound; but PGS is not the configuration recorded, and these are jumps within single steps, which positions 1.91 µm apart cannot confirm. This run shows neither that the bound holds nor that it fails.

A learning tool, which this experiment does not use, is a dependency with a release of its own (Section S11.7): Isaac Lab, for which Isaac Sim stages robots, is released separately. A configuration whose module was trained with

it records the release of the learning tool beside those of the simulator and the physics backend, the settings of the training and the training and evaluation split, and runs the diagnostics of this section again in the configuration that trains, wherever it departs from the one recorded here. No module of the cart's configuration learns, and no learning tool was used.

S11.11 The same declared experiment in two simulators

Sections 11.1 and 11.2 ran one declared experiment in two simulators. This section sets the two runs side by side and asks of each difference between them what it is: a difference in what the simulators built from the same numbers, in the physics that they compute, in the numerical model with which they compute it, or in what a signal of the same name means. Nothing was run again: every number below comes from the results of the two sections.

S11.11.1 What was held the same, and what agrees

The two runs share everything outside the simulator: the cart and the track; the configuration `chapter3` of Chapter 10, with a simulator adapter in place of the exact track; adapters with the same declarations, track coordinate, drive loop, rest feedback, placement and counted steps; and the same comparisons. By the comparison of declarations, the two adapters are interchangeable. Four things differ: MuJoCo compiles an MJCF text, while the stage of Isaac Sim was written from the same parameters; MuJoCo's contacts are soft and its stops yield, while PhysX's contacts are rigid and its limits do not; MuJoCo computes in double precision, PhysX in single; and MuJoCo advances the positions once per step, PhysX's TGS in each of 32 substeps.

Table S11.8 sets the two runs side by side. Where the simulators compute the same physics, they agree with each other and with the cart's model. At rest on the slope, the folded tool's servo yields to the tool's weight by $mg \sin\theta / k_p = 0.162\,\text{mm}$ in both: physics, and the same physics. Accelerating from rest, both lead the nominal motion by $ahT/2$, and in both the lead halves when the step halves: numerics, of the same kind. Both build the same ground, so the braking that each of the 220 labels of the exact run claims is available in both by the same bound, with the same smallest margin. And the configuration of Chapter 3 runs through the core alike in both: a measured speed of the order of nanometres per second, against a funnel that holds a single state and a measurement-error bound of zero, raises an alarm at 0.1 s; the rest feedback holds the cart, no restart is certified, the tool operation is dropped after an overload at 20 s, and there is no physical violation. These outcomes belong to the contract, not to the

simulator: a funnel that holds a single state raises an alarm at any departure from it, however small.

S11.11.2 What differs, and why

The physics of the stops. MuJoCo's soft stops yield, by 0.0612 mm to the folded tool while the cart accelerates and to a reach of up to 0.800 05 m for the unfolded tool, while PhysX's limits hold the tool at its stops. In the cart's model the reach is a mode, 0 or 0.8 m (Section 3.1.1), which PhysX's limits keep more closely.

A numerical difference that the time step resolves. In period 201 the tool unfolds, and its saturated servo brings it to its stop. At 1 ms, MuJoCo ends the period with the tool 0.205 mm short of the stop and PhysX 1.12 mm short. As the step shrinks, MuJoCo's value rises, to 1.07 mm at 0.0625 ms, while PhysX's stays at 1.13 to 1.14 mm. Integrated directly, with the servo's saturation and the tool's weight along the slide, the tool's equation of motion gives a shortfall of 1.15 mm, whether the cart is held or free to recoil. The semi-implicit Euler method, applied to the same equation, gives 2.03, 1.59, 1.37 and 1.26 mm at 1, 0.5, 0.25 and 0.125 ms, where MuJoCo's Euler integrator gave 2.02, 1.58, 1.36 and 1.25 mm. The simulators disagree at the recorded step because MuJoCo's step of 1 ms does not resolve how the servo ends its stroke, while PhysX, which solves the drive implicitly over its 32 substeps, comes within 0.03 mm of the direct integration at every step. Neither value alone would have said which to trust; the study of the time step did.

A numerical difference that the time step does not resolve. At the end of the periods, nine in ten lie within 0.26 mm of the model in MuJoCo and within 0.758 mm in Isaac Sim. As the step shrinks, MuJoCo's differences converge and Isaac Sim's grow, which by the test of Section S11.9.1 makes them numerical. At the recorded step, the traces attribute them, on the firm ground and on the slope, to the rounding of the positions at each of PhysX's 32 substeps (Section S11.6), whose bound grows with the substeps per millisecond: 64 position iterations at 1 ms leave nine periods in ten within 2.42 mm of the model, as the step of 0.5 ms does, and PGS, which advances the positions once per step, brings the 90th percentile to 0.276 mm, close to MuJoCo's. That the growth with the step is the rounding's is our reading: the traces were all at 1 ms, and they did not cover the crest.

A difference of meaning: the reported speed. Both adapters declare the same component v, the speed along the track, and both read it from the velocity of the front axle. In MuJoCo, that velocity behaves as the rate of the position: in each of the 220 periods, the position at the end lies within the band that the speed's differences from the nominal motion allow (Equation (S11.3)). In PhysX, it is

the velocity that the solver reports, which departs from the rate of the position by up to 28.0 mm/s over ten steps, close to the bound of the rounding. Every check that reads the speed changes with it, from the disturbance read from the front axle to the norm of the difference at the end of a period (Table S11.8). The interface names match; the quantities do not. The registry's comparison of declarations cannot see this; only a diagnostic that compares the signal with the quantity whose name it bears, here the rate of the position, can.

An actuator difference, and the test that rules it out. Within a period, outside period 201, the speed departs from the nominal motion by up to 0.347 mm/s in MuJoCo, in period 57 on the crest, and by up to 3.64 mm/s in Isaac Sim, in period 186, backing off the slope. In MuJoCo, doubling the drive loop's speed gain halves the departure over the crest: it is an effect of the adapter's actuation. In Isaac Sim, the loop is the same code with the same gains, and the same change moves its departure over the crest only from 2.56 to 2.48 mm/s: what remains is not the loop's but, by the reading of Section S11.10.4, the reported speed's. A candidate cause was tested by the change that it predicts.

S11.11.3 Which comparisons are matched

A comparison between two simulators is matched when both measure the same quantity under the same declared conditions, so that a difference between them belongs to the simulators. By that test, most of the comparisons above are matched: the sag, the integrator's lead, the margin of the labels' braking and the outcome through the core agree; the tool's stops differ in their physics; its shortfall at the stop agrees once the time step resolves it; and the positions at the ends of the periods differ by an effect of PhysX's numerical model. The comparisons of the speed are not matched, because the quantity differs: MuJoCo reports the rate of the position, PhysX a velocity that departs from it. The speed at the end of a period, the speed within a period and the disturbance read from the speed are therefore matched only in MuJoCo; in Isaac Sim they need the disturbance of the motion itself. The margin of the labels' braking is matched in its bound, which both grounds share, but not in its support, because the bisections, calibrated on MuJoCo's speed, read the jumps of PhysX's reported speed instead of the friction.

Agreement between the two simulators is numerical agreement of declared quantities, not certified execution of a task. In each of them, the run of the configuration of Chapter 3 raises an alarm at 0.1 s, finds no certified restart and does not complete its task. No safety or recoverability result of Chapter 9 applies to these runs, and the restart result, Proposition 9.6, needs a restart plan, which neither run has. Neither simulated cart is a world of the cart's model, because each departs from the motion rule of Chapter 3 that the funnels assume

Table S11.8. The declared experiment of Sections 11.1 and 11.2 in MuJoCo 3.16.0 and in Isaac Sim 6.2.0 with PhysX, the releases used for the runs, at the recorded settings and a step of 1 ms unless stated, grouped by the kind of each difference. The values are those of Table S11.7, of Tables S11.4 and S11.5, and of the text of Sections 11.1 and 11.2; 'our reading' marks an explanation that the evidence supports without establishing it.

Quantity	MuJoCo	Isaac Sim
The same in both		
Folded tool's sag at rest on the slope (mm)	0.162	0.162
Lead after accelerating from rest, $ahT/2$, at 1, 0.5 and 0.25 ms (mm)	0.102, 0.0521, 0.0271	0.100, 0.0498, 0.0261
Least margin of the labels' braking, period 65 (m/s^2); its support	0.232; the bisections	0.232; the tracking and the sled
The configuration of Chapter 3 through the core	one alarm at 0.1 s; no restart; no violation	the same
Physics: a soft stop and a rigid limit		
Folded tool in its stop while accelerating; largest reach	0.0612 mm; 0.800 05 m	1.85×10^{-8} m; 0.800 00 m
Numerics: resolved by a smaller step		
Tool short of its stop at the end of period 201, at 1 and 0.0625 ms, against 1.15 integrated directly (mm)	0.205; 1.07	1.12; 1.14
Numerics: growing as the step shrinks; in PhysX, by our reading, the rounding at each substep		
End of a period, 90th percentile, at 1, 0.5 and 0.25 ms (mm)	0.26, 0.252, 0.228	0.758, 2.42, 6.36; with PGS 0.276
End of a period, largest (mm)	0.388, period 201	1.89, period 49
Start of a period from the exact state: position and reach; speed (mm; mm/s)	0.248; 0.294	1.07; 1.80; with PGS 0.245; 0.400
The 26 cases, beyond 1 mm and 1 mm/s	none	15, up to 2.28 mm in position
Actuation: the drive loop in MuJoCo, not in Isaac Sim		
Speed within a period, outside period 201 (mm/s)	0.347, period 57	3.64, period 186
The same over the crest, at the loop's gain and at twice it (mm/s)	0.347; 0.178	2.56; 2.48
Meaning of a signal: the reported speed		
Reported speed less the rate of the position over ten steps, largest (mm/s)	within the band in every period	28.0; bound 30.5
Disturbance read from the front axle's velocity, tool folded: largest (m/s^2); periods above the bound	0.0339; none of 201	3.02; 195 of 201
Periods in which the speed sets the norm at the end, of 220	12	75

(Sections S11.9.2 and S11.10.4). Agreement between two simulators shows that a result does not belong to one of them; it does not show that the model holds, nor that a task is executed under certificates.

> **Two simulator adapters with the same declared state and actuation can pass every comparison of declarations and still report different quantities under the same names. A comparison across simulators is informative only when each difference is traced to what it is: what the importer built, the physics, the numerical model, or the meaning of a signal. It is matched only where both simulators measure the same quantity, and even then it is numerical agreement of declared quantities, not certified execution of a task.**

CHAPTER S12

Newton, Genesis and numerical credibility: records and details

This chapter of the supplement holds what Chapter 12 moved out of its text: the diagram of its per-period comparison, the releases that its support matrix belongs to, the changes that Newton makes around MuJoCo imposed one at a time, the test of the renderers, the criteria of the checks of the numerical model, a figure of single precision and the time step, the failures found in Chapters 11 and 12 with what reproduces them, and the record of the experiment with its portability report and one reproduced experiment, with some values of the comparisons and checks that the text does not repeat. The releases named are those used for the runs.

S12.1 The per-period comparison

Figure S12.1 draws how each period of the run of Chapter 3 is compared between two simulators (Section 12.1.3).

Between configurations that simulate different carts, the errors of XPBD and of Genesis with its own defaults differ from those of the faithful ports by 0.423 mm and 0.968 mm at the median (Section S12.9.2), and by more than a millimetre in 49 and in 103 of the 220 periods.

S12.2 Releases and the support matrix

The matrix supports, for this experiment, the three faithful ports: they pass the four port checks, run the experiment to its end, and make errors over each period that agree with MuJoCo's to within a few micrometres, outside the tool's last approach to its stop. The other rows stay in the matrix for what they show: the variants with mistakes of conversion, which the port checks or the comparison catch, and the configurations with other models of contact and other solvers, which the experiment does not support.

The matrix belongs to the releases that produced it. The folder of this chapter pins them: Newton 1.7.0 with Warp 1.18.0, MuJoCo 3.15.0 and MuJoCo Warp 3.15.0, and Genesis 1.4.4 with Quadrants 1.3.3, PyTorch 2.14.1 and

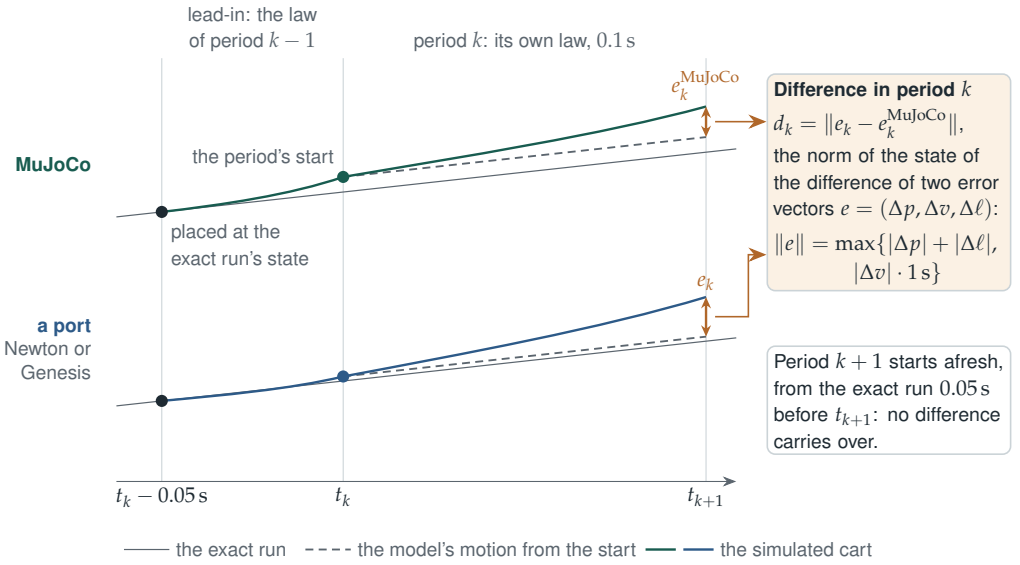


Figure S12.1. ILLUSTRATION How each period of the run of Chapter 3 is compared between two simulators (Section 12.1.3), drawn for one coordinate of the state, with the deviations exaggerated. In each simulator, the cart is placed at the state of the exact run 0.05 s before the period and driven under the previous period's law, so that its contacts carry that law's load. The period's error e_k is the state at its end less the model's nominal motion from the state at which the period began in that simulator. The difference from MuJoCo is the norm of the state, as Section 4.1.2 defines it for the cart, of the difference of the two error vectors. Every period starts afresh, so the comparison measures how two simulators differ over one period, not how far two complete runs would drift apart. Drawn from the procedure of `code/ch11/mujoco_cart.py` and `code/ch12`.

MuJoCo 3.16.0. It also records the complete Python environments in which the results were produced, and each configuration writes the record of the experiment of Table S11.1, with its simulator, solver, options and releases. A second run of every configuration, in the same environments, reproduced every result in its summary; only the times differed.

These simulators change quickly. Newton's predecessor inside Warp was deprecated in Warp 1.8 and removed in Warp 1.10 (Newton Developers, 2026b, FAQ). The defaults that Table 12.1 lists are those of the releases tested: the documentation of Genesis read for Section 12.1 gives 50 iterations as the default of its constraint solver (Genesis Authors, 2026a, `RigidOptions`), where release 1.4.4 uses 25. The release of the code that this book cites therefore keeps these pins, so that it stays executable when later releases of the simulators change their interfaces. A later release of the code adds rows to the matrix rather than replacing the tested ones. A reader who runs the experiment with another

release repeats the port checks first. Here they caught two of the three mistakes of conversion; the third showed in the comparison with a reference run and in one period above the bound of Chapter 5. Isaac Sim has no row. Its workflow (Section 11.2) writes a stage from the design instead of reading this text, it ran on another machine, and by the criteria of the port checks it fails all four, for reasons that Sections S11.6 and S11.10.3 discuss.

The support matrix changes nothing in the guarantee status of Section S11.9.2. The faithful ports reproduce MuJoCo's simulated cart, and with it the conclusions drawn from it; they add no evidence that the tube of Chapter 5 holds beyond what Section S11.9.2 found, and agreement between simulators is no validation on hardware. What they show is how much of the error over one period belongs to the simulator. In the faithful ports it is a few micrometres, outside a transient that the time step does not resolve; when a conversion goes wrong, it reaches 0.108 m/s in the speed at the end of a period.

S12.3 Newton's changes, imposed on MuJoCo itself

Table S12.1 imposes the four changes that Newton makes around MuJoCo (Section S12.9.2) on MuJoCo 3.15.0 itself, one at a time and together. It imposes the second by rounding the parameters of the text to single precision, which emulates Newton's rebuilding of the model but need not match it in every quantity. Imposed together, the changes leave 21 periods in which MuJoCo and Newton still differ by more than a micrometre, in 20 of them by one spacing of the representable positions at the end of the period and in period 190 by two.

S12.4 The renderer

No measurement of the cart comes from a renderer. Its sensors are computed by the engines: the rangefinder of Section 11.1 casts a ray along the axis of its site, and MuJoCo computes it, like its other sensors, independently of the visualizer (Google DeepMind, 2026b, XML Reference). The ports read joint coordinates, velocities and contact forces from the engines. To test that a renderer stays outside both models, the per-period run of Section S11.9.2 was repeated with a camera that draws an image after every step, off screen through Mesa's OSMesa, and compared value for value with the same run without one. With MuJoCo 3.16.0's OpenGL renderer, 36 000 images, one after every step of the comparison, the cart's settling at rest included, left every value of every period unchanged, bit for bit. With Genesis's rasterizer, 1950 images, one after every step of 13 periods that cover every phase of the run, did the same. In these tests, the renderers read the state without changing it.

Table S12.1. The four changes that Newton 1.7.0 makes around MuJoCo, imposed on MuJoCo 3.15.0 itself, one at a time and together: per period, the norm of the state of the difference between two error vectors (Section 12.1.3), its median, and its largest value with its period (m); every period starts afresh. Against Newton: against Newton on MuJoCo's C library, with the number of periods above 1 µm. From `results/layers/mujoco-3.15.0-newton-changes/summary.txt`.

MuJoCo 3.15.0, with	Against MuJoCo 3.15.0	Against Newton
nothing changed	—	9.43×10^{-7}; 4.19×10^{-6}, 198; 67
multiple contact points between convex geoms off	0 in every period	9.43×10^{-7}; 4.19×10^{-6}, 198; 67
the parameters in single precision	3.24×10^{-9}; 3.04×10^{-8}, 32	9.43×10^{-7}; 4.19×10^{-6}, 198; 66
the state in single precision before every step	1.06×10^{-6}; 5.52×10^{-6}, 99	1.22×10^{-7}; 3.98×10^{-6}, 190; 20
the readout of the state in single precision	7.81×10^{-7}; 1.75×10^{-6}, 89	1.61×10^{-7}; 3.84×10^{-6}, 68; 71
all four	9.32×10^{-7}; 5.83×10^{-6}, 99	1.02×10^{-8}; 3.81×10^{-6}, 190; 21

A renderer enters the model when an image is a measurement. A camera, a depth camera or a lidar that is rendered carries the renderer's projection, sampling and arithmetic into its sensor model, and a comparison of renderers then changes that model. Newton's camera sensor, for one, renders the model by ray tracing (Newton Developers, 2026c); MuJoCo 3.16.0 can instead attach its rangefinder to a camera and cast one ray per pixel in the engine (Google DeepMind, 2026b, XML Reference). A visualizer can also act on the simulation: MuJoCo's can write the force with which a user perturbs a body with the mouse into the simulation's data (Google DeepMind, 2026b, Visualization). The cart has neither a rendered sensor nor a user at the mouse, and the record of its experiment names no renderer. None of these comparisons changes the guarantee status of Section S11.9.2.

S12.5 The checks of the numerical model

Table S12.2 gives the checks of the numerical model that Section S12.10.1 applies, with the criteria that the record carries; their outcomes are in Table S12.5.

Outside the faithful configurations, the check of estimation was also run with Newton's XPBD solver: 6 periods lie beyond what the speed signal allows, by up to 57.7 µm (Section S12.10.4).

Table S12.2. The checks of the numerical model that the record of the experiment carries, after `CRITERIA` in `numerics.py`. Tolerances: 1 µm, 1 µm/s, 0.005 m/s², unless the criterion states another.

Check	What changes	What it watches	Criterion
Time step	1 ms, halved twice	the values of the comparison (Table S11.7)	no change of a halving above the tolerance, the integrator's $ahT/2$ set apart and a rate over one step judged by its jump; otherwise the value is unresolved at 1 ms
Constraint solver	a tolerance 10^4 times tighter with ten times the iterations; no warm start	each period's error; the iterations of every step	a change below 1 nm; no step at the iteration limit
Collision detection	nothing: the exact geometry of the ground and the wheels, as MuJoCo compiled them or, for Genesis, as the text gives them	every contact of a wheel on the firm ground, over the crest and on the slope	within 1 µm in distance and 10^{-4} rad in normal
Friction	the time step	the braking that the drive loop sustains; the creep of the encoders	no change of a halving above the tolerance
Limits	the time step	the tool's reach at the end of period 201; its largest reach against the stop	no change of a halving above 1 µm
Estimation	nothing: the speed signal against the position	the position at the end of each period	within 1 µm of what the signal allows (Section S11.10.3)
Arithmetic	single precision	one period at 1 m/s from 5 m	within 1 µm of what the spacing of the numbers predicts

S12.6 Single precision and the time step

Figure S12.2 computes, from the spacing of the positions that single precision represents, why a smaller time step can measure the rounding rather than the integrator (Sections 12.1.2 and S12.10.2).

The arithmetic of Section 12.1.2 in more detail: with MuJoCo's C library, Newton's cart ends one period at 1 m/s 1.31 µm short from 0.5 m, where the spacing is eight times finer and a step leaves 0.216 of it, and 40.1 µm long from

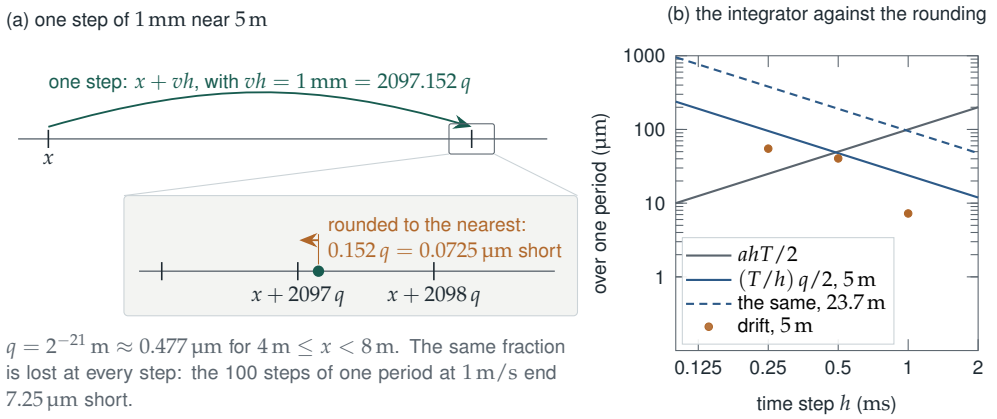


Figure S12.2. ANALYTICAL EXAMPLE Single precision and the time step, from the spacing of the positions that single precision represents (Sections 12.1.2 and S12.10.2). (a) Between 4 and 8 m, representable positions are $q = 2^{-21}$ m apart. A step of 1 mm at 1 m/s is 2097.152 q, and the position after it is rounded to the nearest representable one, $0.152\,q$ short; over the 100 steps of a period the cart ends 7.25 µm short, and Newton's ports end 7.15 µm short (Table S12.5). (b) Over one period, $T = 0.1$ s, the integrator's lead $ahT/2$ of Equation (S11.2), after a commanded acceleration $a = 2\,\text{m/s}^2$ (grey), shrinks with the step h, while the bound $(T/h)\,q/2$ of the rounding (blue) grows; at 23.7 m (dashed), representable positions are four times as far apart as at 5 m. The drift that rounding to the nearest predicts from 5 m (amber) is 7.25 µm short at 1 ms, 40.4 µm long at 0.5 ms and 54.9 µm short at 0.25 ms, where it exceeds the integrator's lead: below some step, halving it measures the rounding, not the integrator. Computed in the figure's source; the measured drifts are in `code/ch12/results/numerics/validation.txt`.

15 m, where the spacing is twice as coarse and a step rounds up by 0.424 of it. With MuJoCo Warp, it ends 1.28 µm short and 40.1 µm long. Genesis with its own defaults, in single precision too, ends 7.20 µm short from 5 m.

S12.7 Failures and their reproduction

A failure belongs in the record when it can be reproduced: the configuration and releases in which it occurs, the smallest run that shows it, a signature that identifies it, and the check that catches it. Table S12.3 lists the thirteen failures that Chapters 11 and 12 found in the configurations that they ran. They are classified as in Section 12.2: mistakes of conversion, which change a model without a declaration; numerical failures, in which the simulated cart is computed wrongly; other models of contact, which are other carts; a limit of MuJoCo's own law of contact, by our reading; a signal and an import that fail; and the unloaded steps of F12 on the crest, left unclassified because their cause is unresolved. What the checks find in the faithful configurations,

among it the check of estimation that Newton fails, is in Table S12.5 and in the record's numerical validation. Each reproduction outside Isaac Sim is part of `numerics.py` and prints its signature, which the script checks; a second run of each reproduced its signature exactly. F11 to F13 need Isaac Sim, and their signatures are read from the results of Section 11.2, the traces of its follow-up among them.

Three things stand out. First, the port checks catch the gross failures, F1 to F4, F6, F7, F11 and F12, whether they change the cart or its computation, but not the mild ones. Newton's gap, F5, showed only in the comparison with MuJoCo and in one period above the bound of Chapter 5. The libccd pipeline, F8, showed only in the check against the exact geometry, in the comparison and in 13 periods above that bound. A port is therefore not trusted on its port checks alone; the record carries the checks of the numerical model too. Second, F9 does not change with the time step: at each step, a wheel rolling freely over the crest at 3 m/s is out of contact for about 39 ms. We read it as belonging to MuJoCo's law of contact rather than to the computation. The time step does not change it, and our reading in Section S11.9.1 traces it to a bias at speed that MuJoCo's documentation describes. It bounds the speeds at which the simulated crest can be trusted, and the certified plans stay below them. Third, F10, first seen on the laptop of Section 11.2, reproduces here, on another machine and another operating system, with the converter alone and without Isaac Sim. It does so in the release that Isaac Sim bundles, 0.6.1, and in release 0.8.0: the converter exits with an error and leaves three files, 1241 bytes in all, whose three prims have neither children nor properties.

The failures of Isaac Sim rest on fewer runs. F11 and F12 were observed in both runs of Section 11.2, on one machine, whose period-by-period tables are identical, byte for byte. F13 was observed at 1 ms in both runs, and at the smaller steps in the second only, where the criterion of the check of friction was applied to a diagnostic; the check itself was not run in Isaac Sim. The record says so, and reproducing these failures needs Isaac Sim. A follow-up on the same machine traced them (Section S11.6). PhysX rounds the positions to single precision at every substep of TGS, which explains F11 on the firm ground and on the slope, where 35 of its 41 periods lie. At 1 ms the encoders follow the rounded motion, and the readings of F13 at 1 and 0.5 ms lie within the bounds of that rounding. The excess load of F12 is the reading of a single step, the mean over 200 steps being the weight to within 0.0034 %. The traces, all at 1 ms and none on the crest, covered neither the unloaded steps on the crest nor F13 at the smaller steps. The reading at 0.25 ms lies 1.9 mm/s beyond its bound, and it and the crest remain open. A reader who runs a reproduction with later releases learns from its signature whether the failure persists, and a failure that

Table S12.3. The failures of the declared experiment found in Chapters 11 and 12, from `results/numerics/failures.txt`, which names the part of `numerics.py` that reproduces each; F11 to F13 need the scripts of Section 11.2 and an RTX GPU.

	Failure; where	Kind	Caught by	Signature
F1	a wheel sinks into the crest; Genesis, MPR	mistake of conversion; numerical failure	check of the crest	292 of 658 steps with a wheel unloaded; 809 mm deep
F2	the state stops being a number at rest; Newton, Featherstone	another model of contact; divergence	every port check	after 2587 steps
F3	the same; Newton, semi-implicit	the same	every port check	after 23 steps
F4	a wheel loses its load on the crest; Newton, the importer's hull	mistake of conversion: the ground	check of the crest	3 of 1213 steps with a wheel unloaded; 1.44 mm deep
F5	other contacts on the crest; Newton, Newton's gap	mistake of conversion: the computation	comparison; bound of Chapter 5	period 59: 71.6 µm from MuJoCo; period 36: 0.0881 m/s^2
F6	almost no braking; Newton, XPBD	another model of contact	check of friction	braking 0.0684 m/s^2
F7	too little braking, a wheel unloaded on the crest; Genesis's defaults	another model of contact, with MPR	checks of friction and the crest	braking 1.01 m/s^2; 97 of 1212 steps with a wheel unloaded
F8	contacts wrong over the crest; MuJoCo, libccd	numerical failure	collision detection; comparison; bound of Chapter 5	loads 18.1 % from a rigid cart's; disturbance 0.232 m/s^2
F9	a wheel leaves the crest, rolling freely at 3 m/s; MuJoCo	MuJoCo's law of contact, by our reading	diagnostics of Section S11.9.1	out of contact for 39, 38 and 38.75 ms at the three steps
F10	the MJCF import fails; mujoco-usd-converter 0.6.1 and 0.8.0	import	the import	'Mesh ground has no file'; exit status 1
F11	the reported speed is not the rate of the position; Isaac Sim	numerical failure: rounded positions	test of the speed signal; two port checks	41 periods beyond it by over 0.01 mm, up to 1.28 mm; none with PGS
F12	loads at rest above the weight; a wheel unloaded on the crest; Isaac Sim	at rest, a signal of single steps; on the crest, unresolved	two port checks	loads 0.296 % above the weight in one step, the weight to within 0.0034 % over 200; 7 steps with a wheel unloaded
F13	the encoders' reading changes with the step; Isaac Sim	numerical failure	criterion of the check of friction	2.57, 4.42 and 17.2 mm/s above the cart's speed at 1, 0.5 and 0.25 ms

a release fixes changes the support matrix as a new row, not by rewriting the old one.

S12.8 The record, the report and one reproduced experiment

The record of an experiment, as Table S11.1 set it out, gains three entries. *Numerical validation*: the checks of Table S12.2, their criteria and their outcomes, among them the values that a check found unresolved. *Failures*: those of Table S12.3, each with the configuration in which it occurs and what reproduces it. *Acceptable numerical variation*: what a reproduction may differ by and still count as one. For this experiment, a second run in the same environment must give every value bit for bit, and so must a run in a new environment built from the requirements files on the same kind of machine. A run on another machine or operating system must give every quoted value at its quoted precision, or within one unit of its last digit. It must also give every period's error vector within 1 nm of the shipped one in double precision, or within two spacings of the representable positions in single precision, the spacing taken at the farthest position of the run. The bound in double precision, a choice, is about thirty times the largest change that the perturbations of the solver above produced, 3.35×10^{-11} m. The bound in single precision is the difference that the emulation of Newton in Section S12.9.2 left. The script writes these entries to `results/numerics/record.txt`.

The portability report, `results/portability-report.txt`, collects for the releases tested what this chapter found: the support matrix, the comparisons of Section 12.2 with the changes of Table S12.1, the outcomes of the checks, the failures with their signatures, and the record's new entries. It names the releases of the simulators, and the code that produced it by the SHA-256 of the code of Chapters 10 to 12. A later release of the code writes a new report beside it rather than replacing it.

The run of Chapter 3 in MuJoCo (Section S11.9.2), against which Section 12.1 compares every port, was reproduced by following its record and the README of its folder. A new Python environment was built from the requirements file, which pins MuJoCo 3.16.0 and leaves NumPy free, and so installed NumPy 2.5.4 where the record names 2.5.3. The processor was not the one of the record: the record gives it as an Intel Xeon at 2.10 GHz, and the new run's record as one at 2.80 GHz; the operating system was the same. The script found all 148 values that its check covers as shipped, and its file of the 220 periods identical to the shipped one, byte for byte; of the record, only the lines that name NumPy and the processor differ. On the same machine, every configuration of the support matrix, rerun in its original environment, reproduced its results. The per-period

files of Newton's five configurations that complete the run, MuJoCo Warp's among them, and of Genesis with MPR and with its defaults are identical to the shipped ones, byte for byte. Featherstone and the semi-implicit solver fail while the cart settles at rest, at the same step as in the shipped runs. Genesis with the options of the text differs in 68 of its values, in the ninth significant digit, its error vectors by at most 1×10^{-13} m. No value of any summary differs, and in the records of the configurations only the lines that name the processor do (`results/numerics/reproduction.txt`). These runs lie within the variation that the record declares acceptable for another machine. They are reproductions from the record by the team that wrote it, on another processor under the same operating system and, for MuJoCo, in a new environment. A run on another operating system, and a reproduction by someone outside that team on hardware of their own, are the tests that Chapter 17 asks for.

Implementation and diagnostic record

The following sections hold the release-specific implementation and diagnostic material of the book's runs. They are records of those runs, not installation advice for later releases.

S12.9 Engine, solver and renderer

A configuration of Table 12.2 combines three things that the name of a simulator runs together. The *engine* holds the model and the state: it imports the model text, keeps the state from one step to the next and computes the sensors. The *solver* advances the state over a step, with its integrator, its constraint solver and the laws of contact that it imposes; the detection of contacts belongs to the engine in some simulators and to the solver in others. The *renderer* turns the state into images. This section states, for twelve comparisons, whether each changes the physical or the numerical model, and how much it changes the cart's error over a period.

S12.9.1 Three layers and two models

Figure S12.3 shows the three layers in each simulator. In MuJoCo, one library is all three (Google DeepMind, 2026b, Visualization); Newton 1.7.0 is an engine with a choice of solvers, MuJoCo among them (Newton Developers, 2026c); and Genesis 1.4.4 builds a scene from the model that MuJoCo compiles, which its rigid solver advances, with or without its compatibility with MuJoCo, an option that its source counts among experimental options intended mainly for debugging and unit tests (Genesis Authors, 2026b).

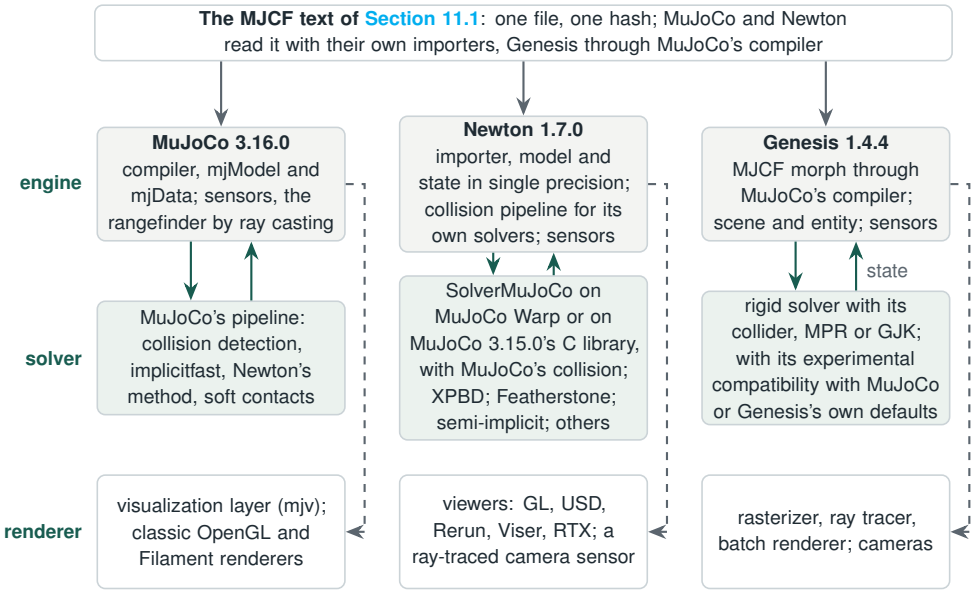


Figure S12.3. ILLUSTRATION Engine, solver and renderer in the three simulators of Section 12.1, for the releases tested. Each engine reads the same MJCF text, MuJoCo and Newton with their own importers and Genesis through MuJoCo's compiler; it holds the state and computes the sensors, and the solver advances the state over each step (solid arrows). A renderer only reads the state (dashed), unless a user perturbs the simulation through a viewer (Section S12.4). In Newton, the solver can be MuJoCo itself, on MuJoCo Warp or on MuJoCo's C library, which then also detects the contacts; Genesis's rigid solver, in its compatibility with MuJoCo, makes errors over each period that differ from MuJoCo's by less than a nanometre in all periods but one (Section 12.1.3). No measurement of the cart comes from a renderer. Drawn from the documentation of MuJoCo 3.16.0 and the sources of Newton 1.7.0 and Genesis 1.4.4 (Google DeepMind, 2026b; Newton Developers, 2026c; Genesis Authors, 2026b).

Two models run through these layers. The *physical model* is what is simulated: the geometry, masses and joints of the text, and the laws that the solver imposes on contact, friction and the actuators, with their parameters, such as the softness of the contacts and the shape of the friction cone. The *numerical model* is how it is computed: the integrator and its time step, the method of the constraint solver with its iterations and tolerance, the algorithm that detects the contacts, the arithmetic and the order of the operations. A change of the numerical model alone leaves the simulated cart the same and changes the error with which it is computed; a change of the physical model gives another cart, and the difference between the two runs is an error of neither. Which of the two a comparison changes is a question about its configurations, which their records and the sources of the releases answer, not about its result.

S12.9.2 What each comparison changes

Table S12.4 sets out twelve comparisons: seven between configurations of Table 12.2, and five with variants outside it, MuJoCo 3.15.0 by itself, Genesis with the options of the text in single precision, and MuJoCo and Genesis with a renderer. Each is the comparison of Section 12.1.3, which measures how a change alters the error over one period, not over a run.

The renderer and the same algorithm. No measurement of the cart comes from a renderer: its sensors, the rangefinder among them, are computed by the engines (Google DeepMind, 2026b, XML Reference). A camera that draws an image after every step left every value unchanged, bit for bit, with MuJoCo's OpenGL renderer in every period of the comparison and with Genesis's rasterizer in 13 periods that cover every phase of the run (Section S12.4). A renderer enters the model only when an image is a measurement, as for a rendered camera or lidar, and the cart has none. A new release of the solver and a reimplementation of its algorithm change the numerical model in principle, and here hardly in fact: MuJoCo 3.15.0 and 3.16.0 differ by 7.83×10^{-12} m at most, and Genesis, in double precision and in its compatibility with MuJoCo, differs from MuJoCo by more than a micrometre only in period 201, in the tool's last approach to its stop.

The arithmetic. Single precision changes the numerical model alone. Genesis in single precision differs from Genesis in double precision by 1.12 µm at the median, in a comparison that changes nothing but the arithmetic. Newton differs from MuJoCo 3.15.0 by 0.943 µm, but it makes four changes around MuJoCo (Newton Developers, 2026c): it disables MuJoCo's multiple contact points between convex geoms; it rebuilds MuJoCo's model from parameters that it holds in single precision; it writes its state, in single precision, into MuJoCo before every step; and it gives the port the positions and velocities of the bodies in single precision. Imposed on MuJoCo 3.15.0 itself, one at a time (Table S12.1), the four changes alter the errors over a period by nothing at all, by 3.24 nm, by 1.06 µm and by 0.781 µm at the median; imposed together, they bring MuJoCo's errors to within 10.2 nm of Newton's at the median, though the two runs still round some positions to different neighbours, which these comparisons do not explain. On this evidence, the micrometres of Section 12.1 belong to single precision in the state and in what the port reads of it, as far as the shares of each change alone and of the four together tell, since no run leaves out one change of the four. These comparisons do not separate MuJoCo Warp's implementation from its arithmetic.

Solvers and their settings. XPBD and Genesis's own defaults change the physical model as well as the numerical one. XPBD imposes its own contacts and friction,

Table S12.4. What each comparison changes, and how much it changes the cart's error over a period: per period, the norm of the state of the difference between two error vectors (Section 12.1.3), its median and its largest value over the 220 periods, or the 13 of the second row (m); every period starts afresh, so these are errors over one period, not whole runs. The models are classified from the configurations, their records and the releases' sources; the differences are measured. (a) An approximate integrator, 25 iterations, a tolerance of 10^{-6}, single precision, MPR, contact pruning and no compatibility with MuJoCo. (b) By MuJoCo's documentation and the test of Section S12.9.2. (c) Separated in Table S12.1. (d) It fails on the crest, a numerical failure (Section S12.10.3).

Comparison	Layer	Physical model	Numerical model	Median	Largest; period
The renderer					
MuJoCo 3.16.0, rendering every step	renderer	same	same	0	0
Genesis, the same, in 13 periods	renderer	same	same	0	0
The same algorithm					
MuJoCo 3.15.0 against 3.16.0	release	same	release	1.66×10^{-16}	7.83×10^{-12}; 61
Genesis against MuJoCo 3.16.0	engine, solver	same	implementation	8.68×10^{-11}	1.74×10^{-4}; 201
The arithmetic					
Newton on MuJoCo's C library against MuJoCo 3.15.0	engine	same	Newton's four changes[c]	9.43×10^{-7}	4.19×10^{-6}; 198
Newton, MuJoCo Warp against the C library	solver	same	implementation and precision	9.97×10^{-7}	2.38×10^{-6}; 46
Genesis, single against double precision	arithmetic	same	precision of the model and the state	1.12×10^{-6}	7.49×10^{-6}; 35
Solvers and their settings					
Newton, XPBD against MuJoCo's C library	solver	contacts, friction, the tool's servo	method	4.23×10^{-4}	3.38×10^{-3}; 199
Genesis, its defaults against the text's options	settings	contacts, friction cone	settings[a]	9.68×10^{-4}	9.21×10^{-3}; 201
Mistakes of conversion					
Newton, the importer's hull against the full hull	model	the ground	same	4.55×10^{-9}	5.71×10^{-4}; 64
Newton, Newton's gap against none	model	same[b]	the contacts' computation	7.77×10^{-7}	7.13×10^{-5}; 59
Genesis, MPR against GJK	settings	same	collision detection[d]	1.86×10^{-5}	0.108; 66

and drives the tool's servo as a compliant constraint without its force limit (Newton Developers, 2026c). Genesis's defaults halve the time constant of the contacts and replace the elliptic friction cone by a pyramidal one, and they also change the integrator, the iterations, the tolerance, the arithmetic and the detection of contacts. Their errors differ from those of the faithful ports by 0.423 mm and 0.968 mm at the median. These comparisons are between two carts: no tolerance on numerical error applies to them, and whether either cart is the right one is a question for the declared assumptions of Chapter 11, such as the friction and the softness that the experiment requires, not for the other simulator. Genesis's defaults, however, also bring MPR, whose failure on the crest is numerical (below), and this comparison does not separate that failure from the change of cart.

Mistakes of conversion. The importer's hull changes the physical model where the ground is curved, and only there: the errors differ from those of the full hull by up to 0.571 mm on the crest. Newton's gap adds no contact that generates force, by MuJoCo's documentation, and so changes at most the numerical model; yet the errors differ by 71.3 µm in period 59, on the crest. MuJoCo itself shows how: computed twice at the same state, without a gap and with one of 0.1 m on every geom, at every step of a motion on the firm ground and of one over the crest, the contacts that act moved by up to 4.0 µm on the firm ground and by up to 0.226 mm on the crest, where their normals turned by up to 1.5×10^{-3} in a component, although the gap added no contact and no constraint. MPR, too, changes the numerical model alone, the algorithm that detects the contacts; on the crest it fails and lets a wheel sink into the ground, and the errors differ by up to 0.108 m in the norm of the state. What fails there is the cart of the text, computed wrongly, not another cart.

Reading a comparison. Of these comparisons, only three change the physical model of this cart: XPBD, Genesis's defaults and the importer's hull. The others change the numerical model alone or, like the renderers, neither model. A change of the numerical model can still fail: MPR's 0.108 m is the largest difference of all, but it is an error in computing the same cart, which a tolerance on numerical error judges and the declared assumptions do not. The support matrix reads accordingly: its faithful ports differ from MuJoCo in the numerical model alone, XPBD and Genesis's defaults simulate other carts, and each row is a claim about an engine, a solver with its settings, and an arithmetic together.

A difference between two simulators says nothing about the controller or the cart until it is known which model the comparison changes. A change of the numerical model alone is an error in computing the same cart, judged by a tolerance; a change of the physical model gives another cart, judged by the declared assumptions.

S12.10 Numerical validation and failures in the record

The two sections before this one compared simulators with one another. This one compares each with itself. A simulated result belongs to the physical model that the record declares only if it survives the changes of the numerical model that leave the physical model as it is: a smaller time step, a tighter solver, another algorithm for the same contacts, another arithmetic. This section states seven such checks with their criteria, applies them to the faithful configurations of Section 12.1, and records what they and the failures found so far show. Numerical validation asks whether the computation solves the declared model. Whether the declared model describes the machine is another question, which no simulator answers.

S12.10.1 Checks of the numerical model

Each check changes one part of the numerical model, as Section 12.2 classified it, or compares the computation with a value that the physical model fixes exactly, and judges the values on which the claims of the record rest by a tolerance that the record states: 1 µm for positions and 1 µm/s for speeds, which are our choices, and $0.005\,\mathrm{m/s^2}$ for accelerations, a tenth of the bound of Chapter 5 and the resolution of the bisection that measures the braking the ground allows. The checks of the *time step*, of *friction* and of *limits* halve the step twice, the integrator's $ahT/2$ set apart; the check of the *constraint solver* tightens its tolerance and drops its warm start; *collision detection* is compared, contact by contact, with the exact geometry of the ground and the wheels; *estimation* compares the speed signal with the position; and *arithmetic* compares one period in single precision with what the spacing of the numbers predicts. The record's criteria are in Table S12.2. A value that a check changes by more than its tolerance is not resolved by the numerical model of the record, and the record reports it with its values under the check, not as a property of the cart. Table S12.5 gives the outcomes in the four configurations that the support matrix supports.

Table S12.5. What the checks find in the faithful configurations. Values at 1, 0.5 and 0.25 ms are given in that order; a change is the larger of the two halvings. The constraint solver of Newton on MuJoCo's C library is MuJoCo's own; the dashes mark checks not run. From `results/numerics/validation.txt`.

Check	MuJoCo 3.16.0	Newton, C library	Newton, MuJoCo Warp	Genesis 1.4.4
Time step	$ahT/2$ as predicted; resolved, except the speed within a period at the crest, which changes by up to 2.68 µm/s	as MuJoCo (2.78 µm/s), and, through rounding, period 40 and the 90th percentile, by 1.48 and 7.50 µm	as on the C library (2.94 µm/s; 1.48, 7.06 µm), and the axle at 0.25 ms: 0.0416 and 0.139 m/s^2	as MuJoCo (2.68 µm/s); MuJoCo's errors to 1×10^{-10} m at the median at every step
Constraint solver	pass: at most 3 iterations; changes up to 3.35×10^{-11} m	MuJoCo's	—	—
Collision detection	native pipeline: pass, 7.63×10^{-16} m; libccd: fail, 0.955 mm	pass: 8.05×10^{-16} m	—	against the text's geometry, GJK: pass, 1.95×10^{-7} m; MPR: fail
Friction	pass: 2.45 m/s^2 and 2.05 mm/s at each step	at 1 ms only: 2.45 m/s^2, 2.05 mm/s	the same, at 1 ms	the same, at 1 ms
Limits	stop: pass, 49.9 to 49.8 µm; last approach unresolved: 0.205, 0.645, 0.888 mm short	stop: 49.6, 49.1, 48.1 µm, unresolved by 1.01 µm; last approach as MuJoCo	as on the C library	stop: pass; last approach unresolved: 0.378, 0.780, 0.964 mm short
Estimation	pass at each step	fail: 13, 5 and 49 periods, by up to 3.81, 2.54 and 42.8 µm	fail: 13, 6 and 50 periods, by up to 3.75, 2.47 and 42.8 µm	pass at each step
Arithmetic	—	one period: 7.15 short, 40.5 long, 54.8 µm short; predicted 7.25, 40.4, 54.9 µm: pass	—	—

S12.10.2 Time steps and arithmetic

In MuJoCo, halving the time step twice changes all but two values of the comparison by less than their tolerances, once the integrator's lead $ahT/2$ (Equation (S11.2)) is set apart: after accelerating from rest, the position error less $ahT/2$ is 2.16, 2.14 and 2.14 µm at the three steps. The two values that are not resolved, the tool's last approach (Section S12.10.3) and the largest speed difference within a period at the crest, which changes by up to 2.68 µm/s, are reported with their values at the three steps. Genesis follows MuJoCo at every step, its errors differing from MuJoCo's at the same step by 8.68×10^{-11}, 8.51×10^{-11} and 8.41×10^{-11} m at the median and by more than a micrometre only in period 201, by a difference that shrinks with the step (Figure S12.4a).

Newton does not follow MuJoCo as the step shrinks: on MuJoCo's C library, its errors differ from MuJoCo's at the same step by up to 4.19, 4.99 and 43.5 µm, and on MuJoCo Warp nearly as much. By the criterion, Newton leaves unresolved two values that MuJoCo resolves, the error of period 40 and the 90th percentile of the errors, both less $ahT/2$, and at 0.25 ms MuJoCo Warp reads a disturbance of 0.139 m/s^2 from the front axle over one step, above the bound of Chapter 5: the finer step makes this value worse. The arithmetic explains why, as Figure S12.2 draws it. Each step rounds the position to the nearest number that single precision represents, and smaller steps round more often. Over one period at 1 m/s from 5 m, the cart on MuJoCo's C library ends 7.15 µm short at 1 ms, 40.5 µm long at 0.5 ms and 54.8 µm short at 0.25 ms, as the spacing of the numbers predicts; in MuJoCo and in Genesis, in double precision, it ends 0.519 µm short at every step. Halving the time step therefore measures the integrator only when the arithmetic resolves the step: in single precision, the rounding that the step adds grows as the error of the integrator shrinks, and below some step the check measures the rounding.

S12.10.3 Contacts, friction and limits

The constraint solver. MuJoCo's Newton solver converges at every one of the 36 000 steps of the comparison in at most three iterations of the hundred allowed. A tolerance ten thousand times tighter, with a thousand iterations, changes no period's error by more than 3.35×10^{-11} m, and neither does starting each step without the previous solution. The conjugate-gradient solver, stopped by the same tolerance, changes the errors by up to 2.34 µm: another method stopped at the same tolerance reaches another point near the solution, so the record names the method as well as the tolerance.

Collision detection. The exact contact of a wheel with the ground, its distance, its normal and its point, follows from plane geometry at each state, since the

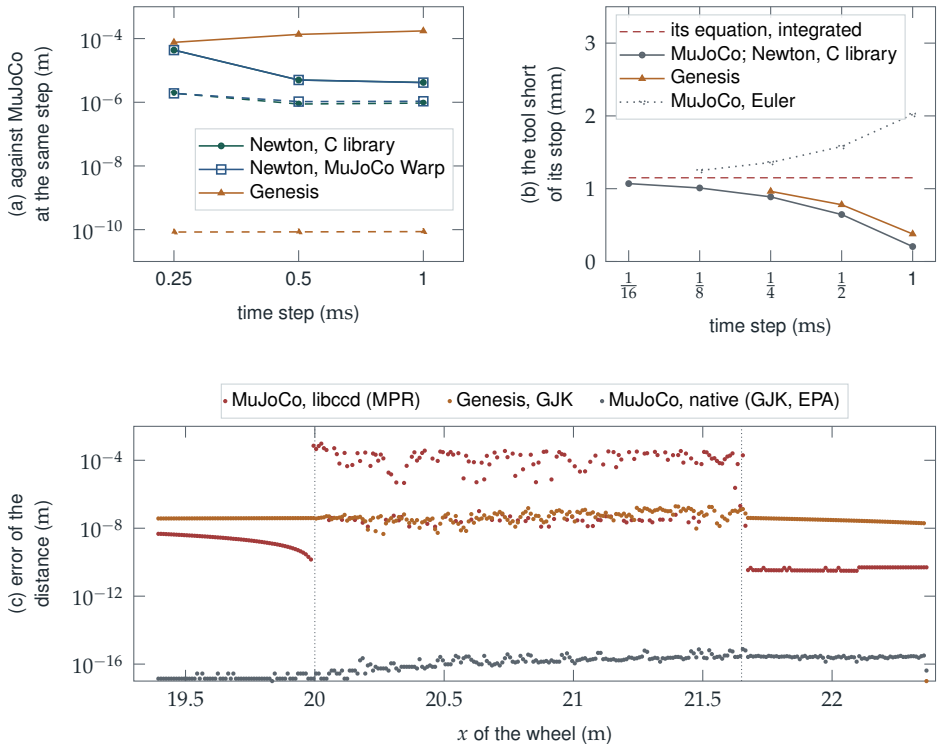


Figure S12.4. SIMULATION Checks of the numerical model. (a) The errors over each period of the faithful ports against MuJoCo's at the same time step, in the norm of the state: largest (solid) and median (dashed) over the 220 periods. Newton's two configurations nearly coincide: each open blue square surrounds a green dot. Genesis agrees with MuJoCo to 1×10^{-10} m at the median at every step, and its largest difference, in period 201, shrinks with the step; Newton's largest differences grow as the step shrinks, as the rounding of single precision would have them. (b) The tool's reach at the end of period 201, short of its stop, against the time step: no configuration resolves it at 1 ms; as the step shrinks, each approaches a shortfall of 1.13 to 1.15 mm, near the value that the tool's equation gives when it is integrated directly (dashed). (c) While the cart brakes at the label over the crest, the largest absolute difference, in each 10 mm of a wheel's path, between the signed distance of a contact of the wheel with the ground and the exact distance from the geometry of the ground and the wheels, as MuJoCo compiled them or, for Genesis, as the text gives them; the dotted lines mark the ends of the crest, and values below 10^{-17} are drawn at 10^{-17}. Data: `steps.csv`, `limits.csv` and `contacts-crest-binned.csv` in `code/ch12/results/numerics`.

ground is the convex hull of the mesh of the text. Over every step of three motions, on the firm ground, over the crest braking at the label, and on the slope, MuJoCo's native pipeline, which runs GJK and EPA, agrees with the exact geometry to 7.63×10^{-16} m in distance, and MuJoCo's C library inside

Newton to 8.05×10^{-16} m; Genesis's GJK, checked against the geometry of the text, agrees to 1.95×10^{-7} m. The check was not run on MuJoCo Warp. The pipelines based on MPR fail it. On the crest, MuJoCo's libccd pipeline, which its documentation describes as legacy (Google DeepMind, 2026b, Computation), errs by up to 0.955 mm in distance (Figure S12.4c), yet the cart passes every port check of Section 12.1.3; its loads depart from those of a rigid cart by up to 18.1 %, and the disturbance read from the front axle exceeds the bound of Chapter 5 in 13 periods over the crest. A tolerance of 10^{-10} for the pipeline, with a thousand iterations, still leaves $0.229\,\mathrm{m/s^2}$: the error is the algorithm's, not that of its convergence. The record must therefore name the collision algorithm with the solver. A check against the exact geometry, run on finitely many motions, can expose an algorithm's failures and establish its agreement in the cases tested, but not its soundness in general.

Friction and limits. In MuJoCo, the braking that the drive loop sustains on the firm ground, $2.45\,\mathrm{m/s^2}$, and the creep of the encoders, 2.05 mm/s, are the same at the three steps: values of the physical model, which the other faithful configurations give at 1 ms. So is the tool's rest against its soft stop in MuJoCo and in Genesis; in Newton it changes by 1.01 µm at the second halving, which we attribute to single precision. The tool's last approach to the stop is not resolved at 1 ms in any configuration (Figure S12.4b). As the step halves, MuJoCo's shortfall grows by 0.440 and 0.243 mm, each increment about half the one before, as at first order. Extrapolated at first order from 0.5 and 0.25 ms, the shortfall is 1.13 mm in MuJoCo and 1.15 mm in Genesis, near the 1.15 mm of the tool's equation of motion integrated directly (Section S11.11.2); the difference of about 0.015 mm has not been traced. The record reports the shortfall as unresolved at 1 ms, between 1.13 and 1.15 mm; the guarantee status does not rest on it. MuJoCo's Runge–Kutta integrator would not resolve it, because with the split step of this adapter MuJoCo integrates with the Euler method (Google DeepMind, 2026b, Simulation).

S12.10.4 Estimation

The check of estimation asks whether the speed signal is the rate of the position, by the test of Equation (S11.3), which such a signal passes, so that passing it is necessary, not sufficient. MuJoCo and Genesis pass it at each step; in Newton, up to 50 periods lie beyond what the signal allows by more than a micrometre, by up to 42.8 µm, and in Isaac Sim 41 by more than 10 µm, by up to 1.28 mm, in both through the rounding of the positions. The measured state also meets the funnels of the plans. Every faithful configuration raises the alarm at 0.1 s, because the state that it measures after the first period differs from the funnel of the initial plan, a single state, by 1.09×10^{-8} m in MuJoCo and in Genesis,

1.53×10^{-8} m in Newton on the C library and 1.16×10^{-7} m on MuJoCo Warp, where the contacts settle and the arithmetic rounds. A measurement-error bound of zero lies below these differences in every simulator tested. A run in simulation that is to keep its certificates needs bounds and funnels that cover them, as a run on hardware needs bounds that cover its sensors.

S12.10.5 Failures and the record

This supplement records the thirteen failures of the declared experiment found in Chapters 11 and 12, each with the configuration and releases in which it occurs, the smallest run that shows it, a signature and the check that catches it (Table S12.3); each reproduction outside Isaac Sim reproduced its signature exactly in a second run. Three findings matter. First, the port checks catch the gross failures, whether they change the cart or its computation, but not the mild ones: Newton's gap showed only in the comparison with MuJoCo and in one period above the bound of Chapter 5, and the libccd pipeline only in the check against the exact geometry, in the comparison and in 13 periods above that bound. A port is therefore not trusted on its port checks alone, and the record carries the checks of the numerical model too. Second, a wheel rolling freely over the crest at 3 m/s in MuJoCo is out of contact for about 39 ms at each time step; we read this as MuJoCo's law of contact rather than the computation, and it bounds the speeds at which the simulated crest can be trusted, which the certified plans stay below. Third, the failed import of Section S11.10.1 reproduces with the converter alone, on another machine and another operating system. The failures of Isaac Sim rest on fewer runs, on one machine.

The record of the experiment gains three entries, the numerical validation, the failures and the variation that a reproduction may show (Section S12.8). The run of Chapter 3 in MuJoCo was reproduced from its record in a new environment, on another processor under the same operating system: the 148 values that its check covers were as shipped, and its file of the 220 periods identical to the shipped one, byte for byte; every configuration of the support matrix, rerun on that processor, reproduced its results. These are reproductions by the team that wrote the record. A run on another operating system, and a reproduction by someone outside that team, are the tests that Chapter 17 asks for. Nothing in this section changes the guarantee status of Section S11.9.2; what it changes is what the record can be trusted for.

A simulated value belongs to the declared model only if it survives the changes of the numerical model that leave the physical model as it is, and the record states which values do. In single precision, a smaller time step can measure the rounding rather than the integrator.

CHAPTER S13

Wheeled robots and the Husky: records and details

This chapter holds what Chapter 13 moved out of its text: the provenance of the Husky's model, the detail of the checks of MuJoCo's contacts, and the records of the chapter's three experiments, with the detail of the runs that the chapter summarizes. Every number is quoted as the summaries of the chapter's three scripts print it (`code/ch13/results`).

S13.1 The task and the asset in detail

Clearpath's macros are in the package `clearpath_platform_description` of the repository `clearpath_common`, and the configuration of the robot's controllers is in `clearpath_control`, both under the BSD 3-Clause licence (Clearpath Robotics, 2026a). The folder `husky/description` holds the files that the A300 needs, copied at the commit `78e3677` of 19 October 2026, release 2.9.18, with the licence and a notice of what was copied and what was not. A file of ours assembles the robot from Clearpath's macros: the A300 with outdoor wheels and both bumpers, without the macros that start ROS controllers. Xacro 2.1.1 expanded it, without ROS, into a URDF of 501 lines, which the folder keeps.

For Gazebo alone, the description gives each wheel a wheel-slip system, which lets the friction force grow with the slip up to its limit. At each step, the system sets the force-dependent slip of the wheel's contact, a velocity per unit of force, to a slip compliance multiplied by the wheel's rim speed and divided by a nominal normal force (Open Robotics, 2026). The description sets the compliance to 0.75, laterally and longitudinally, and the normal force to 113 N. MuJoCo reads no Gazebo elements.

Each suspension beam carries its wheels on two mounts. Apart from the offset of the right beam's mounts (Section 13.1.2), the macros place the two sides symmetrically: the beams' mounts on the chassis at ± 0.192 m and their spacers at ± 15.9 mm. The published description remains available to the code, as `asset.urdf_text(published=True)`. Of the description, the model also leaves out the visual meshes, and the collision meshes of the motors and suspension beams, which have no mass in the description; it keeps

the mass that MuJoCo infers for the bumpers as a variant for the diagnostic (`bumper_mass=True`). The manual's own kinematics use 0.562 m as 'the effective track of the robot' (Clearpath Robotics, 2026b), the wheel separation of the controller's configuration. That configuration also sets two parameters, `preserve_turning_radius` and `publish_wheel_data`, that the controller of the branch for ROS 2 Jazzy does not declare, so the port in `husky/drive.py` has nothing to follow for them.

The robot is 990 mm long and 698 mm wide. In the turn, the lane's outer edge is an arc of radius 3.7 m, and the outward allowance of the robot's centre falls from 0.351 m to 0.318 m over the half length before the turn and rises again over the half length after it. The drivers' integral part stops growing at the torque limit and unwinds while the speed error alone asks for more.

Table S13.1 traces the values that the model uses to the description, the user manual and the controller's configuration. Where they differ, the table says which value the model takes.

S13.2 The contacts in detail

A start that touches. A robot placed with its wheels exactly on the ground gives MuJoCo 3.16.0 one contact per wheel, at a distance of -3.0×10^{-10} m, whose normal points from the wheel upwards, away from the ground. Such a contact can push the wheel only downwards, and gravity already does: in the first step it carries no load, and the robot falls freely by 2.45 µm before the next step finds the regular contacts. Placed 1×10^{-12} m lower, each wheel has its two regular contacts, at the edges of its cylinder, with normals pointing down into the ground. A single cylinder of the wheels' size on a box behaves in the same way, and neither a tolerance of 1×10^{-9} for MuJoCo's convex collision nor its earlier collision pipeline, through the flag `nativeccd`, reports a contact at distance zero. Our reading is that the native collision (Google DeepMind, 2026b, Computation) returns, within its default tolerance of 1×10^{-6}, a penetration whose direction it cannot determine, and that what it returns for a pair at the distance zero is not to be relied on. Here the cost is small: while the robot settles, the largest load on its wheels is 836 N, against 813 N when it is placed 0.1 mm lower, and it sinks as deep. But a start should not rest on a contact whose direction is wrong: the adapter places the robot 0.1 mm below touching, and the script reproduces the case (Table S13.2).

The default contacts in a slide. With MuJoCo's default contacts, a sideways slide from 1 m/s stops within 1.1 % of $v^2/(2\mu g)$, although all four wheels are on the ground in fewer than a quarter of its steps. We read this as a friction that, averaged over the hops, is still μ times the load. A turn is not an average of this

Table S13.1. The Husky A300 in its description, its user manual and its controller's configuration, and the value that the model of Chapter 13 takes. Description: the expanded URDF and `a300.urdf.xacro`; manual: Clearpath Robotics (2026b), with the 40 A h battery; configuration: `diff_4wd.yaml`. Model values from `results/mujoco-summary.txt`.

Quantity	Description	Manual	Configuration	Model
Mass	70 kg chassis, 4 × 2.5 kg wheels	78.5 kg; 93.5 and 105 kg with larger batteries	—	80 kg, the description's; MuJoCo would add 1.238 kg per bumper
Wheel radius	cylinder 0.1651 m	nominal tyre diameter 336 mm; effective tyre radius 162.5 mm, for odometry	0.1625 m	the cylinder; the controller keeps 0.1625 m
Wheelbase	0.512 m	512 mm	—	0.512 m
Track	joints: 0.5468 m, the centre line 9.5 mm to the left; property `wheel_track` 0.562 m, used by no joint	track width 566 mm; kinematics: effective track 0.562 m	separation 0.562 m, multiplier 1.75	0.5658 m, the right wheels' mounts mirrored; the controller keeps its separation
Clearance	—	130 mm	—	135.7 mm under the chassis' mesh at rest
Drive	four continuous wheel joints	four brushless motors, 1080 W in all, 17 A continuous each; no torque stated	50 Hz; limits of speed and acceleration	drivers' speed loops, a torque limit of 30 N m, ours (Section 13.1.3)
Speed	—	2 m/s; off road 1.5 m/s, 1 m/s with up to 50 kg	2 m/s	the route's limit 1 m/s
Sensors	none in the macros copied	a 6-axis IMU as standard	—	wheel encoders and a 6-axis IMU at the chassis' centre, ours
Suspension	rigid mounts	'does not have a suspension system'	—	none
Wheel slip	for Gazebo: lateral and longitudinal slip compliance 0.75, normal force 113 N	—	—	not read by MuJoCo; the declared contacts of Section 13.1.3

kind: in the spin, the median angle between a contact's friction and the reverse of its slip is 28.2° on the asphalt and 18.7° on the wet grass, where Coulomb friction would oppose the slip.

The condition, further. MuJoCo's documentation adds that the motion in the normal direction by which slip begins has 'little practical relevance' for soft contacts, on which an object pushed sideways is expected to 'move up a bit as it starts sliding' (Google DeepMind, 2026b, Computation), and calls the friction dimensions, regularized less than the normal by the factor `impratio`, 'harder' (Google DeepMind, 2026b, XML Reference). With the defaults and $\mu = 0.8$, the condition Equation (S13.2) with the regularizer at rest, where the default impedance has risen with the penetration to 0.913, would predict 0.0181 m/s, against the 0.0214 m/s at which the point mass first loses its contact; with the regularizer at zero penetration it predicts 0.0212 m/s. The declared parameters, whose impedance is the same at every penetration, predict 3.91 m/s, and with their friction damping raised tenfold 0.391 m/s; the point mass loses its contact at 3.91 and 0.391 m/s (Figure 13.2a). At rest, the declared contacts sink by the residual $R_n F$ divided by the stiffness of MuJoCo's reference, $d/(d_w \tau)^2$ (Google DeepMind, 2026b, Modeling), which comes to 0.226 mm, as the simulator shows.

Lazy friction. A contact that nothing drives decelerates its slip at about the rate that the friction's reference asks, bv, as far as friction can give that much. Below a slip speed of about $\mu g/b$, a slide from v_0 therefore ends after about v_0/b instead of $v_0^2/(2\mu g)$. These are approximations, which take the slip's acceleration for the reference itself. MuJoCo's soft constraints realize only a share of the reference: where no other force acts on the slip, $A(A+R)^{-1}$ of it, for the inverse inertia A of the constraints and their regularizer R, which tends to the whole reference only as R tends to zero (Google DeepMind, 2026b, Computation). The regularizer of the friction dimensions, R_n/I, is not zero, and the robot's free slip decays below $\mu g/b$ at a measured rate a few per cent below b: 58.6/s, or $0.976\,b$, on each ground. A slide decelerated by μg down to $\mu g/b$ and by bv below would be 1.7, 1.0 and 0.3 % longer than $v^2/(2\mu g)$ on the asphalt, the gravel and the wet grass; with the measured decay, 1.8, 1.0 and 0.3 %. The simulated slides are 1.50, 0.76 and 0.18 % longer; we have not traced the remainder.

The Coulomb references. The reference of `husky/coulomb.py` is quasi-static. It leaves out the robot's inertia, so that the friction forces and their moment sum to zero, and it takes the loads of the robot at rest. That is exact for a spin in place about the centre of mass, which does not accelerate. With point contacts at the wheels' centres, half a track B and half a wheelbase L apart, a spin with the rims at $\pm v_\mathrm{r}$ turns at $\Omega = Bv_\mathrm{r}/(L^2+B^2)$, whatever μ, as long as the loads balance

about the centre, as the model's do. The model's contacts lie at the edges of the cylinders, half their width, 57.15 mm, on either side of each wheel's centre, with 98.1 N each at rest. The turn of radius 3 m agrees with the quasi-static reference for its rims to 0.17 % on the asphalt and the gravel and to 1.51 % on the wet grass, and the difference is the reference's. The turn's centripetal acceleration, $0.323\,\mathrm{m/s^2}$ on the asphalt and the gravel and $0.328\,\mathrm{m/s^2}$ on the wet grass, is 4.1, 5.5 and 9.5 % of μg, and it shifts load from the inner wheels to the outer: on the asphalt, 381.5 N on the inner side and 403.3 N on the outer, against 392.4 N on each at rest. With the centripetal force and the loads at rest, the reference turns 0.50 to 0.52 % more tightly than the simulated robot; with the centripetal force and the loads measured in the turn, the simulated turns agree with it to 0.01, 0.01 and 0.02 %. The creep under a steady sideways push is the gradual slip that MuJoCo's documentation describes as unavoidable in its soft model (Google DeepMind, 2026b, Overview).

Each parameter does its part. With the impedance at the default 0.9, the spin at 0.98 m/s loses 74.5 % of its rate, with all wheels down in 29 % of the steps. With `impratio` 1, the spin is right, but the slide is 124 % too long and the creep 263 mm/s. With $b = 6/\mathrm{s}$, the slide is 180 % too long. That slide stays entirely below $\mu g/b = 1.31\,\mathrm{m/s}$: its slip decays at $5.605/\mathrm{s}$, $0.934\,b$, and it ends after 0.1782 m. The approximation v_0/b, 0.1667 m, falls short of it by 6.5 %, and v_0 over the measured rate, 0.1784 m, meets it to 0.1 %.

Time steps. Halving the time step from 1 ms changes the stopping distances on the three grounds by 7.5, 5.0 and 1.2 mm; halving it again changes them by 2.3, 1.7 and 0.8 mm. The spins at 0.98 m/s and the turns are the same at the three time steps on each ground, to the digits of Table S13.2. We have not established why the stop is the sensitive case; the slip of the wheels is unlikely to be the reason, since the stop changes least on the wet grass, where the wheels slip most.

S13.3 The record of the Husky in MuJoCo

Table S13.3 is the record of the experiment of Section 13.1, with the items that the book asks of every supported experiment. The drivers' speed loops, which the record lists, keep up with the controller's ramp: while it ramps the speed to 1 m/s at $2\,\mathrm{m/s^2}$, no wheel's speed is more than 0.303 rad/s from its target, 5 cm/s at the rim.

Table S13.2. The contacts of the wheels with the ground, with MuJoCo's default parameters and with the declared ones, against Coulomb references. The spin and the turn are compared with the steady motion that Coulomb friction gives for the model's own contacts and the rims' measured speeds (`husky/coulomb.py`), quasi-static, with the loads at rest and without inertia; the declared turn also with its centripetal force and the loads measured in it. The slide is compared with $v^2/(2\mu g)$. Spins with the rims at 0.49 m/s unless stated; 'wheels down': the share of the steps in which all four wheels touch the ground; the creep 'predicted' by MuJoCo's soft friction, $R_n f_t/(Ib)$. From `results/mujoco-summary.txt`.

Check	Defaults: asphalt; wet grass	Declared: asphalt; gravel; wet grass	Declared, one parameter changed, asphalt
Slip speed v^* of the condition	8.05 mm/s; 12.6 mm/s	1.48; 1.97; 3.37 m/s	—
Spin: yaw rate against Coulomb; wheels down	asphalt −62 % (rims at 0.24 m/s), 33 %; wet grass −31 %, 53 %	−0.05, −0.06, −0.08 %; at 0.98 m/s −0.01, −0.03, −0.06 %; 100 %	impedance 0.9: −74.5 % at 0.98 m/s, 29 %; impratio 1: +0.1 %; $b = 6$/s: −0.1 %
Spin: median angle of friction from the reverse of the slip	28.2°; 18.7°	0.0° on each ground	—
Turn at 1 m/s, $\frac{1}{3}$ rad/s: radius against Coulomb	10.7 m against 3.56 m; 5.94 m against 3.21 m	+0.12; −0.17; −1.51 %; with the centripetal force and the loads of the turn −0.01; −0.01; −0.02 %; wheels down 100 %	—
Sideways slide from 1 m/s against $v^2/(2\mu g)$	0.0644 m against 0.0637 m, +1.06 %; 0.1453 m against 0.1456 m, −0.21 %; wheels down 23 and 17 %	+1.50; +0.76; +0.18 %	impedance 0.9: +1.2 %; impratio 1: +124.4 %; $b = 6$/s: +179.8 %
Creep under a sideways push of 30 % of μmg	—	2.81; 2.11; 1.23 mm/s; predicted 2.83; 2.12; 1.24 mm/s	impedance 0.9: 0.02; impratio 1: 263.33; $b = 6$/s: 25.75 mm/s
Penetration at rest	—	0.226 mm	—
Start exactly touching	one contact per wheel at -3.0×10^{-10} m, normal from the wheel upwards; 1×10^{-12} m lower, two per wheel, normals downwards. A cylinder on a box: the same; none with a tolerance of 1×10^{-9} or without `nativeccd`. Touching, no load in the first step; while settling, at most 836 N, against 813 N 0.1 mm lower		

Table S13.3. The record of the experiment of Section 13.1, after `results/record.txt`, which `husky_mujoco.py` writes.

Item	Record
Model and asset provenance	Clearpath's description, `clearpath_common` at commit `78e3677`, release 2.9.18, BSD 3-Clause; expanded by xacro 2.1.1; the right beam's wheel mounts mirrored (track 0.5658 m, published 0.5468 m); visual elements and the meshes of massless links removed; SHA-256 of the URDF text loaded beginning `d7b2542c25c84fed`; free joint, bumpers without mass, actuators, sensors, contact pairs and yard added by `husky/model.py`
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3 and CPython 3.13.17 (on the first machine of the item Hardware: NumPy 2.5.4 and CPython 3.13.16); Newton solver, at most 100 iterations, tolerance 10^{-8}; elliptic friction cones; impratio 100; wheel–ground pairs `solref` 0.008 1, `solimp` 0.05 0.05 0.001 0.5 2, `solreffriction` -0 -3 ($b = 60/\mathrm{s}$); other contacts MuJoCo's defaults; differ from MuJoCo's defaults: time step, integrator, friction cone, impratio
Time steps	0.5 ms, integrator `implicitfast`; drive controller at 50 Hz, every 40 steps; drivers' loops at every step; the stops, spins and turns repeated at 1 and 0.25 ms
Actuators and sensors	`diff_drive_controller` (ros2_controllers, jazzy, `d71703b`) ported in `husky/drive.py`, configured by `diff_4wd.yaml`: 50 Hz, separation 0.562 m $\times$ 1.75, radius 0.1625 m; drivers, ours: gain 40 N m s/rad, integral time 0.1 s, limit 30 N m; four wheel encoders and a 6-axis IMU
Frames and units	SI units; world x east, y north, z up, the ground at $z = 0$; `base_link` x forward, y left; route coordinates s and d, positive to the left
Uncertainty bounds	frictions of the grounds, the simulator's and not measured: asphalt 0.8, gravel 0.6, wet grass 0.35; no labels yet
Constraints	lane 1.4 m wide on the route, to hold the robot's footprint of 0.990 m $\times$ 0.698 m; pallets 0.1 m outside it; the dock's face at $s = 25$ m; commanded speed at most 1 m/s; controller limits 2 m/s, 2 m/s^2, 4 m/s^2
Terminal behaviour	a stop commanded through the controller, at its deceleration limit
Training and evaluation split	none: nothing learns in this section
Seeds	none: no random numbers
Hardware	one x86-64 Linux machine; the same results, digit for digit, with CPython 3.11.15 (NumPy 2.4.6) and 3.14.6 (NumPy 2.5.4); and again on a second x86-64 Linux machine, with MuJoCo 3.16.0 built from its source at the tag 3.16.0 by clang 18.1.3, which wrote the shipped record; a third, a 2.10 GHz Xeon with the same build and versions, reproduced every summary and data file of the chapter
Guarantee status	none: this section states the task and measures the simulated robot; no plan is certified

Listing S13.1. The contact pair of a wheel with the ground (`husky/model.py`). The constants are the model's: `TIMECONST` 8 ms, `IMPEDANCE` 0.05 and `FRICTION_DAMPING` 60/s; MuJoCo divides the damping of the direct format by d_w, the second element of `solimp`, here equal to the impedance. The code's one comment, which says so, is left out, and two of its lines are broken differently.

```
def pair(spec, geom1, geom2, friction, contacts):
    """An explicit contact pair: the declared parameters of this
    module, or MuJoCo's defaults."""
    p = spec.add_pair(name=f"{geom1} | {geom2}", geomname1=geom1,
                      geomname2=geom2, condim=3)
    p.friction = [friction, friction, 0.005, 0.0001, 0.0001]
    if contacts == "declared":
        p.solref = [TIMECONST, 1.0]
        p.solimp = [IMPEDANCE, IMPEDANCE, 0.001, 0.5, 2.0]
        p.solreffriction = [-0.0, -FRICTION_DAMPING * IMPEDANCE]
    else:
        p.solref = list(DEFAULT_SOLREF)
        p.solimp = list(DEFAULT_SOLIMP)
    return p
```

S13.4 The Husky in Isaac Sim

The script `isaac_husky.py` of `code/ch13` builds the robot of Section 13.1 in Isaac Sim 6.2.0 and runs the diagnostics of that section that apply to PhysX. It imports the same URDF with Isaac Sim's URDF importer, fixed joints merged and the base free, and adds, for every run, a physics scene with the settings of Chapter 11: CPU dynamics, patch friction, no continuous collision detection and no stabilization; an articulation without sleeping, with 32 position iterations; the ground, the frictions of the three grounds, the drive controller of `husky/drive.py` at 50 Hz, and the drivers of Section 13.1.3 as PhysX's angular velocity drives with the integral part added at every step. The wheels' masses and inertias are the description's, as PhysX reports them. Version 5 of the script ran on 28 October 2026, for 3996 s, with no step failed (Table S13.8); its results are in `code/ch13/results/isaac`. Earlier versions were run before it, and the script's documentation describes their runs. Those of versions 1 and 2 were not valid: their wheels were cylinders on a ground that was a box, where PhysX gave each wheel a third point of contact beside the ends of its line of contact (about three per wheel and step in version 5's screen, against two on a plane), and the robot hopped. Version 3 made the ground a plane and found the wide turns; version 4 read PhysX's friction at its anchors and found its direction; version 5 tests the explanation. Only version 5's results are shipped, with the assets that the imports of earlier runs left in its folder `usd`.

What was run, and what counts as valid. The run first screens 19 setups on asphalt: the default (cylindrical wheels on a plane, the solver TGS, a time step of 0.5 ms, one velocity iteration, a contact offset of 2 cm) and 18 that change one or two things from it. It then runs three of them in full on the three grounds: cylinders under TGS, the default; cylinders under PGS; and spheres under TGS. A diagnostic is valid if the robot's base neither hops nor tilts: in motion it stays within 2 mm of its height at rest and within 1° of level, and at rest, after settling, within 1 mm of the height of round wheels touching the ground, with a spread of its height below 0.5 mm and a tilt below 0.2°. Within looser limits, such as 10 mm and 3° in motion, it is marginal, and beyond them invalid. Seventeen of the 19 setups were valid. The two invalid ones, with cylinders on the box as in versions 1 and 2, did not settle in 3 s, and in their stops the base rose by up to 85 mm and tilted by up to 17°. Every diagnostic of the full runs was valid. PhysX's friction was read at its anchors through the tensor interface; summed over the wheels in a sideways slide, its impulse matched the change of the robot's momentum with a factor of 0.999 to 1.004 on the plane, so the data are the forces on the wheels.

Straight motion. Table S13.4 sets the straight motions of the full runs beside MuJoCo's. The stops from 1 m/s end within 6.1 mm of MuJoCo's, and under a sideways push of 30 % of μmg the robot does not creep. A sideways slide from 1 m/s ends 0.4 to 7.7 % short of $v^2/(2\mu g)$, where MuJoCo's ends up to 1.5 % beyond it, so that the slides in Isaac Sim are up to 9.1 % shorter than MuJoCo's. In a forward slide with the drivers commanded to hold the wheels, a diagnostic that MuJoCo's run does not have, the wheels still turned, their rims reaching 0.70 to 0.95 m/s on asphalt and gravel, and the robot slid 47 to 50 % further than $v^2/(2\mu g)$ on asphalt and 12 to 13 % further on gravel; on wet grass, where the rims reached at most 0.45 m/s, the slide ended 1 to 6 % short. Driven at 1 m/s for 10 s, the robot covered 10.1487, 10.1614 and 10.1484 m in the three setups, its odometry counting 1.46, 1.58 and 1.46 % less, close to MuJoCo's 1.51 %. In both, most of the difference is the ratio of the controller's wheel radius, 0.1625 m, to the simulated wheels' (Section 13.1.4); in PhysX, whose wheels do not sink, that ratio alone gives 1.57 %. At rest, the loads averaged over 0.5 s carry the weight, 784.8 N, and their moments match those of the weight. Single steps do not: under TGS the total load of a step swings between about 400 and 1100 N, successive steps alternate, with a correlation of -0.5 to -0.6, and in 40 to 60 % of the steps some wheel carries no load. Under PGS, at rest, every wheel carries load in every step. PhysX's contact forces in one step are therefore no measurement of the load.

Turning. Table S13.5 gives the turns and spins. The Coulomb references are those of `husky/coulomb.py` for the contacts at rest and the rims measured in

Table S13.4. Straight motion of the Husky in the full runs in Isaac Sim, beside MuJoCo's with the declared contacts: the stop from the command at 1 m/s, the sideways slide from 1 m/s and its difference from $v^2/(2\mu g)$, and the creep under a steady sideways push of 30 % of μmg. Data: `code/ch13/results/isaac/isaac-summary.txt` and `code/ch13/results/mujoco-summary.txt`.

Ground	Simulator and setup	Stop (m)	Slide (m)	Against $v^2/(2\mu g)$	Creep (mm/s)
Asphalt	MuJoCo	0.1240	0.0647	+1.50 %	2.81
	Isaac Sim, cylinders, TGS	0.1297	0.0588	−7.66 %	0.00
	Isaac Sim, spheres, TGS	0.1286	0.0593	−6.91 %	0.00
	Isaac Sim, cylinders, PGS	0.1239	0.0610	−4.27 %	0.00
Gravel	MuJoCo	0.1282	0.0856	+0.76 %	2.11
	Isaac Sim, cylinders, TGS	0.1323	0.0796	−6.28 %	0.00
	Isaac Sim, spheres, TGS	0.1308	0.0794	−6.53 %	0.00
	Isaac Sim, cylinders, PGS	0.1257	0.0833	−1.96 %	0.00
Wet grass	MuJoCo	0.1552	0.1459	+0.18 %	1.23
	Isaac Sim, cylinders, TGS	0.1510	0.1381	−5.17 %	0.00
	Isaac Sim, spheres, TGS	0.1491	0.1375	−5.60 %	0.00
	Isaac Sim, cylinders, PGS	0.1515	0.1451	−0.38 %	0.00

each run, as in Section 13.1.3. Under TGS, the turn commanded at 1 m/s and 1/3 rad/s, a radius of 3 m, comes out 8.6 to 10.0 times its Coulomb reference, with the rims at their targets; under PGS, the robot goes almost straight, its turns over 5 km, and on asphalt its right rims fall short of their target. In the spins with the rims at 0.98 m/s, the yaw rate comes within 4.4 % of Coulomb's under TGS. At lower rim speeds, cylinders under TGS sometimes spun far too slowly: with the rims at 0.49 m/s on asphalt, and in the spin commanded at 1 rad/s, whose rims turn at about 0.5 m/s, on asphalt and gravel, they turned at 0.16 to 0.17 rad/s, 80 to 82 % short of Coulomb's rate. Their other spins at that speed, and those of spheres on every ground, came within 8 % of it.

The two-row law. In the PhysX SDK, the solver resolves the friction of a patch of contact along two directions (NVIDIA, 2026e): t_0, along the relative velocity of the two bodies' centres, projected on the plane of contact, which for a wheel on the ground is the velocity of the wheel's centre and not that of the point of contact; and t_1, across it. Each direction is a row of the solver with its own velocity multiplier, a coefficient common to both divided by the contact's response along that direction, the inverse of its effective mass, with the articulation link's constraint-force mixing added. Under TGS, the two impulses of an anchor are cut together to a circle of radius μN, shared by the patch's anchors; under PGS, each is clamped on its own. Where an anchor

Table S13.5. Turning of the Husky in the full runs in Isaac Sim, beside MuJoCo's: the radius of the turn commanded at 1 m/s and 1/3 rad/s (a radius of 3 m), its Coulomb reference for the rims measured, the steady turn of the two-row law for the same contacts and rims, the median angle of PhysX's friction in the turn from the opposite of the slip and from the two-row law's, and the yaw rate of the spin commanded at 1 rad/s. PhysX: Isaac Sim with PhysX, its wheels' colliders and solver as named. The two-row law applies under TGS only. On asphalt under PGS, the right rims turned at 1.01 m/s instead of 1.18 m/s, which makes the Coulomb reference 9.7 m. Data: `code/ch13/results/isaac/isaac-summary.txt` and `code/ch13/results/mujoco-summary.txt`.

Ground	Simulator and setup	Turn (m)	Coulomb (m)	Two-row law (m)	Friction off the slip; off the law	Spin (rad/s)
Asphalt	MuJoCo	3.218	3.214	—	—	0.948
	PhysX, cylinders, TGS	31.229	3.261	26.411	42.7°; 6.5°	0.174
	PhysX, spheres, TGS	31.856	3.179	25.236	41.8°; 5.9°	0.886
	PhysX, cylinders, PGS	5654	9.718	—	44.9°; —	0.660
Gravel	MuJoCo	3.209	3.214	—	—	0.948
	PhysX, cylinders, TGS	29.073	3.182	25.887	43.1°; 5.5°	0.169
	PhysX, spheres, TGS	29.611	3.140	24.924	42.3°; 6.1°	0.916
	PhysX, cylinders, PGS	5018	3.275	—	45.0°; —	0.684
Wet grass	MuJoCo	3.166	3.214	—	—	0.948
	PhysX, cylinders, TGS	27.243	3.182	25.830	43.4°; 5.8°	0.917
	PhysX, spheres, TGS	27.383	3.139	24.915	43.0°; 5.2°	0.939
	PhysX, cylinders, PGS	5100	3.197	—	45.0°; —	0.879

slides steadily under TGS, its friction is therefore anti-parallel to Ds, where s is the slip and $D = \mathrm{diag}\big(1/(W_{00}+c), 1/(W_{11}+c)\big)$ in the frame (t_0, t_1), for the contact's inverse effective masses W_{00} and W_{11} along the two rows and the link's constraint-force mixing c, 0.025 by default; Coulomb's friction is anti-parallel to s. The script computes the effective masses at the foot of a wheel from the description's masses and inertias, with the base free and the wheels free to spin: 1.21 kg along the wheel and 20.02 kg across it, a ratio of 16.5; with c added, 1.17 kg and 13.3 kg, a ratio of 11.4. The script's two-row law takes $c = 0$, so that a sideways slip weighs 16.5 times as much as a forward one. With nothing fitted, this law predicts the direction of PhysX's friction in the turns of the full runs to within 5.2° to 6.5° (median), where the opposite of the slip is 41.8° to 43.4° off, and turns of 24.9 to 26.4 m, against PhysX's 27.2 to 31.9 m (Table S13.5). In the three spins of cylinders that came out far too slow, the law has two steady motions, one near Coulomb's rate and one at 0.19 to 0.21 rad/s, and PhysX settled on the slow one: its friction stood 0.3° to 0.5° from the law's and 41° to

Table S13.6. A test of the two-row law: the turn of the screen on asphalt, cylinders under TGS, with the wheels' inertia about their axles multiplied, the effective masses at a wheel's foot along and across the wheel, the turn of PhysX, the law's for the rims measured, the prediction stated before the run, the Coulomb reference, and the median angle of PhysX's friction from the opposite of the slip and from the law's. Data: `code/ch13/results/isaac/isaac-summary.txt`, group [screen on asphalt, mu 0.8].

Inertia	Effective masses (kg)	PhysX (m)	Law (m)	Predicted (m)	Coulomb (m)	Friction off the slip; off the law
×1	1.21, 20.02	31.519	26.350	—	3.253	42.6°; 4.6°
×2	2.35, 20.02	15.529	14.194	about 14	3.198	40.1°; 3.3°
×8	7.97, 20.02	5.767	5.351	about 5.4	3.181	23.0°; 2.1°
×32	19.79, 20.02	3.625	3.193	about 3.2	3.178	2.3°; 1.4°

42° from the opposite of the slip. With c included, as the source has it, the law fits less well, at 11.7° to 13.1° in the turns and 2.6° to 2.7° in those spins, and its turns, 17.6 to 18.6 m, lie further from PhysX's; why is not settled here.

A test of the law. Raising the wheels' inertia about their axles raises the effective mass along the wheel and leaves it across the wheel, and the law then predicts narrower turns, reaching Coulomb's where the two masses are equal. The script's documentation, saved before the run according to the files' timestamps, predicted turns of about 14, 5.4 and 3.2 m at 2, 8 and 32 times the description's inertia. These setups test the law; they are not models of the robot. Table S13.6 gives the result: the turn narrowed as predicted, and the friction moved towards the opposite of the slip, to 2.3° at 32 times. PhysX's turns remain 8 to 14 % wider than the law's, as in the full runs, where they are 5 to 26 % wider.

The screen. Table S13.7 lists the 19 setups. With round wheels of the description's inertia under TGS, every valid setup turned 7.6 to 10.3 times its Coulomb reference, whatever the time step, from 0.25 to 1.67 ms, the velocity iterations, a friction correlation distance of 0.01 mm, which drops friction anchors sooner as a wheel rolls, or compliant contacts tuned to MuJoCo's sinkage at rest.

Others have met wide turns of this robot in Isaac Sim. In a thread of the NVIDIA developer forum, a user reports a Husky turning on a circle far wider than commanded; a reply reports 'a consistent behavior' after enabling continuous collision detection, which it names as the critical change, with patch friction, 600 steps per second and a physics material on the wheels, and the user replies that it works (NVIDIA Developer Forums, 2023b). In another thread, users report turns wider and yaw rates lower than expected for the Husky and

Table S13.7. The 19 setups screened on asphalt, each changing what its name says from the default (cylindrical wheels on a plane, TGS, a time step of 0.5 ms, one velocity iteration): the verdict, the turn commanded at a radius of 3 m against its Coulomb reference for the rims measured, and the spin with the rims at 0.98 m/s against Coulomb's rate. Data: `code/ch13/results/isaac/isaac-summary.txt`, groups [screen on asphalt, mu 0.8] and [validity].

Setup	Verdict	Turn (m)	Coulomb (m)	Spin against Coulomb
Cylinders on a box, as in version 2	invalid	—	—	—
Cylinders on a plane, the default	valid	31.519	3.253	−3.86 %
Spheres on a plane	valid	33.240	3.248	−4.38 %
Convex wheels on a plane	valid	787.145	18.329	−91.21 %
Spheres on a box	valid	31.393	3.251	−5.02 %
Convex wheels on a box	valid	56.305	28.421	−95.06 %
Cylinders on a box, contact offset 2 mm	invalid	—	—	—
Cylinders, PGS	valid	5755	16.266	−18.79 %
Spheres, PGS	valid	534.011	19.654	−7.16 %
Four velocity iterations	valid	31.902	3.300	−3.72 %
Time step 0.25 ms	valid	47.104	4.559	−7.69 %
Time step 1 ms	valid	32.885	3.228	−3.84 %
Time step 1.67 ms	valid	32.582	3.195	−3.51 %
Time step 1.67 ms, PGS	valid	53 307	3.206	−22.50 %
Friction correlation distance 0.01 mm	valid	32.446	3.328	−3.79 %
Compliant contacts	valid	24.546	3.215	−0.05 %
Wheels' inertia ×2	valid	15.529	3.198	−1.72 %
Wheels' inertia ×8	valid	5.767	3.181	−0.03 %
Wheels' inertia ×32	valid	3.625	3.178	+0.79 %

Jackal, and a reply recommends PGS (NVIDIA Developer Forums, 2023a). This run had patch friction and physics materials on the wheels, and screened 600 steps per second (1.67 ms), but kept continuous collision detection off, as in Chapter 11. Whether it changes the turn here was not tested, and the other differences from those users' models were not examined. Here, PGS made the robot go almost straight.

Convex wheels, faceted, slowed the spin by 91 to 95 %, and in the turn their rims fell short of their targets. In the repeat of the default at 1, 0.5 and 0.25 ms, the step of 0.25 ms widened the turn on asphalt, to 45.4 m (47.1 m in the screen), and left the stops on asphalt and gravel creeping at 4.4 and 4.5 mm/s after 5 s; on gravel and wet grass it narrowed the turn slightly, to 27.4 and 25.7 m.

What the run shows. In this configuration, under TGS, PhysX's friction is not Coulomb's for a sliding contact whose effective masses differ by direction, as

at the foot of a wheel that spins freely: by the two-row law, the two agree only where the slip lies along one of the law's two directions. The runs bear this out. The stops and the odometry agree with MuJoCo's, the sideways slides come within 8 % of $v^2/(2\mu g)$, and the spins with the rims at 0.98 m/s come within 4.4 % of Coulomb's rate, with the friction within about 2° of the opposite of the slip; a skid-steered turn, in which every wheel slides both along and across, comes out about nine times as wide. (The forward slide with the wheels held is no test of the friction law, since the wheels turned although the drivers were commanded to hold them; MuJoCo's run has no such slide.) Under PGS, which clamps each row on its own, the robot goes almost straight. The departure comes from the friction model, not from the Husky; by the law, a skid-steered robot whose wheels spin as freely would turn too wide in the same way, though only this robot was run. What a reader may conclude is narrower than that PhysX is wrong: no real ground obeys Coulomb's law, and this run measured no tyre. It is that the two simulators, configured to model the same robot, disagree by an order of magnitude or more on how it turns, and that only MuJoCo's agrees with the law that its parameters stand for. A turn planned in one of them would not be executed alike in the other. The governed runs of Chapter 13 use MuJoCo, and this run validates none of their turns.

Table S13.8. The record of the run of the script `isaac_husky.py` in Isaac Sim, after its files `record.txt` and `isaac-summary.txt`, group [record], in `code/ch13/results/isaac`.

Item	Record
Model and asset provenance	the URDF of `husky/asset.py`, the same as in MuJoCo (Table S13.3), imported by `isaacsim.asset.importer.urdf` 3.11.12, fixed joints merged, base free; wheels' colliders cylinders as imported, or spheres or convex hulls by setup; masses and inertias as the description gives them, checked through PhysX's tensor interface
Simulator and solver	Isaac Sim 6.2.0, its version file reading 6.2.0-rc.14 as in the release's source (NVIDIA, 2026d); Kit 110.4.0, `omni.physx` 110.4.4; TGS unless a setup says PGS, patch friction, CPU dynamics, MBP broadphase, no CCD, no stabilization; 32 position iterations and the setup's velocity iterations; contact offset 2 cm, rest offset 0; the grounds' frictions static and dynamic, restitution 0
Time steps	0.5 ms unless a setup says otherwise; stops, spins and turns of the default repeated at 1 and 0.25 ms; the drive controller at 50 Hz
Actuators and sensors	the drive controller of `husky/drive.py`; the drivers as PhysX's angular velocity drives, gain 40 N m s/rad, the integral part (0.1 s) added at every step, limit 30 N m; PhysX's friction at its anchors and the joints' torques read through `omni.physics.tensors` 110.4.4
Frames and units	SI units; world z up; base x forward, y left
Uncertainty bounds	frictions of the grounds, the simulator's: asphalt 0.8, gravel 0.6, wet grass 0.35; validity by the base's envelope, as stated above
Constraints, terminal behaviour, split, seeds	as in Table S13.3; no random numbers
Hardware and software	a Windows 11 laptop with an NVIDIA GeForce RTX 4060 Laptop GPU (driver 596.08); CPython 3.12.15, NumPy 2.3.1; script version 5, without options; 3996 s; no step failed
Guarantee status	none: diagnostics only; no plan is certified in Isaac Sim

S13.5 The governed Husky in detail

Table S13.9 gives the crossings of the yard with the five estimators of Section 13.2.1, and Listing S13.2 the recovery continuation and the test of the control package of Section 13.2.4.

The estimators. Dead reckoning estimates the gyro's bias at 0.004 10 rad/s, 9.5×10^{-5} rad/s too large. In the turn of radius 3 m to the right, the robot slides as fast to the left as it slides to the right in the turn to the left, and the centre of rotation lies as far ahead. The estimator with sideslip reads the ground from the yard's map at the estimated position. In the crossing with the map's labels, the robot covers 8.4 mm more than its encoders count at the rolling radius while

it accelerates from rest. In the turn, two effects meet. Counted at the rolling radius, the wheels' turns fall short of the robot's motion, by 0.34 % in the turn of radius 3 m on gravel, which alone would make both estimators count less than the robot covers; and dead reckoning counts the route's arc at the centre line's radius. Over the turn, dead reckoning advances 49.0 mm beyond the robot, and dead reckoning with sideslip 13.3 mm less than it. Across the route, dead reckoning ends the turn with its estimate 83.1 mm inside the robot, and dead reckoning with sideslip 20.8 mm.

The estimators that fail. With the A300's odometry, the robot heads to the right of the second straight and at 19.26 s touches the pallets beside it, at $s = 18.51$ m; at the end its estimate reads 24.3061 m, with the robot at 18.6071 m. With the distance counted at the controller's radius, 1.51 % short of the rolling radius, the estimate falls behind the robot along each straight, by up to 0.206 m. With the gyro's bias unestimated, the heading drifts at 0.004 rad/s, and the robot touches the same pallets at 19.70 s. The one alarm comes at 19.8 s, after the contact, when the estimated heading error lies 0.0699 rad beyond the initial set, 0.1399 m in the norm. The robot stops, restarts once, and comes to rest 1.134 m short of the dock, with its estimate 0.94 m ahead of it.

Seeds. Over the gyro's seeds 1 to 20, dead reckoning exceeds the bound along the route with errors of up to 78.7 mm, and across it with errors of up to 144.1 mm. The seed with the largest error across for dead reckoning with sideslip, 82.3 mm, is seed 5, whose calibration left the largest error of the gyro's bias, -2.24×10^{-4} rad/s; seed 13, with 9.5×10^{-5} rad/s, errs across by 21.6 mm.

The crossing with the map. The robot's front reaches the dock zone at 24.80 s. Before it settles, the robot overshoots its final position by 0.9 mm. The commanded motion comes to rest at the end of period 126, at 25.4 s. At time steps of 1, 0.5 and 0.25 ms, the crossing comes to rest in the dock zone from 25.52, 25.58 and 25.60 s, under plans approved by the box test at 0.10 m, with true clearances at the end of 0.2020, 0.1941 and 0.1900 m, and estimates that err by at most 28.6, 23.3 and 21.6 mm along the route. At each time step, the estimate keeps its bounds, the feedback keeps its own, and the state stays in its tracking box. In the shower with the map's labels, the robot comes to rest 87.5 mm beyond the end of the dry crossing, 0.12 s later.

S13.6 The record of the governed Husky

Table S13.10 is the record of the experiment of Section 13.2, which states its evidence status in full.

Listing S13.2. The recovery continuation, the test of a pair, and the method `profile` of the class `RouteSpeed`, which is the control package `husky.route-speed` (`husky/governed.py`); the docstrings are left out. A pair is a segment, a commanded `Motion` of one period, with its label; `plan.reserve()` is the plan's reserve at the dock (Section 13.2.4), and `level_of` rounds it down to whole centimetres from the floor of 0.1 m.

```
def recovery(labelling, j, s, c):
    pairs = []
    while c > 0.0:
        coast = Motion(s, c, c, T)
        b = min(B_MAX, labelling(j, coast))
        if b <= 0.0:
            raise ValueError("a label without braking")
        m = Motion(s, c, max(0.0, c - b * T), T)
        pairs.append((m, labelling(j, m)))
        s, c = m.end()
        j += 1
    return pairs, s


def pair_failure(m, label):
    top = V_MAX + 1e-12
    if not (0.0 <= m.c0 <= top and 0.0 <= m.c1 <= top):
        return "speed beyond the route's limit"
    rate = (m.c1 - m.c0) / m.T
    if rate > A_MAX + 1e-12:
        return f"acceleration {rate:.3g} m/s^2 beyond {A_MAX:g}"
    if -rate > min(label, B_MAX) + 1e-12:
        return (f"deceleration {-rate:.3g} m/s^2 beyond the "
                f"label {label:.3g}")
    return None


    def profile(self, plan, initial):
        if not _starts_at(plan, initial):
            return Profile.uncertified()
        level = level_of(plan.reserve())
        entries, ok = [], True
        for m, label in reversed(plan.pairs):
            ok = ok and pair_failure(m, label) is None
            entries.append(level if ok else NEG_INF)
        entries.reverse()
        return Profile(entries + [level])
```

Table S13.9. The five estimators across the yard, under the governor with the map's labels and the proposal `husky.cruise`, each with the gyro's seed 13. Errors: the largest distance of the true arc length and offset from their estimates; feedback: the largest distance of the estimated arc length from the commanded one and the largest estimated offset; footprint: the largest offset of a point of the robot's footprint from the centre line, against the lane's half width of 0.7 m; end of the crossing: where the robot ends, whether its state stayed in its tracking box, and the alarms. From `results/route-summary.txt`.

Estimator	Error along; across (mm)	Feedback: along; across (mm)	Footprint (m)	End of the crossing
The A300's odometry	5699; 731	17.6; 7.4	1.119	touches the pallets at 19.26 s; tracking box left; no alarm
With sideslip, at the controller's radius	206; 222	13.5; 36.1	0.641	at rest, 0.0092 m from the dock; tracking box left; no alarm
With sideslip, the gyro's bias unestimated	942; 728	34.0; 39.3	1.148	touches the pallets at 19.70 s; tracking box left; one alarm, at 19.8 s, and one restart; at rest, 1.134 m from the dock
Dead reckoning	47.0; 83.3	7.4; 4.1	0.472	at rest, 0.2567 m from the dock; in the tracking box; no alarm
Dead reckoning with sideslip	23.3; 21.6	7.4; 39.1	0.450	at rest, 0.1941 m from the dock; in the tracking box; no alarm

S13.7 The substitutions in detail

The runs of Section 13.3 have these details, all from the summary `code/ch13/results/variants-summary.txt`.

Pure pursuit at the longest look-ahead. At $L_a = 1.5$ m, from 10.52 to 10.88 s, from 0.17 m before the turn to 0.19 m into it, the robot heads into the turn early. It leaves its tracking box by up to 59.4 mm in the norm, 0.0947 rad to the left of the route's direction at $s = 10.01$ m. From 15.28 to 15.50 s, as it leaves the turn, it leaves its tracking box again, by up to 33.7 mm. Through the turn between, it stays in its tracking box; after it, the robot swings out to 0.1305 m right of the centre line at $s = 15.35$ m, within Δ_d. At the period boundaries at 10.6, 10.8 and 15.4 s, the state lay outside the initial set by 30.8, 27.8 and 33.7 mm in the norm, and the estimate by 31.4, 34.5 and 22.4 mm. The largest estimated

Table S13.10. The record of the experiment of Section 13.2, which follows the record that `husky_route.py` writes, `results/route-record.txt`.

Item	Record
Model and asset provenance	the model, the drive controller, the drivers and the declared contacts of Table S13.3, unchanged
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3 and CPython 3.13.17; the settings of Table S13.3
Time steps	0.5 ms; drive controller, estimator and feedback at 50 Hz; governor every 0.2 s; the crossing with the map repeated at 1 and 0.25 ms
Modules	the core and contract of Chapter 10 (allovi 0.1.0, contract 1.0); control package `husky.route-speed`, proposal `husky.cruise`, predictors `husky.fixed`, `husky.reactive`, `husky.map` and `husky.forecast`, robot `husky.robot`, world `husky.yard`, evaluator `husky.metrics`
Actuators and sensors	the drive controller's velocity command from the feedback of Equation (13.2); encoders exact; gyro, ours: bias 0.004 rad/s, noise 0.001 rad/s per reading at 50 Hz, its bias estimated over 2 s at rest; the estimator with sideslip
Frames and units	SI units; the route coordinates of Table S13.3; the state (s, u, d, e) with the norm of Equation (13.1)
Uncertainty bounds	labels, ours: 0.6 of μg, at most 4 m/s^2; global capability 0.8 m/s^2; the shower, ours: friction 0.15 on the wet grass from 20 s, announced at 10 s; $\varpi = 0.05$ m, ours; the feedback's bounds, ours: 0.02 m, 0.15 m/s, 0.06 m and 0.04 rad
Constraints	the dock and the lane of Section 13.1.1, with a reserve each: the dock's from the floor of 0.1 m, the lane's 0.0586 m for every plan; the commanded speed at most 1 m/s, the task's limit, which no reserve keeps; the robot's speed, by the tracking box's bound, at most 1.25 m/s
Terminal behaviour	the recovery continuation at the labels, then rest; after an alarm, the stop at the global capability, and a restart from rest
Training and evaluation split	none: nothing learns in this section
Seeds	the gyro's noise, NumPy's PCG64, seed 13; seeds 1 to 20 in a repetition
Hardware	one x86-64 Linux machine, the second of Table S13.3
Guarantee status	retrospective consistency only: the same runs set and check the bounds; the tracking box is not a funnel of Definition 4.4, so Theorem 9.5 is not applicable to this package; in the shower with the map's labels, a label and ϖ invalid and the box left by 37.8 mm, inside the reserves; no alarm

heading error, 0.0976 rad, came at 10.70 s, between two boundaries; an alarm would have needed an estimated heading error beyond $\Delta_e + \omega/(2\,\mathrm{m}) = 0.09$ rad at a boundary.

Pure pursuit at 0.5 m, every test again. In each of the seven rows of stops of Table S13.13, the estimate's bounds, the feedback's and the tracking box fare with pure pursuit at 0.5 m as they do with the feedback of Equation (13.2): kept at every valid label, and the estimate's broken and the tracking box left at the map's label in the shower. The crossings with the four predictors, dry and in the shower, have the verdicts of Table 13.1, the one with the map's labels in the shower leaving its tracking box by 37.5 mm, against 37.8 mm. The crossing at time steps of 1 and 0.25 ms keeps every bound, with the robot at rest from 25.24 and 25.60 s. Over the gyro's seeds 1 to 20, every seed keeps the estimate's bounds and the feedback's, with the estimate within 85.1 mm of the true offset, the state in its tracking box, no alarm and no contact. With pure pursuit at 0.5 m, the largest torque comes at $s = 9.93$ m, before the turn.

The proposals. The record of `husky.hurry` differs from the reference's in the proposals of every period, in the candidates from period 125, and in the committed ambitions of periods 125 to 169, 1 for `husky.cruise` and 0 for `husky.hurry`. With `husky.careful`, the front reaches the dock zone at 48.64 s, and the plan in force at the end is certified at 0.11 m.

The payload. The payload is a box of 0.4 m × 0.3 m × 0.1 m. The rolling radius with it is 0.164 96 m. In the crossing, the state stays within 61.5 mm of the centre line and 0.0355 rad of the route's direction. Where the turn begins, the drives' 254 steps at their limit fall within nine updates of the controller ending from 10.78 to 10.94 s. The robot rests in the dock zone from 25.26 s, 0.1902 m from the dock. In the stops on gravel, whose label is 3.53 m/s^2, the drives reach their limit in 16 steps and every bound holds; on the wet grass they stay below it.

Less friction. At 0.19, (S5) holds, at 49.8 mm in the norm; from 0.18, where it fails, the estimate's error along the route also exceeds E_s, at 56.7 mm. The state stays in its tracking box down to 0.18, at 62.4 mm from the commanded arc length, and leaves it from 0.17, by 6.9 mm in the norm; at 0.19 and 0.18 the label is valid while the estimate's bound is not. The estimate's error at rest from 1 m/s grows from 2.6 at 0.35 to 105.9 mm at 0.15. The feedback holds the estimate within 7.1 mm of the commanded arc length, and the drives' torque peaks at 18.1 N m, below their limit. In the crossing at 0.20, the robot rests in the dock zone from 25.34 s, 0.1671 m from the dock, with the estimate's error along the route at 42.9 mm of its 50 mm and the state at 49.5 mm from the commanded arc length. At 0.15, E_s and E_u fail, and (S5) with them, at 115.6 mm in the norm; the state leaves its tracking box by 37.8 mm in its last stop, and the robot rests

0.1067 m from the dock. At 45 period boundaries of that crossing the state lay outside the initial set, by up to 37.4 mm.

Weaker drivers. At 20 N m, the stops' estimate lies up to 31.0 mm from the commanded arc length and 0.210 m/s from the commanded speed; at 13 N m, the state leaves its tracking box by 16.5 mm in the norm; the estimate errs by at most 8.6 mm along the route at every limit. In the crossing at 20 N m, the drives reach their limit in 8 steps where the turn begins. At 15 N m, the estimated heading error reaches 0.0450 rad and the state's 0.0505 rad, at $s = 10.27$ m, and the robot rests 0.1858 m from the dock. At 12 N m, the heading error reaches 0.0970 rad at $s = 10.31$ m, and at the alarm the estimate lies 0.0295 rad beyond the initial set in heading. After the alarm, the feedback asks for a yaw rate from 0.64 to 2.46 rad/s without its integral, and up to 12.15 rad/s with it, since the integral grows while the robot cannot follow; the drive controller passes at most its limit of 2 rad/s. The robot touches the pallets at $s = 11.72$ m, 1.6 s after the alarm, with its footprint 1.088 m from the centre line; the footprint had left the lane 0.82 s after the alarm, at $s = 11.09$ m. The robot then moves backwards, at up to 0.363 m/s, and ends the run at $s = 8.47$ m, its footprint up to 1.449 m from the centre line. In each of the 115 periods after the alarm, the package offered no set to restart from.

A late sensor. The encoders' readings are late from the first update; the gyro's are late once that many readings have been taken, its first reading standing in until then, while the robot starts from rest along a straight. With the readings 20 ms late, the estimate errs by up to 43.1 mm along the route, against 23.3 mm on time. At 40 ms, E_s and E_u fail at 62.9 mm and 0.121 m/s, and (S5) at 62.9 mm in the norm. At 60 ms, the state leaves its tracking box by up to 12.6 mm in the norm, at every update from 11.36 to 25.18 s. With the readings 100 ms late, the clearance from the dock ends at 0.1235 m, and the state lay outside the initial set by up to 51.6 mm. With the compensation, the heading's error at 100 ms takes (S5) to 52.0 mm in the norm, and the robot rests 17.7 mm beyond the point of rest, 0.1949 m from the dock; the heading's largest error comes at $s = 10.11$ m, as it does with the readings 40 ms late, compensated or not.

S13.8 The record of the substitutions

Table S13.11 is the record of the experiment of Section 13.3, which states its evidence status.

Table S13.11. The record of the experiment of Section 13.3, which follows the record that `husky_variants.py` writes, `results/variants-record.txt`.

Item	Record
Model and asset provenance	the model, the drive controller, the drivers and the declared contacts of Table S13.3; with the payload, the manual's 51.5 kg as a box of ours, 0.4 m × 0.3 m × 0.1 m, centred 0.30 m above the ground: 131.5 kg in all
Simulator and solver	as in Table S13.10
Time steps	0.5 ms and the rates of Table S13.10; the crossing with pure pursuit at 0.5 m repeated at 1 and 0.25 ms; the torques recorded at every step
Modules	the reference of Table S13.10; substitutes (`husky/variants.py`): `husky.route-speed-pursuit` at look-aheads of 0.5, 0.75, 1 and 1.5 m, at 0.5 m also with the stops of Table S13.13 and the crossings with the four predictors, at 1.5 m with the stops on the dry grounds; `husky.hurry`; `husky.careful`, over 300 periods
Actuators and sensors	as in Table S13.10; the drivers' limit at 30, 25, 20, 15, 13 and 10 N m in the stops on asphalt and at 20, 15, 12 and 10 N m in the crossing; the estimator's readings late by 1, 2, 3 and 5 updates of 20 ms, and by 2 and 5 with the compensation
Frames and units	as in Table S13.10
Uncertainty bounds	the bounds and labels of Table S13.10, unchanged in every run; the wet grass's true friction 0.35, 0.30, 0.25, 0.22, 0.20, 0.19, 0.18, 0.17, 0.16 and 0.15, from the start, under the map's label
Constraints	as in Table S13.10
Terminal behaviour	as in Table S13.10
Training and evaluation split	none: nothing learns in this section
Seeds	the gyro's noise, NumPy's PCG64, seed 13; seeds 1 to 20 with pure pursuit at 0.5 m
Hardware	one x86-64 Linux machine, the second of Table S13.3
Guarantee status	retrospective consistency as in Table S13.10, without a robust funnel or independent validation, for the reference, `husky.hurry` and pure pursuit at 0.5 m; for the other runs that kept every bound, the evidence of one crossing; for the others, no guarantee, with the obligations of Table 13.2 broken

S13.9 Asset conversion details

The track of the published description. The macro of the suspension beams places both wheel mounts of a beam 9.5 mm to the left of the beam, in the beam's own frame, for the left beam and for the right one alike, and the frames of the two beams are not mirrored. In the published description the left wheels are therefore 0.2829 m from the centre line of the chassis and the right wheels 0.2639 m: a track of 0.5468 m, against the track width of 566 mm in the manual.

We read the offset of the right beam's mounts as a mistake: it contradicts both the manual's track and the symmetry of the rest of the description, and the macro turns the right beam's visual mesh half a turn about the vertical, as one turns a part to mount it on the other side, but not the frame that carries the mounts. Nothing in the sources says that it is a mistake. A simulator finds it without being asked: at rest on level ground, MuJoCo loads the right wheels of the published model with 202.2 N and 202.0 N, the left with 190.2 N and 190.3 N. Mirroring the right beam's mounts gives a track of 0.5658 m, 0.2 mm short of the manual's, and loads of 196.2 N on each wheel, together the weight of the 80 kg robot. The model makes that correction and declares it; the published description remains available to the code.

What MuJoCo makes of the description. MuJoCo fuses the static bodies of a URDF into their parents and discards purely visual elements (Google DeepMind, 2026b, XML Reference), which leaves five bodies, the chassis and the four wheels. Two of its defaults need correcting. The URDF joins the chassis to nothing, and MuJoCo adds no joint between it and the world, so that the robot, loaded as it is, is welded to the ground: the model gives the chassis a free joint. And a body without an inertial element receives the mass of its collision geometry at the density of water (Google DeepMind, 2026b, XML Reference): MuJoCo gives each bumper 1.238 kg, and the robot 82.476 kg. The model gives the bumpers' collision meshes a density of zero.

S13.10 The contact-loss calculation

Why the contacts let go. MuJoCo's documentation says of its convex contact model that 'the only way to initiate slip is to generate some motion in the normal direction', and calls such effects 'numerically small yet undesirable' (Google DeepMind, 2026b, Computation). The solver's source shows how large they can be (Google DeepMind, 2026c). For a contact with the elliptic friction cone, the solver works on the residual between the contact's acceleration and its reference, with a normal part j_n and a tangential part j_t. The contact exerts no force when $j_n \geq \mu\|j_t\|$. When $-I\|j_t\|/\mu < j_n < \mu\|j_t\|$ it slides, with the normal force

$$f_n = \frac{\mu\|j_t\| - j_n}{R_n\,(1 + \mu^2/I)} \tag{S13.1}$$

and the friction μf_n against j_t; below that range it sticks in MuJoCo's soft sense. Here $R_n = (1-d)\,\hat{A}_n/d$ is the regularizer of the normal, set by the impedance d of `solimp` at the contact's penetration and by $\hat{A}_n$, MuJoCo's approximation of the contact's inverse inertia along its normal, and I is the option `impratio`, the factor by which the friction dimensions are less regularized than the normal

(Google DeepMind, 2026b, Computation). In steady motion, the normal residual is the reference's spring on the penetration: zero when the contact just touches, and the more negative the deeper it is. At rest the contact holds its load F with $j_n = -R_n F$. In steady sliding at the slip speed v, j_t is the friction's reference turned round, bv, and by Equation (S13.1) the contact holds the same load with $j_n = \mu b v - (1 + \mu^2/I) R_n F$: the faster the wheel slides, the shallower it sits. When j_n reaches zero the wheel is at the surface, and a sliding contact therefore stays closed only below the slip speed

$$v^* = \frac{(1 + \mu^2/I)\, R_{n0} F}{\mu\, b}, \tag{S13.2}$$

where R_{n0} is the regularizer at zero penetration, with the impedance d_0, the first element of `solimp`. Beyond it, the reference asks for a wheel above the ground: the wheel leaves it, falls back and is thrown up again. Equation (S13.1) is our reading of the solver's code for the cone's middle zone, and Equation (S13.2) is derived from it. The script tests Equation (S13.2) on a point mass of 20 kg pushed at its Coulomb friction, finding by bisection the least slip speed at which it loses its contact. With the defaults and $\mu = 0.8$ the condition predicts 0.0212 m/s and the mass first loses its contact at 0.0214 m/s; with $\mu = 0.35$, 0.0332 and 0.0332 m/s; with the declared parameters below, 3.91 and 3.91 m/s (Figure 13.2a). For the Husky's contacts the defaults give $R_{n0} F = 0.413\,\mathrm{m/s^2}$, and v^* is 8.05 mm/s on asphalt and 12.6 mm/s on wet grass. A skid-steered robot that turns slides its wheels sideways at decimetres per second.

S13.11 Detailed capability and substitution tables

Table S13.12. What the simulated Husky does on each ground of the yard, commanded through its controller: the stop from a commanded 1 m/s, the spin and the turn. 'Commands': the distance of a robot whose wheels followed the controller's commands exactly, without slip, from the speed u_0 before the stop; 'at μg': $u_0^2/(2\mu g)$. The stop beyond the commands is split into what the wheels' turning accounts for, at the rolling radius of the steady drive, and the rest. From `results/mujoco-summary.txt`.

	Asphalt, $\mu = 0.8$	**Gravel,** $\mu = 0.6$	**Wet grass,** $\mu = 0.35$
Deceleration: of the controller's ramp at the ground; μg	4.06; 7.85 m/s^2	4.06; 5.89 m/s^2	4.06; 3.43 m/s^2
Stop: distance, time to rest	0.1240 m, 0.246 s	0.1282 m, 0.253 s	0.1552 m, 0.309 s
Stop: commands; at μg	0.1170; 0.0657 m	0.1170; 0.0876 m	0.1170; 0.1501 m
Stop beyond the commands: by the wheels' turning; the rest	8.5; −1.5 mm	10.0; 1.3 mm	11.1; 27.1 mm
Stop: largest slip of the wheels	0.005 m/s	0.020 m/s	0.178 m/s
Stop at time steps of 1, 0.5, 0.25 ms	0.1165, 0.1240, 0.1263 m	0.1232, 0.1282, 0.1299 m	0.1540, 0.1552, 0.1560 m
Spin commanded at 1 rad/s: yaw rate; W that would give it	0.9478 rad/s; 1.0377 m	0.9476 rad/s; 1.0378 m	0.9475 rad/s; 1.0380 m
Spin commanded at 2 rad/s: largest torque of a wheel	18.6 N m	14.0 N m	8.1 N m
Turn commanded at radius 3 m	3.218 m	3.209 m	3.166 m

Table S13.13. Stops at the labels: from rest, the robot accelerates to the speed shown, holds it for a second and stops as the recovery continuation commands, with the feedback and the estimator with sideslip. Beyond: the true arc length at rest less the commanded point of rest; error at rest: the true arc length less its estimate; bounds: over the four stops, the largest error of the estimate along the route and the largest distance of the estimated arc length from the commanded one, and whether every component of the estimate's bounds and of the feedback's was kept. The shower is that of Section 13.2.5, its friction in force throughout. From `results/route-summary.txt`.

Ground	Label ($\mathrm{m/s^2}$)	Beyond, at 0.25, 0.5, 0.75, 1 m/s (mm)	Error at rest, the same (mm)	Bounds (mm): estimate; feedback
Asphalt	4	+1.6, +3.2, +10.5, +13.5	0.0, 0.0, +3.8, +5.5	8.6; 14.0: kept
Gravel	3.53	+1.6, +3.7, +8.6, +10.0	0.0, +0.5, +4.9, +4.9	8.6; 10.0: kept
Wet grass	2.06	+1.6, +3.7, +5.3, +5.9	0.0, +1.8, +1.8, +2.6	8.6; 7.0: kept
Wet grass, braking at $3\,\mathrm{m/s^2}$, 0.87 of μg	3	+1.6, +8.3, +13.9, +17.3	0.0, +5.1, +11.0, +12.8	14.6; 8.0: kept
Wet grass in the shower, its label	0.883	+0.1, 0.0, −0.9, −2.2	−0.9, −1.9, −2.5, −4.5	4.5; 4.4: kept
Wet grass in the shower, the global capability	0.8	−0.3, −1.4, −1.7, −4.1	−1.4, −2.6, −3.7, −5.8	5.8; 4.4: kept
Wet grass in the shower, the map's label	2.06	+4.0, +27.9, +65.5, +109.5	+2.4, +26.0, +61.6, +105.9	105.9; 6.6: not kept

Table S13.14. Substituted modules on the common task: the crossing with the map's labels and the estimator with sideslip, the reference first. Rest: the instant from which the robot rests in the dock zone, and the clearance of its footprint from the dock at the end. Feedback: the estimate's largest distance from the centre line and from the route's direction, against $F_d = 60\,\text{mm}$ and $F_e = 0.04\,\text{rad}$. State: the same for the state, against $\Delta_d = 160\,\text{mm}$ and $\Delta_e = 0.065\,\text{rad}$. Tracking box: whether the state stayed in its tracking box, or by how far, in the norm, it left it; alarm check: its largest distance, against $\varpi = 50\,\text{mm}$. No crossing raised an alarm or touched an obstacle. From `results/variants-summary.txt`.

Module substituted	Rest (s); clearance (m)	Feedback: across (mm); heading (rad)	State: across (mm); heading (rad)	Tracking box; alarm check (mm)
None: the reference	25.58; 0.1941	39.1; 0.0298	60.0; 0.0310	kept; 0
Pure pursuit, $L_\text{a} = 0.5\,\text{m}$	25.58; 0.1952	14.5; 0.0295	35.3; 0.0262	kept; 0
Pure pursuit, $L_\text{a} = 0.75\,\text{m}$	25.58; 0.1956	28.9; 0.0471	49.1; 0.0438	kept; 0
Pure pursuit, $L_\text{a} = 1\,\text{m}$	25.58; 0.1958	49.2; 0.0641	68.8; 0.0610	kept; 0
Pure pursuit, $L_\text{a} = 1.5\,\text{m}$	25.58; 0.1980	111.7; 0.0976	130.5; 0.0947	left, by 59.4 mm; 34.5
Proposal `husky.hurry`	25.58; 0.1941	39.1; 0.0298	60.0; 0.0310	kept; 0
Proposal `husky.careful`	49.48; 0.2078	9.7; 0.0106	30.0; 0.0136	kept; 0

Table S13.15. The changed assumptions, crossing by crossing, with the map's labels, the proposal `husky.cruise` and the estimator with sideslip. Estimate and feedback: the components of their bounds that the crossing did not keep, at the updates of the controller; in every crossing, (S5), the measurement's bound at the period boundaries, failed exactly where the estimate's bounds did. Tracking box: whether the state stayed in its tracking box, or by how far, in the norm, it left it; alarm check: its largest distance, against $\varpi = 50$ mm; end: the instant from which the robot rests in the dock zone and its clearance from the dock at the end, or what broke the constraints. For the drivers at 12 and 10 N m, the bounds and the tracking box are those before the alarm. From `results/variants-summary.txt`.

Change	Estimate	Feedback	Tracking box	Alarm check (mm)	End
None: the reference	kept	kept	kept	0	25.58 s, 0.1941 m
Payload of 51.5 kg	kept	kept	kept	0	25.26 s, 0.1902 m
Wet grass at $\mu = 0.20$	kept	kept	kept	0	25.34 s, 0.1671 m
Wet grass at $\mu = 0.15$	E_s, E_u	kept	left, by 37.8 mm	0	25.70 s, 0.1067 m
Drivers at 20 N m	kept	kept	kept	0	25.58 s, 0.1915 m
Drivers at 15 N m	kept	F_e	kept	0	25.58 s, 0.1858 m
Drivers at 12 N m	kept	F_e	left, by 63.9 mm	59.1: alarm at 11.0 s	lane left at 11.82 s, pallets touched at 12.60 s
Drivers at 10 N m	kept	F_e	left, by 65.5 mm	60.7: alarm at 11.0 s	lane left at 11.82 s, pallets touched at 12.60 s
Readings 20 ms late	kept	kept	kept	0	25.54 s, 0.1834 m
Readings 40 ms late	E_s, E_u	F_s, F_e	kept	0	25.52 s, 0.1705 m
Readings 60 ms late	E_s, E_u	F_s, F_u, F_e	left, by 12.6 mm	0	25.52 s, 0.1549 m
Readings 100 ms late	E_s, E_u, E_e	F_s, F_u, F_e	left, by 52.1 mm	3.0	25.50 s, 0.1235 m
Readings 40 ms late, compensated	kept	kept	kept	0	25.58 s, 0.1945 m
Readings 100 ms late, compensated	E_e	kept	kept	0	25.36 s, 0.1949 m

CHAPTER S14

Arms and heavy-duty mobile manipulators: records and details

This chapter holds what Chapter 14 of the book moved out of its text: the UR5e's sources, the records of the experiments of Sections 14.1 to 14.3, further checks of the mobile manipulator's rigid model, the record of the lift's demand, the limits that the governed runs' test uses, and the provenance of the governed machine. Every number comes from the results that the scripts of `code/ch14` write.

S14.1 The UR5e in its sources

Section 14.1.1 builds the UR5e in MuJoCo from Universal Robots' description, `ur_description` (Universal Robots, 2026). The folder `arm/description` holds the files of the UR5e in release 3.6.0, the head of the branch for ROS 2 Jazzy, commit `7ce98d8` of 22 October 2026, with the collision meshes and the macro, copied for reference. The description is a set of configuration files that a macro of xacro turns into a URDF: the joints' origins, the links' masses, centres of mass and inertia tensors, the joints' limits and the meshes' placement. Over 200 random poses the model's frame `tool0` lies within 2.0×10^{-10} m of the frame that UR's table of Denavit–Hartenberg parameters gives, and its rotation within 6.0×10^{-10}, the rounding of the angles in the description's files. The links' masses equal those of UR's table, and their centres of mass lie within 2.1×10^{-10} m of UR's.

Three things differ from what UR publishes elsewhere (Table S14.1). First, the inertia tensors of the description's six moving links are measured ones, which replaced approximations by cylinders in April 2025; only the fixed base keeps a cylinder's. UR's table, by contrast, calls the UR5e's inertias negligible and gives zero for its first five links. Second, the description's masses add up to 21.70 kg, against the 20.6 kg with cable of the datasheet. The description itself marks its base's 4 kg with the comment 'This mass might be incorrect'; that matters little while the base stands on a table, and would matter on a mobile base. Third, the description limits the elbow to $\pm 180°$, half its range, to spare planners a divided space, where the datasheet gives $\pm 360°$ for every joint.

MuJoCo Menagerie's model of the UR5e (Zakka et al., 2022) comes from an older description. It holds other masses: 8.393 kg for the upper arm against

UR's 8.058, 2.275 kg for the forearm against 2.846, and 0.1889 kg for the last wrist against 0.365. Its inertias approximate the links by cylinders, as the description's moving links did until April 2025. It also gives every joint a rotor inertia, an armature of 0.1 kg m^2, its own choice, since no description holds one; with it, the loaded arm's largest deceleration is 62.31 rad/s^2 (Table 14.1).

The time step. The overrun of the stop at 1.2 times the capability (Section 14.1.1) is also checked for its time step. At steps of 1, 0.5 and 0.25 ms the overrun is 1.675°, 1.717° and 1.738°; it converges at first order, towards 1.759°, so that the value at 0.5 ms lies 2.4 % below its limit.

Further details of the stops. The capability leaves the feedback no torque to spare: stopped at its own, the empty arm has its base's torque clipped in one update. At 1.2 times the capability with the 5 kg payload, the second wrist is clipped in 41 updates, as the base is, and turns at up to 150.8°/s. With the arm held in its pose, the base under our controller would overrun the commanded rest by 0.843° and 2.197° at 1.2 and 2 times the capability, against the 0.725° and 2.175° of braking at the base's limit all the way to rest: the base's own feedback adds little. When the base's torque leaves its limit, at 0.041 s in the stop at 1.2 times, the base still turns at 1.011 rad/s, against 0.476 rad/s with the pose held, and its farthest angle comes at 0.060 s. Under the empty arm's label with the pose held, the base would have stopped at −64.365° and the payload come within 0.0233 m of the post. In the task, the base joint's own angle lies 15.46° behind the angle of `tool0`, because of the offsets of UR's shoulder and wrists; the empty arm passes over the post, never closer to it than 0.0499 m, and with the payload the clearance at the place is 0.0578 m.

S14.2 The UR5e's stops at its labels

Table S14.2 lists the stops that the labels of Section 14.2.1 command, simulated with the controller of Section 14.1.1. Stopped at the declared label from full speed at the place, the nine payloads of the grid ask at most 0.898 of any joint's torque, and the base follows its command within 0.050°. Outside the declaration, a payload of 6 kg at 0.05 m would keep the share only under a label of 52.75 rad/s^2; under the declared label it asks 0.969 of the base's torque, more than the share but less than the torque, and 4.5 kg at 0.10 m asks 0.932 of the second wrist's. Neither is clipped, and both stop as the label commands. At the global capability's corner, 5 kg at 0.10 m, whose own bound without the share is 55.07 rad/s^2, the second wrist is asked 1.083 of its torque and clipped in 42 updates, and the clearance is 0.0183 m. With 6 kg at 0.10 m, whose own bound without the share is 46.31 rad/s^2, the second wrist is clipped in 56 updates and turns up to 6.108° from the pose; the clearance falls to 0.0112 m from the place,

Table S14.1. The UR5e in UR's publications, in UR's description, in MuJoCo Menagerie's model, and the value that the model of Section 14.1.1 takes. UR: the table of Universal Robots (2018), the datasheet (Universal Robots, 2024) and the article on the joints' torques (Universal Robots, 2015); description: release 3.6.0 (Universal Robots, 2026); Menagerie: commit `f3b6013` (Zakka et al., 2022). Model values from `results/arm-summary.txt`.

Quantity	UR	Description	Menagerie	Model
Kinematics	Denavit–Hartenberg table	joints' origins, from the same values	from an older description	the description's: within 2.0×10^{-10} m of UR's table
Links' masses	3.761, 8.058, 2.846, 1.37, 1.3, 0.365 kg	the same; base 4 kg, 'might be incorrect'	base 4; 3.7, 8.393, 2.275, 1.219, 1.219, 0.1889 kg	the description's
Total	20.6 kg with cable	21.70 kg	20.99 kg	21.70 kg
Inertias	'negligible', zero for links 1 to 5	moving links measured since April 2025, cylinders before; base a cylinder	cylinders	the description's
Joint torques	150 N m at base, shoulder and elbow, 28 N m at the wrists; not all usable in an application	the same	the same, as force limits	the limits of the model's torque actuators
Speeds	180°/s at every joint	the same	—	not enforced; below 180°/s after the start in the five stops
Ranges	±360° at every joint	±360°; elbow ±180°	elbow ±180°	the description's, as MuJoCo's joint limits
Rotor inertia	—	none	armature 0.1 kg m^2 at every joint	none, as the description; Menagerie's as a variant
Controller	not published	—	position servos	ours: computed torque at 1 kHz, torques clipped at the limits

Table S14.2. The UR5e's stops at its labels, simulated from full speed with the controller of Section 14.1.1, for Section 14.2.1: from the place, or from the latest angle that the label allows for a stop that keeps the floor; the largest torque that the controller asks of a joint, as a share of its limit, the joint and the updates in which its torque is clipped, and the same for any other joint asked more than its limit; the overrun of the commanded rest; and the least clearance from the post, negative where the payload strikes it. The payload's centre of mass below the flange. From `results/labels-summary.txt`.

Payload	Label	From	Torque asked; updates clipped	Overrun	Clearance
the nine of the grid	declared	the place	at most 0.898; none	0.000°	0.0184 m
4.95 kg, 0.075 m	declared	−61.072°	0.898, wrist 2; none	0.000°	0.0100 m
6 kg, 0.05 m	declared	the place	0.969, base; none	0.000°	0.0184 m
4.5 kg, 0.10 m	declared	the place	0.932, wrist 2; none	0.000°	0.0184 m
5 kg, 0.10 m	declared	the place	1.083, wrist 2; 42	0.000°	0.0183 m
6 kg, 0.10 m	declared	the place	2.090, wrist 2; 56	0.000°	0.0112 m
6 kg, 0.10 m	declared	−61.072°	2.090, wrist 2; 56	0.000°	0.0028 m
4.95 kg, 0.075 m	empty	the place	3.773, wrist 2; 42; and 3.613, base; 40	2.496°	0.0217 m
4.95 kg, 0.075 m	empty	−63.857°	3.773, wrist 2; 34; and 3.613, base; 35	2.364°	−0.0049 m

and from the latest angle to 0.0028 m. Under the empty arm's label from the place, the heaviest corner asks its second wrist for 3.773 and its base for 3.613 of their torques, which are clipped in 42 and 40 updates; it stops in 4.693° in all, more than the 4.484° that its own full torque would need, but less than the declared label's stop.

S14.3 The records of the two machines

Table S14.3 is the record of the two experiments of Section 14.1, which follows the records that the two scripts write, `results/arm-record.txt` and `hdmm-record.txt`. Neither certifies a plan: they state the task, the models and what the models can do.

S14.4 Further checks of the mobile manipulator's rigid model

The impedance of the contacts. The checks of Sections 14.1.2 and 14.2.2 take MuJoCo's ratio of frictional to normal impedance as 20 rather than its default of 1. At the default, the machine braking at its limit reaching creeps 7.53 mm forward on its contacts in 9.5 s, and its rear contacts drop out at times, when the

Table S14.3. The records of the experiments of Section 14.1, after those that `arm_mujoco.py` and `hdmm_mujoco.py` write.

Item	UR5e	Heavy mobile manipulator
Model and asset provenance	UR's description, release 3.6.0, its files and collision meshes; the macro's mapping in `arm/model.py`; the actuators, payload and post ours	synthetic, every number ours (Table 14.2)
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3 and CPython 3.13.17; PyYAML 6.0.3 for the UR5e's files; the implicitfast integrator; for the mobile manipulator, elliptic friction cones, MuJoCo's default contact softness (solref 0.02, 1; solimp 0.9, 0.95, 0.001) and a frictional impedance ratio of 20	
Time steps	0.5 ms, the controller at 1 kHz; one stop at 1 and 0.25 ms	1 ms; the contact loads settled for 2 and 3 s; for the onsets, settled 2 s, the acceleration raised over 5 s (also at once), held 24 s, tipped if leaning more than 0.1°
Actuators and sensors	torque at the joints, limited to 150 and 28 N m; computed torque, ours; the state from the simulator	the arm rigid; the torques that hold it in its pose computed exactly
Frames and units	SI units; each machine's frame with x forward and z up, the ground or table at $z = 0$; the UR5e's angles those of `tool0` about the base's axis	
Uncertainty bounds	none: every model is nominal, and the capabilities that the section finds carry no margin for the models' errors	
Constraints	no contact with the post; joint ranges, 180°/s, the torque limits	all four wheels on the ground; friction 0.6, 1.5 in the simulated checks
Terminal behaviour	the base joint's commanded stop	none: the section measures limits
Training and evaluation split	none: nothing learns in this section	
Seeds	14, for the random poses of the kinematic check	none
Hardware	one x86-64 Linux machine, a 2.10 GHz Xeon, the third on which the results of Chapter 13 were reproduced	
Guarantee status	none: no plan is certified	none: a synthetic machine

centre of pressure jumps to the front axle. At 5, carrying still drops them, and reaching left raised slowly falls 1.63 mm short. From 10 on the shortfalls are at most 1.08 mm, and between 20 and 50 they change by at most 0.35 mm. At 20 the machine creeps 0.38 mm and keeps its four contacts.

Slow braking, creep and the arm's torques. With the base's deceleration raised slowly, over 5 s, to a braking limit of the declared label with its heaviest payload, the centre of pressure of the contacts' normal forces lies at most 0.64 mm further out than the rigid machine's zero-moment point, the servos yielding a little to

the load. On the least friction that a label admits, a stop from the top speed at each braking limit lets the machine creep at most 0.08 mm on 0.5 and 0.26 mm on 0.3. Through the turns at their speed, braking from the middle of the exit and from where the test binds, and driving off where it binds, the machine creeps at most 0.31 mm and turns on its contacts by at most 0.034 mrad. This holds even reaching, where the test's account of the ground binds. At the braking limits that the arm sets, the slew holds 6.67 kN m, its limit divided by 1.2. At rest the arm holds every pose of its ranges with 300 kg: its largest torques, at full reach, are 27.52 kN m at the lift, 0.918 of its limit, and 9.62 kN m at the tilt, 0.802 of its. Under the global capability the ground's friction of 0.3 limits the braking to 2.94 m/s^2 stowed and carrying, and with the declared payload the brakes still bind carrying, with tipping only just behind them (Table 14.4).

S14.5 The lift and the turns

Table S14.4 is the record that turns the lift of Section 14.2.3 into a demand.

Further checks of the lift. In the run with the base braking at 2.599 m/s^2 from the drive, the centre of pressure comes closest to the edge at 4.26 s, 0.26 s after the lift's end, in a braking begun 0.34 s before that end. Braking from cruising instead, the centre of pressure comes no closer than 0.1111 m. At single instants of that run the centre of pressure lies up to 6.86 mm further out than the rigid machine's zero-moment point for the planned pose at the same instant, 1.79 s into the lift, where that point lies 0.4250 m inside; wherever the centre of pressure lies more than the allowance further out, the rigid machine's point lies at least 0.1428 m inside, above the plan's level. Beyond the horizon, the least distance, 0.1104 m, lies 3.72 mm short of the rigid machine's 0.1141 m; braking at the end pose's own limit there, at 2.67 m/s^2, the lift keeps 0.1012 m. The plan in force, carrying with the arm held, keeps 0.2242 m while braking at 2.599 m/s^2 from the drive, above its 0.22 m, and as much beyond the horizon. In a lift at rest, the arm's motion alone takes the centre of pressure to 0.3407 m, 3.67 mm short of the rigid machine's 0.3444 m. Servos without their offset against gravity sag by 0.782° under the load, and the lift then comes no closer to the edge than 0.1138 m, the sag having lowered the payload. The servos' largest torques in these runs, 27.70 and 9.69 kN m at the lift and the tilt, stay within their limits. With a stop line ahead, the base creeps on its contacts by at most 0.52 mm during the lift, within the allowance.

Further details of the turns. Reaching left with the declared payload, a turn to the right binds: turns of 3 and 5 m allow 1.63 and 2.17 m/s to the right, against 1.86 and 2.37 m/s to the left. With the deceleration rising and fading over 0.6 s each, a slowing takes at least 1.2 s; the slowings to the turns' speeds from the

Table S14.4. The record that turns the mobile manipulator's lift into a demand, after Table S7.1, and the check that the onset admits the lift, with the values of `results/labels-summary.txt`.

Field	Value
Operation	A lift from carrying to reaching over 4 s along a half-cosine, with 243 to 297 kg, at rest or moving
Reserves consumed	The tipping reserve, in metres of the zero-moment point's distance inside the contacts: 0.1029 m at rest and 0.1107 m with the recovery braking at 2.70 m/s^2, in the static model; with a stop line ahead, the clearance, by the front-most point's 0.8592 m
Levels	Tipping: the floor 0.10 m plus 0.1107 m, 0.22 m on whole centimetres; clearance: 0.96 m
Onset	The period in which the plan that begins the lift is committed, as the task plan sets it
Announcement	As the task plan sets it; the lead time must allow the preparation (Theorem 9.14)
Source	Task plan
Check of Assumption 4.13	A plan in force carrying at the level 0.22 m brakes at no more than 2.599 m/s^2, and beyond the forecast horizon 2.572; a plan that begins the lift and brakes there keeps 0.1096 m, and 0.1091 m beyond the horizon; in the simulator, the arm on its servos, the brakes' rise from the drive and a braking begun every 0.1 s, and every 0.02 s near the lift's end, included, the centre of pressure stays 0.1110 m inside, and 0.1104 m beyond the horizon; with the clearance at 0.96 m, the lift leaves 0.1008 m

top speed take no longer, none reaching its pose's braking limit. A turn of 10 m allows the top speed carrying and reaching left, and 2.84 m/s reaching. A recovery that begins in a turn brakes gently: from the speed of the turn of 5 m it stops in 6.43, 4.43 and 5.36 m carrying, reaching and reaching left.

S14.6 The record of the labels, reserves and demands

Table S14.5 is the record of the experiment of Section 14.2, which follows the record that `labels_demand.py` writes, `results/labels-record.txt`. It states labels, reserves and a demand and checks them; it certifies no plan.

S14.7 The governed runs: limits, details and provenance

The limits of the runs' labels. Table S14.6 gives the limits that the test of Section 14.3.1 uses for the labels of the runs, found as in Table 14.4. Those speeds were found for plans that keep their speed through a turn, brake in it at 0.5 m/s^2

Table S14.5. The record of the experiment of Section 14.2, after `labels-record.txt`.

Item	UR5e	Heavy mobile manipulator
Model and asset provenance	the UR5e of Section 14.1.1; its payload a box of 0.1 m hanging from the flange, its mass and centre of mass declared	the synthetic machine of Section 14.1.2; its actuators' limits, labels, floors and lift ours
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3, PyYAML 6.0.3 and CPython 3.13.17; implicitfast; MuJoCo's default contact softness (solref 0.02, 1; solimp 0.9, 0.95, 0.001, 0.5, 2); for the mobile manipulator, elliptic cones, the ratio of frictional to normal impedance 20 (also 1, 5, 10 and 50) and the ground's friction 1.5, so that nothing slides; for the base's creep, the labels' least friction, 0.5 and 0.3; in a turn, the machine in the base's turning frame	
Time steps	0.5 ms, the controller at 1 kHz	1 ms; each run's settling, onsets and holds as `labels-record.txt` lists them
Actuators and sensors	torque at the joints, limited to 150 and 28 N m; the labels use 0.9 of each limit; the controller of Section 14.1.1; the state from the simulator	brakes 4 and drive 1.5 m/s^2, top speed 3 m/s, each change over 0.6 s, a stop fading at rest; turns entered along transitions of 3 m, braking in them at 0.5 m/s^2; slew, lift, tilt 8, 30, 12 kN m, the motion's share counted 1.2 times; servos of 2×10^6 N m/rad, damping 0.05 of it in seconds, without limits, offset against gravity alone (not offset, for comparison), the base's motion not fed forward; the state from the simulator
Frames and units	SI units; the UR5e's angles those of `tool0` about the base's axis; the mobile manipulator's x forward, y left, z up, the ground at $z = 0$	
Uncertainty bounds	declared: 4.05 to 4.95 kg, 0.025 to 0.075 m below the flange; global: up to 5 kg, 0 to 0.10 m	declared: 243 to 297 kg, friction at least 0.5; global: up to 300 kg, friction at least 0.3
Constraints	no contact with the post, the clearance's floor 0.01 m, levels on whole millimetres	the zero-moment point inside the wheels' contacts and the front-most point behind a stop line; floors 0.10 m, an allowance of 5 mm, levels on whole centimetres
Terminal behaviour	the base joint's stop at the label, the other joints holding the carry pose, then rest there	braking within the braking limit to rest with the arm held, or, during a lift, while the lift finishes, in a turn at 0.5 m/s^2; then rest on the brakes
Training and evaluation split	none: nothing learns in this section	
Seeds	none: no random numbers	
Hardware	one x86-64 Linux machine, a 2.10 GHz Xeon	
Guarantee status	none: the section states labels, reserves and a demand and checks them; no plan is certified or governed	

Table S14.6. The limits that the test uses for the labels of the runs: the braking limit on a straight carrying, reaching and while the arm lifts, at the floor and at the lift's requirement for the label's payloads, 0.22 m declared and 0.21 m misdeclared (m/s^2); and the speed at which a plan may take a turn of the yard, radius 5 m, carrying and reaching (m/s), with what binds. The first three labels are those of the runs with the site's survey and the fixed predictor; the next two the oracle's; the last two those of the run whose payload is misdeclared. From `results/governed-summary.txt`.

Label	Braking, carrying	Braking, reaching	Braking, lifting	Turn, carrying	Turn, reaching
243 to 297 kg on 0.5	4.00, 2.59	2.70, 1.35	2.65, 1.29	2.39, slew	1.96, ground
243 to 297 kg on 0.3	2.94, 2.59	2.70, 1.35	2.65, 1.29	1.95, ground	1.46, ground
0 to 300 kg on 0.3, global	2.94, 2.57	2.67, 1.31	2.61, 1.26	1.95, ground	1.45, ground
243 to 297 kg on 0.6				2.39, slew	2.04, slew
243 to 297 kg on 0.35				2.13, ground	1.61, ground
180 to 220 kg on 0.5	4.00, 3.41	3.58, 2.33	3.52, 2.27	2.56, slew	2.20, slew
180 to 220 kg on 0.3				2.15, ground	1.73, ground

or drive off in it from rest; a plan of the governed runs may also change its speed in a turn within that bound, below the turn's speed, which is why the test also evaluates each segment's own states.

Further details of the governed runs. A plan that rests on the last straight must keep its front-most point at least the floor and the allowance behind the stop line, with the point moved by the funnel's creep and the worst corner of its bounds on the arm: 0.1060 m reaching and 0.1104 m carrying. At the states that the script evaluates, a slew moves the zero-moment point by up to 0.641 m per radian, sideways, but changes its distance from the nearest edge, the front one, by at most 0.038 m per radian: the side edges lie at least 231 mm farther. At full reach with 297 kg, at rest, the funnel's bound of 0.005 rad on the slew moves the point 3.14 mm sideways, within the slew's 4.25 mm in the tipping norm. Measured at the states that the script evaluates, the worst corner of the funnel's bounds on the arm costs 0.93 mm held on a straight, 0.66 mm at a state of a turn, and 0.93 mm about a pose of a lift's path, less than the bounds of Proposition 14.2. With the fixed predictor, the cap at the slower 1.45 m/s makes the candidate that follows the proposal, committed in period 28, brake at up to 2.58 m/s^2 in period 29; that candidate needs no clip and is certified, so period 28 counts no intervention. The gentle package's level in force falls to 0.11 m only in periods 50 and 51, in which the head, which follows the proposal, brakes hard for the second turn. In the reference run the base turns on its contacts by

at most 0.026 mrad. With the misdeclared payload, the lift's requirement for the declaration is 0.21 m; taking the first turn at 2.20 m/s, the machine asks the slew for 0.941 of its limit, and the servos, offset for the declared 200 kg, sag a little further under the heavier payload.

The provenance of the governed machine. Table S14.7 lists each part of the governed mobile manipulator of Section 14.3, where it comes from, and what has been checked of it.

S14.8 The record of the governed runs

Table S14.8 is the record of the experiment of Section 14.3, after the record that `hdmm_governed.py` writes, `results/governed-record.txt`.

S14.9 Bounds for the two reserve norms and the reserve proof

The constants of the two reserve norms (Lemma 14.1, Table 14.5) and the proof of Proposition 14.2 are in Section 14.3.2 of the book, with the proposition that they support.

Table S14.7. What the governed mobile manipulator of Section 14.3 rests on: each part, where it comes from, and what has been checked of it in Chapter 14. None has been checked on a physical machine.

Part	Source	Checked
The machine's masses, lengths, poses	ours, synthetic (Section 14.1.2)	the rigid model's thresholds against MuJoCo's machine (Section 14.1.2)
Actuators: brakes, drive, rise, arm's limits	ours	that the plans keep them, in the test; the servos' torques in the simulator (Sections 14.2.2 and 14.3)
The ground's friction and its survey	ours; the survey lists less than the grounds give	the labels' validity against the world's oracle (Section 14.3)
The axles' share of the ground's force	ours, an assumption about the machine (Section 14.2.2)	not checked: MuJoCo's contacts share the forces otherwise, one wheel sliding while the axle holds (Section 14.3.5)
The rigid model and the allowances	MuJoCo's inverse dynamics of the model; the allowances ours	the levels reserves of the rigid machine (Proposition 14.2); the simulated machine within the allowances at the braking limits, the turns and the lifts run (Section 14.2.2), and within the room that Assumption 14.3 allows in every plan executed unchanged (Section 14.3.3); not in the run without the governor
The funnel's bounds	ours	measured in every run with valid labels and every plan executed unchanged (Section 14.3.3), with room; with the misdeclared payload, once just beyond the bound
The norms	ours, each weight at least its constant of Lemma 14.1	the constants against the rates measured at the states that the script evaluates (Table 14.5)
The payload's declaration	ours	a misdeclared payload (Section 14.3.5)
The servos and the measurement	ideal position servos without torque limits; the measurement exact	not checked: idealizations of the simulation (Section 14.3.6)
The simulator's contacts	MuJoCo 3.16.0, default softness, the impedance ratio of Section 14.2.2	the ratio's effect on creep and shortfall (Section S14.4); not against tyres
The yard, the task, the proposals and predictors	ours; the proposals scripted	the checks of Section S10.10.2

Table S14.8. The record of the governed runs of Section 14.3, after `governed-record.txt`.

Item	Value
Model and asset provenance	the synthetic mobile manipulator of Section 14.1.2, every number ours; its labels, limits, floors, lift and allowance of 5 mm with the arm held those of Section 14.2.2; the yard, the grounds and their survey, the task, the modules, their funnel's bounds, the allowance of 10 mm before a lift's end, and the runs ours; no parameter identified from a machine
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3 and CPython 3.13.17; implicitfast, elliptic cones, MuJoCo's default contact softness, the ratio of frictional to normal impedance 20; each wheel's contact on the friction of the ground under it; the machine held in the base's frame, which turns with the route
Time steps	1 ms; the governor's period 0.6 s; 100 periods, after 2 s of settling at rest
Actuators and sensors	those of Section 14.2.2, the acceleration within $\pm 0.5\,\mathrm{m/s^2}$ in a turn; position servos offset for the payload believed; the measurement the simulated state, exact
Frames and units	SI units; the state's arc length that of the base's middle along the route; the base's x forward, y left, z up
Uncertainty bounds	the labels' payload intervals and frictions; the funnel's bounds: the creep 1 mm; the arm held, the slew, the lift and the tilt within 0.005, 0.002 and 0.002 rad; lifting, within those of a pose on the path from 4 % of it behind the planned pose to 0.1 % ahead; the norms of Equation (14.4), each weight at least its constant of Lemma 14.1, the state's norm the larger of the two
Constraints	the tipping reserve and the clearance at the stop line, floors 0.10 m, the allowances 5 mm with the arm held and 10 mm before a lift's end, levels on whole centimetres; the drop zone 0.5 m
Terminal behaviour	the recovery of `hdmm.route-arm`, then rest on the brakes; after an alarm, the rest feedback at the global capability's limits and a restart at rest
Configurations	the eleven runs of Table 14.6, and the lift announced one period before its onset and at it
Training and evaluation split	none: the proposals are scripted, and no learned module is evaluated
Seeds	none: no random numbers
Hardware	one x86-64 Linux machine, a 2.10 GHz Xeon
Guarantee status	retrospective checks of a rigid-model reserve calculation: containment and Assumption 14.3 set and checked in sample, without a robust funnel or independent validation; the premises for invoking Theorem 9.5 for new runs remain unestablished; ideal position servos without torque limits, exact commanded braking and drive acceleration, and exact measurements; no physical machine

CHAPTER S15

A governed quadruped: records and details

This chapter holds what Chapter 15 of the book moved out of its text: what the Go2's model file declares, the numbers of its control package, the touchdowns on two contact models, the provenance of the numbers of the governed lift, the record of its experiment, and the Apollo's run without one of its adaptations.

S15.1 The Go2 as published

Table S15.1 gives what `go2.xml` of MuJoCo Menagerie declares at commit `f3b6013` of 23 October 2026, and what the book changes (Section 15.1.1). Menagerie's README states that the model is derived from the URDF description that Unitree publishes, and lists the steps of the derivation: the meshes converted, common properties gathered into defaults, a free joint added to the trunk, and the contacts of the feet softened 'to approximate the effect of rubber', with the solver's ratio of frictional to normal impedance increased to reduce slipping. The file last changed in May 2026, when every model's file was reformatted; the folder's later change concerns only its MJX variant, which the book does not use.

S15.2 The package's numbers

Table S15.2 lists the numbers of the Go2's control package and of its task (Section 15.2.1), and the status of each: declared, ours, measured in the runs that test it, or an assumption.

S15.3 Touchdowns on two contact models

Table S15.3 gives the eight touchdowns of Section 15.2.3: FR lowered onto the ground from the hold at the requirement's place, at four speeds, on Menagerie's soft feet and on MuJoCo's default impedance, which lets a loaded foot sink 0.2 mm instead of 12.6 mm.

Table S15.1. What `go2.xml` of MuJoCo Menagerie, commit `f3b6013`, declares, and what the book changes (`quad/model.py`). From `results/model-summary.txt`.

Item	Value
Bodies and joints	13 bodies: the trunk and, for each leg, a hip, a thigh and a calf; a free joint and 12 hinges; MuJoCo's state has 19 coordinates and 18 velocities
Masses	15.206 kg in all; the trunk 6.921 kg, a leg 2.071 kg
Motors	12 torque motors, within 23.7 N m (abduction, hip) and 45.43 N m (knee)
Joints	a front leg's ranges −1.0472 to 1.0472, −1.5708 to 3.4907 and −2.7227 to −0.837 76 rad; damping 2 N m s/rad, armature 0.01 kg m^2, friction loss 0.2 N m
Feet	spheres of radius 0.022 m at the calves' ends; friction 0.8 sliding, 0.02 torsional, 0.01 rolling; six contact dimensions; impedance (solimp) 0.015, 1, 0.022, 0.5, 2, softer than MuJoCo's default; priority over the ground's parameters
Other collision geometries	19 boxes, cylinders and spheres on the trunk and the legs; one contact dimension, friction 0.6
Solver	elliptic friction cones; ratio of frictional to normal impedance 100; Euler integration; MuJoCo's default time step, 2 ms
The book's changes	the 33 visual meshes deleted, the masses, inertias, centres of mass and collision geometries unchanged; a ground plane added; the time step 1 ms

S15.4 Provenance

The robot's model is Menagerie's, unchanged but for its visual meshes, and Menagerie's is derived from Unitree's description, not identified from a robot. The stance controller, the reduced model and its constants, the floors, the allowances, the funnel's bounds, the label's bound, the task, the modules and the runs are ours. The allowances and the funnel's bounds were set from the runs of Section 15.2 and are then checked on the same runs, the sweep and every plan executed unchanged. This is an in-sample check: it holds in the simulator, on the motions and pushes run, at Menagerie's contacts, and motions and pushes held out from the setting, or a robot, would have to repeat it. Like the mobile manipulator's, this is evidence about the implementation, not about a physical robot.

S15.5 The record

Table S15.4 is the record of the experiment of Section 15.2, after the record that `quad_governed.py` writes, `results/governed-record.txt`.

Table S15.2. The numbers of the control package `go2.stance` and the task (`quad/governed.py`, `quad/reduced.py`), and their status. From `results/governed-summary.txt`.

Quantity	Value	Status
The governor's period; the operation	0.2 s; 15 periods, the lift 60 mm, the touchdown's commanded descent 0.05 m/s	ours
The reduced model	the feet's places, home, $z_c = 0.2484$ m	declared, from the settled stance
The floors; the grid; C_3; the lift's demand	f3 and f4 20 mm; 5 mm; 60 mm, at least 52 mm for home; 80 mm	ours; the demand derived
The global capability	a push of 10 N on the trunk	ours
The funnel's bounds	the centre of mass 8 mm, FR 20 mm	ours, measured
The allowance	5 mm, and 3 mm per newton of push	ours, measured
The limits	acceleration 0.25 m/s^2; friction 0.5; torques counted 1.5 times	ours; friction declared
The touchdown	the ground under FR within 10 mm of where the schedule expects it, and the state in the funnel from FR's first contact until it carries its share	an assumption, Section 15.2.3
The rest	four feet, powered	an assumption, Section 15.2.5

Table S15.3. FR lowered onto the ground from the hold at the requirement's place, at four speeds, on Menagerie's soft feet and on MuJoCo's default impedance: the contact's speed; FR's largest force in the first 50 ms; the largest jump of the centre of pressure in that time; the centre of mass's largest deviation from its place in 0.2 s; and whether FR leaves the ground again within 0.1 s. FR's share on four feet is 17.2 N (17.1 on the stiffer contacts). From `results/governed-summary.txt`.

Contact	Speed (m/s)	FR's force (N)	Jump (mm)	Centre of mass (mm)	Leaves again
Menagerie's soft feet	0.050	0.2	0.2	0.2	no
	0.101	0.8	1.2	0.2	no
	0.201	0.9	1.8	0.4	yes
	0.389	0.5	3.9	0.6	yes
MuJoCo's default impedance	0.051	9.5	9.6	2.4	no
	0.101	18.0	23.0	2.4	no
	0.201	31.7	45.8	2.3	no
	0.388	25.1	40.3	2.2	yes

Table S15.4. The record of the experiment of Section 15.2, after `results/governed-record.txt`.

Item	Value
Model and asset provenance	MuJoCo Menagerie's `unitree_go2/go2.xml`, commit `f3b6013`, BSD-3-Clause, derived by Menagerie from Unitree's URDF; its visual meshes deleted, a ground plane added; the stance controller, the reduced model, the task, the modules and the runs ours
Simulator and solver	MuJoCo 3.16.0 with NumPy 2.5.3, SciPy 1.18.1 and CPython 3.13.17; `go2.xml`'s options: Euler, elliptic cones, a ratio of frictional to normal impedance of 100, the feet's soft contacts
Time steps	1 ms; the governor's period 0.2 s; 50 periods, after 3 s of settling
Actuators and sensors	the twelve torque motors of `go2.xml`, within 23.7, 23.7 and 45.43 N m; the stance controller at every step; the measurement the simulated state, exact
Frames and units	SI units; world x forward, y left, z up; the state's place along the shift line from home towards the incentre of the triangle FL, RL, RR
Uncertainty bounds	the labels' bound on a push on the trunk, 10 N; the funnel's bounds, the centre of mass 8 mm and FR 20 mm; the allowance 5 mm and 3 mm per newton of push
Constraints	f4, the zero-moment point inside the rectangle, at its floor of 20 mm; f3, inside the triangle, enlarged by $C_3 = 60$ mm while FR carries its share, floor 20 mm, grid 5 mm; the lift's demand 80 mm
Terminal behaviour	the rest on four feet, powered, a rest under the assumption that power and control last; after an alarm the rest feedback, which lowers FR at once, and a restart at rest; the shutdown construction run apart, not certified
Configurations	the ten runs of Table 15.2, the 72 runs of the sweep, the touchdowns of Table S15.3, and the shutdown
Training and evaluation split	none: no learned module
Seeds	none: no random numbers
Hardware	one x86-64 Linux machine, a 2.80 GHz Xeon
Guarantee status	retrospective simulation consistency: the reduced model's allowances and tracking bounds are set and checked in sample; universal containment, invariant rest and independent validation remain unestablished; touchdown requires a further contact argument, and standing assumes power and control; no physical robot

S15.6 The Apollo without the beam's reference following the pelvis

Of the three adaptations that one Apollo needs to carry the beam alone (Section 15.3.2), the third, the beam's reference following the pelvis sideways, is needed. Without it (`humanoid_carry.py --without-follow`), the left wrist's roll actuator reached its torque limit in the first loaded step, and the

right wrist's in the weight's shift after it. Eleven of the legs' 12 actuators reached theirs, one in the first loaded step and ten in the second. In the second, at 40.11 s, the beam was more than 0.15 m from the external impedance's compliant frame, which the repository's mission counts as the girder lost.

S15.7 An ungovened humanoid illustration

One humanoid, simulated. A run of a humanoid carrying a load shows how small that reserve can be. The Team-Impedance-Control repository (Shahna, 2026) simulates Apptronik Apollo humanoids, MuJoCo Menagerie's model of the robot (Zakka et al., 2022), that carry a girder together under a two-loop controller: an external and an internal impedance in the outer loops, with a QP that shares the load among the contacts, penalizing lightly a contact's wrench outside its cone shrunk by margins and heavily one outside the cone itself or torques beyond 85 % of the actuators' limits; and an inner loop that turns a fitted reference into the joints' efforts.

We ran its mission with one robot (`code/ch15/humanoid_carry.py`; Figure S15.1). The Apollo walks two gait cycles of 0.25 m to an aluminium beam with the cross-section of an HEA 200 (Stahlwerk Thüringen, 2026), 1.2 m long and 17.46 kg, below the 25 kg that it is reported to lift (Oitzman, 2023), puts its palms under the beam, takes its weight off two trestles, carries it four cycles, a metre, sets it on two others and steps back. What one robot needs that the repository's team does not have is ours: a station for one robot; the pelvis shifted over the stance foot before a loaded step as far as before an unloaded one, 95 % of the way, where the repository's team shifts a loaded robot's 25 %; and the beam's reference following the pelvis sideways. The third is needed: without it, the beam was more than 0.15 m from the external impedance's compliant frame at 40.11 s, in the second loaded step, which the repository's mission counts as the girder lost (Section S15.6).

With the beam's reference following the pelvis, the run completed in 91.06 s, with 14 lift-offs and 14 touchdowns, in a quasi-static gait. In single support the zero-moment point is the centre of pressure of the stance sole, 200 mm long and 85 mm wide (Figure S15.2). Unloaded, the point stayed at least 11.0 mm inside the sole's edges; carrying, at least 19.2 mm on the left foot. On the right foot it ran to the sole's front edge in the first loaded step: of the samples taken every 10 ms from 33.67 to 34.05 s, 34 found it within 0.1 mm of that edge, and the other five within 2.3 mm of the sole's edges, while the right ankle's pitch actuator used 96.2 % of its torque limit, the run's largest share, above the 85 % at which the QP's penalty begins. In the three loaded steps on that foot that followed, the point came within 3.9 to 4.9 mm of the sole's edges. The robot did

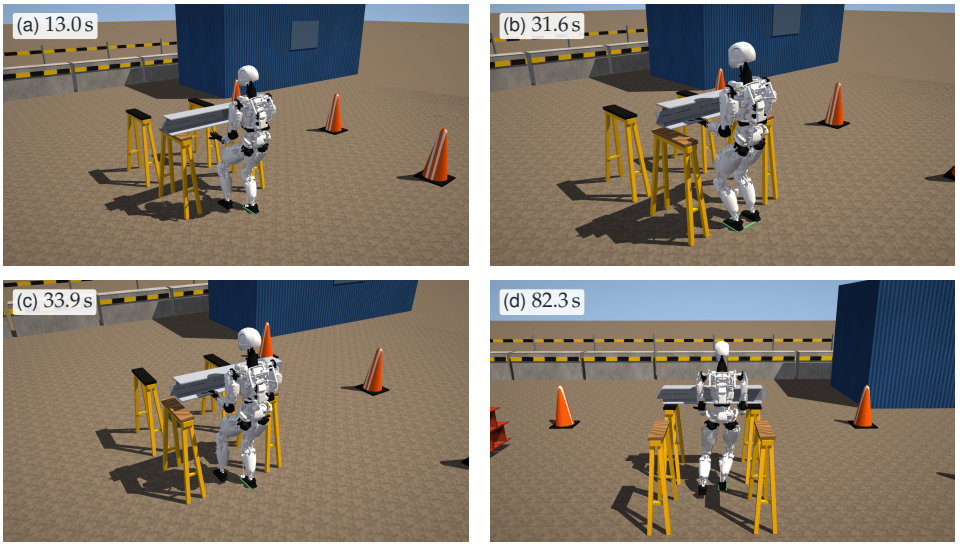


Figure S15.1. SIMULATION Four frames of a video of one Apptronik Apollo carrying an aluminium beam of the HEA 200 section, 1.2 m and 17.46 kg, under the two-loop controller of the Team-Impedance-Control repository, outside the governor: (a) walking to the beam, on its right foot; (b) the beam lifted off its trestles; (c) the first loaded step, on the right foot, the zero-moment point at the sole's front edge; (d) the beam set down on the other trestles, stepping back. Green: the soles that carry the robot; white: the zero-moment point, with a pole over it; orange: a ring round a foot after its touchdown. `humanoid_video.py` draws the record of `humanoid_carry.py` in the repository's scene; the drawing simulates nothing. Video: `https://github.com/Mehdi-Heydari-Shahna/Physical-AI-Together/releases/download/v1.0/apollo-carry.mp4`.

not fall. But a floor such as the Go2's, 20 mm, is more than the gait kept even unloaded; and carrying, at the edge, the reduced reserve with which a governor could certify such motions was none.

Every touchdown is a contact made: the soles touched at up to 0.060 m/s and carried up to 305 N within 50 ms. The run is an empirical demonstration, on the repository's model and contacts, with its load-sharing QP solved by SciPy's NNLS in place of daqp, the same problem (`code/ch15/humanoid/qp.py`). The repository's decoupling and decay certificates concern its model in a fixed mode, and its contact simulations provide, in its words, 'numerical evaluations rather than a certificate for the complete hybrid mission' (Shahna, 2026); this mission makes 28 contact transitions of the feet.

S15.8 Detailed claims ledger

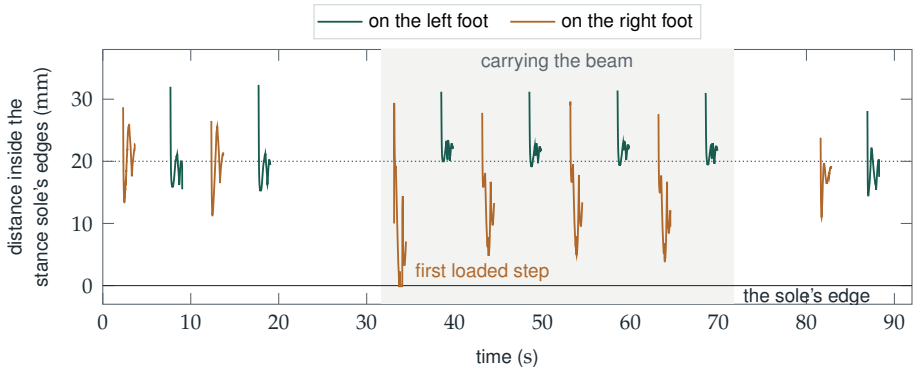


Figure S15.2. SIMULATION The margin of balance of the Apollo of Figure S15.1 over its mission: in single support, the distance of the zero-moment point, the stance sole's centre of pressure, inside the edges of the sole, 200 by 85 mm, every 10 ms, on the left foot (green) and on the right (amber); nothing in double support. Shaded: carrying the beam. Unloaded, the distance stays at least 11.0 mm; carrying, at least 19.2 mm on the left foot, while on the right it reaches the sole's front edge in the first loaded step and comes within 3.9 to 4.9 mm of the edges in the three loaded steps that follow. Dotted: 20 mm, the floor of the Go2's families. Data: `code/ch15/results/humanoid-trace.csv`.

Table S15.5. The claims of this chapter, their labels, what each rests on, and what would change the label. The conditional model claims do not establish a certificate for the simulated robot: the tracking bounds and allowances were set and checked in sample, and universal containment and rest invariance remain unestablished.

Claim	Label	Rests on	What would change it
The governor commits only plans that its test certifies, and prepares for the lift's demand	implementation behaviour and conditional lead-time calculation	the core of Chapter 10; the selection rule; Theorem 9.14 with Assumption 4.13, which the operation's first candidate meets by construction	–
While the labels are valid and the funnels contain the motion, the zero-moment point keeps the floor inside its polygon in the pairs before FR is lowered: the stance, the shift, the unloading, the lift and the hold	formal conditional claim, conditional on the measured bounds	the safety theorem of Chapter 9; the reduced model; the test; the tracking bounds and allowances, checked in sample	robust containment, rest invariance and independent validation
The same in the pairs from the lowering of FR on: the lowering, the touchdown and the reload	research extension; its runs an empirical demonstration	the touchdown's assumption, the band and containment from contact on, assumed (Section 15.2.3); runs at the band's edges and, apart, under the label's pushes, impacts to 0.093 m/s	a hybrid argument for the contact made (below)

Claim	Label	Rests on	What would change it
Lift-off of an unloaded foot	formal conditional claim, conditional on the measured bounds, as a stance pair's	the schedule unloading the foot before lifting it; the measured funnel	–
The simulated Go2 stays in the funnels, and its centre of pressure within the allowances	empirical demonstration	the reference, pushed, unannounced and band runs, 72 pushes, 98 plans executed unchanged, on Menagerie's soft feet, in sample	a bound proved for the controller and the contacts, runs held out, or measurements on a robot
The motors' limits as `go2.xml` states them	established for the static torques at the segments' ends, counted 1.5 times; the commands an empirical demonstration	the test's torque check, its factor 1.5 unmeasured; the runs	a bound on the commands within the funnel; for a real motor, labels or constraints from data
Standing as the plans' rest	an assumption of the package and of the operational design domain, that power and control last; its runs an empirical demonstration	runs of finite length, in sample	the battery and the motors' heat in the state; a proof of the rest's invariance
The shutdown	an empirical demonstration, one run without pushes; its certificate a research extension	one simulated run	a test for a changing height, a certificate for the uncontrolled contact, and the lying robot's constraints under the global capability
Dynamic locomotion, humanoids, Kangaroo	research extension	the Apollo's run, an empirical demonstration	the results below

CHAPTER S16

Comparisons that identify the contribution: records and details

This chapter holds what Chapter 16 moved out of its text: the record of the anticipation benchmark and of its runs, the paired statistics of Section 16.2.2 with the script that computes them, the diagnoses of the comparison with equal information world by world, the failed runs, and the counts behind the interaction of the substitution grid.

S16.1 The record of the anticipation benchmark

Table S16.1. The record of the anticipation benchmark, which follows the record that `cart_benchmark.py` writes, `results/benchmark-record.txt`.

Item	Record
Model and asset provenance	the cart of Chapter 3, its modules of Chapter 10 and the survey's calibration of Section 7.3.2; the benchmark's modules, the preview planner and the evaluator, ours
Simulator and solver	none: the motion in exact rational arithmetic; CPython 3.13.17
Time steps	the governor's period 0.1 s; 320 periods
Actuators and sensors	the command bound $2\,\mathrm{m/s^2}$; the measurement exact
Frames and units	SI units; the position along the track from the start
Uncertainty bounds	the global capability, braking of $0.5\,\mathrm{m/s^2}$; the survey's offset, 0.40 m, calibrated in Section 7.3.2
Constraints	$p + \ell \leq 25\,\mathrm{m}$, the floor 0.5 m; the operation's reserve 1.3 m
Terminal behaviour	rest at standstill, the brakes engaged; after an alarm, maximal braking and a restart from rest
Configurations	C1 to C15 of Table 16.1
Training and evaluation split	no learned module; the calibration worlds of Section 7.3.2, seeds 1001 to 1100, set the offset, and the benchmark's worlds serve nothing else
Seeds	the worlds 16001 to 16100, through `random.Random(seed)`
Hardware	x86_64, Linux
Guarantee status	A1 tests the safety theorem under its assumption, valid executed labels, as the oracle assesses them; the other hypotheses concern useful performance, measured and not guaranteed

S16.2 The runs of the benchmark and their timing

The script ran four times. The first decided the hypotheses; the reporting of each statement was then added, and the second and third runs repeated every value; the fourth followed the correction of the summary's heading for the data of Figure 16.2 (`HYPOTHESES.md`, Deviations). Every value that a run reported was the same in the runs that reported it, and the files of runs, failed runs and traces of the first, the third and the fourth run were the same byte for byte; those of the second were not kept.

The fourth run, whose times `benchmark-summary.txt` reports and Section 16.3.2 quotes, was made with the script's two worker processes (`--jobs 2`); in it, fixed conservatism had the slowest median and 99th percentile. In the third run, the largest decision took longer than the period of 100 ms in two configurations only, and the largest decision of ALLOVI with one candidate took 74.9 ms (`benchmark-summary-run3.txt`).

S16.3 When ALLOVI began the operation on time

Figure S16.1 places each valid world of the anticipation benchmark by its lead time and the slip of its true onset, and marks whether ALLOVI began the operation on time there; Section 16.3.1 explains the three groups of late runs.

S16.4 Paired statistics

Section 16.2.2 compares configurations of the anticipation benchmark world by world. Counts describe the fixed suite; p-values additionally require the independent-draw generator model discussed in Section 16.2.2. These statistics were computed after the initial analysis; they are not among the hypotheses and decision rules of Section 16.1. Listing S16.1 computes them from the shipped file of runs, with the standard library alone. In that file, `delay` is the number of periods by which the operation began after its true onset, empty if it never began, so that the operation is on time when the delay is 0, and `in_work_zone` is the number of periods of the run, of 0.1 s each, at whose start the cart's front is at or beyond the work zone's edge, 24 m (the evaluator of `bench/modules.py`). Whether a world is valid is a property of the world, the same in every configuration. An exact McNemar test is the two-sided binomial test, with probability one half, of the worlds in which only one configuration was on time; an exact sign test is the same test of the worlds in which a difference is positive or negative, the zeros left out. With no discordant world, as for C14 against C11, there is nothing to test, and the script prints $p = 1$.

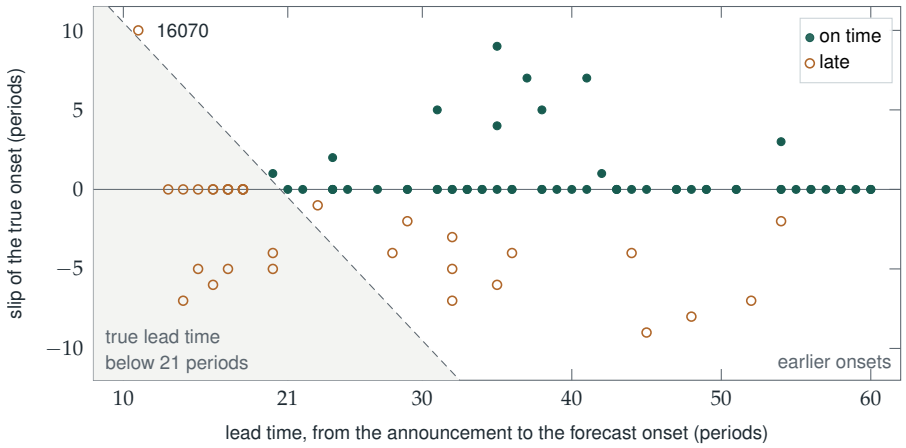


Figure S16.1. SIMULATION When ALLOVI (C6) began the operation on time in the 94 valid worlds: each world's lead time against the slip of the true onset, positive when the onset comes later than forecast. Filled: on time; open: late; worlds with the same lead time and slip coincide and have the same outcome. Shaded: a true onset fewer than 21 periods after the announcement, too soon for the back-away; ALLOVI and the oracle of the demand (C9) are late in exactly these 18 worlds. Lower right: the 13 earlier onsets, where ALLOVI begins at the forecast onset, late by the slip. World 16070 is the one late run that neither group contains (Section 16.2.2). Data: `code/ch16/results/benchmark-runs.csv`.

Listing S16.1. Paired statistics of the anticipation benchmark, computed after the initial analysis from `results/benchmark-runs.csv`; run in `code/ch16` with Python 3.8 or later.

```
# Paired statistics of the anticipation benchmark, computed after
# the initial analysis from the shipped file of runs. Run in
# code/ch16 with Python 3.8 or later; standard library only.
import collections
import csv
from math import comb

by = collections.defaultdict(dict)
for r in csv.DictReader(open("results/benchmark-runs.csv")):
    by[r["config"]][int(r["world"])] = r
valid = [w for w in sorted(by["C6"])
         if by["C6"][w]["valid"] == "yes"]


def exact(n, k):
    """Two-sided exact binomial test of k of n, probability 1/2."""
    if n == 0:
        return 1.0
    k = min(k, n - k)
```

```
    tail = sum(comb(n, i) for i in range(k + 1))
    return min(1.0, 2 * tail / 2**n)


def mcnemar(a, b):
    """Exact McNemar test of the operation on time (delay 0)."""
    on = {c: {w for w in valid if by[c][w]["delay"] == "0"}
          for c in (a, b)}
    only_a, only_b = on[a] - on[b], on[b] - on[a]
    p = exact(len(only_a) + len(only_b), len(only_a))
    print(f"{a} v {b}: both {len(on[a] & on[b])}, "
          f"neither {len(valid) - len(on[a] | on[b])}, "
          f"only {a} {len(only_a)}, "
          f"only {b} {len(only_b)} {sorted(only_b)}")
    print(f"  exact McNemar p = {p:.2g}")


def sign(a, b, column="in_work_zone"):
    """Exact sign test of the paired differences of a column."""
    d = [int(by[a][w][column]) - int(by[b][w][column])
         for w in valid]
    pos, neg = sum(x > 0 for x in d), sum(x < 0 for x in d)
    print(f"{column}, {a} - {b}: positive {pos}, "
          f"zero {len(d) - pos - neg}, negative {neg}")
    print(f"  exact sign test p = {exact(pos + neg, pos):.2g}; "
          f"mean {sum(d) / len(d):.2f} periods")


print("valid worlds:", len(valid))
for a, b in (("C6", "C11"), ("C14", "C11"), ("C6", "C12")):
    mcnemar(a, b)
sign("C6", "C11")
```

With the shipped file, it prints:

```
valid worlds: 94
C6 v C11: both 62, neither 31, only C6 0, only C11 1 [16070]
  exact McNemar p = 1
C14 v C11: both 63, neither 31, only C14 0, only C11 0 []
  exact McNemar p = 1
C6 v C12: both 3, neither 31, only C6 59, only C12 1 [16070]
  exact McNemar p = 1.1e-16
in_work_zone, C6 - C11: positive 20, zero 74, negative 0
  exact sign test p = 1.9e-06; mean 0.69 periods
```

S16.5 The differences under equal information

Table S16.2 sets every difference that A6 counted beside the mechanism that the diagnosis after the run found, by rerunning the worlds concerned (`cart_diagnosis.py`); Section 16.2.2 summarizes them. They are diagnoses, not results that the benchmark tested.

Table S16.2. The valid worlds in which the runs that A6 compares differ, what differs, and the mechanism that the diagnosis after the run found. Data: `results/benchmark-summary.txt` and `results/diagnosis-summary.txt`.

Comparison	Worlds	What differs	Mechanism, by the diagnosis
C11, the filter with the planner, against C6	20 of 94	the time in the work zone, longer with C6 in all 20; the operation's period in one	the planner's sufficient rule, 16 worlds; a lead time too short, 3; an onset later than forecast, one (16070)
C14, ALLOVI with the planner, against C11	8 of 94	two periods more in the work zone with C14	the governor stopped the retreat for one period beyond 23.7 m
C13, ALLOVI with the late planner, against C6	1 of 94	the operation's period, in 16070: C13 on time, C6 21 periods late	the late planner's retreat decided the selection where the requirement no longer could
C12, the filter with the late planner, against C6	46 of 46	with no slip and a lead time of 27 periods or more, C12 begins later, as A6 predicted	by design: the planner retreats six periods late

The planner's rule is sufficient, the governor's test exact. In the 16 worlds in which ALLOVI spends one or two periods longer in the work zone, the planner's cart began to back away one period earlier than ALLOVI's in 13 and two periods earlier in 3. At the start of the period in which the operation began, the planner's cart was at rest in all 16. ALLOVI's cart was at rest in one and still moving backwards in the others, its front at or before 23.7 m in all 16.

After a lead time too short. In the three worlds whose lead time, 13 or 14 periods, was too short for any preparation, both configurations postpone the operation by 21 periods; in world 16020, whose onset came early, this follows a first postponement to the forecast onset. The planner retreats once and completes its retreat, and its cart waits outside the work zone for the postponed onset. ALLOVI's governor prepares as far as the forecast onset allows; then the overdue requirement leaves the choice to the proposal 'hold', and the cart stops where it is, still in the work zone, until the postponed onset makes it back away again.

In worlds 16002, 16020 and 16083, ALLOVI's cart rested at 24.0075, 24.0087 and 24.0000 m for seven periods each, while the planner's waited at 23.695, 23.696 and 23.650 m. This is the selection rule doing what it was designed to do, following the proposal for as long as the requirement can still be met later.

An onset later than forecast. In world 16070, ALLOVI was not ready at the forecast onset, and its requirement then applied from the first pair on. The cart stopped at 24.20 m, in the work zone, and held there from period 205 to period 218, through the true onset in period 212. There the plan in force held 0.80 m, and the operation was postponed by 21 periods, to period 233. This is the one world in which equal information favoured the filter. The mechanism, as the diagnosis found it, is a property of an overdue requirement, which cannot steer a choice within a period, combined with a proposal method that asks to hold.

The exact planner under ALLOVI. In the 8 worlds in which the exact planner stayed two periods longer in the work zone under ALLOVI than under the filter, the cart first moved backwards in the same period under both; under ALLOVI it then came to rest for one period beyond 23.7 m and backed away again, which under the filter it never did. The decision records show why. The governor committed the planner's first period of retreat, followed, as every candidate is, by the generator's recovery continuation, which brakes every motion to rest before it backs away. In the next period, a candidate that continued the retreat would have come to rest one period later, and its back-away would have held less than the required reserve at the onset. The candidates that braked first held it, so the governor committed one of them. The cart stopped and backed away on the governor's schedule: with its front at or before 23.7 m by the onset, the governor's requirement, rather than at rest there, the planner's. Why the stop came in these 8 of the 16 worlds in which the planner's rule made it leave the work zone one or two periods earlier than ALLOVI with the naive method, and not in the other 8, was not established.

S16.6 The failed runs

`results/benchmark-failures.csv` lists every failed run of the anticipation benchmark (Section 16.3.3). The task fails in every run of the filter (100 of 100 in each of C1 to C3, the operation never begun), of fixed conservatism (100, never in the work zone, and in two worlds that are not valid never begun either) and of reactive preparation (99 late and one never begun). In the other configurations, the operation began late or never in 35 runs of ALLOVI, 35 without the capability forecast and 35 with the oracle of the capability; in 21 with the oracle of the demand and 20 with both oracles; in 34 of the filter with the planner and 95 with the late planner; in 34 of ALLOVI with either

planner; and in 35 with one candidate per period. The valid worlds among these are those of Table 16.5; the others are the worlds that are not valid. With the runs that executed an invalid label and began on time, the failed runs of these configurations, C6 to C15, number 37, 35, 35, 23, 20, 36, 96, 36, 36 and 35.

Table S16.3 gives the six worlds whose survey placed the ramp's start beyond the calibrated offset. In worlds 16042 and 16059, the eight survey configurations other than fixed conservatism and ALLOVI with one candidate per period each executed one invalid label. In 16064, these eight and ALLOVI with one candidate did; in 16077, all ten survey configurations did; and in 16043 and 16100, only fixed conservatism did. In every one of the 37, the executed segment commanded more braking than the ground allowed somewhere on it.

Table S16.3. The six worlds whose survey placed the ramp's start beyond the calibrated offset: the survey's error, and, of the ten configurations that use the survey, those that executed an invalid label, raised an alarm and restarted, and the least true clearance among them, rounded down to the millimetre. Data: `results/diagnosis-summary.txt`.

World	Survey error	Invalid executed label, of 10	Alarms	Restarts	Least true clearance
16042	0.42 m	8	8	8	0.500 m
16043	0.50 m	1	1	1	1.250 m
16059	0.50 m	8	8	8	0.528 m
16064	0.44 m	9	8	8	0.500 m
16077	0.54 m	10	10	2	0.450 m
16100	0.49 m	1	1	1	1.200 m
All six		37	36	28	

S16.7 The interaction in the substitution grid

The diagnosis of Section 16.2.4 reran the braking curve and the expert planner, which need no training, in the 20 evaluation worlds with each package (`results/diagnosis-summary.txt`). With the contraction-metric package, 755 of the braking curve's 1876 periods before arrival and 792 of the expert planner's 1921 asked on the ramp for the declared braking, rounded to 0.399 $\mathrm{m/s^2}$, and the governor committed none of them in full. The MPC package, which declares the same braking but whose head is certified for more, committed 586 of the braking curve's 729 such requests in full, and 689 of the expert planner's 764.

CHAPTER S17

How readers add their own research: records and details

This chapter holds what Chapter 17 moved out of its text: the helpers that its two worked contributions reuse beyond the templates, the documented evaluation tasks, the detail of the diagnoses of the failed hypotheses and of the worlds that are not valid, and the record of the reproductions that the author made.

S17.1 The helpers that the two contributions reuse

The two contributions of Section 17.1.2 reuse code beyond the interfaces of the templates. The demand predictor reads two constants of the cart's model, `OPERATION_LEVEL` and `ce.EPS_MIN`, and the schedule of the cart's task, which the configuration passes to it. The control package extends the benchmark's package and calls helpers of the cart's modules: the private class `cart.control._Labels`, and the plans, constants and functions of the example of Chapter 3 that `cart.model` exposes as `ce`. Following the path showed that the templates did not document these helpers, which a reader outside the team would need. `code/ch10/templates/HELPERS.md` documents each of them, with its units, its invariants and an example that runs as a test, and shows how to run and check a module in the cart's configurations.

Following the path with a module of one's own. The two contributions serve as templates for steps 6 to 9 of Figure S17.1, as the templates of Chapter 10 do for steps 2 to 5. A reader's comparison can start from a copy of `contribute.py`. Its function `benchmark_configuration` builds C6 with one module replaced, its function `run` runs one configuration in one world, and its function `hypotheses` applies each decision rule through a small class, `Rule`, that counts the cases of a statement and its exceptions. The hypotheses go into a file such as `HYPOTHESES.md`, dated, before the first run, and the seeds must be fresh, drawn by `code/ch16/bench/worlds.py`. The script's table of expected values, filled from the first run, then lets `--check` confirm every value of a second run. The method card starts from `code/ch17/cards/TEMPLATE.md`, with the two filled cards beside it as examples.

S17.2 Documented evaluation tasks

Table S17.1 lists the tasks of the book that are documented well enough for a contribution to be evaluated on them (Section 17.2.3): each has the rules that draw its worlds, its seeds, a record, and a script whose `--check` option verifies every value of its summary.

Table S17.1. Documented evaluation tasks of this book: what each tests, how its worlds are drawn, and the script that runs it. The seeds are those of the scripts.

Task	What it tests	Worlds	Script
The cart's approach and operation (Chapter 16)	anticipation, capability and demand forecasts, proposal methods, control packages for the cart	`code/ch16/bench/worlds.py`; seeds 16001 to 16100, and 16101 to 16200 in Section 17.1.2	`code/ch16/cart_benchmark.py`; `code/ch17/contribute.py`
The disturbed cart (Chapter 6)	proposal methods and learners, crossed with control packages	evaluation worlds 1000 to 1019; training seeds 1 to 3	`code/ch06/cart_learning.py`; `code/ch16/cart_substitution.py`
The survey's coverage (Section 7.3)	calibrated capability forecasts	calibration seeds 1001 to 1100; held-out 2001 to 4000	`code/ch07/cart_forecasts.py`
The cart in four simulators (Chapters 11 and 12)	simulator adapters and ports	one world; the support matrix's releases	`code/ch11`, `code/ch12`
The Husky's yard (Chapter 13)	a wheeled robot's modules and their substitutions	the yard and its shower; twenty seeds of the gyro	`code/ch13`
The heavy machine's yard (Chapter 14)	a working machine's labels, payload and demand	the yard of Section 14.3.1	`code/ch14`
The Go2's lift of a foot (Chapter 15)	a legged robot's certificate and its boundary	the reference and its variants	`code/ch15`

S17.3 The diagnoses of F4 and E4

F4, the full reverse. F4 did not hold in any of its 58 worlds. The selection compares the candidates first by the lesser of the requirement and the level that each holds from the onset on, then by ambition, and breaks the remaining ties in

favour of a fresh candidate and then of the smallest index (Equation (4.2)). The candidates that follow the proposal end, as every blend does, in the generator's own recovery continuation, whose back-away needs 21 periods. They therefore stop holding the requirement in the same period as under C6, and in all 58 worlds the governor then commits an ambition below 1 in the same period as under C6. From the next period on, it committed the full reverse, for 14 periods in each world. But the full reverse begins with the same command as the generator's back-away, the strongest that the ramp certifies, and in all 58 worlds the executed commands were those of C6 until the operation began. A candidate of ambition 0 is committed only when no candidate of higher ambition holds as much of the requirement, that is, once the candidates that follow the proposal no longer hold it; to delay the preparation, the shorter back-away would have to be the continuation of every blend, which is a different package and a new hypothesis.

E4, the early window. E4 predicted that a window which cannot be met makes the early window late in every world without a slip whose lead time is 21 to 30 periods; it was late in 7 of the 11, and on time, as C6 was, in four, with lead times of 27, 28, 27 and 30 periods. With a lead time of 27 periods or more, the window's onset, 10 periods before the forecast one, came after the period of commitment and the back-away's 16 periods of acceleration. The requirement then applied from the first pair on, the first key tied, and the proposal 'hold' decided, as E4 expected; but holding meant braking, which the back-away was about to do. The cart came to rest at or before 23.7 m and began the operation on time. With 21 to 26 periods, the tie came while the cart was still accelerating: it braked to rest beyond 23.7 m, waited there through the onset, and was postponed once or twice. In this diagnosis, the mechanism of world 16070 was at work in all 11 worlds, and E4 erred in its cost, which came only where the tie came before the back-away began to brake. In these runs, the lead times at which the window cost the operation ended between 26 and 27 periods, inside the band of 21 to 30 that E4 named. Two more worlds, with lead times of 22 and 21 periods and onsets two periods and one period later than forecast, outside E4's subset, went the same way: the early window was postponed twice where C6 was on time.

S17.4 The worlds that are not valid

In the five worlds of the family 16101 to 16200 that are not valid, each of the three configurations of Table S17.3 executed an invalid label, and each of these runs raised an alarm. Two of them restarted, in worlds 16107 and 16197, whose surveys erred by 0.47 and 0.57 m. In the other three, 16120, 16116

and 16144, whose surveys erred by 0.44, 0.50 and 0.59 m, no restart could be certified; the true clearance fell to 0.48 or 0.49 m, below the floor but without a violation, and the operation never began, in all three configurations alike (`code/ch17/results/contribution-diagnosis.txt`).

S17.5 The reproductions made by the author

In the author-reported reproduction of the earlier source snapshot, every check of the repository's package that needs neither a graphics processor nor Newton or Genesis was run again from a fresh copy of its archive. The chapters that need Python alone ran with CPython 3.12.3, another release than those of the records, and wrote the same files as shipped, byte for byte, except for the lines that name the interpreter, the wall-clock times of the decisions of Chapter 16, and a code hash in the report of Section S12.8, which named an earlier version of the code and has been corrected: the second degree. The chapters in MuJoCo ran in the environment of their records, the three scripts of Chapter 13, the governed run of Chapter 15 and the checks of Section 12.3 on another processor than their records name, which is the third degree for them. Every computed value was as expected. One check, of a variant of Section 12.2, still failed, because it also compares the release of Python that its record names, 3.13.16 against 3.13.17. The variant with a renderer did not run, because that machine lacked PyOpenGL and OSMesa. Once the code of that edition was final, the checks of Chapters 3, 10, 14 and 17, the reports of Chapter 12 and the validation of Section 6.2.4 ran once more from a fresh copy of its archive and wrote the same files as shipped, but for the validation's running times; in the other chapters, only comments and documentation had changed since the first run.

Every script checks every value that the text quotes from its summary, which holds most of them; the few counted from a data file, and wall-clock times, are not checked. Four runs of the anticipation benchmark repeated every value that each reported (Section S16.2), and a second run of the substitution grid and of each script of Section 17.1.2 repeated every value of the first. The run of Chapter 3 in MuJoCo was reproduced from its record in a new environment, with another release of NumPy and on another processor, all its quoted values as quoted and its file of periods identical byte for byte, and every configuration of the support matrix, rerun on that processor, reproduced its results (Section S12.8). The previous version of the governed run of Chapter 14 ran on two machines, and the present version, which writes its data files unchanged, on one. All of this was done by the author.

S17.6 Detailed contribution records

The material below records the implementation paths and author-reported runs behind Chapter 17. Commands and release-specific details are companion material; their preservation does not turn the local protocols into externally timestamped preregistrations.

S17.7 Procedural record: Contribution paths

A contribution enters ALLOVI through one module, and the module decides what the contribution must establish and what it can claim. This section gives the path for each of the five kinds of module that a reader can change, and follows two of them to the end.

Five paths

Every path begins with the four steps of Section S10.10.4: copy the template of the role, register the module, write down how each obligation of the role is established, and run the check suite with the module in its role. It ends with hypotheses recorded before evaluation, a comparison with a configuration that differs in the contributed module only, in worlds that did not suggest the hypotheses, and a report of every outcome, the failures included, with a method card (Section 17.2.2). Figure S17.1 sets out the ten steps, with the file or command of the book's code that each uses; Table 17.1 gives, for each role, what the contribution changes and supplies, the evidence it needs and what it can claim.

The paths differ most in what a reader may claim. A proposal method or a demand forecast cannot weaken a guarantee, because neither enters the test, so it claims performance and needs no proof; the burden is a fair comparison. A control method or a capability forecast enters the test, so its obligations come first, and a passed check discharges none of them (Section S10.10.1). A governor variant changes the steps on which the proofs of Chapter 9 rest, so its guarantees are its own to prove (Section 17.1.3). The book has worked four of the five paths: a proposal method and a capability forecast, the eager method and the survey predictor with a margin, through the first four steps only, the evidence that each kind needs being given for other modules (Section 16.2.4 and Chapter 7); a control method and a demand forecast to the end, below. No variant of the governor was implemented, because its proofs must come first.

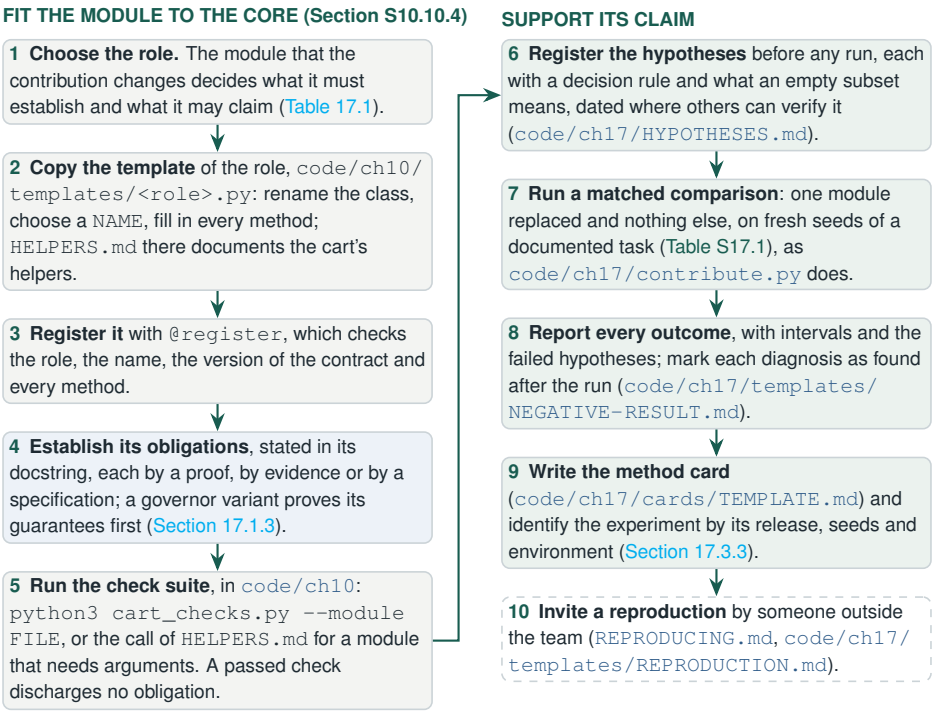


Figure S17.1. ILLUSTRATION The path of a contribution, with the files and commands of the book's code at each step. Steps 1 to 5 fit the module to the core: the four steps of Section S10.10.4, preceded by the choice of the role; steps 6 to 10 support what the contribution claims. Only step 4 depends on the role (Table 17.1). The two contributions of Section 17.1.2 took steps 1 to 9 with two shortfalls that Table 17.2 records: their hypotheses carry no verifiable timestamp and no rule for an empty subset, and the code's release, v1.0, has no archive identifier yet. No module of the book has taken step 10.

Two contributions, worked to the end

The *full reverse*, `reader.full-reverse`, a control method, is the control package of Chapter 16 with one more candidate, of ambition 0, whenever a requirement is pending: brake a forward motion to rest, drive towards the start at the strongest command that the labels certify until the front is at or before the requirement's place, the position 25 m less the required reserve, and brake to rest. Section 9.3.2 showed that a generator that offers it meets the operation with 19 periods of lead time instead of 21. The *early window*, `reader.early-window`, a demand forecast, is the task plan's demand predictor with the requirement's onset moved 10 periods before the forecast onset, the earliest that the task plan says the operation may come. By Corollary 9.15, a reserve held at an onset announced early is kept until the operation is due: a remedy for the 13 worlds

of Section 16.3.1 whose onset came earlier than forecast after a lead time of 21 periods or more.

Both modules, in `code/ch17/contrib`, were written for this chapter by the author against the core's public interface, and both reuse helpers of the cart's code beyond the templates, one of them a private class. Following the path showed that the templates did not document these helpers, and `code/ch10/templates/HELPERS.md` now documents each, with an example that runs as a test (Section S17.1). The hypotheses were recorded in `code/ch17/HYPOTHESES.md`, reportedly before the first run of `contribute.py`, which applies each decision rule as written there; as in Chapter 16, the files alone cannot prove that order, and the current file includes later editorial references; these do not establish an independently timestamped pre-run version (Section 17.3.3). `contribute_diagnosis.py`, written after that run, examines the hypotheses that failed and the effects that no hypothesis stated, and everything that it reports is a diagnosis. Post-run edits and diagnoses are described by the author; the local files alone do not independently verify the pre-run text.

Each contribution replaces one module of C6, ALLOVI as Chapter 16 configured it, and nothing else, in a family of 100 worlds that no earlier chapter used, seeds 16101 to 16200, drawn by the rules of Section 16.1.2; 95 of them are valid. Their lead time is 31 periods or more in 66 of the valid worlds, 21 to 30 in 14, 19 or 20 in one and 18 or fewer in 14; in 25 the onset slips, later than forecast 12 times and earlier 13 times.

The path, step by step. Steps 1 to 5 of Figure S17.1 went as they should, but step 5 needed a call of its own. Both modules registered, and both passed all 13 checks of the suite in the configuration of the forecasting run of Chapter 3. The command of step 5, `cart_checks.py --module`, cannot take either file: it creates each module without arguments, but the demand predictor needs the task, and it does not find the benchmark's package, which the control package's file imports; `HELPERS.md` gives the call that both need. The full reverse adds candidates and no verdicts, so the test, the rest and its feedback are inherited unchanged from the package of Chapter 16, whose test is exact; the early window carries no obligation, because a requirement never enters the test. Neither claims a new guarantee; both claim performance, in the hypotheses of Table S17.2.

The full reverse. In the world of Chapter 3, the full reverse began the operation on time when it was announced 21, 20 or 19 periods before its onset, and not with 18; C6 only with 21. This is the lead time that Section 9.3.2 computed, now from a registered module, and the only case of F2: because that world suggested the result, the case repeats it rather than testing it. The family's one valid world with a lead time of 19 or 20 periods, world 16179 with 20, has an onset that slips,

Table S17.2. The hypotheses of the two contributions as `HYPOTHESES.md` records them, in short: the cases to which each decision rule applied, the exceptions and the verdict. A rule supports a hypothesis only if it holds without exception in its subset; a statement whose subset is empty is untested, which applies to the second statement of F2. Data: `code/ch17/results/contribution-summary.txt`.

Id	Hypothesis	Cases	Exceptions	Verdict
F1	With valid executed labels, no alarm and the floor kept	95	none	supported
F2	On time with 19 periods of lead time and not with 18, in the world of Chapter 3; on time with 19 or 20, without a slip, where C6 is not	1	none	supported by its rule; in substance untested
F3	Never later than C6, and the same arrival	95	none	supported
F4	Without a slip and with 21 periods or more: both on time, and one or two periods more in the work zone	58	58	not supported
E1	With valid executed labels, no alarm and the floor kept	95	none	supported
E2	An onset 1 to 10 periods earlier, with 31 periods or more: on time, where C6 is not	11	none	supported
E3	Without a slip and with 31 periods or more: both on time, exactly 10 periods fewer in the work zone	47	none	supported
E4	Without a slip and with 21 to 30 periods: later than C6, which is on time	11	4	not supported
E5	The same arrival as C6 in every world	100	none	supported

so the second statement of F2 was tested in no world. F2 is therefore supported by its rule, as written, and untested in substance; a rule should say what an empty subset means. In world 16179, whose onset came six periods early, both configurations missed the true onset; the full reverse began the operation at the forecast onset, late by the slip, where C6 was postponed again by 21 periods, the one world of the 95 in which the two differ in the operation's period. F1 and F3 held. F4 failed in all 58 of its worlds: the full reverse spent exactly as many periods in the work zone as C6. By the diagnosis, the candidates that follow the proposal end, as every blend does, in the generator's own recovery continuation, whose back-away needs 21 periods, so they stop holding the requirement in the same period as under C6, and the full reverse, committed from the next period on, begins with the same command as the generator's back-away, so that the executed commands were those of C6 until the operation began. To delay the preparation, the shorter back-away would have to be the continuation of every blend, a different package and a new hypothesis (Section S17.3).

The early window. The early window met E1, E2, E3 and E5: in all 11 worlds with an earlier onset and a lead time of 31 periods or more, it began on time where C6 was late, at the cost of exactly ten periods in the work zone in the 47 worlds without a slip and with that lead time, and the approach was unchanged in all 100. E4 predicted, from the mechanism found in world 16070, that a window which cannot be met makes the early window late in every world without a slip whose lead time is 21 to 30 periods; it was late in 7 of the 11, and on time, as C6 was, in four, with lead times of 27, 28, 27 and 30 periods. By the diagnosis, the tie of an overdue requirement was at work in all 11, and its cost depended on where the cart first came to rest (Figure S17.2). With a lead time of 27 periods or more, the window's onset came after the period of commitment and the back-away's 16 periods of acceleration, so that holding meant braking and the cart came to rest at or before 23.7 m; with 21 to 26, the tie came while the cart was still accelerating, and it braked to rest beyond 23.7 m and waited there through the onset. That the boundary lies between 26 and 27 periods in other worlds is a new hypothesis.

What no hypothesis stated was the largest effect, found by reading every outcome after the run. In all 15 valid worlds whose lead time was 20 periods or fewer, the early window began the operation later than C6, and in the 12 of them without a slip it was postponed three times where C6 was postponed once. The task plan announces a postponed onset 21 periods ahead, enough for a back-away from rest; the early window moves the requirement 10 periods earlier, to 11 periods after the postponement, which is too few, and the same tie stops the cart again beyond 23.7 m. Over the valid worlds, the early window began the operation on time in 71, against C6's 69, earlier than C6 in 11, later in 24 and in the same period in 60; its 24 late runs were late by a median of 6.3 s, against 2.1 s for C6's 26. A predictor that moved only the first forecast would avoid this, which is a new hypothesis, untested here.

What the two paths show. Neither contribution changed a guarantee, as their paths predicted (Table S17.3). Every run whose executed labels were valid kept the floor without an alarm, and in each configuration the five runs that executed an invalid label, in the five worlds that are not valid, raised an alarm, and two of them restarted. In the other three worlds, whose surveys erred by 0.44, 0.50 and 0.59 m, no restart could be certified, the true clearance fell to 0.48 or 0.49 m, below the floor but without a violation, and the operation never began, in all three configurations alike (Section S17.4). The full reverse helped in the one world whose lead time it was designed for; the early window helped where it was meant to and cost more than its hypotheses foresaw where it was not. Both contributions were made by the author, with the book's code at hand, so they test the path, not the documentation.

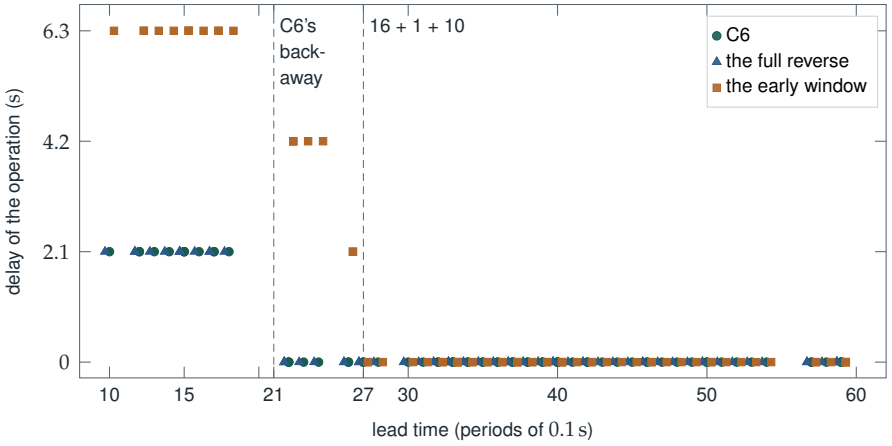


Figure S17.2. SIMULATION The delay with which the operation begins against the lead time, in the 70 valid worlds of the family 16101 to 16200 whose onset does not slip. C6 is late with 18 periods or fewer and on time with 22 or more, the smallest such lead time above 18 in this family; its back-away needs 21 (Section 9.3.2). The full reverse coincides with C6 in every one of these worlds. The early window is on time with 27 periods or more, which the diagnosis after the run explains: one period of commitment, the back-away's 16 periods of acceleration and the window of 10. With 21 to 26 periods it is postponed once or twice where C6 is on time, and with 18 or fewer, three times where C6 is postponed once. Marks of the contributions are offset sideways by 0.3 period. Data: `code/ch17/results/contribution-leads.csv`.

Table S17.3. The two contributions beside C6 over the 95 valid worlds of the family 16101 to 16200: the operation on time, as a count out of 95; the late runs and their median delay; and the finite-suite medians of arrival and time in the work zone. These summaries describe the fixed seeded suite, as in Section 16.3.1. Over all 100 worlds: runs with an invalid executed label, alarms, restarts and violations. Data: `code/ch17/results/contribution-summary.txt`.

Configuration	On time, of 95	Late; delay, s	Arrival, s	In the work zone, s	Invalid; alarms; restarts; violations
C6, ALLOVI	69	26; 2.1	8.9	13.0	5; 5; 2; 0
The full reverse	69	26; 2.1	8.9	13.0	5; 5; 2; 0
The early window	71	24; 6.3	8.9	12.2	5; 5; 2; 0

A recorded hypothesis can fail, and the failure belongs in the report. A diagnosis found after evaluation is an exploratory explanation. A claim suggested by it needs a prospectively timestamped protocol and fresh worlds for confirmatory testing; an edit to the old protocol cannot supply that history.

A variant of the governor

A variant of the governor changes a step that every application shares: admission, selection, the fallback or commitment. Every guarantee of Chapter 9 rests on these steps, so a variant inherits none of them. Its path begins with the proofs: it states what it changes, proves each guarantee that it claims, under the assumptions of Assumption 9.1 or stated others, and gives a counterexample for each that it gives up; only then come the check suite (Section S10.10.2) and the comparisons. Two variants suggested by this book are research extensions in the sense of Section 15.3.1. A selection that broke the tie of Section 16.2.2, for instance by preferring, among the candidates that tie on the first key, the one that holds the requirement from the earliest pair, might keep a robot preparing where the proposal asks it to hold, in every application at once; whether Theorem 9.14 and Corollary 9.15, whose proofs compare the first key of the committed plan with the fallback's, still hold must be proved again. A governor that coordinates several robots, with plans that depend on one another, lies outside the results altogether (Section 9.3.6).

S17.8 Procedural record: A contribution checklist

A contribution is usable by others only if what they need to use it, check it and compare with it is written down. This section collects those things into a checklist and a method card, and names the documented tasks on which a contribution can be evaluated.

What every contribution states

The checklist of Table 17.2, in the book, lists the seven items: what to state, where this book states it for its own modules, and how a reader checks it. They are what a reader needs from any method that is meant to be combined with others; what the contract adds is a fixed list of obligations for each role, so that a reader knows in advance which ones a contribution must address.

Two items deserve a warning. The semantics include the resolution of every number that a module rounds: the interaction of Section 16.2.4 came from requests rounded past the braking that a package declared. And the measured limitations include the hypotheses that failed, with their runs, which a report is

tempted to leave out; Table 16.7 keeps them, and a contribution's report should too.

The method card

A method card puts the seven items in one place, in a fixed order. The template, `code/ch17/cards/TEMPLATE.md`, has twelve fields: identity, what the module does, semantics, obligations, assumptions and scope, a runnable example, evidence, measured limitations, reproducibility, provenance, status, and how to cite it. The status gives each claim one of the three labels of Section 15.3.1, an established certificate, an empirical demonstration or a research extension, and says whether the results have been reproduced outside the team that wrote the module. The two contributions of Section 17.1.2 have filled cards in the same folder, and Exercise 17.1 asks for one more. A card is written for a reader who has the book's code and nothing else.

Open, documented evaluation tasks

A contribution is evaluated on a task that its readers can run too. Seven tasks of this book are documented well enough for that; the online supplement lists them with their worlds and scripts (Table S17.1). Each has the rules that draw its worlds, its seeds, a record, and a script whose `--check` option verifies every value of its summary; the few values that the book counts from a data file, and the wall-clock times, are not checked. The tasks are open in that every rule, seed and script is in the book's code, and open in the legal sense: release v1.0 of the code is under the MIT License (Section 17.3.3). A new contribution should use fresh seeds, drawn by the same rules, for any claim whose hypotheses were written after seeing results on the book's seeds, as Section 17.1.2 does for every statement but the first of F2.

S17.9 Procedural record: Reproductions, negative results and releases

A contribution becomes part of a shared body of work when others can reproduce it, when its failures are recorded as carefully as its successes, and when the experiment that produced it can still be found and run after the software has changed. This section gives a form to each of the three.

Independent reproductions

A reproduction obtains the results of an experiment again from its record and its code alone; in its strongest form, it is made by someone who did not

produce them. It tests the record and the code, not the claims: a reproduction of a simulated result in the same simulator confirms the computation, not the physics, and a replication, the same hypothesis tested on new worlds or another robot, is a different and stronger test. Reproductions come in degrees, and a report should say which it reached:

1. the script's own check, rerun where it was run;
2. a run in a new environment built from the record's pins;
3. a run on another machine or operating system, within the variation that the record allows (Section S12.8);
4. a run by someone outside the team that produced the results.

The author reached the first degree for every script whose results the book reports; the second for the run of Chapter 3 in MuJoCo and, from a fresh copy of the code's archive with another release of CPython, for the chapters that need Python alone; and the third, on another processor under the same operating system, for that run in MuJoCo, the configurations of the support matrix, the three scripts of Chapter 13, the governed run of Chapter 15, the checks of Section 12.3 and the previous version of the governed run of Chapter 14. Section S17.5 records these runs and what differed in each.

No independent reproduction exists yet: no one outside the team has reproduced a result of this book or contributed a module to it, so neither the fourth degree nor the chapter output that the plan of the book sets has been reached. The repository holds a kit for the first person to try. At its top, `REPRODUCING.md` gives each chapter's environments, commands, running times and the line that each check prints; `CONTRIBUTING.md` takes a module of one's own along the path of Figure S17.1 to a pull request; and two forms of the issue tracker, `.github/ISSUE_TEMPLATE/reproduction-report.md` and `.github/ISSUE_TEMPLATE/contribution.md`, ask for the fields of a reproduction report and of the checklist of Section 17.2. The kit has been tried only by the author; release v1.0, under the MIT License, lets others use it (Section 17.3.3). No outside reproduction report is available.

`code/ch17/templates/REPRODUCTION.md` gives the form of a reproduction report: the experiment and its release; who reproduced it, whether from outside the team, and the degree reached; the environment, the commands and the line that each check printed; the values compared, every difference with its cause, and the verdict; and what was not reproduced, and why. A failed reproduction is reported in the same form, and is as useful: it shows what the record left out.

Negative results

A author-recorded hypothesis that its rule does not support is a result. Left unreported, it is found again by the next researcher, at the same cost, and the reports that remain overstate what works. This book reports its own: A5, A6 and B2 to B4 of Chapter 16, and F4 and E4 of the two worked contributions. `code/ch17/templates/NEGATIVE-RESULT.md` gives the form: the hypothesis word for word, with its rule, where and when it was recorded; the configurations, the family of worlds and the code; the decision, with every exception and the statements that did hold; the diagnosis, marked as found after the run; and what follows. The diagnosis is the part most easily misread: it becomes a result only when a hypothesis that it suggests is prospectively fixed and tested on worlds that did not suggest it, as Section 17.1.2 does with a diagnosis of Section 16.2.2, which, tested as E4 on new worlds, predicted the outcome in 7 of 11.

Releases, citations and deprecations

Software changes, and an experiment run with one release may not run, or not give the same values, with the next. `code/RELEASES.md` states how the code of this book keeps experiments identifiable.

What identifies an experiment. An experiment is identified by the release that contains its script and the command that ran it; the version of the contract and the name and contract of every module of its configurations; the seeds of its worlds; the environment, with every package and its version, pinned wherever the script needs packages; and the summary file that its check verifies. The records that the scripts write collect these items (Table S16.1), and experiments that depend on each other are tied by comparing their files byte for byte. The current protocol files include post-run edits and no external timestamp; their current contents do not establish a pre-run version. Preserve an immutable, externally timestamped protocol before a new confirmatory run.

Versions and deprecations. A change to the methods of a role, to what they receive or return, or to their meaning changes the major version of the contract, and the registry refuses a module with another major version (Section S10.9.5); an addition that leaves every module valid changes the minor version. A module's name is never reused for a different behaviour. A later release adds results beside the earlier ones, and a corrected result stays in the release that produced it, with the correction saying what changed and why. A module or script to be removed is marked deprecated, with the release in which it will be removed, and stays in at least that release and the next.

A verifiable release. Others can verify a release only if it carries five things: a licence, which says on what terms the code and its data may be used; a tagged version, which names exactly the code that produced the results; an archive of that version with a persistent identifier, which keeps it available and citable if the repository moves; checksums of the archived files, with which a reader confirms that a copy is the archived one; and continuous integration that runs the checks that need only a CPU, so that every change is known to leave the shipped results as they are. Release v1.0 of the book's code carries the first two: the MIT License, which `LICENSE` states for the code and its results (the third-party descriptions that it includes keep their own), and the tag v1.0, which `code/RELEASES.md` lists. It has no archive identifier yet, so the book's citation metadata gives none; no checksums are published beyond git's own hashes of its files; and no continuous integration runs the checks. The book describes the following release material: the `--check` of every script, the records with their pins, the shipped results and the kit of Section 17.3.1.

Citations. A result is cited by the release that produced it, with its archive identifier, and a contributed module also by its method card. `CITATION.cff`, at the top of the repository (`https://github.com/Mehdi-Heydari-Shahna/Physical-AI-Together`), gives the citation of the book and its code.

> **A contribution is complete when another researcher can run it, check every value of its report and compare with it without asking its author: its semantics, obligations, examples, reproducibility, provenance and limitations stated in a method card; its hypotheses fixed and externally timestamped before its runs; its failures reported as results; and its experiment identified by a release that is licensed, archived and can still be run and checked.**

CHAPTER S19

Rendering and media provenance

The book's figures show simulator renderings from the reported runs. Path-traced renderings of the same states were proposed as presentation assets; none has been made, so this chapter holds no gallery and claims no path-tracing run.

Rendering an existing state record changes its visual presentation. It neither repeats its physics nor establishes label validity, robust funnels, real-time execution or task completion. A media release should identify the state record, camera and rendering configuration, and should state whether it reran the physics or only rendered recorded poses.

Some simulator panels of the book are static images taken from earlier renderings of the same runs. A recovery manifest records, for each panel, the document and page it came from, its crop and its checksum. The book's build uses these panels and the native diagrams; it does not regenerate experimental data, and the plotting sources are kept for a build with the complete experiment assets.

APPENDIX SA

Safety standards and regulation: records and details

SA.1 The status of the standards and acts cited

The four tables of this section provide a dated inventory of the standards and acts that Appendix A cites, with its status checked to 28 October 2026, its scope and, in the appendix's reading, its relevance to the book. They record the state at a stated date. Public scope and status descriptions do not substitute for review of licensed standard clauses; no row establishes product applicability or conformity.

Table SA.1. The law and the standards of machinery safety and functional safety cited in Appendix A, with their status on 26 October 2026; the Directive and its list are repealed, and the Regulation applies, from 20 January 2027. The relevance is the appendix's reading.

ID and edition	Status	Scope	Relevance
Directive 2006/42/EC	repealed in 2027	EU machinery law	machines placed before 2027
Regulation (EU) 2023/1230, amended by (EU) 2024/2748	applies from 2027	EU machinery law	every robot here
Regulation (EU) 2024/2847	Art. 14 from 11 Sep. 2026; generally 11 Dec. 2027	cybersecurity of products with digital elements	assess scope and reporting
Directive (EU) 2024/2853, corrected May 2026	national transposition by 9 Dec. 2026	liability for defective products	products from 9 Dec. 2026
Decisions (EU) 2023/1586, 2026/546, 2026/2015	list repealed in 2027	the Directive's list	a presumption to 2027
Request M/605, C(2025) 129	adopted January 2025	standards requested	no list yet
ISO 12100:2010, ed. 1	current; to be revised	risk assessment (type A)	all the robots
ANSI B11.0-2023	current	US type-A analogue	the US route
ISO/TR 22100-4, -5	published 2018, 2021	IT security; learning	labels; learning
IEC 60204-1, ed. 6.1	current	stop categories	the rest as a stop
NFPA 79-2024	current	US counterpart	the US route
ISO 13850:2015, ed. 3	current; to be revised	emergency stop	outside the governor
ISO 14118:2017, ed. 2	current	unexpected start-up	restart after alarm
ISO 13849-1:2023, ed. 4	current	PL of control parts	the safety-rated part
ISO 13849-2:2012, ed. 2	current; being revised	validation	the safety-rated layer
IEC 62061, ed. 2.2 (2026)	current	SIL of control systems	the alternative to PL
IEC 61508-1 to -7, ed. 2.0	current; being revised	basic functional safety	complex software
IEC 61800-5-2:2016, ed. 2.0	current; 2007 version cited	safety functions of drives	beneath the governor
ISO 13855:2024, ed. 3	current	positioning of safeguards	stopping time
IEC 61496-1, -2, -3	current (2020, 2025)	protective equipment	not outdoors
IEC TS 62998-1:2019	current	safety-related sensors	the Husky outdoors

Table SA.2. The standards for industrial, mobile, service, legged and humanoid robots cited in Appendix A, with their status on 26 October 2026. The relevance is the appendix's reading.

ID and edition	Status	Scope	Relevance
ISO 10218-1:2025, ed. 3	current; cited 2026	industrial robots	the UR5e; arms
ISO 10218-2:2025, ed. 2	current; cited 2026	applications and cells	the UR5e's stop
ANSI/A3 R15.06-2025	published 2025	US adoption of ISO 10218	the US route
ISO/TS 15066:2016, ed. 1	to be revised	collaborative operation	a speed cap
ISO 8373:2021, ed. 3	current; to be revised	vocabulary of robotics	the robots' names
ISO 3691-4:2023, ed. 2	current; to be revised	driverless trucks	the Husky as a truck
ANSI/A3 R15.08-1, -2, -3	current	industrial mobile robots	the Husky; Type C
ANSI/ITSDF B56.5-2024	effective 2025	guided vehicles (US)	the Husky in the US
ISO/WD 26058-1	working draft	industrial mobile robots	a future ISO route
ISO 18646-1, -2, -5	current	service-robot performance	tests of labels
ISO 13482:2014, ed. 1	to be revised	personal care robots	outside industry
ISO/FDIS 13482, ed. 2	at its final vote	service robots	carrying loads
ISO/CD 25785-1	committee draft	dynamically stable robots	the Go2, the Apollo
IEEE study group report	a report, 2025	a pathway for humanoids	levels of response
ISO 22166-1:2021, ed. 1	to be revised	modular service robots	a format for labels

Table SA.3. The standards for earth-moving, agricultural and lifting machinery cited in Appendix A, with their status on 26 October 2026. The relevance is the appendix's reading.

ID and edition	Status	Scope	Relevance
ISO 20474-1:2017, ed. 2	current	earth-moving machinery	outside the EU
EN 474-1:2022	cited, restricted	earth-moving machinery	the EU route
ISO 17757:2019, ed. 2	current	autonomous machine systems	the wider system
ISO 19014-1, -2, -4, -5:2026	2026; Part 3 pending	functional safety (MPL)	whether a governor is safety-related
ISO 15998:2008	withdrawn 2022	replaced by ISO 19014	historical only
ISO 21815-1 to -3	current	collision avoidance	an added governor
ISO 16001:2017, ed. 2	current	object detection	detection time
ISO 13766-1, -2:2018	current	EMC	the electronics
ISO 4413:2010, ed. 3	current; cited	hydraulic systems	the hardware beneath
ISO 18497-1 to -4:2024	current; cited 2026	autonomous agriculture	the Husky outdoors
ISO 25119-1 to -4	being revised	agricultural control	an analogue of MPL
ISO 4305:2014	current	mobile crane stability	mobile manipulator

Table SA.4. The law, the standards and the guidance on artificial intelligence and autonomy cited in Appendix A, with their status on 26 October 2026. The relevance is the appendix's reading.

ID and edition	Status	Scope	Relevance
Regulation (EU) 2024/1689	high-risk rules from 2027 and 2028	the AI Act	a learned proposal method
Regulation (EU) 2026/1744	in force since July 2026	the Digital Omnibus on AI	AI in machinery
EN 18286:2026; Decision (EU) 2026/2274	cited 14 October 2026	AI quality management	the only presumption
ISO/IEC TR 5469:2024, ed. 1	published	functional safety and AI	the book's pattern
ISO/IEC TS 22440-1 to -3	committee drafts	AI in functional safety	the likely reference
ISO/IEC 23894:2023	published	AI risk management	labels and risk
ISO/IEC 42001:2023	published	AI management systems	little for a machine
ISO/IEC 22989:2022	published	AI concepts and terms	terminology
ISO/IEC TS 8200:2024	published	controllability of AI	recovery as transfer
ISO/IEC 24029-2:2023	published	robustness of networks	a learned estimator
ISO 26262:2018, ed. 2	current; being revised	road-vehicle safety	an analogue
ISO 21448:2022, ed. 1	to be revised	intended functionality	an invalid label
ISO/PAS 8800:2024	published	AI in road vehicles	an analogue
UL 4600, ed. 3 (2023)	current	autonomous products	labels as indicators
IEEE 7001-2021	active	transparency	the governor's records
IEEE 7009-2024	active	fail-safe design	the certified recovery
NIST AI RMF 1.0 (2023)	current; being revised	voluntary AI framework	monitoring, shut-down

SA.2 Routes to conformity under the Machinery Regulation

Table SA.5 gives the procedures by which the conformity of a machine is assessed under the Machinery Regulation, by its category, for Appendix A.1.3.

Table SA.5. The procedures by which conformity is assessed under Article 25 of Regulation (EU) 2023/1230, by the category of the machine in its Annex I, with the entries of that annex closest to this book; paraphrased from the text in the Official Journal, which Appendix A.1.3 cites. The modules are those of the Regulation's Annexes VI to X.

Category	Entries relevant to this book	Procedures
Annex I, Part A	safety components with fully or partially self-evolving behaviour using machine learning that ensure safety functions (item 5); machinery with such embedded systems, not placed on the market independently, for those systems only (item 6)	EU type-examination (module B) with internal production control (module C); full quality assurance (module H); or unit verification (module G)
Annex I, Part B	protective devices designed to detect the presence of persons (item 15); logic units to ensure safety functions (item 17); roll-over and falling-object protective structures (items 18 and 19)	internal production control (module A) only if harmonized standards or common specifications covering every relevant requirement are applied; otherwise B with C, H or G
Not in Annex I	most machinery; the embedded systems of item 6 follow Part A	internal production control (module A)

SA.3 Per-robot source map

This table records design questions for future products; the examples remain simulation observations with the limitations of the main text.

Table SA.6. Candidate standards and drafts to investigate for a product based on Part V. The mapping is the author's analysis from public source descriptions, not an assessment of applicability or licensed clauses. The simulation observations are not product validation; Table A.2 states the general limits.

Standard	Function or quantity	Related simulation or analysis	From elsewhere
The Husky A300 (Chapter 13)			
Regulation (EU) 2023/1230, Annex III, 3.3.3	braking and stopping under every allowed condition of load, speed, ground and gradient	the stops at each ground's label (Table S13.13); the crossing under the shower's forecast (Section 13.2.5)	the risk assessment; an enclosed zone or detection of persons

Standard	Function or quantity	Related simulation or analysis	From elsewhere
ISO 3691-4:2023 as a truck; ANSI/A3 R15.08-1 and -2 as a mobile robot	the stops; speeds by zone; stability	a recovery's deceleration set by the label; the reserves before the dock and in the lane (Section 13.2.4)	tests of the detection of persons; the required levels; type tests
IEC TS 62998-1:2019	the detection of persons outdoors	none: the alarm check is no protective device	a safety-rated sensor system
The UR5e arm (Chapter 14)			
ISO 10218-1 and -2:2025; ANSI/A3 R15.06-2025	stopping distance with a declared payload; the setting of the payload	the simulated candidate recovery; the declared interval's label, its stop and its latest start (Table 14.3); the empty arm's label as a counterexample	the measurement of Annex H, which funnels cannot replace; the levels of Annex C
ISO/TS 15066; ISO 10218-2, Annexes L and M	S_s in Equation (SA.1); a speed cap from Equation (SA.2)	a simulated stop as a candidate S_s; the payload label in μ	biomechanical limits; measured forces
The heavy-duty mobile manipulator (Chapter 14)			
ANSI/A3 R15.08-2-2023, Type C; R15.06 for its arm	braking and turning without tipping	the tipping reserve as a constraint; the braking limits by pose and label (Table 14.4)	type tests of stability; identified masses and tyres
Regulation, Annex III, 1.3.1, 3.4.1 and 4.1.2.1; ISO 4305 as an analogue	stability against overturning, in motion and when lifting	the reserve and its levels; the misdeclared payload as a failure (Section 14.3.5)	the dynamic case: a research extension
ISO 19014:2026 and ISO 17757:2019, by its intended use	whether a governor that stops or vetoes is safety-related	the failure types read as a governor's; the nominal correctness of the recovery	the MCSSA; the system's hazard analysis
The Go2 and the Apollo (Chapter 15)			
ISO/CD 25785-1, a draft whose content is not public	a stability margin; the behaviour on a loss of power	the margin of the zero-moment point as a constraint with a reserve; the lift as a demand; the guarantee boundary (Table 15.3)	the published standard; tests of falls and of power loss
ISO 13482:2014 and ISO/FDIS 13482	stability control; instability while carrying loads	the Go2's empirical stance and lift-off with bounds fitted to the reported runs; the Apollo's carry, under an external controller, as an empirical demonstration	stability tests on the robot
IEC 60204-1; ISO 13850; ISO 14118	stopping and restart; the stored energy of the raised body and of the beam	the powered standing rest and simulated shutdown, without a validated stop-category assignment (Section 15.2.5)	a certified shutdown; an assessment of the fall; an analysis of the rest as a safe state
ISO 10218-1:2025, for a humanoid's arms	the arms' stopping; mobility excluded	as for the UR5e	the platform's balance

SA.4 Public-source notes on the robot families

These dated notes retain the publicly described scopes and the technical analogies behind the concise orientation in Appendix A. They are an attributed source map from the catalogues, previews and named commentaries cited; the licensed clause texts were not reviewed. Clausal summaries and draft details

must therefore be checked against the applicable full edition before use in a product assessment. The notes establish no conformity finding, no stop-category assignment for an example and no gap in the actual requirements when a public summary is silent.

SA.4.1 Industrial robots and their applications

An industrial arm such as the UR5e of Chapter 14 is partly completed machinery, which becomes a machine only when it is integrated, with its tool and its task, into an application.

ISO 10218-1:2025, the third edition, replaces that of 2011 (International Organization for Standardization, 2025a). Its industrial robot is an automatically controlled, reprogrammable, multipurpose manipulator, programmable in three or more axes, fixed in place or to a mobile platform, but the standard does not cover the hazards of the platform's own motion; the payload is the mass of everything attached to the manipulator, the end-effector and the workpiece included. Its public table of contents places the setting of the payload (5.1.15), the parameterization of safety functions (5.3.5), stopping (5.4) and further safety functions (5.5), among them the monitored standstill and the limits on stopping time and on stopping distance, new in 2025; a normative Annex C sets out the safety functions, about 38 in the final draft as ISO/TC 299 reports, not all mandatory or at PL d, with a target of PL d with category 3, or SIL 2 with a hardware fault tolerance of 1, and a normative Annex H measures stopping time and distance. ISO 10218-2:2025, the second edition, covers the robot application and the robot cell over their life (International Organization for Standardization, 2025b); it incorporates most of ISO/TS 15066, and its annexes treat the performance of safety functions (Annex C), the separation distance (Annex L) and limits for contact by clamping and by impact (Annex M). Both parts have been cited under the Directive since 7 September 2026 (European Commission, 2026a). With the revision, only an application can be collaborative; the safety-rated monitored stop became a safety function, the monitored standstill, which follows a stop of category 2; robots fall into two classes, Class I having a low capability to transfer energy, with limits that secondary sources give from drafts but that could not be verified; and requirements on cybersecurity were added as far as they bear on safety. ANSI/A3 R15.06-2025 adopts both parts as its Parts 1 and 2 and adds a national Part 3 for the users of robot cells; Parts 1 and 2 went on sale on 9 September 2025, and the edition with all three parts was published on 29 October 2025 (Association for Advancing Automation, 2025). ISO 8373:2021 is the vocabulary of robotics (International Organization for Standardization, 2021b).

ISO/TS 15066:2016 supplemented the collaborative requirements of the editions of 2011 (International Organization for Standardization, 2016); it has been marked 'to be revised' since 26 June 2025, and ISO lists as its replacement ISO/AWI 15066-1, a new project on biomechanical thresholds and data. In speed and separation monitoring, the robot keeps at least a protective separation distance from a person, which the specification, as reproduced in peer-reviewed work, writes as

$$S_p = S_h + S_r + S_s + C + Z_d + Z_r : \tag{SA.1}$$

the person's travel while the robot reacts and stops (S_h), the robot's travel while it reacts (S_r), its stopping distance (S_s), the intrusion before detection (C), and the uncertainties of the person's and of the robot's positions (Z_d, Z_r); we did not verify that Annex L of ISO 10218-2:2025 has the same form. In power and force limiting, a robot of effective mass m_R that strikes a body region of mass m_H and stiffness k at the speed v stores its kinetic energy $\frac{1}{2}\mu v^2$, with $\mu = (1/m_H + 1/m_R)^{-1}$, as $\frac{1}{2}F^2/k$ at the force F, so that a limit $F_{\max}$ on the force caps the speed:

$$v \le \frac{F_{\max}}{\sqrt{\mu k}}. \tag{SA.2}$$

As the literature describes the specification, its limits of force and pressure for regions of the body rest on this model, with the effective mass taken as half the moving mass plus the payload, so that a heavier payload lowers the permissible speed; Behrens and colleagues, of the Fraunhofer IFF and its partners, state that most of these limits are preliminary, because they rest on unverified data from the literature. In the book's terms, Equation (SA.1) is a reserve, with Z_r in the place of the bound on the measurement error, and Equation (SA.2) turns a limit on force into a cap on speed that consumes a validated payload label (Table SA.6).

SA.4.2 Mobile robots and mobile manipulators

A robot that travels can strike a person in its path, and it can tip over or fail to brake on the ground that it crosses. The Husky of Chapter 13 and the base of the heavy-duty mobile manipulator of Chapter 14 could fall under the standards for driverless trucks or for industrial mobile robots, or, as research platforms, under neither.

ISO 3691-4:2023, the second edition, gives safety requirements, and the means of verifying them, for driverless industrial trucks, which carry or tow loads, and for their systems, autonomous mobile robots among them; it excludes public roads and zones open to the public (International Organization for Standardization, 2023). Its table of contents shows clauses on stops, braking, speed and stability, with a tilting-platform test, and on the detection of persons

in the path, and an annex that determines the rated capacity. As TÜV Rheinland described its edition of 2020, a truck must stop before its rigid parts or its load touch a stationary person, its safety functions having minimum required levels under ISO 13849-1. EN ISO 3691-4:2023 is cited under the Directive (European Commission, 2023), and the standard is being revised. The US series ANSI/A3 R15.08 covers industrial mobile robots: Part 1 for manufacturers (2020, reaffirmed in 2026), Part 2 for integrators (2023) and Part 3 for users (2026) (Association for Advancing Automation, 2020, 2023, 2026). Its Type C robot is a mobile manipulator, whose manipulator, according to SICK, must comply with R15.06, while R15.08 addresses the hazards of integrating it with the platform; as the IEEE study group's report reads Part 1, a robot must remain stable in travel, and the only cause of instability considered is too heavy a payload (IEEE Humanoid Study Group, 2025). ANSI/ITSDF B56.5-2024 is the US standard for driverless, automatic guided industrial vehicles (Industrial Truck Standards Development Foundation, 2024), and ISO/WD 26058-1, the working draft of an ISO type-C standard for industrial mobile robots, excludes robots whose stability is actively controlled (International Organization for Standardization, 2025c). The ISO 18646 series gives test methods for the performance of service robots, not meant for verifying safety, in wheeled locomotion indoors (Part 1, 2016), navigation (Part 2, 2024) and legged locomotion (Part 5, 2026) (International Organization for Standardization, 2026a): in the book's terms, protocols for measuring what a capability label declares.

A mobile manipulator falls between these standards: ISO 10218 covers its arm but not the platform's motion, ISO 3691-4 covers trucks, The Robot Report has described R15.08-2 as the first standard to define safety considerations for mobile manipulators, and in 2013 NIST found that B56.5 did not address an onboard robot that moves while the base moves. In the book's terms, whether the Husky falls under ISO 3691-4 depends on whether it carries or tows loads as a truck, which a deployment must settle. Tipping while braking and turning with a load is not among the requirements that we could verify, and what the clause of ISO 3691-4 on speed and stability asks could not be verified, so the book's dynamic margin against tipping is a research extension, not a method of compliance.

SA.4.3 Earth-moving and other heavy machines

Heavy working machines have families of standards of their own, written mostly for machines with an operator on board. Which family would apply to a machine such as the heavy-duty mobile manipulator of Chapter 14 depends on its intended use, which the book does not fix for its synthetic machine.

ISO 20474 is the international type-C series for earth-moving machinery; its Part 1, of 2017, gives the general requirements and leaves autonomous machines to ISO 17757 (International Organization for Standardization, 2017b). Of the European series EN 474, the one cited under the Directive, EN 474-1:2022 gives no performance requirements for the safety-related functions of control systems, and since 13 March 2026 it has been cited with a restriction on the test loads of two of its lifting clauses (European Committee for Standardization, 2022; European Commission, 2026b). ISO 17757:2019 gives safety requirements for autonomous and semi-autonomous machines in earth-moving and mining and for the systems they belong to, infrastructure and software included; we found no European adoption (International Organization for Standardization, 2019a). The ISO 19014 series, the functional-safety standard of earth-moving machinery, was reissued on 9 October 2026, except Part 3, which was under publication; it replaced ISO 15998:2008, withdrawn in 2022, and its parts of 2026 do not apply to machines made before them (International Organization for Standardization, 2026b, 2008). Its machine control system safety analysis (MCSSA) decides which control systems are safety-related, and what machine performance level (MPL), from a to e, each needs; in the edition of 2018, the analysis considered failure types such as a failure to apply, an uncommanded apply, an incorrect rate and an incorrect direction, and whether this is unchanged in 2026 could not be verified. ISO 21815 specifies collision warning and avoidance for earth-moving and mining machinery by slowing, stopping or inhibiting motion, without automatic steering, its Part 2 an interface through which an independently integrated device requests the machine to slow or stop; ISO 16001:2017 sets the performance of object detection systems, with a detection time and continuous self-checking (International Organization for Standardization, 2022, 2017a). ISO 13766-1 and -2:2018 set electromagnetic compatibility, with requirements for functional safety in parts at PL b or higher, and ISO 4413:2010 covers hydraulic systems (International Organization for Standardization, 2018, 2010). Outside earth-moving, ISO 18497-1 to -4:2024 give design principles for automated and autonomous agricultural machinery, with obstacle protection and the prevention of unintended excursions beyond an autonomous operating zone; their European adoptions were added to the Directive's list in September 2026, and ISO 25119 gives the functional safety of agricultural control systems, with agricultural performance levels (International Organization for Standardization, 2024; European Commission, 2026a; International Organization for Standardization, 2019b). For a machine that lifts, the nearest analogues of a rated capacity are, in our reading, those of cranes: ISO 4305:2014 verifies the stability of a mobile crane by calculation, on a firm,

level surface, and so statically (International Organization for Standardization, 2014b).

In the book's terms. Under ISO 19014, the MCSSA would decide whether a governor that can stop or veto a heavy machine's motion is safety-related; the failure types read as a governor's, a failure to apply as an unsafe proposal not blocked and an uncommanded apply as a spurious recovery, and the book's certified recoveries address only their own nominal correctness. In the system of ISO 17757, whose hazard analysis also covers infrastructure, communications and the people who enter the zone, the book contributes nothing beyond such a governor, and the interface of ISO 21815-2 is a precedent for adding one to an existing machine. ISO 18497 is the most explicit published framework here for an autonomous ground vehicle outdoors, although the Husky is not an agricultural machine. This is our reading.

SA.4.4 Service, legged and humanoid robots

A legged robot or a humanoid does not simply stand still when its power is cut: it may fall. According to an IEEE study group, the specialized standards for robots assume a base that is fixed or statically stable; the standard written for such robots in industry is still a draft, and the standard for service robots is being revised.

ISO 13482:2014 gives requirements for personal care robots, namely mobile servant robots, physical assistant robots and person carriers, and excludes industrial and medical robots (International Organization for Standardization, 2014a); its requirements for safety-related control cover stopping, speed, environmental sensing and stability control, and, according to the IEEE study group, it does not address self-balancing robots or a changing base of support. Its revision, ISO/FDIS 13482, *Safety requirements for service robots*, began its final vote on 15 September 2026 (International Organization for Standardization, 2026d); its enquiry draft of 2024 named among its hazards instability during travel, while carrying loads and in a collision, and the final text may differ. ISO/CD 25785-1 is the draft of safety requirements for dynamically stable industrial mobile robots, legged, wheeled or otherwise, whose stability is actively controlled (International Organization for Standardization, 2026c), in our reading the standard that will apply most directly to the Go2 and the Apollo in a factory. It is a committee draft, whose comments closed on 8 July 2026, with no date set for its later stages; A3, which took part in the work, explains that actively balanced robots need power continuously to stay upright. The IEEE study group's *Pathway Study for Future Humanoid Standards*, of September 2025, is a technical report and not a standard (IEEE Humanoid Study Group, 2025). It notes that a humanoid with its joints braked and its power off is not necessarily safe, since 'it must

be assumed that a locked pose is unstable'; it ranks the responses to a fault in levels, from cutting the power or braking (levels 0 and 1) to operating at all times so that a failure or a loss of power endangers no one (level 4), with curling up and planning a safe state from perception between them; and it recommends test methods and metrics for stability, online measurement among them, and safety standards whose thresholds can be built into a controller's design. ISO 22166-1:2021, on modular frameworks and modules of service robots, states that it is not a safety standard (International Organization for Standardization, 2021a).

In the book's terms. For a robot that balances, removal of actuation can itself create a hazard. The Go2 example ends its candidate plans standing, with power and control assumed available (Section 15.2.5). The book's legged results are research extensions towards these documents, not compliance with them. This is our reading.

References

Association for Advancing Automation. ANSI/A3 R15.08-1-2020 (R2026), Industrial mobile robots – Safety requirements – Part 1: Requirements for the industrial mobile robot. American National Standard, 2020, reaffirmed 2026, 2020. URL https://webstore.ansi.org/standards/ria/ansia3r15082020r2026. Accessed 26 October 2026.

Association for Advancing Automation. ANSI/A3 R15.08-2-2023, Industrial mobile robots – Safety requirements – Part 2: Requirements for IMR system(s) and IMR application(s). American National Standard, 2023, 2023. URL https://webstore.ansi.org/standards/ria/ansia3r15082023. Accessed 26 October 2026.

Association for Advancing Automation. ANSI/A3 R15.06-2025 and ANSI/A3 R15.06-3-2025, Industrial robots and robot systems – Safety requirements (Parts 1, 2 and 3). American National Standard, published 2025-10-29, 2025. URL https://webstore.ansi.org/standards/ria/ANSIA3R15062025PartsPart. Accessed 26 October 2026.

Association for Advancing Automation. ANSI/A3 R15.08-3-2026, Industrial mobile robots – Safety requirements – Part 3: Use of IMR applications. American National Standard, published 2026-04-23, 2026. URL https://webstore.ansi.org/standards/ria/ansia3r15082026. Accessed 26 October 2026.

Cheng Chi and Diffusion Policy Contributors. Diffusion Policy: The authors' code. Source code, commit `5ba07ac`, 2024. URL https://github.com/real-stanford/diffusion_policy. Accessed 27 October 2026.

Clearpath Robotics. clearpath_common, release 2.9.18. Source code, commit `78e3677`, 2026a. URL https://github.com/clearpathrobotics/clearpath_common. Accessed 23 October 2026.

Clearpath Robotics. Husky A300 user manual. Online documentation, 2026b. URL https://docs.clearpathrobotics.com/docs_robots/outdoor_robots/husky/a300/user_manual_husky. Last updated 2 July 2026; accessed 23 October 2026.

European Commission. Commission implementing decision (EU) 2023/1586 of 26 July 2023. *Official Journal of the European Union*, L 194, 2 August 2023, 2023. URL https://eur-lex.europa.eu/eli/dec_impl/2023/1586/oj/eng. The list of harmonized standards for machinery under Directive 2006/42/EC, as amended up to Decision (EU) 2026/2015; accessed 26 October 2026.

European Commission. Commission implementing decision (EU) 2026/2015 of 4 September 2026. *Official Journal of the European Union*, L series, 7 September 2026, 2026a. URL https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?

uri=OJ:L_202602015. Amends the list of harmonized standards under Directive 2006/42/EC and repeals Decision (EU) 2023/1586 with effect from 20 January 2027; accessed 26 October 2026.

European Commission. Commission implementing decision (EU) 2026/546 of 12 March 2026. *Official Journal of the European Union*, L series, March 2026, 2026b. URL https://www.boe.es/doue/2026/546/L00001-00006.pdf. Amends the list of harmonized standards under Directive 2006/42/EC; accessed 26 October 2026.

European Committee for Standardization. EN 474-1:2022, Earth-moving machinery – Safety – Part 1: General requirements. European standard, 2022-03-29, 2022. URL https://standards.iteh.ai/catalog/standards/cen/84021cbc-6757-4970-856a-4ea8e5a34871/en-474-1-2022. Accessed 26 October 2026.

Genesis Authors. Genesis world documentation. Official documentation, latest version (release 1.4.4), 2026a. URL https://genesis-world.readthedocs.io/en/latest/. Pages Introduction, `gs.morphs.MJCF` and `RigidOptions` (API reference); accessed 22 October 2026.

Genesis Authors. Genesis world, release 1.4.4. Source code, Python package `genesis-world`, 2026b. URL https://github.com/Genesis-Embodied-AI/genesis-world. Accessed 22 October 2026.

Google DeepMind. MuJoCo changelog. Official documentation, stable version, 2026a. URL https://mujoco.readthedocs.io/en/stable/changelog.html. Listing version 3.17.0 of 22 October 2026 after version 3.16.0 of 14 October 2026; accessed 27 October 2026.

Google DeepMind. MuJoCo documentation, version 3.16.0. Official documentation, 2026b. URL https://mujoco.readthedocs.io/en/3.16.0/. Pages Overview, Computation, Modeling, Simulation, Visualization, XML Reference and Changelog, the last listing version 3.16.0 of 14 October 2026; accessed 22, 23 and 27 October 2026, on the last date in this versioned copy.

Google DeepMind. MuJoCo, release 3.16.0. Source code, tag `3.16.0`, commit `a892750`, 2026c. URL https://github.com/google-deepmind/mujoco. Accessed 23 October 2026.

Nicklas Hansen. TD-MPC2: Official implementation. Source code, commit `332f19e`, 2026. URL https://github.com/nicklashansen/tdmpc2. Accessed 27 October 2026.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

IEEE Humanoid Study Group. A pathway study for future humanoid standards. Technical report, IEEE Robotics and Automation Society study group, September 2025, 2025. URL https://www.therobotreport.com/wp-content/uploads/2025/09/IEEE-Humanoid-Report-of-Future-Standards-Development.pdf. Not a standard; accessed 26 October 2026.

Industrial Truck Standards Development Foundation. ANSI/ITSDF B56.5-2024, Safety standard for driverless, automatic guided industrial vehicles and automated functions

of manned industrial vehicles. American National Standard, 2024, effective 16 December 2025, 2024. URL https://www.itsdf.org/cue/b56-standards.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 15998:2008, Earth-moving machinery – Machine-control systems (MCS) using electronic components – Performance criteria and tests for functional safety. International standard, first edition, 2008-04; withdrawn 2022-06-14, 2008. URL https://www.iso.org/standard/28559.html. Withdrawn; accessed 26 October 2026.

International Organization for Standardization. ISO 4413:2010, Hydraulic fluid power – General rules and safety requirements for systems and their components. International standard, third edition, 2010-11, 2010. URL https://www.iso.org/standard/44781.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 13482:2014, Robots and robotic devices – Safety requirements for personal care robots. International standard, first edition, 2014-02, 2014a. URL https://www.iso.org/standard/53820.html. Current on 26 October 2026, to be replaced by its second edition; accessed 26 October 2026.

International Organization for Standardization. ISO 4305:2014, Mobile cranes – Determination of stability. International standard, third edition, 2014-03-20, with Amendment 1:2016, 2014b. URL https://www.iso.org/standard/57220.html. Accessed 26 October 2026.

International Organization for Standardization. ISO/TS 15066:2016, Robots and robotic devices – Collaborative robots. Technical specification, first edition, 2016-02, 2016. URL https://www.iso.org/standard/62996.html. Current on 26 October 2026, to be replaced by ISO/AWI 15066-1; accessed 26 October 2026.

International Organization for Standardization. ISO 16001:2017, Earth-moving machinery – Object detection systems and visibility aids – Performance requirements and tests. International standard, second edition, 2017-11, 2017a. URL https://www.iso.org/standard/63688.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 20474-1:2017, Earth-moving machinery – Safety – Part 1: General requirements. International standard, second edition, 2017-07-07, 2017b. URL https://www.iso.org/standard/60734.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 13766-1:2018 and 13766-2:2018, Earth-moving and building construction machinery – Electromagnetic compatibility (EMC) of machines with internal electrical power supply. International standards, first editions, 2018-04: Part 1, General EMC requirements; Part 2, Additional EMC requirements for functional safety, 2018. URL https://www.iso.org/standard/67347.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 17757:2019, Earth-moving machinery and mining – Autonomous and semi-autonomous machine system safety. International standard, second edition, 2019-07, 2019a. URL https://www.iso.org/standard/76126.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 25119-1 to -4, Tractors and machinery for agriculture and forestry – Safety-related parts of control systems. International standards: Parts 1, 3 and 4 of 2018 and Part 2 of 2019, with amendments of 2020, 2019b. URL https://www.iso.org/standard/69025.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 22166-1:2021, Robotics – Modularity for service robots – Part 1: General requirements. International standard, first edition, 2021-02, 2021a. URL https://www.iso.org/standard/72715.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 8373:2021, Robotics – Vocabulary. International standard, third edition, 2021-11, 2021b. URL https://www.iso.org/standard/75539.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 21815, Earth-moving machinery – Collision warning and avoidance. Series: Part 1, General requirements, 2022; ISO/TS 21815-2, On-board J1939 communication interface, 2021; Part 3, 2023; Parts 4 and 5 under publication in 2026, 2022. URL https://www.iso.org/standard/77302.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 3691-4:2023, Industrial trucks – Safety requirements and verification – Part 4: Driverless industrial trucks and their systems. International standard, second edition, 2023-06-12, 2023. URL https://www.iso.org/standard/83545.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 18497-1 to -4:2024, Agricultural machinery and tractors – Safety of partially automated, semi-autonomous and autonomous machinery. International standards in four parts, first editions, 2024-07-31, 2024. URL https://www.iso.org/standard/82684.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 10218-1:2025, Robotics – Safety requirements – Part 1: Industrial robots. International standard, third edition, 2025-02, 2025a. URL https://www.iso.org/standard/73933.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 10218-2:2025, Robotics – Safety requirements – Part 2: Industrial robot applications and robot cells. International standard, second edition, 2025-02, 2025b. URL https://www.iso.org/standard/73934.html. Accessed 26 October 2026.

International Organization for Standardization. ISO/WD 26058-1, Safety requirements for industrial mobile robots – Part 1: Industrial mobile robots. Working draft, stage 20.60 on 2025-12-18, 2025c. URL https://www.iso.org/standard/92460.html. A draft, not a standard; accessed 26 October 2026.

International Organization for Standardization. ISO 18646-1:2016, -2:2024 and -5:2026, Robotics – Performance criteria and related test methods for service robots. International standards: Part 1, Locomotion for wheeled robots, first edition, 2016-09-16; Part 2, Navigation, second edition, 2024-01-22; Part 5, Locomotion for legged robots,

first edition, 2026-07-24, 2026a. URL https://www.iso.org/standard/86850.html. Accessed 26 October 2026.

International Organization for Standardization. ISO 19014-1, -2, -4 and -5:2026, Earth-moving machinery – Functional safety. International standards: Parts 1, 2 and 4, second edition, and Part 5, first edition, all 2026-10-09; Part 3, second edition, under publication, 2026b. URL https://www.iso.org/standard/87988.html. Accessed 26 October 2026.

International Organization for Standardization. ISO/CD 25785-1, Robotics – Safety requirements for dynamically stable industrial mobile robots (legged, wheeled, or other forms of locomotion) – Part 1: Robots. Committee draft, stage 30.60 on 2026-07-08, 2026c. URL https://www.iso.org/standard/91469.html. A draft, not a standard; accessed 26 October 2026.

International Organization for Standardization. ISO/FDIS 13482, Robotics – Safety requirements for service robots. Final draft international standard, second edition, FDIS ballot from 2026-09-15, 2026d. URL https://www.iso.org/standard/83498.html. A draft, not a standard; accessed 26 October 2026.

Newton Developers. mujoco-usd-converter, releases 0.6.1 and 0.8.0. Source code, Python package `mujoco-usd-converter`, 2026a. URL https://github.com/newton-physics/mujoco-usd-converter. Accessed 22 October 2026.

Newton Developers. Newton documentation. Official documentation, stable version (release 1.7.0), 2026b. URL https://newton-physics.github.io/newton/stable/. Pages Frequently Asked Questions and ModelBuilder (API reference); accessed 22 October 2026.

Newton Developers. Newton, release 1.7.0. Source code, Python package `newton`, 2026c. URL https://github.com/newton-physics/newton. Accessed 22 October 2026.

NVIDIA. PhysX SDK documentation. Official documentation, version 5.6.1, 2025a. URL https://nvidia-omniverse.github.io/PhysX/physx/5.6.1/. Pages Rigid Body Dynamics, Articulations and Advanced Collision Detection; accessed 22 October 2026.

NVIDIA. PhysxSchema API reference. API documentation of the USD schemas of PhysX, latest version, last updated 4 February 2025, 2025b. URL https://docs.omniverse.nvidia.com/kit/docs/omni_usd_schema_physics/latest/. Classes PhysxCollisionAPI and PhysxSceneAPI; accessed 22 October 2026.

NVIDIA. Isaac Lab 3.0.0. Python package `isaaclab` on PyPI, 2026a. URL https://pypi.org/project/isaaclab/. Released 19 October 2026; accessed 22 October 2026.

NVIDIA. Isaac Sim documentation. Official documentation, release 6.2.0, 2026b. URL https://docs.isaacsim.omniverse.nvidia.com/6.2.0/. Pages Home, Release notes, Requirements and Cross-product compatibility; accessed 22 October 2026.

NVIDIA. Isaac Sim 6.2.0.0. Python package `isaacsim` on PyPI, 2026c. URL https://pypi.org/project/isaacsim/. Released 13 October 2026; accessed 22 October 2026.

NVIDIA. Isaac Sim, release 6.2.0. Source code, tag `v6.2.0`, commit `74a9356`, 2026d. URL https://github.com/isaac-sim/IsaacSim. Accessed 22 October 2026.

NVIDIA. PhysX SDK: Contact friction under TGS and PGS. Source code: `DyTGSContactPrep.cpp`, `DyArticulationContactPrep.cpp` and `DySolverConstraints.cpp`, at commit `b97365c` of branch `main` (version 5.12.0) and at tag `110.3-omni-and-physx-5.9.1`, 2026e. URL https://github.com/NVIDIA-Omniverse/PhysX. Folder `physx/source/lowleveldynamics/src`; accessed 29 October 2026.

NVIDIA. PhysX SDK. Source code, branch `main`: `PxSimpleTypes.h`; `DyTGSDynamics.cpp` and `DyFeatherstoneArticulation.cpp` at commit `b97365c`, 2026f. URL https://github.com/NVIDIA-Omniverse/PhysX. Folders `physx/include/foundation` and `physx/source/lowleveldynamics/src`; accessed 22 and 23 October 2026.

NVIDIA Developer Forums. Skid-steered behavior for robots. Forum thread, Isaac Sim category, 2023a. URL https://forums.developer.nvidia.com/t/skid-steered-behavior-for-robots/263112. Started 14 August 2023; accessed 29 October 2026.

NVIDIA Developer Forums. Wheeled robot incorrect behavior. Forum thread, Isaac Sim category, 2023b. URL https://forums.developer.nvidia.com/t/wheeled-robot-incorrect-behavior/245133. Started 6 March 2023; accessed 29 October 2026.

Mike Oitzman. Apptronik unveils Apollo humanoid robot. The Robot Report, 23 August 2023, 2023. URL https://www.therobotreport.com/apptronik-unveils-apollo-humanoid-robot/. Accessed 26 October 2026.

Open Robotics. Gazebo Sim 8: WheelSlip class reference. API documentation, version 8.15.0, 2026. URL https://gazebosim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1WheelSlip.html. Accessed 23 October 2026.

Pixar Animation Studios. Universal Scene Description: UsdPhysics classes. API documentation, development version, 2026. URL https://openusd.org/dev/api/class_usd_physics_drive_a_p_i.html. Classes UsdPhysicsDriveAPI and UsdPhysicsScene; accessed 22 October 2026.

Python Software Foundation. random — generate pseudo-random numbers. The Python Standard Library, version 3.14, 2026. URL https://docs.python.org/3.14/library/random.html. Section “Notes on Reproducibility”; accessed 21 October 2026.

Antonin Raffin and Stable-Baselines3 Contributors. Stable-Baselines3, release 2.9.2. Source code, tag `v2.9.2`, commit `0cdcd20`, 2026. URL https://github.com/DLR-RM/stable-baselines3. Accessed 27 October 2026.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL https://jmlr.org/papers/v22/20-1364.html.

Mehdi Heydari Shahna. Team-Impedance-Control. Source code, 2026. URL https://github.com/Mehdi-Heydari-Shahna/Team-Impedance-Control. Commit eb530ea, 19 October 2026; no project-wide licence; its Apollo model from MuJoCo Menagerie under the Apache License 2.0.

Mehdi Heydari Shahna, Joongheon Kim, and Jouni Mattila. RoboCompiler: Graph-native compilation of closed-chain robots for consistent modeling, control, and simulation. arXiv preprint arXiv:2609.35717, 2026a. arXiv version 2 of 29 September 2026; the description in Section 10.4 is taken from the manuscript in the code repository at commit a4e4437.

Mehdi Heydari Shahna, Joongheon Kim, and Jouni Mattila. RoboCompiler. Source code, 2026b. URL https://github.com/Mehdi-Heydari-Shahna/RoboCompiler. Commit a4e4437, 10 October 2026; Apache License 2.0.

Stahlwerk Thüringen. European wide flange beams: Dimensions and section properties. Data sheet, undated, 2026. URL https://stahlwerk-thueringen.de/files/144215F3B6C. Dimensions to EN 10365:2017; accessed 26 October 2026.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032, 2024. arXiv version 4 of 2 November 2025.

Universal Robots. Max. joint torques, CB3 and e-Series. Support article, 2015. URL https://www.universal-robots.com/articles/ur/robot-care-maintenance/max-joint-torques-cb3-and-e-series/. Last modified 26 August 2015; accessed 24 October 2026.

Universal Robots. DH parameters for calculations of kinematics and dynamics. Support article, 2018. URL https://universal-robots.com/articles/ur/parameters-for-calculations-of-kinematics-and-dynamics. Last modified 4 October 2018; accessed 24 October 2026.

Universal Robots. UR5e technical specifications. Datasheet, 2024. URL https://universal-robots.com/media/1807465/ur5e_e-series_datasheets_web.pdf. Dated 08/2023, updated December 2024; accessed 24 October 2026.

Universal Robots. Universal_Robots_ROS2_Description (ur_description), release 3.6.0. Source code, commit 7ce98d8, 2026. URL https://github.com/UniversalRobots/Universal_Robots_ROS2_Description. Accessed 24 October 2026.

Kevin Zakka, Yuval Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. Source code, commit `f3b6013`, 2022. URL https://github.com/google-deepmind/mujoco_menagerie. Accessed 24 October 2026.